

Containerisierung: Docker & Kubernetes

Agenda

1. Docker-Grundlagen

- [Übersicht Architektur](#)
- [Was ist ein Container ?](#)
- [Was sind container images](#)
- [Container vs. Virtuelle Maschine](#)
- [Was ist ein Dockerfile](#)
- [Dockerfile - image kleinhalten](#)
- [Übung: Java REST-API mit Multi-Stage Dockerfile](#)
- [Docker installieren \(Ubuntu, apt\)](#)
- [Bind-Mounts](#)
- [Bind-Mounts: Berechtigungen \(Rocky/SELinux\)](#)
- [Docker Security Overview](#)
- [Image-Scan mit docker scan \(snyk\)](#)

2. Kubernetes - Überblick

- [12-Factor-App - Design Prinzipien fuer Cloud Native Anwendungen](#)
- [Warum Kubernetes, was macht Kubernetes](#)
- [Aufbau Allgemein](#)
- [Kubernetes Architektur Deep-Dive](#)
- [Ausbaustufen Kubernetes](#)
- [Wann macht Kubernetes Sinn, wann nicht?](#)
- [Aufbau mit helm, OpenShift, Rancher \(RKE\), micro8s](#)
- [Welches System ? \(minikube, micro8s etc.\)](#)
- [Installer für grosse Cluster](#)
- [Installation - Welche Komponenten from scratch](#)

3. Kubernetes (Use Cases)

- [Kubernetes Use Cases](#)

4. Kubernetes Cluster ausrollen

- [Kubernetes Cluster mit terraform und bash-scripting ausrollen](#)

5. Kubernetes - Upgrade

- [Wie Upgrade ich Kubernetes in kubeadm](#)

6. kubectl

- [kubectl einrichten mit namespace](#)
- [kubectl cheatsheet kubernetes](#)
- [kubectl mit verschiedenen Clustern arbeiten](#)
- [2 kubeconfig - Dateien zusammenführen](#)

7. Kubernetes Praxis API-Objekte

- [Das Tool kubectl \(Devs/Ops\) - Spickzettel](#)
- [kubectl example with run](#)
- [Bauen einer Applikation mit Resource Objekten](#)
- [Anatomie einer Webanwendungen](#)
- [kubectl/manifest/pod](#)
- [ReplicaSets \(Theorie\) - \(Devs/Ops\)](#)
- [kubectl/manifest/replicaset](#)
- [Deployments \(Devs/Ops\)](#)
- [kubectl/manifest/deployments](#)
- [Debugging](#)
- [Netzwerkverbindung zum Pod testen](#)
- [Services \(Devs/Ops\)](#)
- [kubectl/manifest/service](#)
- [DaemonSets \(Devs/Ops\)](#)
- [ConfigMap Example MariaDB](#)
- [ConfigMap Example nginx.conf](#)
- [Secrets Example MariaDB](#)
- [Connect to external database](#)

8. Kubernetes Ingress (Grundlagen)

- [Hintergrund Ingress](#)

9. Kubernetes Ingress (Traefik)

- [Install Traefik-IngressController](#)
- [Ingress mit traefik](#)
- [pathType ImplementationSpecific vs. Prefix bei traefik](#)
- [ingress mit traefik, letsencrypt und cert-manager](#)
- [cert-manager HTTP-01 Solver: Wo landet die well-known-Datei?](#)

- [Traefik errors-Middleware: eigenen HTTP-Code/Body bei Backend-Fehlern liefern](#)

10. Kubernetes Ingress (HA Proxy)

- [Install HA Proxy-IngressController](#)
- [Ingress mit ha proxy](#)

11. Kubernetes Praxis (Stateful Sets)

- [Hintergrund statefulsets](#)
- [Example stateful set](#)

12. Kubernetes Secrets und Encrypting von z.B. Credentials

- [Credentials in Kubernetes verwenden - welche Möglichkeiten gibt es?](#)
- [Kubernetes secrets Typen](#)
- [Sealed Secrets - bitnami](#)
- [Exercise Sealed Secret mariadb](#)
- [Overview Hashicorp Vault](#)
- [Alternative zu Hashicorp Vault - Fork OpenBao](#)
- [registry mit secret auth](#)
- [Kubernetes secrets mit sops \(mariadb\)](#)
- [Vergleich: GitLab CI/CD vs. SOPS vs. Vault](#)

13. Hashicorp Vault

- [Architektur einfach erklärt](#)
- [Übersicht Vault in Kubernetes](#)
- [Übung: Vault Agent Injection](#)

14. Kubernetes API-Objekte (Teil 2)

- [Jobs](#)
- [Cronjobs](#)
- [DaemonSet - einfaches Beispiel](#)
- [Daemonset with HostPort](#)
- [Daemonset with HostNetwork](#)

15. Kubernetes Praxis

- [Befehle in pod ausführen - Übung](#)
- [Welche Pods mit Namen gehören zu einem Service](#)

16. Security

- [ServiceLinks nicht in env in Pod einbinden](#)
- [Pod with unprivileged user](#)
- [Pod ohne capabilities starten. Funktioniert !](#)

17. Helm (Kubernetes Paketmanager)

- [Helm - Was kann Helm](#)
- [Helm Grundlagen](#)
- [Helm Warum ?](#)
- [Helm Example](#)
- [Installation, Upgrade, Uninstall helm-Chart exercise - simple \(mariadb-cloudpirates\)](#)
- [Installation, Upgrade, Uninstall helm-Chart exercise \(wordpress-groundhog2k\)](#)
- [Helm Exercise with nginx](#)
- [Helm Spickzettel](#)

18. Helm Charts erstellen und analysieren

- [Eigenes Helm-Chart erstellen](#)
- [Chart zur Analyse runterladen und entpacken](#)
- [Eigenes Chart in eine Registry pushen \(helm push\)](#)
- [Wie starte ich am besten einfach - mit eigenem Projekt](#)

19. Helm Spezial: Umgang mit Einrückungen

- [Whitespaces meistern mit "-"](#)
- [Exercise Whitespaces](#)
- [Named Templates](#)

20. Helm Advanced

- [Helm Dependencies Exercise](#)

21. Helm - Fehleranalyse

- [Beispiel Cloudpirates - helm chart nginx](#)

22. Gitlab ci/cd (auch mit helm)

- [Helm mit gitlab ci/cd ausrollen](#)
- [Übung: Unit Tests im Merge Request Dashboard](#)
- [Exercise: Docker Image bauen und in Registry pushen](#)

23. Helpful plugins

- [Use shortnames for kubectl - commands](#)

24. Kubernetes Debugging

- [Probleme über Logs identifiziert - z.B. non-root image](#)
- [Übung: FE zu Backend Verbindungen debuggen mit kubectl debug und NetworkPolicy](#)
- [Übung: Service-Verbindungsprobleme debuggen mit kubectl debug \(ohne NetworkPolicy\)](#)
- [Übung: nginx als Nicht-Root-User startet nicht - CrashLoopBackOff mit describe/logs/exec debuggen](#)
- [Übung: Pod laeuft, Anwendung kaputt - Fehler nur in den Logs sichtbar \(403/502\)](#)
- [Übung: CrashLoopBackOff, aber die Logs sind sauber - OOMKilled \(137\) und Completed \(0\) ueber Exit Codes finden](#)

25. Kubernetes RBAC

- [Kubernetes RBAC - was darf Traefik](#)

26. Weiter lernen

- [Lernumgebung](#)
- [Kubernetes Doku - Bestimmte Tasks lernen](#)
- [Kubernetes Videos mit Hands On](#)

27. Kubernetes Storage (CSI)

- [Überblick Persistent Volumes \(CSI\)](#)
- [Liste der Treiber mit Features \(CSI\)](#)
- [Übung Persistent Storage](#)
- [Beispiel mariadb](#)

28. Kubernetes Installation

- [k3s installation](#)

29. Kubernetes Monitoring

- [Prometheus Monitoring Server \(Overview\)](#)
- [Prometheus Metriktypen \(Counter, Gauge, Histogram, Summary\)](#)
- [Achtung: Bitte kein Prometheus-Agent verwenden](#)
- [Prometheus / Grafana Stack installieren](#)
- [Prometheus / Grafana mit Ingress, Letsencrypt und BasicAuth \(inkl. Alertmanager\)](#)
- [Übung: Prometheus UI und PromQL](#)
- [PromQL: rate\(\)](#)
- [PromQL: irate\(\)](#)
- [PromQL: rate\(\) vs. irate\(\)](#)
- [Übung: Custom Metriken mit eigener Demo-App](#)
- [Demo-App Source Code: app.py](#)
- [Demo-App Source Code: Dockerfile](#)
- [Demo-App Source Code: requirements.txt](#)
- [Vergleich: Splunk vs. CheckMK vs. Prometheus/Grafana](#)

30. Kubernetes Monitoring (checkmk)

- [Checkmk Server mit Docker einrichten - Script \(umfangreich\) mit letsencrypt und ssl](#)
- [Checkmk in Kubernetes einrichten \(checkmk raw\)](#)
- [Checkmk in Kubernetes einrichten \(checkmk enterprise/cloud edition\)](#)
- [Checkmk Raw vs. Enterprise in Bezug auf Kubernetes](#)
- [Kubernetes Dashboards](#)
- [Prometheus-Metriken in Checkmk integrieren](#)
- [Übung: Kubernetes API Health als HTTP Active Check](#)

31. Kubernetes Performance

- [Heap analyse](#)

32. Kubernetes QoS / HealthChecks / Live / Readiness

- [Quality of Service - evict pods](#)
- [LiveNess/Readiness - Probe / HealthChecks](#)
- [Übung: Readiness Probe mit HTTP](#)
- [Taints / Tolerations](#)

33. Installation mit microk8s

- [Schritt 1: auf 3 Maschinen mit Ubuntu 24.04 LTS](#)
- [Schritt 2: cluster - node2 + node3 einbinden - master ist node 1](#)
- [Schritt 3: Remote Verbindung einrichten](#)

34. Installation mit kubeadm

- [Schritt für Schritt mit kubeadm](#)

35. Kubernetes Netzwerk (Antrea / NSX)

- [Netzwerkspezialität bei NSX mit Antrea](#)

36. Tipps & Tricks

- [Pods bleiben im terminate-mode stehen](#)

37. Datenbanken

- [MariaDB Operator](#)
- [HA mit dem Postgres Operator -> cloudnativePG umsetzen](#)
- [Graph-Datenbank neo4j installieren mit helm](#)
- [Graph-Datenbank neo4j Cluster mit Operator installieren](#)

Backlog

1. Podman

- [Podman vs. Docker](#)

2. Kubernetes Installation

- [Kubernetes Installation mit Proxmox und kubespary](#)

3. Kubernetes Workloads

- [Welche Wege gibt es Kubernetes Workloads auszurollen](#)

4. Kubernetes Infrastructure (Performance)

- [Performance of etcd for setup](#)

5. Kubernetes - Überblick

- [Liste wichtiger/sinnvoller Client-Tools](#)

6. Kubernetes - Hochverfügbarkeit

- [Strategien für Hochverfügbarkeit](#)

7. Kubernetes - Authentication

- [oidc mit kubectl](#)
- [traefik authentication mit oidc](#)

8. Kubernetes Deployment - Internals

- [Strategy when creating and terminating pods in Deployment - RollingUpdate - maxSurge - maxUnavailability](#)

9. Kubernetes Ingress (Eingehender Traffic ins Cluster)

- [Wann LoadBalancer, wann Ingress](#)

10. Kubernetes Ingress (Nginx - deprecated)

- [Ingress Controller auf Digitalocean \(doks\) mit helm installieren](#)
- [Documentation for default ingress nginx](#)
- [Beispiel Ingress](#)
- [Beispiel mit Hostnamen](#)
- [Beispiel Deployment mit Ingress und Hostnamen](#)
- [Achtung: Ingress mit Helm - annotations](#)
- [Permanente Weiterleitung mit Ingress](#)

11. ServiceMesh

- [Why a ServiceMesh ?](#)
- [How does a ServiceMesh work? \(example istio\)](#)
- [istio vs. ingress](#)
- [istio security features](#)
- [istio-service mesh - ambient mode](#)
- [Performance comparison - baseline, sidecar, ambient](#)

12. Kubernetes Ingress

- [Ingress HA-Proxy Sticky Session](#)
- [Nginx Ingress Session Stickyness](#)
- [https mit ingressController und Letsencrypt](#)

13. Kubernetes Pod Termination

- [LifeCycle Termination](#)
- [preStopHook](#)
- [How to wait till a pod gets terminated](#)

14. Kubernetes Security

- [Best practices security pods](#)
- [Best practices in general](#)
- [Images in kubernetes von privatem Repo verwenden](#)

15. Kubernetes Monitoring/Security

- [Überwachung, ob Images veraltet sind, direkt in Kubernetes](#)

16. Helm (IDE - Support)

- [Kubernetes-Plugin IntelliJ](#)

- [IntelliJ - Helm Support Through Kubernetes Plugin](#)
17. Helm - Charts entwickeln
- [Unser erstes Helm Chart erstellen](#)
 - [Wie starte ich am besten - Übung](#)
18. Helm und Kustomize kombinieren
- [Helm und Kustomize kombinieren](#)
19. LoadBalancer on Premise (metallb)
- [MetalLB](#)
20. Kubernetes Verlässlichkeit erreichen
- [Keine 2 pods auf gleichem Node - PodAntiAffinity](#)
21. Metrics-Server / Größe Cluster
- [Metrics-Server mit helm installieren](#)
 - [Speichernutzung und CPU berechnen für Anwendungen](#)
22. Kubernetes -> High Availability Cluster (multi-data center)
- [High Availability multiple data-centers](#)
 - [PodAntiAffinity für Hochverfügbarkeit](#)
 - [PodAffinity](#)
23. Kubernetes -> etcd
- [etcd - cleaning of events](#)
 - [etcd in multi-data-center setup](#)
24. Kubernetes Storage
- [Praxis. Beispiel \(Dev/Ops\)](#)
25. Kubernetes Netzwerk
- [Kubernetes Netzwerke Übersicht](#)
 - [DNS - Resolution - Services](#)
 - [Kubernetes Firewall / Cilium Calico](#)
 - [Sammlung istio/mesh](#)
26. Kubernetes NetworkPolicy (Firewall)
- [Kubernetes Network Policy Beispiel](#)
 - [Übung: NetworkPolicy - Pod-Traffic absichern](#)
 - [Übung: FE zu Backend Verbindungen debuggen mit kubectl debug und NetworkPolicy](#)
27. Kubernetes Autoscaling
- [Kubernetes Autoscaling](#)
28. Kubernetes Secrets / ConfigMap
- [Configmap Example 1](#)
 - [Secrets Example 1](#)
 - [Änderung in ConfigMap erkennen und anwenden](#)
29. Kubernetes RBAC (Role based access control)
- [RBAC Übung kubectl](#)
30. Kubernetes Operator Konzept
- [Überblick](#)
31. Kubernetes Deployment Strategies
- [Deployment green/blue canary rolling update](#)
 - [Praxis-Übung A/B Deployment](#)
32. Kubernetes Monitoring
- [Prometheus / blackbox exporter](#)
 - [Kubernetes Metrics Server verwenden](#)
33. Tipps & Tricks
- [Netzwerkverbindung zum Pod testen](#)
 - [Debug Container neben Container erstellen](#)
 - [Debug Pod auf Node erstellen](#)
34. Kubernetes Administration /Upgrades
- [Kubernetes Administration / Upgrades](#)
 - [Terminierung von Container vermeiden](#)
 - [Praktische Umsetzung RBAC anhand eines Beispiels \(Ops\)](#)
35. Documentation (Use Cases)

- [Case Studies Kubernetes](#)
- [Use Cases](#)

36. Interna von Kubernetes

- [OCI Container Images Standards](#)

37. Andere Systeme / Verschiedenes

- [Kubernetes vs. Cloudfoundry](#)
- [Kubernetes Alternativen](#)
- [Hyperscaler vs. Kubernetes on Premise](#)

38. Lokal Kubernetes verwenden

- [Kubernetes in ubuntu installieren z.B. innerhalb virtualbox](#)
- [minikube](#)
- [rancher for desktop](#)

39. Microservices

- [Microservices vs. Monolith](#)
- [Monolith schneiden/aufteilen](#)
- [Strategic Patterns - wid monolith praktisch umbauen](#)
- [Literatur von Monolith zu Microservices](#)

40. Extras

- [Install minikube on wsl2](#)
- [kustomize - gute Struktur für größere Projekte](#)
- [kustomize with helm](#)

41. Documentation

- [References](#)
- [Tasks Documentation - Good one !](#)

42. AWS

- [External Secrets Operator \(ESO\) mit AWS Secrets Manager + KMS einrichten](#)
- [ESO mit AWS Secrets Manager - Secret als Volume-Mount](#)

43. Documentation for Settings right resources/limits

- [Goldilocks](#)

Backlog

1. Kubernetes - Überblick

- [Allgemeine Einführung in Container \(Dev/Ops\)](#)
- [Microservices \(Warum ? Wie ?\) \(Devs/Ops\)](#)
- [Wann macht Kubernetes Sinn , wann nicht ?](#)
- [Aufbau Allgemein](#)
- [Aufbau mit helm , OpenShift , Rancher \(RKE\) , microk8s](#)
- [Welches System ? \(minikube , microk8s etc.\)](#)
- [Installation - Welche Komponenten from scratch](#)

2. Kubernetes - microk8s (Installation und Management)

- [Installation Ubuntu - snap](#)
- [Remote-Verbindung zu Kubernetes \(microk8s\) einrichten](#)
- [Create a cluster with microk8s](#)
- [Ingress controller in microk8s aktivieren](#)
- [Arbeiten mit der Registry](#)
- [Installation Kubernetes Dashboard](#)

3. Kubernetes Praxis API-Objekte

- [Das Tool kubectl \(Devs/Ops\) - Spickzettel](#)
- [kubectl example with run](#)
- [Arbeiten mit manifests \(Devs/Ops\)](#)
- [Pods \(Devs/Ops\)](#)
- [kubectl/manifest/pod](#)
- [ReplicaSets \(Theorie\) - \(Devs/Ops\)](#)
- [kubectl/manifest/replicaset](#)
- [Deployments \(Devs/Ops\)](#)
- [kubectl/manifest/deployments](#)
- [Services \(Devs/Ops\)](#)
- [kubectl/manifest/service](#)
- [DaemonSets \(Devs/Ops\)](#)
- [IngressController \(Devs/Ops\)](#)
- [Hintergrund Ingress](#)
- [Documentation for default ingress nginx](#)
- [Beispiel Ingress](#)

- [Beispiel mit Hostnamen](#)
- [Achtung: Ingress mit Helm - annotations](#)
- [Permanente Weiterleitung mit Ingress](#)
- [ConfigMap Example](#)

4. Kubernetes - ENV - Variablen für den Container setzen

- [ENV - Variablen - Übung](#)

5. Kubernetes - Arbeiten mit einer lokalen Registry (microk8s)

- [microk8s lokale Registry](#)

6. Kubernetes Praxis Scaling/Rolling Updates/Wartung

- Rolling Updates (Devs/Ops)
- Scaling von Deployments (Devs/Ops)
- [Wartung mit drain / uncordon \(Ops\)](#)
- [Ausblick AutoScaling \(Ops\)](#)

7. Kubernetes Storage

- Grundlagen (Dev/Ops)
- Objekte PersistentVolume / PersistentVolumeClaim (Dev/Ops)
- [Praxis. Beispiel \(Dev/Ops\)](#)

8. Kubernetes Networking

- [Überblick](#)
- Pod to Pod
- Webbasierte Dienste (Ingress)
- IP per Pod
- Inter Pod Communication ClusterDNS
- [Beispiel NetworkPolicies](#)

9. Kubernetes Paketmanagement (Helm)

- [Warum ? \(Dev/Ops\)](#)
- [Grundlagen / Aufbau / Verwendung \(Dev/Ops\)](#)
- [Praktisches Beispiel bitnami/mysql \(Dev/Ops\)](#)

10. Kustomize

- [Beispiel ConfigMap - Generator](#)
- [Beispiel Overlay und Patching](#)
- [Resources](#)

11. Kubernetes Rechteverwaltung (RBAC)

- Warum ? (Ops)
- [Wie aktivieren?](#)
- Rollen und Rollenzuordnung (Ops)
- Service Accounts (Ops)
- [Praktische Umsetzung anhand eines Beispiels \(Ops\)](#)

12. Kubernetes Backups

- [Kubernetes Backup](#)
- [Kasten.io overview](#)

13. Kubernetes Monitoring

- [Debugging von Ingress](#)
- [Ebenen des Loggings](#)
- [Working with kubectl logs](#)
- [Built-In Monitoring tools - kubectl top pods/nodes](#)
- [Protokollieren mit Elasticsearch und Fluentd \(Devs/Ops\)](#)
- [Long Installation step-by-step - Digitalocean](#)
- Container Level Monitoring (Devs/Ops)
- [Setting up metrics-server - microk8s](#)

14. Kubernetes Monitoring (CheckMK)

- [CheckMK RAW - Kubernetes Monitoring einrichten](#)
- [CheckMK Enterprise Features für Kubernetes](#)
- [Kubernetes Dashboards in CheckMK](#)

15. Kubernetes Security

- [Grundlagen und Beispiel \(Praktisch\)](#)

16. Kubernetes GUI

- [Rancher](#)
- [Kubernetes Dashboard](#)

17. Kubernetes CI/CD (Optional)

- Canary Deployment (Devs/Ops)

- [Blue Green Deployment \(Devs/Ops\)](#)

18. Tipps & Tricks

- [Ubuntu client aufsetzen](#)
- [bash-completion](#)
- [Alias in Linux kubectl get -o wide](#)
- [vim einrückung für yaml-dateien](#)
- [kubectl spickzettel](#)
- [Alte manifests migrieren](#)
- [X-Forward-Header-For setzen in Ingress](#)

19. Übungen

- [Training 23.-25.09.2026: Pad-Zusammenfassung \(alle Übungen\)](#)
- [übung Tag 3](#)
- [übung Tag 4](#)

20. Fragen

- [Q and A](#)
- [Kubernetes und Ansible](#)

21. Documentation

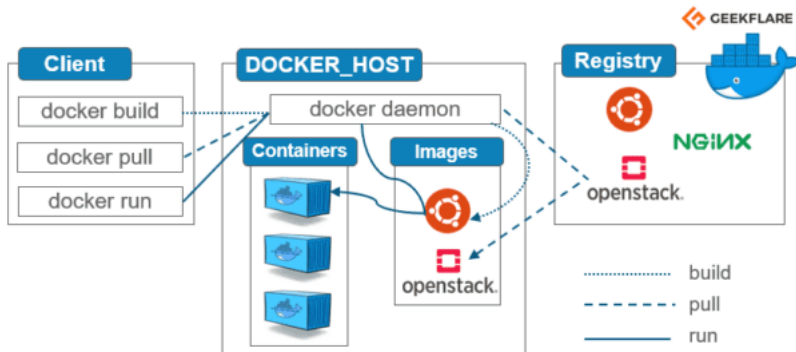
- [Kubernetes mit VisualStudio Code](#)
- [Kube Api Ressources - Versionierungsschema](#)
- [Kubernetes Labels and Selector](#)

22. Documentation - Sources

- [controller manager](#)

Docker-Grundlagen

Übersicht Architektur



Was ist ein Container ?

- vereint in sich Software
- Bibliotheken
- Tools
- Konfigurationsdateien
- keinen eigenen Kernel
- gut zum Ausführen von Anwendungen auf verschiedenen Umgebungen
- Container sind entkoppelt
- Container sind voneinander unabhängig
- Können über wohldefinierte Kommunikationskanäle untereinander Informationen austauschen
- Durch Entkopplung von Containern:
 - o Unverträglichkeiten von Bibliotheken, Tools oder Datenbank können umgangen werden, wenn diese von den Applikationen in unterschiedlichen Versionen benötigt werden.

Anwendungsfälle

- Unterschiedliche Versionen einer Applikation (z.B. MariaDB-Server) auf einem Linux-System betreiben
- Gute Skalieren zu können (Beispiel: Bestellanzahl steigt (wir brauchen bei Ressourcen für Shop-Katalog und Warenkorb), aber nicht für Registrierung
 - o Sprachagnostik /ein Service in python, einer in Rust

Was sind container images

- Container Image benötigt, um zur Laufzeit Container-Instanzen zu erzeugen
- Bei Docker werden Docker Images zu Docker Containern, wenn Sie auf einer Docker Engine als Prozess ausgeführt werden
- Man kann sich ein Docker Image als Kopiervorlage vorstellen.
 - o Diese wird genutzt, um damit einen Docker Container als Kopie zu erstellen

Container vs. Virtuelle Maschine

VM's virtualisieren Hardware
Container virtualisieren Betriebssystem

Was ist ein Dockerfile

Grundlagen

- Textdatei, die Linux - Kommandos enthält
 - o die man auch auf der Kommandozeile ausführen könnte
 - o Diese erledigen alle Aufgaben, die nötig sind, um ein Image zusammenzustellen
 - o mit docker build wird dieses image erstellt

Beispiel

```
FROM node:12-alpine
RUN apk add --no-cache python2 g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
## übersetzt: node src/index.js
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Jetzt wird gebaut...

```
cd buildtest
docker build -t trainerimage:1.0 .
```

Dockerfile - image kleinhalten

- Delete all files that are not needed in image

Example

```
### Delete files needed for installation
### Right after the installation of the necessary
## Variante 2
## nano Dockerfile
FROM ubuntu:22.04
RUN apt-get update && \
    apt-get install -y inetutils-ping && \
    rm -rf /var/lib/apt/lists/*
## CMD ["/bin/bash"]
```

Example 2: Start from scratch

- <https://codeburst.io/docker-from-scratch-2a84552470c8>

Übung: Java REST-API mit Multi-Stage Dockerfile

Hintergrund

Ein Multi-Stage-Build trennt das **Bauen** einer Anwendung vom **Ausführen**:

- Stage 1 (`build`): enthaelt den vollen JDK-Compiler, uebersetzt den Java-Code
- Stage 2: enthaelt nur ein schlankes JRE + die fertige `.class` -Datei

Der Compiler, Quellcode und alle Build-Tools landen NICHT im finalen Image - das Image wird kleiner und hat eine kleinere Angriffsflaeche.

Schritt 1: Verbindung zum Docker-Host

```
ssh -i ~/.ssh/id_ed25519_nopass root@docker11.t3isp.de
```

Schritt 2: Arbeitsverzeichnis anlegen

```
mkdir -p ~/java-api-<dein-name>
cd ~/java-api-<dein-name>
```

Schritt 3: Die REST-API (Main.java)

Kein Framework, kein Maven noetig - nur der eingebaute `com.sun.net.httpserver` aus dem JDK. Zwei Endpunkte: `/api/health` und `/api/hello` .

```
## vi Main.java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;

public class Main {

    public static void main(String[] args) throws IOException {
        int port = 8080;
        HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);

        server.createContext("/api/health", new HealthHandler());
        server.createContext("/api/hello", new HelloHandler());

        server.setExecutor(null);
        server.start();
        System.out.println("REST API laeuft auf Port " + port);
    }

    static class HealthHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String json = "{\"status\":\"UP\"}";
            sendJson(exchange, 200, json);
        }
    }
}
```

```

    }
}

static class HelloHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
        String host = "unknown";
        try {
            host = InetAddress.getLocalHost().getHostName();
        } catch (Exception e) {
            // ignore, bleibt "unknown"
        }
        String json = "{ \"message\": \"Hallo von Java!\", \"host\": \"" + host + "\" }";
        sendJson(exchange, 200, json);
    }
}

static void sendJson(HttpExchange exchange, int status, String json) throws IOException {
    byte[] body = json.getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().set("Content-Type", "application/json; charset=utf-8");
    exchange.sendResponseHeaders(status, body.length);
    try (OutputStream os = exchange.getResponseBody()) {
        os.write(body);
    }
}
}
}

```

Schritt 4: Das Multi-Stage Dockerfile

```

## vi Dockerfile
## Stage 1: Build - hier steht der komplette JDK-Compiler zur Verfuegung
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY Main.java .
RUN javac Main.java

## Stage 2: Runtime - nur die fertige .class-Datei + schlankes JRE
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/Main*.class .
EXPOSE 8080
CMD ["java", "Main"]

```

Wichtig: `javac` erzeugt fuer jede innere Klasse eine eigene `.class`-Datei (`Main.class` , `Main$HealthHandler.class` , `Main$HelloHandler.class`). Deshalb im `COPY --from=build` das Muster `Main*.class` verwenden, nicht nur `Main.class` - sonst startet der Container mit `NoClassDefFoundError` .

Schritt 5: Image bauen

```
docker build -t java-api-<dein-name>:1.0 .
```

Schritt 6: Container starten

Auf dem geteilten Docker-Host wuerden feste Ports (`-p 8080:8080`) zwischen den Teilnehmern kollidieren. Deshalb den Host-Port von Docker zufaellig vergeben lassen:

```
docker run -d --name java-api-<dein-name> -p 8080 java-api-<dein-name>:1.0
docker port java-api-<dein-name> 8080/tcp

```

Schritt 7: API testen

Den Port aus Schritt 6 einsetzen:

```
curl http://localhost:<port>/api/health
curl http://localhost:<port>/api/hello

```

Erwartete Ausgabe:

```

{"status":"UP"}
{"message":"Hallo von Java!","host":"<container-id>"}

```

Schritt 8 (optional): Groessenvergleich mit Single-Stage

Zum Vergleich ein Image ohne Multi-Stage bauen (JDK + Compiler bleiben mit drin):

```

## vi Dockerfile.singlestage
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY Main.java .
RUN javac Main.java

```

```
EXPOSE 8080
CMD ["java", "Main"]
```

```
docker build -f Dockerfile.singlestage -t java-api-<dein-name>-singlestage:1.0 .
docker images java-api-<dein-name>
docker images java-api-<dein-name>-singlestage
```

Im Test (docker11.t3isp.de, eclipse-temurin:21-*--alpine): Multi-Stage-Image ca. **74 MB**, Single-Stage-Image ca. **184 MB** - mehr als doppelt so gross, nur weil Compiler und Build-Werkzeuge mitgeschleppt werden.

Aufraeumen

```
docker rm -f java-api-<dein-name>
docker rmi java-api-<dein-name>:1.0 java-api-<dein-name>-singlestage:1.0
```

Docker installieren (Ubuntu, apt)

Walkthrough

```
sudo apt-get update
sudo apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release

sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Läuft der Dienst (dockerd)

```
systemctl status docker
```

docker-compose ?

```
## herausfinden, ob docker compose installieren
docker compose version
```

Bind-Mounts

Example 1

```
## andere Verzeichnis als das Heimatverzeichnis von root funktionieren aktuell nicht mit
## snap install docker
## wg. des Confinements
docker run -d -it --name devtest --mount type=bind,source=/root,target=/app nginx:latest
docker exec -it devtest bash
/# cd /app
```

Example 2 with /home/kurs

```
## testen wo der DocumentRoot
docker run --rm -p 80:80 -it --name=nginx-test nginx bash
```

Bind-Mounts: Berechtigungen (Rocky/SELinux)

Step: 1

```
## als unprivilegierter Nutzer kurs
cd
mkdir -p /home/kurs/nginx/html
cd /home/kurs/nginx/html
echo "hallo welt" > index.html
sestatus
docker run --rm -it --mount type=bind,source=/home/kurs/nginx/html,target=/usr/share/nginx/html --name=nginx-test nginx bash
exit
```


Step: 2

```
## Adjust permissions
cd /home/kurs/nginx/
sudo chown -R kurs:kurs html
## Neue Verzeichnisse und Dateien werden mit der Gruppe kurs angelegt
chmod -R g+s html
setfacl -d -m g::rwx html
docker run --rm -it --mount type=bind,source=/home/kurs/nginx/html,target=/usr/share/nginx/html --name=nginx-test nginx bash
## in container
cd /usr/share/nginx/html
mkdir rootdata
cd rootdata
touch foo
```

Docker Security Overview

Run container under specific user:

```
## user with id 40000 does not need to exist in container
docker run -it -u 40000 alpine

## user kurs needs to exist in container (/etc/passwd)
docker run -it -u kurs alpine
```

Default capabilities

- Set everytime a new container is started as default
- <https://github.com/moby/moby/blob/master/profiles/seccomp/default.json>

Run container with less capabilities

```
cd
mkdir captest
cd captest
```

```
nano docker-compose.yml
```

```
services:
  nginx:
    image: nginx
    cap_drop:
      - CHOWN
```

```
docker compose up -d
## start and exits
docker compose ps
##
docker exec -it captest_nginx_1 bash
##/ touch /tmp/foo; chown 10000 /tmp/foo

## what happened -> wants to do chown, but it is not allowed
docker logs captest_nginx_1
```

```
docker compose down
```

Reference:

- https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- <https://www.redhat.com/en/blog/secure-your-containers-one-weird-trick>
- man capabilities

Image-Scan mit docker scan (snyk)

Prerequisites

You need to be logged in on docker hub with docker login
(with your account credentials)

Example

```
## Snyk (docker scan)
docker help scan
docker scan --json --accept-license dockertrainereu/jm-hello-docker > result.json
```

Kubernetes - Überblick

12-Factor-App - Design Prinzipien fuer Cloud Native Anwendungen

- Das sind best-practices

Die 12-Factors stammen von Heroku und beschreiben, wie eine App aussehen muss, damit sie sich problemlos in einer Cloud-Plattform betreiben lässt – also genau das, was Kubernetes heute von einem Workload erwartet.

- Ursprünglich entwickelt von heroku 2011
- Ursprünglich gedacht für cloud-native apps
- Auch gut für microservices anwendbar

Anwendung

- Checkliste: Gilt das für meinen Service ?

Hier ist die Tabelle der Twelve-Factor App Principles:

#	Prinzip	Beschreibung
1	Codebase	Versionsverwaltetes Code-Repository
2	Dependencies	Abhängigkeiten sollten extern verwaltet werden. (spielte zur Zeit von heroku eine Rolle, weil Software auf dem Host ausgeführt wurde. Man soll sich also nicht darauf verlassen, was auf dem Host existiert. Bei Docker/Kubernetes ist das bereits im Container-Image selbst geregelt. Man könnte also sagen, die Regel ist bei Docker-Images ohnehin erfüllt.
3	Config	Konfiguration als Umgebungsvariablen
4	Backing Services	Datenbanken, Messaging etc. als externe Ressourcen
5	Build, Release, Run	Drei unabhängige Deployment-Schritte
6	Stateless Processes	Zustandslose, unabhängige Prozesse zum guten Skalieren
7	Port Binding	App bindet direkt an Port
8	Concurrency	Apps sollten in Module aufgeteilt werden zur einfachen Skalierung
9	Disposability	Schneller Start und einfaches Herunterfahren
10	DEV/PROD Parity	Entwicklungsumgebung und Produktion möglichst ähnlich
11	Logs	Logs als Event-Streams behandeln (wie bei docker / kubernetes)
12	Admin Processes	Admin-Aufgaben als One-off-Prozesse Einmalige Aktion möglichst gescrriptet und versioniert und sie sollten in der gleichen Umgebung umgesetzt werden. Job/Cronjob/initContainer

Warum Kubernetes, was macht Kubernetes

Ausgangslage

- Ich habe jetzt einen Haufen Images, aber:
 - Wie bekomme ich die auf die Systeme.
 - Und wie halte ich den Verwaltungsaufwand in Grenzen.
- Lösung: Kubernetes -> ein Orchestrierungstool

Hintergründe

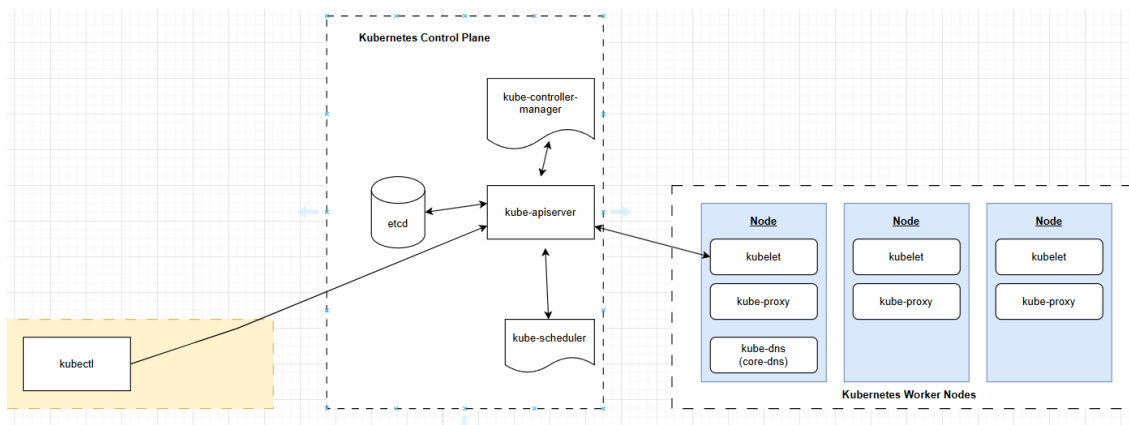
- Gegenüber Virtualisierung von Hardware - x-fache bessere Auslastung
- Google als Ausgangspunkt (Borg)
- Software 2014 als OpenSource zur Verfügung gestellt
- Optimale Ausnutzung der Hardware, hunderte bis tausende Dienste können auf einigen Maschinen laufen (Cluster)
- Immutable - System
- Selbstheilend

Wozu dient Kubernetes

- Orchestrierung von Containern
- am gebräuchlichsten aktuell Docker -Images

Aufbau Allgemein

Schaubild



Komponenten / Grundbegriffe

Control Plane (Master)

Aufgaben

- Der Control Plane (Master) koordiniert den Cluster
- Der Control Plane (Master) koordiniert alle Aktivitäten in Ihrem Cluster
 - Planen von Anwendungen
 - Verwalten des gewünschten Status der Anwendungen
 - Skalieren von Anwendungen
 - Rollout neuer Updates.

Komponenten des Masters

ETCD

- Verwalten der Konfiguration und des Status des Clusters (key/value - pairs)

KUBE-CONTROLLER-MANAGER

- Zuständig für die Überwachung der Stati im Cluster mit Hilfe von endlos loops.
- kommuniziert mit dem Cluster über die kubernetes-api (bereitgestellt vom kube-api-server)

KUBE-API-SERVER

- provides api-frontent for administration (no gui)
- Exposes an HTTP API (users, parts of the cluster and external components communicate with it)
- REST API

KUBE-SCHEDULER

- assigns Pods to Nodes.
- scheduler determines which Nodes are valid placements for each Pod in the scheduling queue (according to constraints and available resources)
- The scheduler then ranks each valid Node and binds the Pod to a suitable Node.
- Reference implementation (other schedulers can be used)

Nodes

- Nodes sind virtuelle oder physische Maschinen auf denen die notwendigen Kubernetes-System-Komponenten (Software)
- Worker Nodes (Knoten) sind die Arbeiter (Maschinen), die Anwendungen ausführen
- Ref: <https://kubernetes.io/de/docs/concepts/architecture/nodes/>

Pod/Pods

- Pods sind die kleinste verwaltbare Einheit, die in Kubernetes erstellt und verwaltet werden können.
- Ein Pod (übersetzt Gruppe) ist eine Gruppe von einem oder mehreren Containern
 - gemeinsam genutzter Speicher- und Netzwerkressourcen
 - Befinden sich immer auf dem gleichen virtuellen Server

Node (Minion) - components

General

- On the nodes we will rollout the applications

kubelet

Node Agent that runs on every node (worker)
Er stellt sicher, dass Container in einem Pod ausgeführt werden.

Kube-proxy

- Läuft auf jedem Node
- = Netzwerk-Proxy für die Kubernetes-Netzwerk-Services.
- Kube-proxy verwaltet die Netzwerkkommunikation der Services innerhalb des Clusters

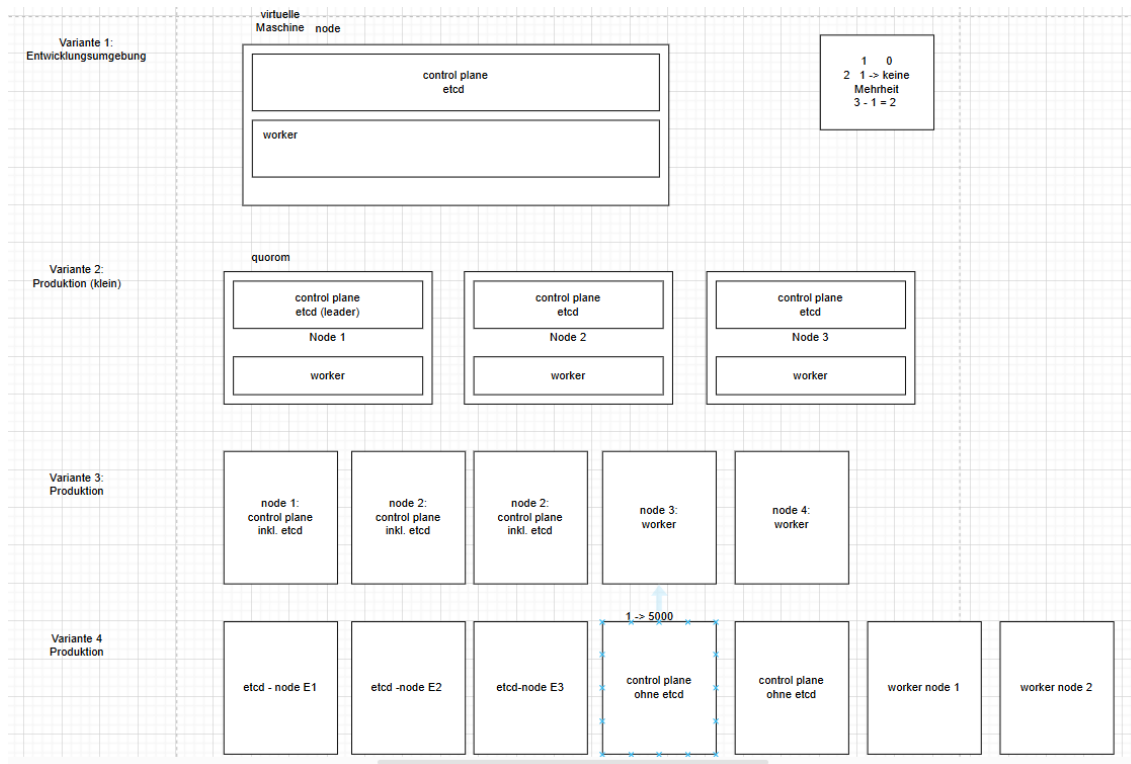
Referenzen

- <https://www.redhat.com/de/topics/containers/kubernetes-architecture>

Kubernetes Architektur Deep-Dive

- <https://github.com/jmetzger/training-kubernetes-advanced/assets/1933318/1ca0d174-f354-43b2-81cc-67af8498b56c>

Ausbaustufen Kubernetes



Wann macht Kubernetes Sinn, wann nicht?

Wann nicht sinnvoll ?

- Anwendung, die ich nicht in Container "verpackt" habe
- Spielt der Dienstleister mit (Wartungsvertrag)
- Kosten / Nutzenverhältnis (Umstellen von Container zu teuer)
- Anwendung lässt sich nicht skalieren
 - z.B. Bottleneck Datenbank
 - Mehr Container bringen nicht mehr (des gleichen Typs)

Wo spielt Kubernetes seine Stärken aus ?

- Skalieren von Anwendungen.
- bessere Hochverfügbarkeit out-of-the-box
- Heilen von Systemen (neu starten von Containern)
- Automatische Überwachung (mit deklarativem Management) - ich beschreibe, was ich will
- Neue Versionen auszurollen (Canary Deployment, Blue/Green Deployment)

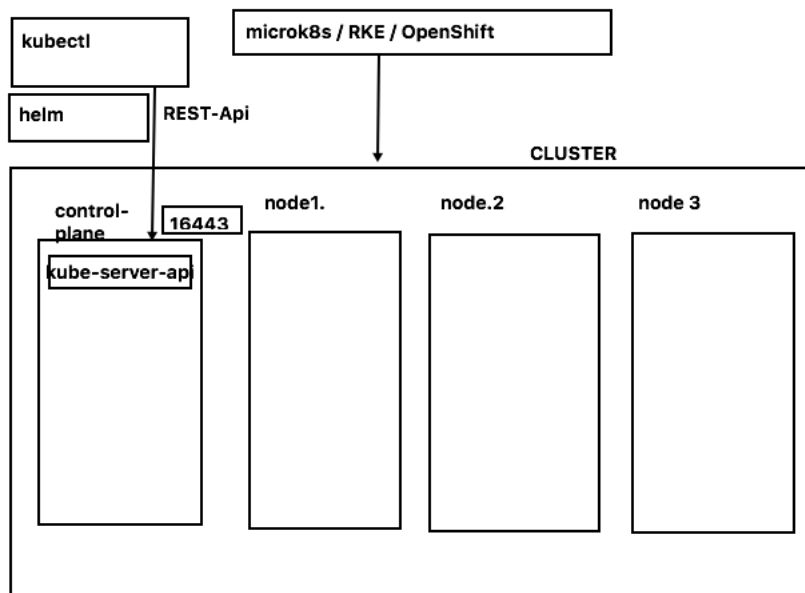
Mögliche Nachteile

- Steigert die Komplexität.
- Debugging wird u.U. schwieriger
- Mit Kubernetes erkaufe ich mir auch, die Notwendigkeit.
 - Über adequate Backup-Lösungen nachzudenken (Moving Target, Kubernetes Aware Backups)
 - Bereitstellung von Monitoring
 - Bereitstellung Observability (Log-Aggregierungslösung, Tracing)

Klassische Anwendungsfällen (wo Kubernetes von Vorteil)

- Webbasierte Anwendungen (z.B. auch API's bzw. Web)
- Ausser Problematik: Session StickyNess

Aufbau mit helm,OpenShift,Rancher(RKE),microk8s



Welches System ? (minikube, micro8ks etc.)

Überblick der Systeme

General

kubernetes itself has not convenient way of doing specific stuff like creating the kubernetes cluster.

So there are other tools/distri around helping you with that.

Kubeadm

General

- The official CNCF (<https://www.cncf.io/>) tool for provisioning Kubernetes clusters (variety of shapes and forms (e.g. single-node, multi-node, HA, self-hosted))
- Most manual way to create and manage a cluster

microk8s

Prerequisites:

- at least 4 GB of ram per maschine

General

- Created by Canonical (Ubuntu)
- Runs on Linux
- Runs only as snap
- (In the meantime it is also available for Windows/Mac)
- HA-Cluster (control plane)

Production-Ready ?

- Short answer: YES

Quote canonical (2020) :

MicroK8s is a powerful, lightweight, reliable production-ready Kubernetes distribution. It is an enterprise-grade Kubernetes distribution that has a small disk and memory footprint while offering carefully selected add-ons out-the-box, such as Istio, Knative, Grafana, Cilium and more. Whether you are running a production environment or interested in exploring K8s, MicroK8s serves your needs.

Ref: <https://ubuntu.com/blog/introduction-to-microk8s-part-1-2>

Advantages

- Easy to setup HA-Cluster (multi-node control plane)
- Easy to manage

Disadvantages

- Nicht so flexible wie kubeadm

- z.B. freie Wahl des CNI - Providers (z.B. Calico)
- nicht so flexibel bei speziell config (z.B. andere IP-Ranges)

minikube

Disadvantages

- Not usable / intended for production

Advantages

- Easy to set up on local systems for testing/development (Laptop, PC)
- Multi-Node cluster is possible
- Runs on Linux/Windows/Mac
- Supports plugin (Different name ?)

k3s (wsl oder virtuelle Maschine)

- sehr schlank.
- lokal installierbar (eine node, ca 5 minuten)
- ein einziges binary
- <https://docs.k3s.io/quick-start>

kind (Kubernetes-In-Docker)

General

- Runs in docker container

For Production ?

```
Having a footprint, where kubernetes runs within docker
and the applikations run within docker as docker containers
it is not suitable for production.
```

Installer für grosse Cluster

Tanzuh (vmware)

- Lizenzkosten

Alternative (Cluster API)

- 1 Management Cluster
- jedes weiteres wird vom Management Cluster ausgerollt.
- Beschreibung Deines Cluster als Konfiguration
- feststehende images für die Basis des Clusters

Nachteile:

- nur auf der Kommandozeile
- keinen Support

Rancherlabs Ranger (SuSE)

- Grafische Weboberfläche (Management GUI)
 - mit Docker oder aber auch in einem cluster (z.B. helm - Chart)
- kann eine oder mehrere Cluster verwalten
 - Installation: k3s
 - Installation: RKE2

OpenStack (Alternative: vmware) - OpenSource

```
* API für OpenStack (Nutzung dieser API über Terraform oder OpenTofu) - > Terraform -> Infrastruktur as code. (.tf)
```

Schritt 1: virtuellen Maschinen ausrollen.

Schritt 2: Kubernetes ausrollen

```
* Ansible (leichter bestimmte zu Konfigurieren)
* kubeadm
```

Proxmox

Schritt 1: virtuellen Maschinen ausrollen.

Schritt 2: Kubernetes ausrollen

```
* Kubespray (verwendet auch ansible aber direkt auf ansible abgestimmt)
* Ansible (leichter bestimmte zu Konfigurieren)
* kubeadm
```

Talos

```
* Installation mit iso
* immutable
```

Installation - Welche Komponenten from scratch

Step 1: Server 1 (manuell installiert -> microk8s)

```
## Installation Ubuntu - Server

## cloud-init script
## s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)

## Server 1 - manuell
## Ubuntu 20.04 LTS - Grundinstallation

## minimal Netzwerk - öffentlichen IP
## nichts besonderes eingerichtet - Standard Digitalocean

## Standard vo Installation microk8s
lo                UNKNOWN          127.0.0.1/8 ::1/128
## public ip / interne
eth0              UP                164.92.255.234/20 10.19.0.6/16 fe80::c:66ff:fec4:cbce/64
## private ip
eth1              UP                10.135.0.3/16 fe80::8081:aaff:feaa:780/64

snap install microk8s --classic
## namensaufloesung fuer pods
microk8s enable dns

## Funktioniert microk8s
microk8s status
```

Steps 2: Server 2+3 (automatische Installation -> microk8s)

```
## Was macht das ?
## 1. Basisnutzer (11trainingdo) - keine Voraussetzung für microk8s
## 2. Installation von microk8s
##.>>>>>> microk8s installiert <<<<<<<
## - snap install --classic microk8s
## >>>>>> Zuordnung zur Gruppe microk8s - notwendig für bestimmte plugins (z.B. helm)
## usermod -a -G microk8s root
## >>>>>> Setzen des .kube - Verzeichnisses auf den Nutzer microk8s -> nicht zwingend erforderlich
## chown -R -R microk8s ~/.kube
## >>>>>> REQUIRED .. DNS aktivieren, wichtig für Namensauflösungen innerhalb der PODS
## >>>>>> sonst funktioniert das nicht !!!
## microk8s enable dns
## >>>>>> kubectl alias gesetzt, damit man nicht immer microk8s kubectl eingeben muss
## - echo "alias kubectl='microk8s kubectl'" >> /root/.bashrc

## cloud-init script
## s.u. MITMICROK8S (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)
##cloud-config
users:
  - name: 11trainingdo
    shell: /bin/bash

runcmd:
  - sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config
  - echo " " >> /etc/ssh/sshd_config
  - echo "AllowUsers 11trainingdo" >> /etc/ssh/sshd_config
  - echo "AllowUsers root" >> /etc/ssh/sshd_config
  - systemctl reload sshd
  - sed -i '/11trainingdo/c
11trainingdo:$6$HeLUJW3a$4xSfDFQjKWfAoGkZF3LFAxM4hgl3d6ATbr2kEu9zMOfwLxkYMO.AJF526mZONwdmsm9sg0tCBK1.SYbhs52u70:17476:0:99999:7:::
/etc/shadow
  - echo "11trainingdo ALL=(ALL) ALL" > /etc/sudoers.d/11trainingdo
  - chmod 0440 /etc/sudoers.d/11trainingdo

  - echo "Installing microk8s"
  - snap install --classic microk8s
  - usermod -a -G microk8s root
  - chown -f -R microk8s ~/.kube
  - microk8s enable dns
  - echo "alias kubectl='microk8s kubectl'" >> /root/.bashrc

## Prüfen ob microk8s - wird automatisch nach Installation gestartet
## kann eine Weile dauern
microk8s status
```

Step 3: Client - Maschine (wir sollten nicht auf control-plane oder cluster - node arbeiten)

```
Weiteren Server hochgezogen.
Vanilla + BASIS

## Installation Ubuntu - Server

## cloud-init script
## s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)

## Server 1 - manuell
## Ubuntu 20.04 LTS - Grundinstallation

## minimal Netzwerk - öffentlichen IP
## nichts besonderes eingerichtet - Standard Digitalocean

## Standard vo Installation microk8s
lo                UNKNOWN          127.0.0.1/8 ::1/128
## public ip / interne
eth0              UP                164.92.255.232/20 10.19.0.6/16 fe80::c:66ff:fec4:cbce/64
## private ip
eth1              UP                10.135.0.5/16 fe80::8081:aaff:feaa:780/64

##### Installation von kubectl aus dem snap
## NICHT .. keine microk8s - keine control-plane / worker-node
## NUR Client zum Arbeiten
snap install kubectl --classic

##### .kube/config
## Damit ein Zugriff auf die kube-server-api möglich
## d.h. REST-API Interface, um das Cluster verwalten.
## Hier haben uns für den ersten Control-Node entschieden
## Alternativ wäre round-robin per dns möglich

## Mini-Schritt 1:
## Auf dem Server 1: kubeconfig ausspielen
microk8s config > /root/kube-config
## auf das Zielsystem gebracht (client 1)
scp /root/kubeconfig 11trainingdo@10.135.0.5:/home/11trainingdo

## Mini-Schritt 2:
## Auf dem Client 1 (diese Maschine) kubeconfig an die richtige Stelle bringen
## Standardmäßig der Client nach eine Konfigurationsdatei sucht in ~/.kube/config
sudo su -
cd
mkdir .kube
cd .kube
mv /home/11trainingdo/kube-config config

## Verbindungstest gemacht
## Damit feststellen ob das funktioniert.
kubectl cluster-info
```

Schritt 4: Auf allen Servern IP's hinterlegen und richtigen Hostnamen überprüfen

```
## Auf jedem Server
hostnamectl
## evtl. hostname setzen
## z.B. - auf jedem Server eindeutig
hostnamectl set-hostname n1.training.local

## Gleiche hosts auf allen server einrichten.
## Wichtig, um Traffic zu minimieren verwenden, die interne (private) IP

/etc/hosts
10.135.0.3 n1.training.local n1
10.135.0.4 n2.training.local n2
10.135.0.5 n3.training.local n3
```

Schritt 5: Cluster aufbauen

```
## Mini-Schritt 1:
## Server 1: connection - string (token)
microk8s add-node
## Zeigt Liste und wir nehmen den Eintrag mit der lokalen / öffentlichen ip
## Dieser Token kann nur 1x verwendet werden und wir auf dem ANDEREN node ausgeführt
```



```
## microk8s join 10.135.0.3:25000/e9cdaa1b5d6d24461c8643cdf107837/bcad1949221a

## Mini-Schritt 2:
## Dauert eine Weile, bis das durch ist.
## Server 2: Den Node hinzufügen durch den JOIN - Befehl
microk8s join 10.135.0.3:25000/e9cdaa1b5d6d24461c8643cdf107837/bcad1949221a

## Mini-Schritt 3:
## Server 1: token besorgen für node 3
microk8s add-node

## Mini-Schritt 4:
## Server 3: Den Node hinzufügen durch den JOIN-Befehl
microk8s join 10.135.0.3:25000/09c96e57ec12af45b2752fb45450530c/bcad1949221a

## Mini-Schritt 5: Überprüfen ob HA-Cluster läuft
Server 1: (es kann auf jedem der 3 Server überprüft werden, auf einem reicht
microk8s status | grep high-availability
high-availability: yes
```

Ergänzend nicht notwendige Skripte

```
## cloud-init script
## s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)

## Digitalocean - unter user_data reingepastet beim Einrichten

##cloud-config
users:
  - name: 11trainingdo
    shell: /bin/bash

runcmd:
  - sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config
  - echo " " >> /etc/ssh/sshd_config
  - echo "AllowUsers 11trainingdo" >> /etc/ssh/sshd_config
  - echo "AllowUsers root" >> /etc/ssh/sshd_config
  - systemctl reload sshd
  - sed -i '/11trainingdo/c
11trainingdo:$6$HeLUJW3a$4xSfDFQjKWfAoGkZF3LFAxM4hgl3d6ATbr2kEu9zMOfwLxkYMO.AJF526mZONwdmsm9sg0tCBK1.SYbhS52u70:17476:0:99999:7:::
/etc/shadow
  - echo "11trainingdo ALL=(ALL) ALL" > /etc/sudoers.d/11trainingdo
  - chmod 0440 /etc/sudoers.d/11trainingdo
```

Kubernetes (Use Cases)

Kubernetes Use Cases



Online-Shop — Black Friday Autoscaling

Autoscaling

Situation: shop.example.de läuft 24/7. An normalen Tagen: 5 Replicas des checkout-Pods. Um 00:00 am Black Friday gehen innerhalb von 2 Minuten 10× mehr Requests ein. Der HPA bemerkt CPU > 70 %, skaliert auf 40 Replicas — kein Neustart, kein Downtime. Um 04:00 Uhr morgens wieder auf 5.

HPA

Deployment

KEDA

PodDisruptionBudget



CI/CD — Ephemere Build-Agents

Build-Pipelines

Situation: 20 Entwickler pushen gleichzeitig. GitLab startet pro Pipeline einen eigenen Pod (2 CPU, 4 Gi RAM), der nach dem Build wieder verschwindet. Keine idle Jenkins-Node kostet Geld, kein "geht nur auf Maschine 3 wegen lokalem Docker-Cache"-Problem.

GitLab Runner

Job

ServiceAccount

ResourceLimits



Multi-Tenant SaaS — Kunden-Isolation

Multi-Tenancy

Situation: Eine HR-Software hostet 200 Firmenkunden auf einem Cluster. Jeder Kunde bekommt einen eigenen Namespace mit NetworkPolicy (kein east-west-Traffic zwischen Kunden), eigenem ResourceQuota und separaten Secrets. Kein Kunde sieht Daten des anderen.

Namespace

NetworkPolicy

ResourceQuota

RBAC

I



ML-Training — GPU-Jobs auf Abruf

Batch-Workloads

Situation: Ein KI-Startup trainiert Modelle über Nacht. Tagsüber laufen die GPU-Nodes idle. Kubernetes startet via Job den Trainingslauf, requestet `nvidia.com/gpu: 2`, und gibt die Nodes danach für andere Workloads frei. Kein fest gebuchter GPU-Server.

Job

NodeSelector

Tolerations

PVC



Rolling Update — Deployment ohne Downtime

Deployments

Situation: Version 2.1.0 soll deployed werden — freitags um 16:00 Uhr. Kubernetes tauscht Pod für Pod aus: erst wenn der neue Pod `ready` ist (readinessProbe grün), wird der alte terminiert. Nutzer merken nichts. Rollback bei Fehler: ein Befehl, fertig.

RollingUpdate

readinessProbe

maxSurge

kubectl rollout undo

Kubernetes Cluster ausrollen

Kubernetes Cluster mit terraform und bash-scripting ausrollen

- ausgerollt mit terraform (binary ist installiert) - `snap install --classic terraform`
- beinhaltet
 1. 1 controlplane
 2. 2 worker nodes
 3. metallb mit ips der Nodes (hacky but works)
 4. ingress mit wildcard-domain: `*.tlx.do.t3isp.de`
 5. cert-manager mit helmfile sync

Prerequisites

```
### Nothing
## in /tmp/.env ist die Umgebungsvariable wie folgt gesetzt
export TF_VAR_do_token=HIER_MUSS_WAS_DRIN_STEHEN
```

Walkthrough

```
cd
git clone https://github.com/jmetzger/training-istio-kubernetes-stack-do-terraform.git
cd install
cat /tmp/.env
source /tmp/.env
terraform init
terraform apply -auto-approve
##
helmfile sync
```

Hinweis

```
## Sollte es nicht sauber durchlaufen
## einfach nochmal
terraform apply -auto-approve

## Wenn das nicht geht, einfach nochmal neu
terraform destroy -auto-approve
terraform apply -auto-approve
```

Kubernetes - Upgrade

Wie Upgrade ich Kubernetes in kubeadm

- <https://github.com/jmetzger/training-kubernetes-einfuehrung/blob/main/kubernetes-upgrade/upgrade.md>

kubectl

kubectl einrichten mit namespace

config einrichten

```
cd
mkdir -p .kube
cd .kube
cp /tmp/config config
ls -la
## Alternative: nano config befüllen
## das bekommt ihr aus Eurem Cluster Management Tool
```

```
kubectl cluster-info
```

Arbeitsbereich konfigurieren

```
NS=jochen # hier tragt ihr euren eigenen Namen ein z.B. NS=peter
```

```
kubectl create ns $NS
kubectl get ns
kubectl config set-context --current --namespace $NS
kubectl get pods
```

```
## Beispiel
## kubectl create ns jochen
## kubectl get ns
## kubectl config set-context --current --namespace jochen
## kubectl get pods
```

kubectl cheatsheet kubernetes

- <https://kubernetes.io/docs/reference/kubectl/cheatsheet/>

kubectl mit verschiedenen Clustern arbeiten

```
### Zwei config in KUBECONFIG env variable
cd ~/.kube
mv config config.kubeadm
export KUBECONFIG=~/.kube/config.kubeadm:~/.kube/config.doks
```

```
## wir sehen beide configs
kubectl config view

kubectl config view --flatten > config
unset KUBECONFIG

## alle Contexti anzeigen lassen
kubectl config get-contexts

kubectl config use-context do-fra1-cluster-bka
kubectl cluster-info
kubectl get nodes

kubectl config use-context kubernetes-admin@kubernetes
kubectl cluster-info
kubectl get nodes
```

2 kubeconfig - Dateien zusammenführen

```
cp ~/.kube/config ~/.kube/config.bak

## Merge (KUBECONFIG mit beiden Dateien setzen)
KUBECONFIG=~/.kube/config1:~/.kube/config2 kubectl config view --flatten > ~/.kube/config-merged

## Als neue config verwenden
mv ~/.kube/config-merged ~/.kube/config
```

Jetzt context anzeigen und auswählen

```
kubectl config get-contexts
kubectl config use-context <context-name>
```

Kubernetes Praxis API-Objekte

Das Tool kubectl (Devs/Ops) - Spickzettel

Hilfe

```
## Hilfe zu befehl
kubectl help config
## Hilfe nächste Ebene
kubectl config set-context --help
```

Allgemein

```
## Zeige Informationen über das Cluster
kubectl cluster-info

## Welche Ressourcen / Objekte gibt es, z.B. Pod
kubectl api-resources
kubectl api-resources | grep namespaces

## Hilfe zu object und eigenschaften bekommen
kubectl explain pod
kubectl explain pod.metadata
kubectl explain pod.metadata.name
```

namespaces

```
kubectl get ns
kubectl get namespaces

## namespace wechseln, z.B. nach Ingress
kubectl config set-context --current --namespace=ingress
## jetzt werden alle Objekte im Namespace Ingress angezeigt
kubectl get all,configmaps

## wieder zurückwechseln.
## der standardmäßige Namespace ist 'default'
kubectl config set-context --current --namespace=default
```

Arbeiten mit manifesten

```

kubectrl apply -f nginx-replicaset.yml
## Wie ist aktuell die hinterlegte config im system
kubectrl get -o yaml -f nginx-replicaset.yml

## Änderung in nginx-replicaset.yml z.B. replicas: 4
## dry-run - was wird geändert
kubectrl diff -f nginx-replicaset.yml

## anwenden
kubectrl apply -f nginx-replicaset.yml

## Alle Objekte aus manifest löschen
kubectrl delete -f nginx-replicaset.yml

## Recursive Löschen
cd ~/manifests
## multiple subfolders subfolders present
kubectrl delete -f . -R

```

Ausgabeformate / Spezielle Informationen

```

## Ausgabe kann in verschiedenen Formaten erfolgen
kubectrl get pods -o wide # weitere Informationen
## im json format
kubectrl get pods -o json
## eine Werte rausziehen
## key ist tls.crt (wichtig escapen => \.), sonst funktioniert das nicht
kubectrl get secrets example-tls -o jsonpath='{.data.tls.crt}' | base64 -d

## gilt natürluch auch für andere kommandos
kubectrl get deploy -o json
kubectrl get deploy -o yaml

## Label anzeigen
kubectrl get deploy --show-labels

```

Zu den Pods

```

## Start einen pod // BESSER: direkt manifest verwenden
## kubectrl run podname image=imagename
kubectrl run nginx image=nginx

## Pods anzeigen
kubectrl get pods
kubectrl get pod

## Pods in allen namespaces anzeigen
kubectrl get pods -A

## Format weitere Information
kubectrl get pod -o wide
## Zeige labels der Pods
kubectrl get pods --show-labels

## Zeige pods mit einem bestimmten label
kubectrl get pods -l app=nginx

## Status eines Pods anzeigen
kubectrl describe pod nginx

## Pod löschen
kubectrl delete pod nginx
## Löscht alle Pods im eigenen Namespace bzw. Default
kubectrl delete pods --all

## Kommando in pod ausführen
kubectrl exec -it nginx -- bash

```

Deployments

```

kubectrl -n ingress rollout restart deployment traefik
ik

```

Alle Objekte anzeigen

```
## Nur die wichtigsten Objekte werden mit all angezeigt
kubectl get all
## Dies, kann ich wie folgt um weitere ergänzen
kubectl get all,configmaps

## Über alle Namespaces hinweg
kubectl get all -A
```

Logs

```
kubectl logs <container>
kubectl logs <deployment>
## e.g.
## kubectl logs -n namespace8 deploy/nginx
## with timestamp
kubectl logs --timestamps -n namespace8 deploy/nginx
## continuously show output
kubectl logs -f <pod>
## letzten x Zeilen anschauen aus log anschauen
kubectl logs --tail=5 <your pod>
```

CRD

```
kubectl get crd
```

Referenz

- <https://kubernetes.io/de/docs/reference/kubectl/cheatsheet/>

kubectl example with run

Example (that does work)

```
## Synopsis (most simplistic example)
## kubectl run NAME --image=IMAGE_EG_FROM_DOCKER
## example
kubectl run nginx --image=nginx:1.23

kubectl get pods
## on which node does it run ?
kubectl get pods -o wide
```

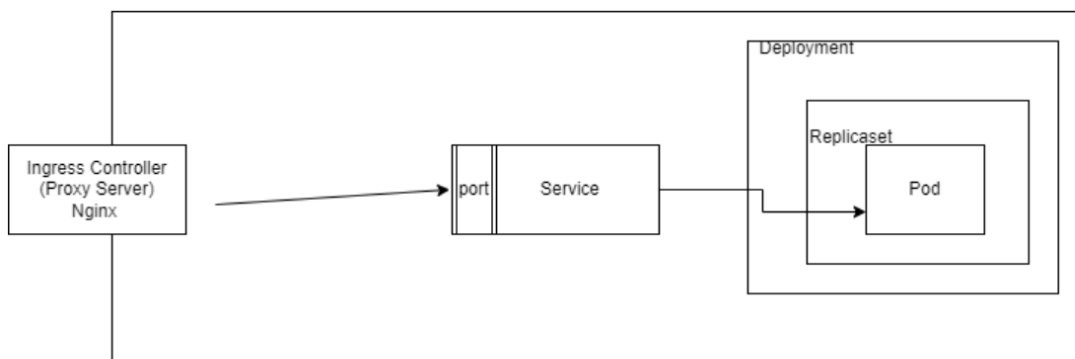
Example (that does not work)

```
kubectl run testpod --image=foo2
## ErrImagePull - Image konnte nicht geladen werden
kubectl get pods
## Weitere status - info
kubectl describe pods testpod
```

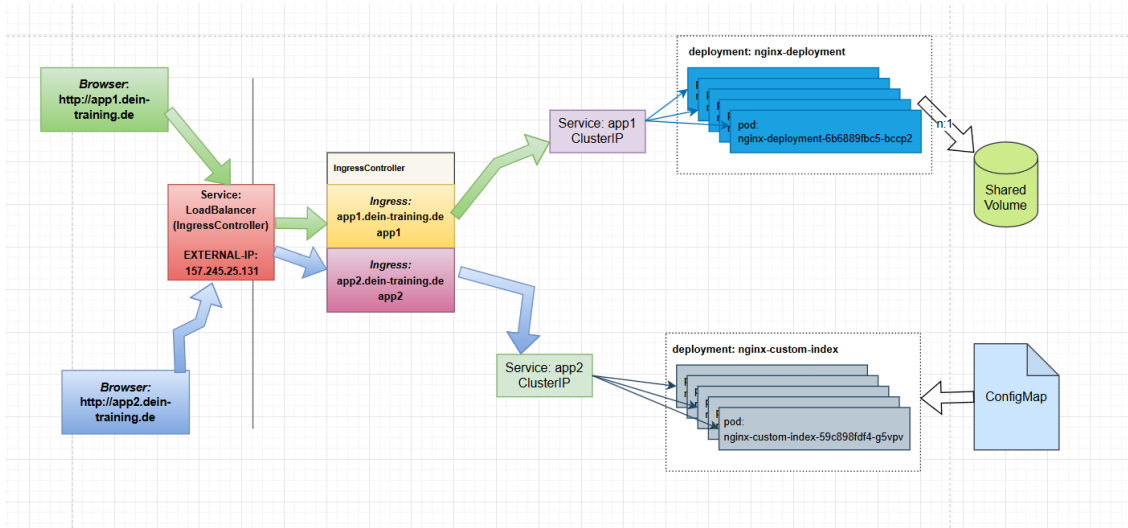
Ref:

- <https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#run>

Bauen einer Applikation mit Resource Objekten



Anatomie einer Webanwendungen



kubectl/manifest/pod

Walkthrough

```
cd
mkdir -p manifests
cd manifests/
mkdir -p 01-web
cd 01-web
nano nginx-static.yml
```

```
## vi nginx-static.yml

apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-web
  labels:
    webserver: nginx
spec:
  containers:
    - name: web
      image: nginx:1.23
```

```
kubectl apply -f nginx-static.yml
```

```
kubectl get pod/nginx-static-web -o wide
kubectl describe pod nginx-static-web
## show config
kubectl get pod/nginx-static-web -o yaml
## seitenweise anzeigen
kubectl get pod/nginx-static-web -o yaml | less
```

Aufräumen

```
kubectl delete -f .
```

kubectl/manifest/replicaset

Walkthrough Erstellen

```
cd
mkdir -p manifests
cd manifests
mkdir 02-rs
cd 02-rs
nano rs.yml
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replica-set
spec:
  replicas: 5
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      name: template-nginx-replica-set
      labels:
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:1.23
          ports:
            - containerPort: 80
```

```
kubectl apply -f .
kubectl get all
kubectl get pods -l tier=frontend
kubectl get pods --show-labels
## name anpassen
kubectl describe pod/nginx-replica-set-lpkbs
```

Pod löschen, was passiert

```
## kubectl delete po nginx-r<TAB>
## einfach einen pod raussuchen und löschen
## z.B.
kubectl delete po nginx-replica-set-xg8jp
```

```
## gucken, welches sind die neuesten ?
kubectl get pods
```

Walthrough Skalieren

```
nano rs.yml
```

```
## Ändern
## replicas: 5
## -> ändern in
## replicas: 8
```

```
kubectl apply -f .
kubectl get pods
```

Aufräumen des replicasets

```
kubectl delete -f .
```

kubectl/manifest/deployments

Prepare

```
cd
mkdir -p manifests
cd manifests
mkdir 03-deploy
cd 03-deploy
nano nginx-deployment.yml
```

```
## vi nginx-deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
```



```

  app: nginx
  replicas: 8 # tells deployment to run 8 pods matching the template
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginxinc/nginx-unprivileged:1.28
        ports:
        - containerPort: 8080

```

```
kubectl apply -f .
```

Explore

```
kubectl get all
```

Optional: Change image - Version

```
nano nginx-deployment.yml
```

Version 1: (optical nicer)

```

## Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
## danach
kubectl apply -f . && watch kubectl get pods

```

Version 2:

```

## Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
## danach
kubectl apply -f . && kubectl get all && kubectl get pods -w

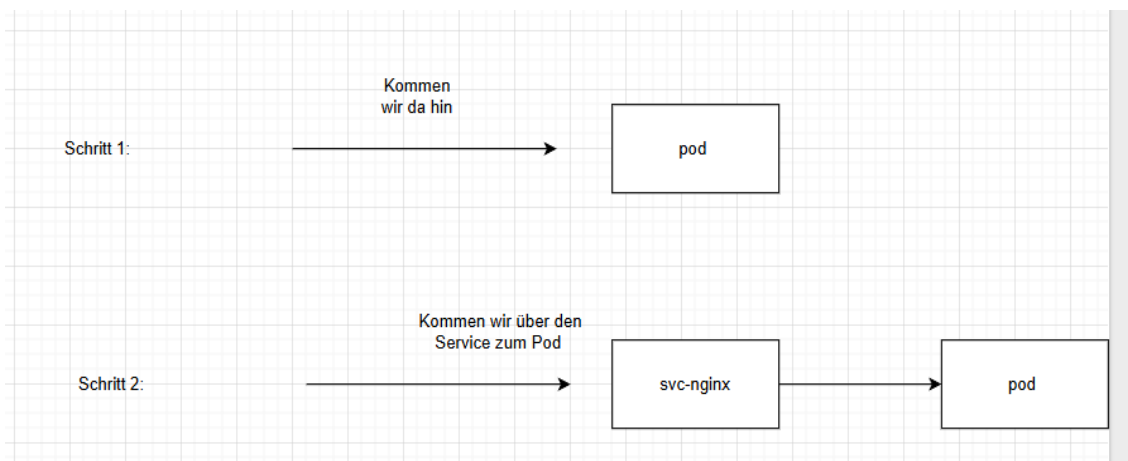
```

Netzwerkverbindung zum Pod testen

Situation

Managed Cluster und ich kann nicht auf einzelne Nodes per ssh zugreifen

Was wollen wir testen (auf der Verbindungsebene) ?



Behelf: Eigenen Pod starten mit busybox

```

## der einfachste Weg
kubectl run podtest --rm -it --image busybox

```

```

## Alternative
kubectl run podtest --rm -it --image busybox -- /bin/sh

```

Example test connection

```
## wget befehl zum Kopieren
ping -c4 10.244.0.99
wget -O - http://10.244.0.99
```

```
## -O -> Output (grosses O (buchstabe))
kubectl run podtest --rm -ti --image busybox -- /bin/sh
/ # wget -O - http://10.244.0.99
/ # exit
```

kubectl/manifest/service

Warum Services ?

- Wenn in einem Deployment bei einem Wechsel des images neue Pods erstellt werden, erhalten diese eine neue IP-Adresse
- Nachteil: Man müsste diese dann in allen Applikation ständig ändern, die auf die Pods zugreifen.
- Lösung: Wir schalten einen Service davor !

Hintergrund IP-Wechsel

```
cln1@client:~/manifests/04-service$ kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE      IP             NODE
web-nginx-69cc9f8d49-9r6wb         1/1      Running   0           5m37s    10.108.0.200   node
-p8s6u    <none>    <none>
web-nginx-69cc9f8d49-z6fxf         1/1      Running   0           5m37s    10.108.0.25    node
-n8s6r    <none>    <none>
```

- Image-Version wurde jetzt in Deployment geändert, Ergebnis:

```
cln1@client:~/manifests/04-service$ kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE      IP             NODE
web-nginx-754794c6d6-8pj99         1/1      Running   0           8s       10.108.1.42    node-p
8s68    <none>    <none>
web-nginx-754794c6d6-swckk         1/1      Running   0          10s      10.108.0.121   node-p
8s6r    <none>    <none>
```

Example I : Service with ClusterIP

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 04-service
cd 04-service
```

Schritt 2: Deployment erstellen

```
nano deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
spec:
  selector:
    matchLabels:
      web: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        web: my-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
nano service.yml
```

```
apiVersion: v1
kind: Service
```

```

metadata:
  name: svc-nginx
spec:
  type: ClusterIP
  ports:
  - port: 80
    protocol: TCP
  selector:
    web: my-nginx

```

```

kubectl apply -f .
## wie ist die ClusterIP ?
kubectl get all
kubectl get svc svc-nginx
## Find endpoints / did svc find pods ?
kubectl describe svc svc-nginx

```

Schritt 3: Deployment löschen

```

kubectl delete -f deploy.yml
## Keine endpunkte mehr
kubectl describe svc svc-nginx

```

Schritt 4: Deployment wieder erstellen

```

kubectl apply -f .
## Endpunkte wieder da
kubectl describe svc svc-nginx

```

Example II : Short version (NodePort)

```

## Wo sind wir ?
## cd; cd manifests/04-service

```

```

nano service.yml
## in Zeile type:
## ClusterIP ersetzt durch NodePort

kubectl apply -f .
## NodePort ab 30.000 ausfindig machen
kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
svc-nginx	NodePort	10.109.24.227	<none>	80:32708/TCP	18h

```

kubectl get nodes -o wide

```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
node-p8s68	Ready	<none>	25h	v1.33.1	10.135.0.72	139.59.128.36	Debian GNU/Linux 12 (bookworm)
node-p8s6c	Ready	<none>	25h	v1.33.1	10.135.0.73	64.226.125.3	Debian GNU/Linux 12 (bookworm)
node-p8s6u	Ready	<none>	25h	v1.33.1	10.135.0.74	159.223.24.231	Debian GNU/Linux 12 (bookworm)

```

## im client Externe NodeIP und NodePort verwenden
curl http://164.92.193.245:32708

```

Example II : Service with NodePort (long version)

```

nano deploy.yml

```

```

## you will get port opened on every node in the range 30000+
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
spec:
  selector:

```

```

    matchLabels:
      web: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        web: my-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx
          ports:
            - containerPort: 80

```

```
nano service.yml
```

```

apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
  labels:
    run: svc-my-nginx
spec:
  type: NodePort
  ports:
    - port: 80
      protocol: TCP
  selector:
    web: my-nginx

```

```
kubectl apply -f .
```

Example III: Service mit LoadBalancer (ExternalIP)

```

cd; cd manifests/04-service
nano service.yml
## in Zeile type:
## NodePort ersetzt durch LoadBalancer

kubectl apply -f .
kubectl get svc svc-nginx

kubectl describe svc svc-nginx
## hier heisst das nicht External-IP ->
## sondern

```

```

---
LoadBalancer Ingress: 129.212.254.6 (VIP)
Port: <unset> 80/TCP

```

```

kubectl get svc svc-nginx -w
## spätestens nach 5 Minuten bekommen wir eine externe ip
## z.B. 41.32.44.45

curl http://41.32.44.45

```

Example getting a specific ip from loadbalancer (if supported)

```

apiVersion: v1
kind: Service
metadata:
  name: svc-nginx2
spec:
  type: LoadBalancer
  # this line to get a specific ip if supported
  loadBalancerIP: 10.34.12.34
  ports:
    - port: 80
      protocol: TCP
  selector:
    web: my-nginx

```

Ref.

- <https://kubernetes.io/docs/concepts/services-networking/connect-applications-service/>

ConfigMap Example MariaDB

Schritt 1: configmap

```
cd
mkdir -p manifests
cd manifests
mkdir cftest
cd cftest
nano 01-configmap.yml
```

```
### 01-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
  name: mariadb-configmap
data:
  # als Wertepaare
  MARIADB_ROOT_PASSWORD: 11abc432
  TEST_CASE: "47"
```

```
kubectl apply -f .
kubectl describe cm mariadb-configmap
kubectl get cm
kubectl get cm mariadb-configmap -o yaml
```

Schritt 2: Deployment

```
nano 02-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:10.11
          envFrom:
            - configMapRef:
                name: mariadb-configmap
```

```
kubectl apply -f .
kubectl get pods
kubectl exec -it deploy/mariadb-deployment -- bash
```

```
env
env | grep ROOT
env | grep TEST
exit
```

Schritt 3: Service for mariadb

```
nano 03-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: mariadb
spec:
  type: ClusterIP
  ports:
    - port: 3306
```

```
protocol: TCP
selector:
  app: mariadb
```

```
kubectl apply -f 03-service.yml
```

Schritt 4: client aufsetzen

```
nano 04-client.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-client
spec:
  selector:
    matchLabels:
      app: ubuntu
  replicas: 1 # tells deployment to run 2 pods matching the template
  template: # create pods using pod definition in this template
    metadata:
      labels:
        app: ubuntu
    spec:
      containers:
      - name: service
        image: ubuntu
        command: [ "/bin/sh" , "-c", "tail -f /dev/null" ]
        envFrom:
        - configMapRef:
            name: mariadb-configmap
```

```
kubectl apply -f 04-client.yml
```

```
## im client
kubectl exec -it deploy/mariadb-client -- bash
apt update; apt install -y mariadb-client iputils-ping
```

Schritt 5: mysql-zugang von aussen erstellen

```
kubectl exec -it deploy/mariadb-deployment -- bash
```

```
mysql -uroot -p$MARIADB_ROOT_PASSWORD
```

```
## innerhalb von mysql
create user ext@'%' identified by '11abc432';
grant all on *.* to ext@'%';
```

Schritt 6: mysql von client aus testen

```
kubectl exec -it deploy/mariadb-client -- bash
```

```
mysql -uext -p$MARIADB_ROOT_PASSWORD -h mariadb
```

```
show databases;
```

Important Sidenode

- If configmap changes, deployment does not know
- So kubectl apply -f deploy.yml will not have any effect
- to fix, use stakater/reloader: <https://github.com/stakater/Reloader>

ConfigMap Example nginx.conf

Schritt 1: configmap

```
cd
mkdir -p manifests
cd manifests
mkdir nginx-conf
cd nginx-conf
nano 01-configmap.yml
```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  nginx.conf: |
    events {
      worker_connections 1024;
    }
    http {
      server {
        listen 80;
        location / {
          return 200 "Hello from ConfigMap!\n";
          add_header Content-Type text/plain;
        }
        location /health {
          return 200 "OK";
        }
      }
    }
  }
}

```

```

kubectl apply -f .
kubectl describe cm nginx-config
kubectl get cm
kubectl get cm nginx-config -o yaml

```

Schritt 2: Pod

```
nano 02-pod.yml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: custom-nginx
spec:
  containers:
    - name: nginx
      image: nginx:alpine
      volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf
      volumes:
        - name: nginx-config
          configMap:
            name: nginx-config

```

```

kubectl apply -f .
kubectl get pods
kubectl exec -it custom-nginx -- sh

```

```

cd /etc/nginx
cat nginx.conf
exit

```

Schritt 3: busybox connection

```

## wir brauchen die pod-ip
kubectl get pods custom-nginx -o wide
kubectl run -it --rm podtest --image=busybox

```

```

## in der shell
wget -O - <ip-von-oben-aus-schritt-3>

```

```
exit
```

Secrets Example MariaDB

Schritt 1: secret

```

cd
mkdir -p manifests
cd manifests

```

```
mkdir secrettest
cd secrettest
```

```
kubectl create secret generic mariadb-secret --from-literal=MARIADB_ROOT_PASSWORD=11abc432 --dry-run=client -o yaml > 01-secrets.yml
```

```
kubectl apply -f .
kubectl get secrets
kubectl get secrets mariadb-secret -o yaml
```

Schritt 2: Deployment

```
nano 02-deploy.yml
```

```
##deploy.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:latest
          envFrom:
            - secretRef:
                name: mariadb-secret
```

```
kubectl apply -f .
```

Testing

```
## Führt den Befehl env in einem Pod des Deployments aus
kubectl exec deployment/mariadb-deployment -- env
## eigentlich macht er das:
## kubectl exec mariadb-deployment-c6df6f959-q6swp -- env
```

Important Sidenote

- If configmap changes, deployment does not know
- So kubectl apply -f deploy.yml will not have any effect
- to fix, use stakater/reloader: [Stakater reloader](#)

Connect to external database

Prerequisites

- MariaDB - Server is running on digitalocean in same network as doks (kubernetes) - cluster (10.135.0.x)
- DNS-Entry for mariadb-server.t3isp.de -> pointing to private ip: 10.135.0.9

Variante 1:

Schritt 1: Service erstellen

```
cd
mkdir -p manifests
cd manifests
mkdir 05-external-db
cd 05-external-db
nano 01-external-db.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: dbexternal
spec:
  type: ExternalName
  externalName: mariadb-server.t3isp.de
```



```
kubectl apply -f 01-external-db.yml
```

Schritt 2: configmap anlegen oder ergänzen

```
## Ergänzen
## unter data zwei weitere Zeile
### 01-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
  name: mariadb-configmap
data:
  # als Wertepaare
  MARIADB_ROOT_PASSWORD: 11abc432
  DB_USER: ext
  DB_PASS: 11dortmund22
```

```
kubectl apply -f 01-configmap.yml
```

```
## client deployment gelöscht
kubectl delete -f 04-client.yml
kubectl apply -f 04-client.yml
kubectl exec -it deploy/mariadb-client -- bash
```

```
## Im client
apt update; apt install -y mariadb-client iputils-ping
```

Schritt 3: Service testen

```
kubectl exec -it deploy/mariadb-client -- bash
```

```
## im container verbinden mit mysql
mysql -u$DB_USER -p$DB_PASS -h dbexternal
```

```
## im verbundenen MySQL-Client
show databases;
```

Variante 2:

```
cd
mkdir -p manifests
cd manifests
mkdir 05-external-db
cd 05-external-db
nano 02-external-endpoint.yml
```

Kubernetes Ingress (Grundlagen)

Hintergrund Ingress

Ref. / Dokumentation

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

Kubernetes Ingress (Traefik)

Install Traefik-IngressController

```
helm repo add traefik https://traefik.github.io/charts

helm upgrade -n ingress --install traefik traefik/traefik --version 40.3.0 --create-namespace --skip-crds --reset-values

kubectl -n ingress get pods
kubectl -n ingress get svc
helm -n ingress status traefik

## Use special crds helm chart instead, because it does not deploy crds for gateway-api by default
## We get an error on digitalocean doks
## Das funktioniert in neuen version nicht mehr weil last-applied zu gross waere
## helm -n ingress upgrade --install traefik-crds traefik/traefik-crds --version 1.18.0 --reset-values
## Workaround
helm template traefik-crds traefik/traefik-crds | kubectl -n ingress apply --server-side -f -
```

Ingress mit traefik

Step 1: Walkthrough

```
cd
mkdir -p manifests
cd manifests
mkdir abi
cd abi
```

```
nano apple-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apple-app
  labels:
    app: apple
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apple
  template:
    metadata:
      labels:
        app: apple
    spec:
      containers:
        - name: web
          image: hashicorp/http-echo
          args:
            - "--text=apple-<euer-name>"
```

```
nano apple-svc.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: apple-service
spec:
  type: ClusterIP
  selector:
    app: apple
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

```
nano banana-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: banana-app
  labels:
    app: banana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: banana
  template:
    metadata:
      labels:
        app: banana
    spec:
      containers:
        - name: web
          image: hashicorp/http-echo
          args:
            - "--text=banana-<euer-name>"
```

```
nano banana-svc.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: banana-service
spec:
  type: ClusterIP
  selector:
    app: banana
  ports:
    - port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

Step 2: Testing connection by podIP and Service

```
kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox
```

```
/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>
```

Step 3: Walkthrough

```
nano ingress.yml
```

```
## Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: "<euername>.appv2.do.t3isp.de"
      http:
        paths:
          - path: /apple
            backend:
              serviceName: apple-service
              servicePort: 80
          - path: /banana
            backend:
              serviceName: banana-service
              servicePort: 80
```

```
## ingress
kubectl apply -f ingress.yml
```

Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

Step 4: Find the problem

Fix 4.1: Fehler: no matches kind "Ingress" in version "extensions/v1beta1"

```
## Gibt es diese Landkarte überhaupt
kubectl api-versions
## auf welcher Landkarte/Gruppe befindet sich Ingress jetzt
kubectl explain ingress | head
## -> jetzt auf networking.k8s.io/v1
```

```
nano ingress.yml
```

```
## auf apiVersion: extensions/v1beta1
## wird -> networking.k8s.io/v1
```

```
kubectl apply -f .
```

Fix 4.2: Bad Request unknown field serviceName / servicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
```

```
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
```

```
nano ingress.yml
```

```
## Wir ersetzen
## serviceName: apple-service
## durch:
## service:
##   name: apple-service

## das gleiche für banana
```

```
kubectl apply -f .
```

Fix 4.3. BadRequest unknown field servicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
## number
kubectl explain ingress.spec.rules.http.paths.backend.service.port
```

```
## neue Variante sieht so aus
backend:
  service:
    name: apple-service
    port:
      number: 80
## das gleich für banana-service
```

```
kubectl apply -f .
```

Fix 4.4. pathType must be specified

```
## Was macht das ?
kubectl explain ingress.spec.rules.http.paths.pathType
```

```
paths:
- path: /apple
  pathType: Prefix
  backend:
    service:
      name: apple-service
      port:
        number: 80
- path: /banana
  pathType: Exact
  backend:
    service:
      name: banana-service
      port:
        number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
```

Step 5: bereits fertige Lösung

```
nano ingress.yml
```

```
## Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: traefik
  rules:
  - host: "<euername>.appv2.do.t3isp.de"
    http:
      paths:
      - path: /apple
        pathType: Prefix
```

```

      backend:
        service:
          name: apple-service
          port:
            number: 80
- path: /banana
  pathType: Exact
  backend:
    service:
      name: banana-service
      port:
        number: 80

```

```

## ingress
kubectl apply -f ingress.yml
kubectl describe ingress

```

Step 6: Testing

```

## mit describe herausfinden, ob er die services gefunden hat
kubectl describe ingress example-ingress

```

```

## Im Browser auf:
## hier euer Name
http://jochen.appv2.do.t3isp.de/apple
http://jochen.appv2.do.t3isp.de/apple/
http://jochen.appv2.do.t3isp.de/apple/foo
http://jochen.appv2.do.t3isp.de/banana
## geht nicht
http://jochen.appv2.do.t3isp.de/banana/nix

```

pathType ImplementationSpecific vs. Prefix bei traefik

Frage aus dem Training: Wie verhält sich `pathType: ImplementationSpecific` bei Traefik konkret, im Unterschied zu `Prefix` und `Exact` ?

Kurze Antwort vorab (Details und Beleg weiter unten):

- Laut Kubernetes-Spec ist `ImplementationSpecific` **nicht spezifiziert** — jeder Ingress-Controller darf hier selbst entscheiden, wie er matcht.
- `Prefix` ist dagegen spezifiziert: Kubernetes verlangt Pfadsegment-Grenzen (`/apple` `matcht` `/apple` , `/apple/` , `/apple/foo` , aber **nicht** `/applebee`).
- Traefik übersetzt intern **beide** pathTypes (`Prefix` UND `ImplementationSpecific`) auf denselben Router-Matcher `PathPrefix(...)` . Und dieser Matcher macht standardmäßig einen reinen String-Präfix-Vergleich **ohne** Segmentgrenze — d.h. `/apple` `matcht` auch `/applebee` . In der Praxis verhalten sich `Prefix` und `ImplementationSpecific` bei Traefik also **identisch**, und keiner von beiden folgt standardmäßig der Kubernetes-Spec für `Prefix` .

Das testen wir jetzt live.

Step 1: Walkthrough - Deployment und Service

```

cd
mkdir -p manifests
cd manifests
mkdir pathtype
cd pathtype

```

```

nano apple-deploy.yml

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: apple-app
  labels:
    app: apple
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apple
  template:
    metadata:
      labels:
        app: apple
    spec:
      containers:
        - name: web
          image: hashicorp/http-echo
          args:
            - "-text=apple-<euer-name>"

```

```
nano apple-svc.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: apple-service
spec:
  type: ClusterIP
  selector:
    app: apple
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

Step 2: Walkthrough - Ingress mit Prefix und ImplementationSpecific nebeneinander

Damit wir die beiden pathTypes direkt vergleichen können, bauen wir zwei Hosts auf demselben Ingress, die auf denselben Pfad `/apple` und denselben Service zeigen - einmal mit `pathType: Prefix`, einmal mit `pathType: ImplementationSpecific`.

```
nano ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: pathtype-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: "prefix-<euername>.apppathtype.do.t3isp.de"
      http:
        paths:
          - path: /apple
            pathType: Prefix
            backend:
              service:
                name: apple-service
                port:
                  number: 80
    - host: "implspec-<euername>.apppathtype.do.t3isp.de"
      http:
        paths:
          - path: /apple
            pathType: ImplementationSpecific
            backend:
              service:
                name: apple-service
                port:
                  number: 80
```

```
kubectl apply -f ingress.yaml
kubectl describe ingress pathtype-ingress
```

Step 3: Testing - matcht /applebee bei Prefix wirklich anders als bei ImplementationSpecific?

Wir testen für beide Hosts dieselben vier Pfade: `/apple`, `/apple/`, `/apple/foo` (sollten laut Spec bei `Prefix` alle matchen) und `/applebee` (sollte laut Spec bei `Prefix` **nicht** matchen, weil `bee` kein eigenes Pfadsegment ist, sondern einfach an `apple` drangehängt).

```
## Host mit pathType: Prefix
curl http://prefix-<euername>.apppathtype.do.t3isp.de/apple
curl http://prefix-<euername>.apppathtype.do.t3isp.de/apple/
curl http://prefix-<euername>.apppathtype.do.t3isp.de/apple/foo
curl http://prefix-<euername>.apppathtype.do.t3isp.de/applebee
```

```
## Host mit pathType: ImplementationSpecific
curl http://implspec-<euername>.apppathtype.do.t3isp.de/apple
curl http://implspec-<euername>.apppathtype.do.t3isp.de/apple/
curl http://implspec-<euername>.apppathtype.do.t3isp.de/apple/foo
curl http://implspec-<euername>.apppathtype.do.t3isp.de/applebee
```

Ergebnis: **Alle acht curls liefern `apple-<euer-name>` zurück** - inklusive `/applebee` bei beiden Hosts. Traefik unterscheidet hier standardmäßig gar nicht zwischen `Prefix` und `ImplementationSpecific`. Das ist kein Trainingsfehler, sondern dokumentiertes Traefik-Verhalten (siehe Referenzen unten): Traefik matcht `PathPrefix` per einfachem String-Vergleich (`strings.HasPrefix`), ohne auf Pfadsegment-Grenzen zu achten.

Step 4: Bonus - echtes Kubernetes-konformes Prefix-Matching aktivieren

Seit Traefik v3.5 gibt es dafür die Provider-Option `strictPrefixMatching` (per Default `false`). Aktiviert man sie in den Helm-Values des Traefik-Providers, matcht `PathPrefix` dann Kubernetes-konform pfadsegmentweise - und zwar für **beide** pathTypes gleichermaßen, weil Traefik intern weiterhin nicht zwischen `Prefix` und `ImplementationSpecific` unterscheidet:

```
## traefik helm values.yaml (Auszug)
additionalArguments:
  - "--providers.kubernetesingress.strictprefixmatching=true"
```

```
helm upgrade -n ingress traefik traefik/traefik --version 40.3.0 --reuse-values -f values.yaml
```

Mit aktiviertem `strictPrefixMatching` würde `/applebee` bei beiden Hosts aus Step 3 **nicht** mehr matchen (404), `/apple`, `/apple/` und `/apple/foo` weiterhin schon.

Fazit

- `pathType: Exact` -> Traefik-Matcher `Path(...)` (exakter Treffer, siehe [04-ingress-traefik-with-hostnames-deployment.md](#)).
- `pathType: Prefix` **und** `pathType: ImplementationSpecific` -> beide landen bei Traefik auf demselben Matcher `PathPrefix(...)`, der standardmäßig ein reiner String-Präfix-Vergleich ohne Segmentgrenze ist. Kubernetes-konformes Verhalten für `Prefix` gibt es bei Traefik nur als Opt-in über `strictPrefixMatching`.
- Für Teilnehmer heißt das praktisch: Verlasst euch bei Traefik nicht darauf, dass `pathType: Prefix` automatisch an Pfadsegmenten stoppt - testet es wie in Step 3, oder aktiviert `strictPrefixMatching` wenn ihr euch auf die Kubernetes-Spec verlassen wollt.

Referenzen

- [Kubernetes Ingress: Path types \(Exact, Prefix, ImplementationSpecific\)](#)
- [Traefik Dok: Kubernetes Ingress Provider - strictPrefixMatching](#)
- [Traefik Dok: Kubernetes Ingress Routing Configuration](#)
- [GitHub Issue traefik/traefik #11200: Prefix-Matching entsprach nicht der Kubernetes-Spec](#)
- [GitHub PR traefik/traefik #11203: Einführung von strictPrefixMatching](#)
- Quellcode zum Nachvollziehen (Version dieses Trainings, Chart 40.3.0 = Traefik v3.7.4): [pkg/provider/kubernetes/ingress/kubernetes.go](#) (Funktionen `loadRouter` und `buildRule`) und [pkg/muxer/http/matcher.go](#) (Funktion `pathPrefix`, macht `strings.HasPrefix(req.URL.Path, path)`).

ingress mit traefik, letsencrypt und cert-manager

Prerequisites

- abi-projekt muss existieren

Trainer: Schritt 1: cert-manager installieren

```
helm repo add jetstack https://charts.jetstack.io
helm upgrade --install cert-manager jetstack/cert-manager \
--namespace cert-manager --create-namespace \
--version v1.19.2 \
--set crds.enabled=true \
--reset-values
```

- Ref: <https://artifactory.io/packages/helm/cert-manager/cert-manager>

Trainer: Schritt 2: Create ClusterIssuer (gets certificates from Letsencrypt)

```
cd
mkdir -p manifests/cert-manager
cd manifests/cert-manager
nano cluster-issuer.yaml
```

```
## cluster-issuer.yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    # Email-Adresse ändern - example.com ist nicht erlaubt
    email: your-email@example.com
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
      - http01:
          ingress:
            class: traefik
```

```
kubectl apply -f .
## Should be True
kubectl get clusterissuer
```

Schritt 3: Ingress-Objekt mit TLS erstellen

```
cd
cd manifests/abi
```

```
nano ingress.yml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - <dein-name>.appv2.do.t3isp.de
    secretName: example-tls

  rules:
  - host: "<dein-name>.appv2.do.t3isp.de"
    http:
      paths:
      - path: /apple
        pathType: Prefix
        backend:
          service:
            name: apple-service
            port:
              number: 80
      - path: /banana
        pathType: Exact
        backend:
          service:
            name: banana-service
            port:
              number: 80
```

```
kubectl apply -f .
```

- Interessent, der cert-manager erstellt kurz ein Ingress - Objekt

```
cm-acme-http-solver-vf2hm <none> jochen.app.do.t3isp.de 174.138.100.89 80 10s
example-ingress traefik jochen.app.do.t3isp.de 174.138.100.89 80, 443 16m
tlnl@client:~/manifests/abi$ kubectl get ingress
NAME CLASS HOSTS ADDRESS PORTS AGE
example-ingress traefik jochen.app.do.t3isp.de 174.138.100.89 80, 443 17m
tlnl@client:~/manifests/abi$ kubectl get certificate
NAME READY SECRET AGE
example-tls True example-tls 85s
```

Schritt 4: Herausfinden, ob Zertifikate erstellt werden

```
kubectl describe certificate example-tls
```

```
## muss auf True stehen
kubectl get cert
```

NAME	READY	SECRET	AGE
example-tls	True	example-tls	39m

```
## Certificate Request
kubectl get cr
## da ist das Zertifikat drin
kubectl get secret example-tls
kubectl get orders
```

Debugging

- Solange das Zertifikat nicht bestätigt bei der ACME-Anfrage (Challenge), seht ihr das noch unter


```
kubectl get challenges
```

Verschlüsselungstiefe erhöhen

- Eintragen in die ingress - resource (z.B. manifests/abi/ingress.yaml)
- Standardmäßig 2048bit

```
# Hier legst du die Verschlüsselungstiefe fest
cert-manager.io/private-key-algorithm: "RSA"
cert-manager.io/private-key-size: "4096"
```

Schritt 5: Testen

- Aufruf der Subdomain im Browser (mit https): z.B. <https://ipchen.app.do.t3isp.de/banana>

Ref:

- <https://hbayraktar.medium.com/installing-cert-manager-and-nginx-ingress-with-lets-encrypt-on-kubernetes-fe0dff4b1924>

cert-manager HTTP-01 Solver: Wo landet die well-known-Datei?

Frage aus dem Training: Beim `http-01` -Check erstellt cert-manager kurz ein eigenes Ingress-Objekt (siehe Screenshot in <https://letsencrypt-ingress-traefik.md>). Wo genau liegt die Datei unter `/.well-known/acme-challenge/<token>`, und wie kommt der Traffic über Traefik dahin?

Hinweis: Diese Übung ist noch nicht live auf einem Cluster verifiziert (siehe TEST-PFLICHT im Skill `workshop-training`). Bevor sie im Training als "fertig getestet" verwendet wird, einmal mit einem laufenden Trainings-Cluster (Traefik + cert-manager + echte Subdomain unter `do.t3isp.de`) durchgehen und Step 3/4 mit echtem Output ergänzen.

Vorwissen (Kurzfassung)

Es gibt **keine Datei auf einem Volume**. cert-manager baut für die Dauer der Challenge (meist nur wenige Sekunden) drei temporäre Objekte im **Namespace des Certificates**:

1. Einen **Solver-Pod** (`quay.io/jetstack/cert-manager-acmesolver`) - ein kleiner Go-HTTP-Server, der auf Port `8089` genau eine Route beantwortet: `GET /.well-known/acme-challenge/<token>`. Die Antwort (Token + Fingerprint des ACME-Account-Keys) wird zur Laufzeit aus den Challenge-Parametern berechnet, nicht aus einer Datei gelesen.
2. Einen **Service** (ClusterIP), der auf diesen Pod zeigt.
3. Ein **Ingress** (Name `cm-acme-http-solver-<hash>`) mit genau einer Pfad-Regel auf `/.well-known/acme-challenge/<token>` -> Service.

Traefik sieht dieses Ingress ganz normal über seinen Watch auf die Ingress-API und baut daraus einen Router mit dem Matcher `PathPrefix(...)` - denselben Mechanismus, den wir in [path-type-implementationspecific.md](#) schon für `Prefix / ImplementationSpecific` angeschaut haben.

Das prüfen wir jetzt live nach - Voraussetzung ist die abgeschlossene Übung <https://letsencrypt-ingress-traefik.md> (cert-manager

- ClusterIssuer laufen bereits, ein Ingress mit `cert-manager.io/cluster-issuer: "letsencrypt-prod"` existiert).

Step 1: Vorbereitung - Watch-Fenster aufmachen

Der Solver läuft nur für den Zeitraum der Challenge (Sekunden bis wenige Minuten). Damit man ihn überhaupt sieht, muss man **parallel zum Auslösen** der Challenge schon einen Watch laufen haben.

```
cd
mkdir -p manifests/http01-solver-watch
cd manifests/http01-solver-watch
```

In einem eigenen Terminal (oder `tmux -Split`) parallel laufen lassen, im Namespace, in dem euer `Certificate / Ingress` mit dem `cert-manager.io/cluster-issuer` -Annotation liegt:

```
kubectl get pod,svc,ingress -w
```

In einem zweiten Terminal parallel:

```
kubectl get challenges -w
```

Step 2: Challenge auslösen

Am einfachsten: Ein neues `Certificate` (oder ein Ingress mit `cert-manager.io/cluster-issuer`) anlegen, für das noch kein gültiges Secret existiert - z.B. mit einer neuen Subdomain aus der vorherigen Übung:

```
nano ingress.yml

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: solver-test-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - solvertest-<dein-name>.appv2.do.t3isp.de
    secretName: solver-test-tls
  rules:
  - host: "solvertest-<dein-name>.appv2.do.t3isp.de"
```

```

http:
  paths:
    - path: /apple
      pathType: Prefix
      backend:
        service:
          name: apple-service
          port:
            number: 80

```

```
kubectl apply -f ingress.yml
```

Step 3: Beobachten, was cert-manager anlegt

Im Watch-Fenster aus Step 1 sollte kurz aufblitzen (Reihenfolge: Pod -> Service -> Ingress):

```

NAME                                READY   STATUS    RESTARTS   AGE
pod/cm-acme-http-solver-xxxxx      1/1     Running   0           3s

NAME                                TYPE          CLUSTER-IP   PORT(S)
service/cm-acme-http-solver-xxxxx  ClusterIP     10.x.x.x      8089/TCP

NAME                                CLASS          HOSTS
ingress.networking.k8s.io/cm-acme-http-solver-xxxxx  traefik        solvertest-<dein-name>.appv2.do.t3isp.de

```

Solange die Objekte existieren, direkt reinschauen:

```
kubectl get ingress cm-acme-http-solver-xxxxx -o yaml
```

Wichtig zu sehen: `spec.rules[0].http.paths[0].path` ist exakt `/.well-known/acme-challenge/<token>` - derselbe Token wie in

```
kubectl get challenges -o yaml
```

unter `spec.token` / `spec.key`.

Von außen (falls schnell genug) lässt sich der Pfad sogar direkt abrufen:

```
curl http://solvertest-<dein-name>.appv2.do.t3isp.de/.well-known/acme-challenge/<token-aus-challenge>
```

Antwort ist der `key` -Wert aus der Challenge (Token + Fingerprint des Account-Keys), **nicht** Inhalt einer Datei.

Step 4: Aufräumen durch cert-manager selbst beobachten

Nach erfolgreicher Validierung (Status der Challenge wechselt auf `valid`) räumt cert-manager Pod, Service und Ingress automatisch wieder ab - kein manuelles Löschen nötig:

```

kubectl get challenges
kubectl get pod,svc,ingress
## cm-acme-http-solver-* sollte nach kurzer Zeit weg sein
kubectl get certificate solver-test-tls
## READY: True

```

Aufräumen (eigene Testobjekte)

```

kubectl delete ingress solver-test-ingress
kubectl delete secret solver-test-tls

```

Zusammenfassung

Frage	Antwort
Wo liegt <code>/.well-known/acme-challenge/<token></code> physisch?	Nirgends als Datei - wird vom Solver-Pod zur Laufzeit aus Token+Key berechnet und per HTTP-Handler ausgeliefert.
In welchem Namespace laufen Solver-Pod/Service/Ingress?	Im selben Namespace wie das Certificate, nicht im <code>cert-manager</code> -Namespace.
Wer routet den Traffic dahin?	Ein von cert-manager temporär erzeugtes Ingress-Objekt, das Traefik ganz normal über seinen Ingress-Watch aufnimmt und als <code>PathPrefix</code> -Router einrichtet.
Wie lange existieren die Objekte?	Nur für die Dauer der Challenge (typ. Sekunden bis wenige Minuten), danach automatisches Cleanup.
Kann cert-manager statt eines neuen Ingress ein bestehendes patchen?	Ja, über <code>solvers[0].http01.ingress.name</code> im Issuer/ClusterIssuer statt eines neu erzeugten Ingress.

Wichtige Fallstricke

- Der `http-01`-Check läuft immer über Port 80/HTTP, unabhängig davon, dass am Ende ein HTTPS-Zertifikat rauskommt - Traefik-EntryPoint für Port 80 muss von außen erreichbar sein.
- DNS muss extern auflösen, da Let's Encrypt von außen validiert.
- Das Zeitfenster zum Beobachten ist kurz - ohne laufenden `-w` -Watch verpasst man die Objekte meistens.

Referenzen

- [cert-manager Doku: HTTP-01 Challenges](#)
- [cert-manager Doku: ACME Issuer](#)
- Quellcode acmesolver: `cmd/acmesolver` und `pkg/util/solver`
- [pathype-implementationsspecific.md](#) - wie Traefik das von cert-manager erzeugte Ingress intern routet.

Traefik errors-Middleware: eigenen HTTP-Code/Body bei Backend-Fehlern liefern

Frage aus dem Training: Kann man bei Traefik festlegen, mit welcher HTTP-Message/welchem Statuscode geantwortet wird, wenn ein Service bzw. dessen Endpunkte nicht erreichbar sind - und kann die Antwort dabei auch einen **anderen** Code liefern als der ursprüngliche Fehler?

Hinweis: Diese Übung ist noch nicht live auf einem Cluster verifiziert (siehe TEST-PFLICHT im Skill `workshop-training`). Vor dem Einsatz im Training einmal real durchspielen und den Beispiel-Output ersetzen.

Wichtiger Hintergrund zuerst: 404 vs. 503 bei Traefik

Bevor man mit der `errors` -Middleware arbeitet, muss man verstehen, **wann Traefik welchen Code liefert** - sonst testet man am falschen Fall vorbei (siehe Zusammenfassung unten, das war ein echter Stolperstein im Training):

Situation	Was Traefik macht	Code
Service-Selektor matcht von Anfang an keine Pods (0 Endpoints)	Traefik baut für Host/Pfad gar keinen Router	404 ("no router matched" - derselbe Fall wie unbekannter Host)
Router existiert (Service hatte/hat Endpoints), Verbindung zum Pod schlägt fehl	Router+Service existieren, Backend nicht erreichbar	502/503 von Traefik
Pod läuft und ist erreichbar, antwortet aber selbst mit einem Fehlercode (z.B. Wartungsmodus)	Ganz normale Anfrage, Antwort kommt vom Pod	Code vom Pod, nicht von Traefik

Für die `errors` -Middleware ist nur Fall 2 und 3 relevant: Die Middleware hängt an einem Router, der tatsächlich existiert. Beim reinen 404-Fall (Zeile 1) gibt es keinen Router, an den man eine Middleware hängen könnte - dafür bräuchte man einen separaten Catch-all-Router, der ist nicht Teil dieser Übung.

Step 1: Vorbereitung

```
cd
mkdir -p manifests/errors-middleware
cd manifests/errors-middleware
```

Step 2: Backend, das erreichbar ist, aber bewusst mit 503 antwortet

Damit der Router wirklich existiert (siehe Hintergrund oben), braucht das Backend einen laufenden, erreichbaren Pod - der antwortet einfach selbst mit 503:

```
nano broken-app.yml

apiVersion: v1
kind: ConfigMap
metadata:
  name: broken-nginx-conf
data:
  default.conf: |
    server {
      listen 80;
      location / {
        return 503 "Simulierter Ausfall\n";
      }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: broken-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: broken-app
  template:
    metadata:
      labels:
        app: broken-app
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          volumeMounts:
            - name: conf
              mountPath: /etc/nginx/conf.d
          volumes:
```

```

- name: conf
  configMap:
    name: broken-nginx-conf
---
apiVersion: v1
kind: Service
metadata:
  name: broken-service
spec:
  selector:
    app: broken-app
  ports:
    - port: 80
      targetPort: 80

```

Step 3: Ingress, der auf den Service zeigt

```
nano broken-ingress.yml
```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: broken-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: broken-<dein-name>.appv2.do.t3isp.de
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: broken-service
                port:
                  number: 80

```

```
kubectl apply -f .
```

Step 4: Vorher testen - noch ohne Middleware

```

kubectl get endpoints broken-service
## sollte eine IP zeigen - Pod ist erreichbar

curl -i http://broken-<dein-name>.appv2.do.t3isp.de/

```

Erwartete Antwort:

```

HTTP/1.1 503 Service Unavailable
Simulierter Ausfall

```

Das ist jetzt tatsächlich der 503-Fall (Router existiert, Pod antwortet selbst mit 503) - im Unterschied zum 404-Fall aus der Hintergrund-Tabelle.

Step 5: Ersatz-Service bauen, der bewusst einen anderen Code liefert

```
nano fallback-page.yml
```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: fallback-nginx-conf
data:
  default.conf: |
    server {
      listen 80;
      location / {
        return 200 "Kein Backend verfuegbar - Ersatzantwort mit Code 200 statt 503\n";
        add_header Content-Type text/plain;
      }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fallback-page
spec:

```

```

replicas: 1
selector:
  matchLabels:
    app: fallback-page
template:
  metadata:
    labels:
      app: fallback-page
  spec:
    containers:
      - name: nginx
        image: nginx:alpine
        volumeMounts:
          - name: conf
            mountPath: /etc/nginx/conf.d
        volumes:
          - name: conf
            configMap:
              name: fallback-nginx-conf
---
apiVersion: v1
kind: Service
metadata:
  name: fallback-page-svc
spec:
  selector:
    app: fallback-page
  ports:
    - port: 80
      targetPort: 80

```

Die Zeile `return 200 "...";` legt fest, mit welchem Code am Ende geantwortet wird. Für einen anderen Code (z.B. 410 Gone) einfach `return 410 "...";` schreiben.

```
kubectl apply -f fallback-page.yml
```

Step 6: Middleware anlegen, die 503 abfängt

```
nano middleware.yml
```

```

apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: fallback-on-503
spec:
  errors:
    status:
      - "503"
  service:
    name: fallback-page-svc
    port:
      number: 80
  query: "/"

```

```
kubectl apply -f middleware.yml
```

Step 7: Middleware am Ingress anhängen

Ein normales `networking.k8s.io/Ingress` kennt Middlewares nicht direkt - Traefik liest sie über eine Annotation aus. Format: `<namespace-der-middleware>-<name-der-middleware>@kubernetescrd` .

```
nano broken-ingress.yml
```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: broken-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.middlewares: <dein-namespace>-fallback-on-503@kubernetescrd
spec:
  ingressClassName: traefik
  rules:
    - host: broken-<dein-name>.appv2.do.t3isp.de
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:

```

```
service:
  name: broken-service
  port:
    number: 80
```

```
kubectl apply -f broken-ingress.yml
```

Step 8: Testen - Antwort sollte jetzt vom Fallback-Service kommen

```
curl -i http://broken-<dein-name>.appv2.do.t3isp.de/
```

Erwartete Antwort jetzt:

```
HTTP/1.1 200 OK
Content-Type: text/plain
...
Kein Backend verfuegbar - Ersatzantwort mit Code 200 statt 503
```

Statt `503` kommt jetzt `200` (oder der Code, den ihr in Step 5 in `return` gesetzt habt) beim Client an - Traefik hat die komplette Antwort (Code + Body) durch die des Fallback-Service ersetzt.

Aufräumen

```
kubectl delete -f .
```

Zusammenfassung

- Der ursprüngliche Fehler-Code (hier 503) ist bei Traefik **hartverdrahtet** - es gibt kein Config-Feld, das ihn direkt umschreibt.
- Die `errors` -Middleware fängt Antworten in einem konfigurierten Statusbereich ab und ersetzt sie komplett durch die Antwort eines eigenen Services.
- Welcher Code am Ende beim Client ankommt, bestimmt **der Fallback-Service selbst** (hier über `return 200 ...;` in der Nginx-Config) - nicht die Middleware-Konfiguration.
- Ist der Fallback-Service nicht erreichbar, fällt Traefik auf den ursprünglichen Code zurück.
- **Achtung 404 vs. 503:** Diese Middleware wirkt nur, wenn der Router überhaupt existiert. Matcht ein Service-Selektor von Anfang an keine Pods (0 Endpoints), baut Traefik gar keinen Router - das Ergebnis ist dann ein einfaches 404, an dem keine Router-gebundene Middleware greift.

Referenzen

- [Traefik Doku: errors-Middleware](#)
- [Traefik Doku: Kubernetes Ingress - Middleware-Annotation](#)
- [path-type-implementation-specific.md](#) - wie Traefik Router aus Ingress-Objekten baut (relevant für das Verständnis, warum ein fehlender Router keine Middleware durchläuft).

Kubernetes Ingress (HA Proxy)

Install HA Proxy-IngressController

```
cd
mkdir -p helm-values/ingress-haproxy
cd helm-values/ingress-haproxy
```

```
nano values.yaml
```

```
controller:
  service:
    type: LoadBalancer
```

```
helm repo add haproxytech https://haproxytech.github.io/helm-charts
helm upgrade -n ingress-haproxy --install ingress-haproxy haproxytech/kubernetes-ingress --version 1.47.4 --reset-values -f values.yaml
```

Ref:

- <https://artifacthub.io/packages/helm/haproxytech/kubernetes-ingress>

Ingress mit ha proxy

Step 1: Walkthrough

```
cd
mkdir -p manifests
cd manifests
mkdir abi
cd abi
```

```
nano apple-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apple-app
  labels:
    app: apple
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apple
  template:
    metadata:
      labels:
        app: apple
    spec:
      containers:
        - name: apple-app
          image: hashicorp/http-echo
          args:
            - "-text=apple-<euer-name>"
```

nano apple-svc.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: apple-service
spec:
  type: ClusterIP
  selector:
    app: apple
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678 # Default port for image
```

kubectl apply -f .

nano banana-deploy.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: banana-app
  labels:
    app: banana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: banana
  template:
    metadata:
      labels:
        app: banana
    spec:
      containers:
        - name: apple-app
          image: hashicorp/http-echo
          args:
            - "-text=banana-<euer-name>"
```

nano banana-svc.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: banana-service
spec:
  type: ClusterIP
  selector:
    app: banana
  ports:
    - port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

Step 2: Testing connection by podIP and Service

```
kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox
```

```
/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>
```

Step 3: Walkthrough

```
nano ingress.yml
```

```
## Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: haproxy
  rules:
    - host: "<euername>.app.do.t3isp.de"
      http:
        paths:
          - path: /apple
            backend:
              serviceName: apple-service
              servicePort: 80
          - path: /banana
            backend:
              serviceName: banana-service
              servicePort: 80
```

```
## ingress
kubectl apply -f ingress.yml
```

Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

Step 4: Find the problem

Fix 4.1: Fehler: no matches kind "Ingress" in version "extensions/v1beta1"

```
## Gibt es diese Landkarte überhaupt
kubectl api-versions
## auf welcher Landkarte/Gruppe befindet sich Ingress jetzt
kubectl explain ingress | head
## -> jetzt auf networking.k8s.io/v1
```

```
nano ingress.yml
```

```
## auf apiVersion: extensions/v1beta1
## wird -> networking.k8s.io/v1
```

```
kubectl apply -f .
```

Fix 4.2: Bad Request unkown field ServiceName / ServicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
```

```
nano ingress.yml
```

```
## Wir ersetzen
## serviceName: apple-service
## durch:
## service:
##   name: apple-service
```



```
## das gleiche für banana
```

```
kubectl apply -f .
```

Fix 4.3. BadRequest unknown field servicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service.port
## number
kubectl explain ingress.spec.rules.http.paths.backend.service.port
```

```
## neue Variante sieht so aus
backend:
  service:
    name: apple-service
    port:
      number: 80
## das gleich für banana-service
```

```
kubectl apply -f .
```

Fix 4.4. pathType must be specified

```
## Was macht das ?
kubectl explain ingress.spec.rules.http.paths.pathType
```

```
paths:
  - path: /apple
    pathType: Prefix
    backend:
      service:
        name: apple-service
        port:
          number: 80
  - path: /banana
    pathType: Exact
    backend:
      service:
        name: banana-service
        port:
          number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
```

Step 5: Testing

```
## mit describe herausfinden, ob er die services gefunden
kubectl describe ingress example-ingress
```

```
## Im Browser auf:
## hier euer Name
http://jochen.app.do.t3isp.de/apple
http://jochen.app.do.t3isp.de/apple/
http://jochen.app.do.t3isp.de/apple/foo
http://jochen.app.do.t3isp.de/banana
## geht nicht
http://jochen.app.do.t3isp.de/banana/nix
```

Kubernetes Praxis (Stateful Sets)

Hintergrund statefulsets

Why ?

- stable network identities (always the same name across restarts) in contrast to deployments

```
Name:      web-0.nginx
Address 1: 10.244.1.6
```

```
Name:      web-1.nginx
Address 1: 10.244.2.20
```

The Pods' ordinals, hostnames, SRV records, and A record names have not changed, but the IP addresses associated with the Pods may have changed.

Features

- **Scaling Up:** Ordered creation on scaling (web-2 till ready then web-3 till ready and so on)

```
StatefulSet controller created each Pod sequentially
with respect to its ordinal index,

and it waited for each Pod's predecessor to be Running and Ready

before launching the subsequent Pod
```

- **Scaling Down:** last created pod is torn down firstly, till finished, then the one before

The controller deleted one Pod at a time,
in reverse order with respect to its ordinal index,
and it waited for each to be completely shutdown before deleting the next.

- **VolumeClaimTemplate** (In addition if the pod is scaled the copies will have their own storage)
 - **Plus:** When you delete it, it gets recreated and claims the same persistentVolumeClaim

```
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
```

- **Update Strategy:** RollingUpdate / OnDelete
- **Feature:** Staging an Update with Partitions
 - <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/#staging-an-update>
- **Feature:** Rolling out a canary
 - <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/#rolling-out-a-canary>

Reference

- <https://kubernetes.io/docs/concepts/workloads/controllers/statefulsets/>

Example stateful set

Schritt 1:

```
cd
mkdir -p manifests
cd manifests
mkdir sts
cd sts
```

```
nano 01-svc.yml
```

```
## vi 01-svc.yml
## Headless Service - no ClusterIP
## Just used for name resolution of pods
## web-0.nginx
## web-1.nginx
## nslookup web-0.nginx
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: ClusterIP
  ports:
  - port: 80
    name: web
  clusterIP: None
  selector:
    app: nginx
```

```
nano 02-sts.yml
```

```
## vi 02-sts.yml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  ## name des statefulset wird nachher für den dns-namen verwendet
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: registry.k8s.io/nginx-slim:0.24
          ports:
            - containerPort: 80
              name: web-nginx
```

```
kubectl apply -f .
```

Schritt 2: Auflösung Namen.

```
kubectl run --rm -it podtester --image=busybox
```

```
## In der shell
## web ist der name des statefulsets
ping web-0.nginx
ping web-1.nginx
exit
```

```
## web-0 / web-1
kubectl get pods -o wide
kubectl get sts web
kubectl delete sts web
kubectl apply -f .
kubectl run --rm -it podtest --image=busybox
```

```
## in the shell
## gleicher namer, aber andere IP als beim letzten Ping
ping web-0.nginx
exit
```

```
kubectl describe svc nginx
```

Schritt 3: Aufräumen

```
kubectl delete -f .
```

Referenz

- <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/>

Kubernetes Secrets und Encrypting von z.B. Credentials

Credentials in Kubernetes verwenden - welche Moeglichkeiten gibt es?

Ein Passwort, API-Key oder Token muss irgendwie in den Container. Kubernetes bietet dafür mehrere Wege — nicht alle sind gleich sicher. Diese Seite gibt den groben Überblick, bevor es an die Details (Secret-Typen, Sealed Secrets, SOPS, Vault) geht.

1. Die vier Grundwege

 Wege für Credentials in einen Container

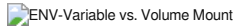
Weg	Beispiel	Wann sinnvoll
env mit festem value	value: "s3cret"	Nie für echte Secrets — landet im Manifest/Git
env mit valueFrom.secretKeyRef	einzelner Key aus einem Secret als eine ENV-Variable	Wenn nur 1-2 Variablen gebraucht werden

<code>envFrom.secretRef</code>	alle Keys eines Secrets werden zu ENV-Variablen	Viele Variablen auf einmal (siehe Beispiel)
<code>volumeMounts</code> (Secret als Datei)	Secret wird unter <code>/etc/secret/...</code> gemountet	Sicherer, wenn die App auch Dateien lesen kann

Praktisches Beispiel für `valueFrom.secretKeyRef` und `envFrom.secretRef`: siehe [Übung: ENV-Variablen aus Secrets](#).

2. ENV-Variable vs. Datei (Volume Mount) — der Sicherheitsunterschied

ENV-Variablen sind bequem, aber sie "kleben" am Prozess: jeder Sub-Prozess erbt sie, und sie sind leicht auslesbar. Ein Secret als Volume-Mount ist die etwas sicherere Wahl.



3. Und wo kommt das Secret-Objekt selbst her?

Beide Wege (ENV oder Volume) setzen voraus, dass es bereits ein Kubernetes- `Secret` -Objekt gibt. Wie dieses sicher **erzeugt und verwaltet** wird, ist eine eigene Frage:

- [Kubernetes Secret-Typen](#) — was ein natives `Secret` -Objekt überhaupt ist (nur base64, nicht verschlüsselt!)
- [Sealed Secrets \(Bitnami\)](#) — Secret verschlüsselt in Git ablegen, Controller entschlüsselt im Cluster
- [SOPS + Age/KMS](#) — Secret-Datei lokal/CI entschlüsseln, dann `kubectl apply`
- [HashiCorp Vault](#) — zentrales Secret-Management, Injection direkt in den Pod (ganz ohne natives `Secret` -Objekt möglich)
- [Vergleich der Ansätze](#) — GitLab CI/CD vs. SOPS vs. Vault
- [AWS Secrets Manager + KMS](#) — siehe Schaubild unten

4. AWS Secrets Manager + KMS an Kubernetes anbinden

Wenn die Secrets bereits in AWS Secrets Manager liegen (dort per KMS verschlüsselt), ist der gängige Weg der **External Secrets Operator (ESO)**: er läuft im Cluster, holt sich über eine eng begrenzte IAM-Rolle (IRSA) periodisch den aktuellen Wert aus Secrets Manager und legt daraus ein ganz normales Kubernetes- `Secret` an — das dann wie in Abschnitt 1 per `env / envFrom / Volume` genutzt wird.



Warum ESO die bevorzugte Wahl ist:

Kriterium	External Secrets Operator (ESO)	AWS Secrets Store CSI Driver
Verbreitung / Doku	Sehr weit verbreitet, viele Backends (nicht nur AWS)	AWS-spezifisch, weniger verbreitet
GitOps-fähig	Ja — Manifest referenziert nur die ARN	Ja — ähnliches Prinzip
Erzeugt natives <code>Secret</code> -Objekt	Ja → funktioniert mit <code>env/envFrom</code>	Optional (Sync-Feature), Standard ist reiner Volume-Mount
Ohne persistentes <code>Secret</code> -Objekt	Nein, per Design	Ja — etwas kleinere Angriffsfläche

Für die meisten Fälle (v.a. wenn ENV-Variablen gebraucht werden) ist ESO der pragmatischste Weg. Nur wenn bewusst **kein** Kubernetes- `Secret` -Objekt im Cluster persistiert werden soll, lohnt sich der CSI Driver.

Konkretes Setup Schritt für Schritt (Helm-Installation, IAM-Rolle, `SecretStore`, `ExternalSecret`): [External Secrets Operator mit AWS Secrets Manager + KMS einrichten](#).

Kurz zusammengefasst

Frage	Antwort
Darf ein Secret-Wert im Manifest stehen (<code>value: "..."</code>)?	Nein — landet im Klartext in Git/kubectl-Historie
Ist ein Kubernetes- <code>Secret</code> an sich schon "sicher"?	Nein — nur base64-kodiert, nicht verschlüsselt
ENV-Variable oder Volume-Mount?	Volume-Mount ist sicherer (kein Leak via <code>env/proc</code>)
Wie bekomme ich das Secret sicher ins Cluster?	Sealed Secrets, SOPS oder Vault — je nach Anforderung

Kubernetes secrets Typen

Welche Arten von Secrets gibt es ?

Built-in Type	Usage
<code>Opaque</code>	arbitrary user-defined data
<code>kubernetes.io/service-account-token</code>	ServiceAccount token
<code>kubernetes.io/dockercfg</code>	serialized <code>~/.docker/cfg</code> file
<code>kubernetes.io/dockerconfigjson</code>	serialized <code>~/.docker/config.json</code> file
<code>kubernetes.io/basic-auth</code>	credentials for basic authentication
<code>kubernetes.io/ssh-auth</code>	credentials for SSH authentication
<code>kubernetes.io/tls</code>	data for a TLS client or server
<code>bootstrap.kubernetes.io/token</code>	bootstrap token data

- Ref: <https://kubernetes.io/docs/concepts/configuration/secret/#secret-types>

Sealed Secrets - bitnami

2 Komponenten

- Sealed Secrets besteht aus 2 Teilen
 - kubeseal, um z.B. die Passwörter zu verschlüsseln
 - Dem Operator (ein Controller), der das Entschlüsseln übernimmt

Schritt 1: Walkthrough - Client Installation (als root)

```
curl -OL "https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.34.0/kubeseal-0.34.0-linux-amd64.tar.gz"
tar -xvzf kubeseal-0.34.0-linux-amd64.tar.gz kubeseal
sudo install -m 755 kubeseal /usr/local/bin/kubeseal
```

Schritt 2: Walkthrough - Server Installation mit kubectl client

```
helm repo add bitnami-labs https://bitnami-labs.github.io/sealed-secrets/
helm upgrade --install sealed-secrets --namespace kube-system bitnami-labs/sealed-secrets --version 2.18.0 --reset-values
```

Schritt 3: Walkthrough - Verwendung (als normaler/unprivilegierter Nutzer)

Übung ist hier zu finden:

[Beispiel mit kubeseal arbeiten](#)

Wie kann man sicherstellen, dass nach der automatischen Änderung des Secretes, der Pod bzw. Deployment neu gestartet wird ?

- <https://github.com/stakater/Reloader>

Ref:

- Controller: <https://github.com/bitnami-labs/sealed-secrets/releases/>

Exercise Sealed Secret mariadb

Prerequisites: MariaDB secrets done

[MariaDB Secret](#)

Based on mariadb secrets exercise

```
cd
cd manifests/secrettest
```

```
## Cleanup
kubectl delete -f 02-deploy.yml
kubectl delete -f 01-secrets.yml
## rm
rm 01-secrets.yml
```

```
## Öffentlichen Schlüssel zum Signieren holen
kubeseal --fetch-cert --controller-namespace=kube-system --controller-name=sealed-secrets > pub-sealed-secrets.pem
cat pub-sealed-secrets.pem
```

```
## Secret - config erstellen mit dry-run, wird nicht auf Server angewendet (nicht an Kube-API-Server geschickt)
kubectl create secret generic mariadb-secret --from-literal=MARIADB_ROOT_PASSWORD=11abc432 --dry-run=client -o yaml > 01-top-secret.yaml
cat 01-top-secret.yaml
```

```
kubeseal --format=yaml --cert=pub-sealed-secrets.pem < 01-top-secret.yaml > 01-top-secret-sealed.yaml
cat 01-top-secret-sealed.yaml
```

```
## Ausgangsfile von dry-run löschen
rm 01-top-secret.yaml
```

```
## Ist das secret basic-auth vorher da ?
kubectl get secrets mariadb-secret
```

```
kubectl apply -f .
```

```
## Kurz danach erstellt der Controller aus dem sealed secret das secret
kubectl get secret
```

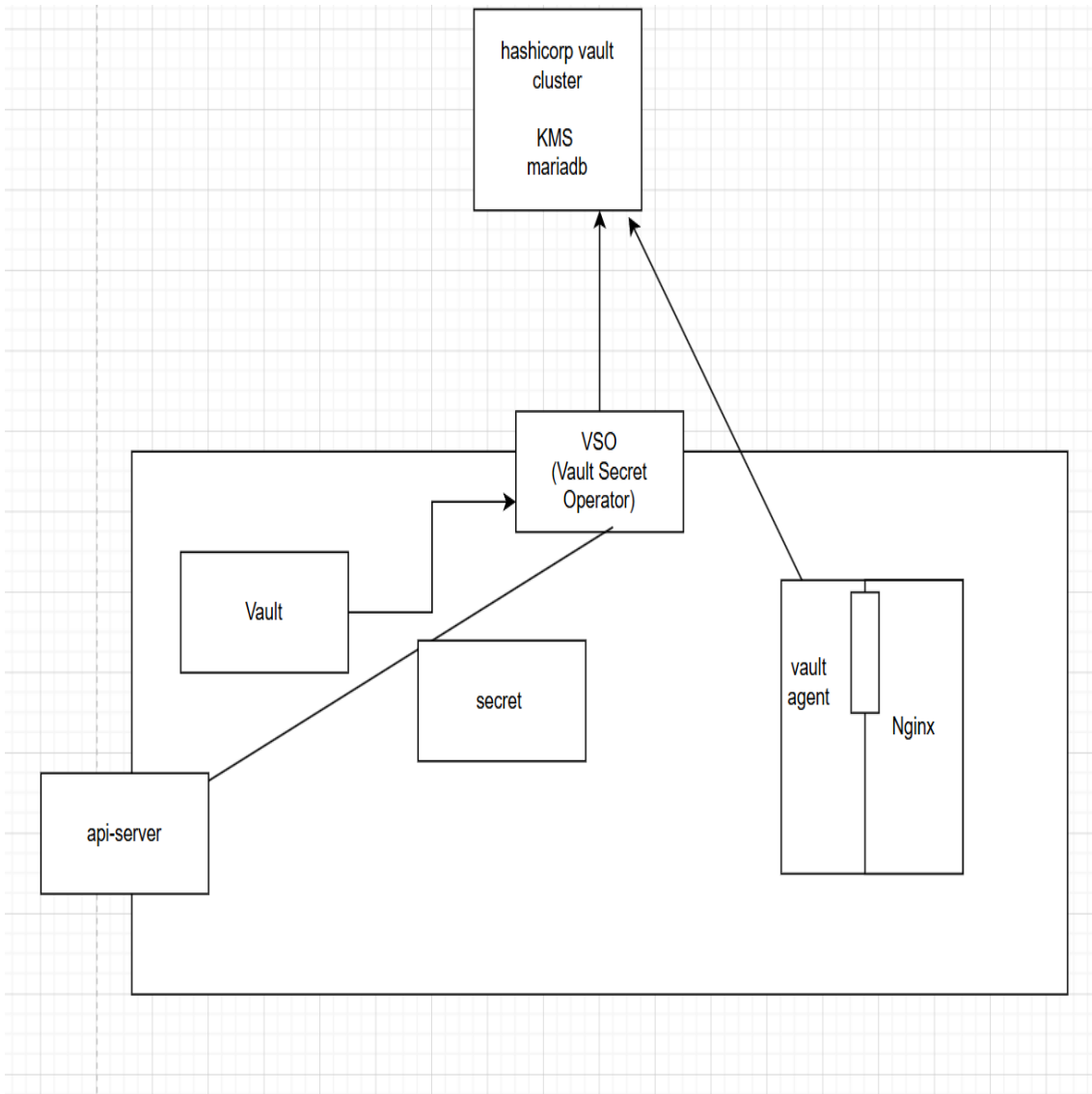
```
kubectl get sealedsecrets
kubectl get secret mariadb-secret -o yaml
```

```
kubectl exec -it deploy/mariadb-deployment -- env | grep ROOT
kubectl delete -f 01-top-secret-sealed.yaml
```

```
kubectl get secrets
kubectl get sealedsecrets
```

Overview Hashicorp Vault

Zentrale Externer Server mit 3 Nodes (Produktion)



3-Wege für Kubernetes Daten zu bekommen

- VSO (Vault Secrets Operator)
- SideCar Injection
- Volumes

VSO

- Ich bestücke eine neue CRT mit dem Wunsch eines Credentials "Vault Static Secret"

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata:
  name: webapp-config
  namespace: default
spec:
  # Reference to VaultAuth in another namespace
  vaultAuthRef: vault-secrets-operator-system/default

  # Vault mount path (where the secret engine is mounted)
  mount: secret
```

```
# Path to the secret within the mount
path: webapp/config

# Type of secret engine
type: kv-v2

# Destination Kubernetes secret configuration
destination:
  create: true
  name: webapp-secret
  type: Opaque

# How often to refresh the secret from Vault
refreshAfter: 30s
```

Nachteil

- Das automatisch erstellte Secret wird in etc gespeichert, solange wie das VaultStaticSecret existiert

Vault Sidecar Injector

Vorteile

- Sicherste Variante
- Es wird kein Secret erstellt, passwort wird direkt im Pod zur Verfügung gestellt (in einer Datei)

Nachteile

- Relativ viele Einträge im Pod über Annotations zu machen, damit das funktioniert
- Overhead über SideCar (weil jeder Pod ein Sidecar bekommt)
- Bekommt mit, wenn sich das Passwort ändert

Volumes

Alternative zu Hashicorp Vault - Fork OpenBao

- <https://openbao.org/>

registry mit secret auth

- <https://kubernetes.io/docs/tasks/configure-pod-container/pull-image-private-registry/>

Kubernetes secrets mit sops (mariadb)

Warum SOPS?

Kubernetes Secrets sind **base64-kodiert, nicht verschlüsselt** – jeder mit Cluster-Zugriff kann sie lesen:

```
kubectl get secret mariadb-secret -o jsonpath='{.data.MARIADB_ROOT_PASSWORD}' | base64 -d
## → 11abc432
```

Das Problem: Wie speichert man Secrets **sicher in Git**? SOPS löst das – nur verschlüsselte Dateien ins Repo, Klartext existiert nie auf Disk.

SOPS Plugin-Support

GitOps

Tool	Integration	Wie
Flux CD	Nativ	decryption.provider: sops in Kustomization
ArgoCD	Plugin	argocd-vault-plugin oder helm-secrets

Helm

Tool	Integration	Wie
helm-secrets	Plugin	helm secrets upgrade ... -f secrets.enc.yaml
Helmfile	Nativ	secrets: Block in helmfile.yaml

CI/CD

Tool	Integration	Wie
GitLab CI	Manuell	SOPS_AGE_KEY als masked Variable
GitHub Actions	Action	getsops/sops-action
Jenkins	Manuell	Credentials Plugin + Shell

Key Backends

Backend	Typisch für

Age	Self-hosted, einfachste Option
AWS KMS	AWS-Umgebungen
GCP KMS	GCP-Umgebungen
OpenBao/Vault	Self-hosted Enterprise
PGP	Legacy, nicht empfohlen

Fazit: SOPS ist der De-facto-Standard für verschlüsselte Secrets in GitOps-Workflows – besonders in Kombination mit **Flux CD + Age** für Self-hosted Setups.

Wie funktioniert SOPS?

SOPS verschlüsselt **nur die Values**, nicht die Keys – die Dateistruktur bleibt lesbar:

```
## Klartext
db_password: geheim123

## Mit SOPS verschlüsselt
db_password: ENC[AES256_GCM,data=xyz...,tag=abc...,type=str]
sops:
  age: [...]
  lastmodified: "2024-01-01T00:00:00Z"
  mac: "..."
```

SOPS generiert pro Datei einen zufälligen **Data Encryption Key (DEK)**:

```
Encrypt: Datei → SOPS → DEK (zufällig) → verschlüsselt Values
          ↓
          DEK selbst wird mit Age/KMS verschlüsselt
          und in der Datei gespeichert

Decrypt: SOPS liest verschlüsselten DEK → Age/KMS entschlüsselt DEK → Values entschlüsseln
```

Was ist Age?

age steht für "**A**ctually **G**ood **E**ncryption" – moderner Ersatz für PGP/GPG.

Age verwendet ein asymmetrisches Schlüsselpaar:

```
Public Key → in .sops.yaml → darf jeder sehen → zum Verschlüsseln
Private Key → ~/sops-age.key → geheim halten → zum Entschlüsseln
```

Generiert mit:

```
age-keygen -o ~/sops-age.key
## Public key: age1xyz... ← direkt ausgegeben
```

Lab: Mariadb Secret mit SOPS verschlüsseln

Voraussetzungen

Installation: **age & sops**

```
## age installieren
sudo apt-get update
sudo apt-get install -y age

## sops installieren (aktuelle Version)
SOPS_VERSION=$(curl -s https://api.github.com/repos/getsops/sops/releases/latest | grep tag_name | cut -d '"' -f 4)
curl -LO https://github.com/getsops/sops/releases/download/${SOPS_VERSION}/sops-${SOPS_VERSION}.linux.amd64
sudo mv sops-${SOPS_VERSION}.linux.amd64 /usr/local/bin/sops
sudo chmod +x /usr/local/bin/sops

## Verifizieren
age --version
sops --version
```

Schritt 1: Age Key generieren

```
age-keygen -o ~/sops-age.key
cat ~/sops-age.key
```

Frage: Welche zwei Bestandteile siehst du in der Datei?


```
## Public Key für nächsten Schritt extrahieren
grep "public key" ~/sops-age.key
```

Schritt 2: SOPS konfigurieren

```
cd
mkdir -p manifests/secrettest-sops
cd manifests/secrettest-sops
```

.sops.yaml erstellen – Public Key von oben eintragen:

```
cat > .sops.yaml <<EOF
creation_rules:
  - path_regex: .*secrets.*\.yaml
    age: <DEIN-PUBLIC-KEY>
EOF
```

Schritt 3: Secret erstellen und verschlüsseln

Unverschlüsselte Secret-Datei erstellen:

```
kubect1 create secret generic mariadb-secret \
--from-literal=MARIADB_ROOT_PASSWORD=11abc432 \
--dry-run=client -o yaml > 01-secrets.yaml
```

Verschlüsseln:

```
export SOPS_AGE_KEY_FILE=~/.sops-age.key
sops -e 01-secrets.yaml > 01-secrets.enc.yaml

## Originaldatei löschen - nie committen!
rm 01-secrets.yaml
```

Ergebnis ansehen:

```
cat 01-secrets.enc.yaml
```

Frage: Welche Felder wurden verschlüsselt, welche nicht? Warum ist `metadata.name` noch lesbar?

Schritt 4: Deployment erstellen

```
nano 02-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:latest
          envFrom:
            - secretRef:
                name: mariadb-secret
```

Schritt 5: Entschlüsseln und deployen

```
export SOPS_AGE_KEY_FILE=~/.sops-age.key

## Entschlüsseln und direkt anwenden (kein temporäres File auf Disk)
```

```
sops -d 01-secrets.enc.yaml | kubectl apply -f -

## Deployment anwenden
kubectl apply -f 02-deploy.yaml
```

Schritt 6: Verifizieren

```
kubectl get secrets
kubectl get secrets mariadb-secret -o yaml
kubectl get pods
```

```
## Secret im Pod prüfen
kubectl exec deployment/mariadb-deployment -- env | grep MARIADB
```

Frage: Was siehst du bei `kubectl get secrets mariadb-secret -o yaml` unter `data` ? Ist das verschlüsselt?

Schritt 7: Datei bearbeiten

```
## SOPS öffnet direkt im Editor, entschlüsselt temporär im RAM
sops 01-secrets.enc.yaml
```

Passwort ändern, speichern – SOPS re-verschlüsselt automatisch.

```
## Neu deployen
sops -d 01-secrets.enc.yaml | kubectl apply -f -
kubectl exec deployment/mariadb-deployment -- env | grep MARIADB
```

Cleanup

```
kubectl delete deployment mariadb-deployment
kubectl delete secret mariadb-secret
```

Wichtige Regeln

✔ 01-secrets.enc.yaml	In Git committen – safe
✔ .sops.yaml	In Git committen – nur Public Key
✘ 01-secrets.yaml	Niemals committen
✘ ~/.sops-age.key	Niemals committen

.gitignore :

```
## Unverschlüsselte Secrets blockieren
*secrets.yaml
```

Important Sidenote

- If secret changes, deployment does not know → Stakater Reloader verwenden
- Der Age Private Key ist das einzige was zur Entschlüsselung nötig ist – sicher aufbewahren!
- In CI/CD: Private Key als masked Variable (`SOPS_AGE_KEY` in GitLab)

Vergleich: GitLab CI/CD vs. SOPS vs. Vault

Überblick: Drei Ansätze im Vergleich

Kriterium	GitLab CI/CD Secrets	SOPS + Age/KMS	HashiCorp Vault
Zweck	Pipeline-Variablen	Secrets in Git verschlüsseln	Zentrales Secret-Management
Speicherort	GitLab-Datenbank	Git (verschlüsselt)	Vault-Server
Verschlüsselung	AES-256 in DB	AES-256-GCM pro Datei	AES-256-GCM + Shamir's Secret Sharing
Audit-Trail	Nein	Nein	Ja – jeder Zugriff wird geloggt
Dynamic Secrets	Nein	Nein	Ja (z.B. DB-Passwörter mit TTL)
Secret Rotation	Manuell	Manuell	Automatisch
Zugriffskontrolle	Grob (Projekt/Branch)	Wer den privaten Key hat	Feingranular per Policy
Kubernetes-Integration	Nur via env vars	Via SOPS-Operator / Flux	VSO, Sidecar-Injector, Volumes

GitOps-fähig	Schlecht	Sehr gut	Gut
Compliance	Schwach	Mittel	Stark (SOC2, PCI-DSS)
Betriebskomplexität	Minimal	Gering	Hoch

Sind GitLab CI/CD Secrets sicher?

Technisch: GitLab verschlüsselt Variablen mit AES-256-GCM in PostgreSQL.

Aber: Encryption Key und verschlüsselte Daten liegen auf **demselben Server**:

GitLab-Server

/etc/gitlab/gitlab.rb

→ secret_key_base = "abc..."

PostgreSQL

→ encrypted_value = "xyz..."

← Encryption Key

← Verschlüsselte Secrets

Root-Zugriff auf den Server = Zugriff auf alles

Was "masked" und "protected" wirklich bedeuten

Option	Was es tut	Was es NICHT schützt
Masked	Wert in Logs → [MASKED]	Nicht gegen <code>echo \$VAR base64</code>
Protected	Nur auf protected Branches	Nicht gegen Maintainer-Zugriff
File-Type	Secret als Datei statt Env-Var	Liegt trotzdem im Runner-Filesystem

Angriffsvektoren in der Pipeline

```
## Masking lässt sich leicht umgehen:
- echo $SECRET           # [MASKED] - geblockt
- echo $SECRET | base64  # Klartext sichtbar!
- env | grep SECRET      # alle Secrets sichtbar
- cat /proc/self/environ # Env-Vars aus Prozess-Speicher
```

Nachteile im Detail

GitLab CI/CD Secrets

Nachteil	Warum problematisch
Secrets landen als Env-Vars im Prozess	Jede Library kann <code>env</code> lesen
Kein Audit-Trail	Kein Nachweis wer wann welches Secret verwendet hat
GitLab-Admin = God Mode	Sieht alle Secrets aller Projekte
Keine Rotation	Abgelaufene Secrets müssen manuell getauscht werden
Nicht GitOps-fähig	Secrets außerhalb von Git → Drift möglich
Scope nur grob	Kein "Job A darf nur Secret X"

SOPS + Age/KMS

Nachteil	Warum problematisch
Private Key = Single Point of Failure	Verloren → Datenverlust, gestohlen → Totalverlust
Kein Audit-Trail	SOPS loggt nicht wer wann entschlüsselt hat
Keine Dynamic Secrets	Passwörter sind statisch, kein automatisches Ablaufen
Key-Rotation aufwändig	Alle Dateien müssen re-verschlüsselt werden (<code>sops updatekeys</code>)
Decrypted Secret muss irgendwo hin	Am Ende landet es doch als Kubernetes-Secret (base64)
Fehleranfällig	<code>secrets.yaml</code> versehentlich committen → Plaintext in Git-History für immer

HashiCorp Vault

Nachteil	Warum problematisch
Hohe Betriebskomplexität	HA-Cluster, Unsealing, TLS, Backup - braucht dediziertes Ops-Team
Unsealing ist kritisch	Nach Neustart: Vault versiegelt → alle abhängigen Services down

Single Point of Failure	Vault down → keine neuen Secrets abrufbar
Lizenzkosten (Enterprise)	Namespaces, HSM, DR → kostenpflichtig
Steile Lernkurve	Auth-Methoden, Policies, Secret Engines, Leases
Netzwerkabhängigkeit	Pod wartet auf Vault beim Start → Startup-Reihenfolge kritisch
Vault selbst = Angriffsziel	Speichert alle Secrets → hochattraktiv für Angreifer

Wann welches Tool?

GitLab CI/CD Secrets
└─ ✓ Einfache Projekte, Dev-Umgebungen
└─ ✓ Nicht-kritische Credentials (z.B. Test-Tokens)
└─ ✗ Produktions-DBs, API-Keys mit hohem Impact, Multi-Team
SOPS + Age/KMS
└─ ✓ GitOps-Workflows (Flux, ArgoCD)
└─ ✓ Secrets sicher in Git versionieren
└─ ✓ Self-hosted ohne Vault-Infrastruktur
└─ ✗ Dynamic Secrets, Audit-Trail, viele Teams
HashiCorp Vault
└─ ✓ Enterprise, Compliance (SOC2, PCI-DSS)
└─ ✓ Viele Services und Teams
└─ ✓ Dynamic Secrets mit TTL (z.B. DB-Credentials)
└─ ✓ Vollständiger Audit-Trail
└─ ✗ Kleines Team ohne dediziertes Infra-Team

Typische Kombination in der Praxis

GitLab CI/CD Variable: SOPS_AGE_KEY (masked + protected)
┆
▼
SOPS entschlüsselt Secrets aus Git
┆
▼
kubectl apply → Kubernetes Secret (base64)
┆
▼
Pod nutzt Secret als Env-Variable oder Volume

Vault ergänzt dieses Setup wenn **dynamische Secrets** oder **Compliance-Anforderungen** hinzukommen.

Diskussionsfragen

1. Warum schützt "masked" in GitLab nicht vollständig vor Secret-Leaks?
2. Was passiert wenn der Age Private Key verloren geht?
3. In welchem Szenario würdet ihr Vault trotz der hohen Komplexität einsetzen?
4. Was ist der Unterschied zwischen einem statischen und einem dynamischen Secret?

Hashicorp Vault

Architektur einfach erklärt

Die Idee: ein Tresorraum fuer Geheimnisse

Stell dir Vault wie den Tresorraum einer Bank vor. Statt Goldbarren liegen darin Geheimnisse: Passwoerter, Datenbank-Zugaenge, Zertifikate. Niemand schreibt diese Geheimnisse mehr in den Programmcode oder in Konfigurationsdateien - wer eines braucht, geht zum Tresor und fragt danach.



Die 4 Schritte

1. **Ausweis-Kontrolle (Authentifizierung):** Deine App meldet sich an und beweist, wer sie ist. Dafuer bekommt sie einen Ausweis - den **Token**.
2. **Regel-Check (Policy):** Vault schaut in seine Regelliste: Was darf dieser Ausweis sehen? Die App bekommt nur genau die Geheimnisse, die fuer sie erlaubt sind - nicht mehr.
3. **Schublade oeffnen (Secret Engine):** Jede Art von Geheimnis liegt in einer eigenen Schublade. Manche Schubladen geben gespeicherte Passwoerter heraus, andere erzeugen sogar frische Zugangsdaten, die nach kurzer Zeit automatisch wieder ungeltig werden.
4. **Antwort:** Die App bekommt ihr Geheimnis und kann damit z.B. auf die Datenbank zugreifen.

Die wichtigsten Begriffe uebersetzt

Fachbegriff	Einfach gesagt
Token	Ausweis, den man nach dem Anmelden bekommt

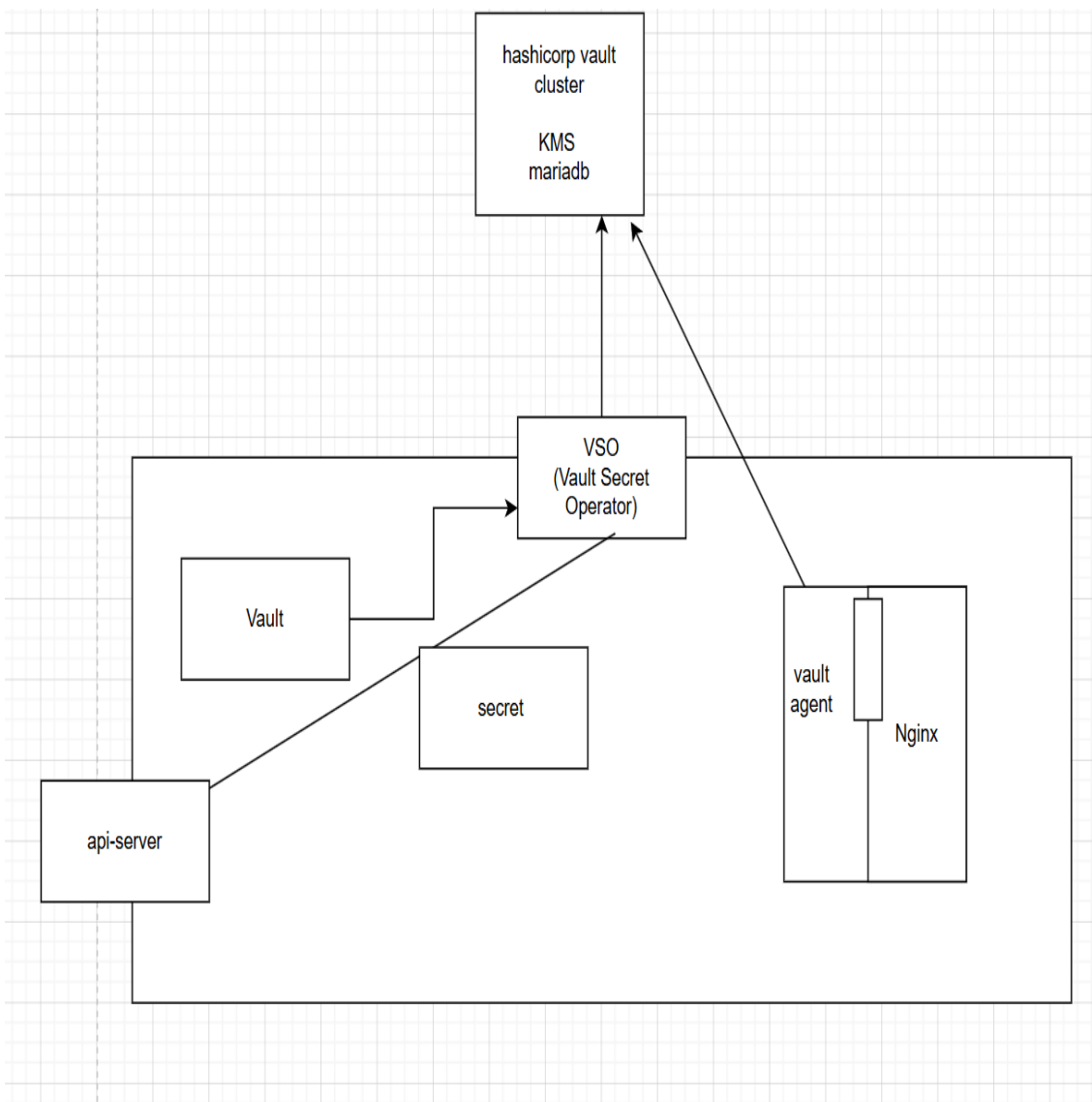
Policy	Regelliste: wer darf was sehen
Secret Engine	Schublade fuer eine bestimmte Art von Geheimnis
Sealed / Unseal	Tresor verriegelt / Tresor aufschliessen
Audit Log	Besucherbuch: wer hat wann welches Geheimnis geholt

Warum der Aufwand?

- Passwoerter stehen nicht mehr im Code oder in Git - dort werden sie am haeufigsten gestohlen.
- Alle Geheimnisse liegen an EINEM zentralen Ort und sind dort verschlüsselt.
- Jeder Zugriff wird protokolliert - man sieht, wer wann was geholt hat.
- Geheimnisse lassen sich zentral austauschen oder sperren, ohne dass irgendwo Code angefasst werden muss.

Uebersicht Vault in Kubernetes

Zentrale Externer Server mit 3 Nodes (Produktion)



3-Wege für Kubernetes Daten zu bekommen

- VSO (Vault Secrets Operator)
- SideCar Injection
- Volumes

VSO

- Ich bestücke eine neue CRT mit dem Wunsch eines Credentials "Vault Static Secret"

```

apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata:
  name: webapp-config
  namespace: default
spec:
  # Reference to VaultAuth in another namespace
  vaultAuthRef: vault-secrets-operator-system/default

  # Vault mount path (where the secret engine is mounted)
  mount: secret

  # Path to the secret within the mount
  path: webapp/config

  # Type of secret engine
  type: kv-v2

  # Destination Kubernetes secret configuration
  destination:
    create: true
    name: webapp-secret
    type: Opaque

  # How often to refresh the secret from Vault
  refreshAfter: 30s

```

Nachteil

- Das automatisch erstellte Secret wird in etcd gespeichert, solange wie das VaultStaticSecret existiert

Vault Sidecar Injector

Vorteile

- Sicherste Variante
- Es wird kein Secret erstellt, passwort wird direkt im Pod zur Verfügung gestellt (in einer Datei)

Nachteile

- Relativ viele Einträge im Pod über Annotations zu machen, damit das funktioniert
- Overhead über SideCar (weil jeder Pod ein Sidecar bekommt)
- Bekommt mit, wenn sich das Passwort ändert

Volumes

Uebung: Vault Agent Injection

Hintergrund

Was ist der Vault Agent Injector?

Der Vault Agent Injector ist ein **Mutating Webhook** in Kubernetes. Das bedeutet: Jede neue Pod-Definition wird automatisch abgefangen und veraendert, bevor der Pod wirklich startet — ohne dass du deinen Application-Code anfassen musst.



Vault Agent Injector Mutating Webhook

Die Annotation `vault.hashicorp.com/agent-inject: "true"` im Pod-Manifest reicht aus, damit der Injector zwei neue Container in jeden Pod einschleust:

- **vault-agent-init** (Init Container): laeuft einmalig beim Start, holt das Secret aus Vault
- **vault-agent** (Sidecar Container): laeuft dauerhaft neben deiner App, erneuert Leases

Wie laeuft die Authentifizierung ab?

Der Pod muss sich bei Vault beweisen, dass er berechtigt ist, das Secret zu lesen. Das passiert ueber den **Kubernetes ServiceAccount Token** (JWT) — vollautomatisch.



Vault Kubernetes Auth Ablauf

Vault fragt Kubernetes per **TokenReview** ob der JWT-Token gueltig ist und prueft dann, ob der ServiceAccount und der Namespace zur konfigurierten Role passen.

Wo landet das Secret?



Secret Location Vergleich

Das Besondere: Es wird **kein Kubernetes Secret Objekt** erstellt. Das Secret existiert nur als Datei im Arbeitsspeicher des Pods (`/vault/secrets/config`) und taucht weder in `kubect1 get secrets` auf noch wird es in etcd gespeichert.

Voraussetzung: Vault laeuft im Cluster (Trainer-Setup, einmalig)

```

helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update

```

```
helm install vault hashicorp/vault \
--namespace vault \
--create-namespace \
--set "server.dev.enabled=true" \
--set "server.dev.devRootToken=root" \
--set "injector.enabled=true" \
--wait
```

Kubernetes Auth aktivieren:

```
kubectl exec -n vault vault-0 -- vault auth enable kubernetes

kubectl exec -n vault vault-0 -- vault write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc.cluster.local:443"
```

Vault laeuft im Dev-Modus: Root-Token `root` , kein TLS, In-Memory-Storage.

Schritt 1: Verzeichnis anlegen und Namen setzen

```
cd
mkdir -p manifests
cd manifests
mkdir vault-injection
cd vault-injection
```

Namen einmalig setzen — alle folgenden Befehle nutzen automatisch `$NAME` :

```
NAME=jochen
```

Schritt 2: Namespace und ServiceAccount erstellen

```
kubectl create namespace vault-$NAME
kubectl create serviceaccount vault-auth -n vault-$NAME
```

Pruefe:

```
kubectl get serviceaccount vault-auth -n vault-$NAME
```

Schritt 3: Secret in Vault anlegen

Vault laeuft als Pod im Namespace `vault` . Alle Vault-Befehle werden per `kubectl exec` ausgefuehrt.

```
kubectl exec -n vault vault-0 -- vault kv put secret/$NAME/config \
  username="dbuser" \
  password="supersecret123"
```

Pruefen ob das Secret gespeichert wurde:

```
kubectl exec -n vault vault-0 -- vault kv get secret/$NAME/config
```

Erwartete Ausgabe:

```
===== Secret Path =====
secret/data/<dein-name>/config

===== Data =====
Key      Value
---      -
password supersecret123
username dbuser
```

Schritt 4: Vault Policy erstellen

Die Policy legt fest, auf welche Pfade zugegriffen werden darf.

```
kubectl exec -n vault vault-0 -- /bin/sh -c "
cat > /tmp/$NAME-policy.hcl << 'EOF'
path \"secret/data/$NAME/config\" {
  capabilities = [\"read\"]
}
EOF
vault policy write $NAME-policy /tmp/$NAME-policy.hcl
"
```

Policy pruefen:

```
kubectl exec -n vault vault-0 -- vault policy read $NAME-policy
```

Erwartete Ausgabe:

```
path "secret/data/<dein-name>/config" {
  capabilities = ["read"]
}
```

Schritt 5: Vault Role erstellen

Die Role verbindet den Kubernetes ServiceAccount mit der Policy.

```
kubectl exec -n vault vault-0 -- vault write auth/kubernetes/role/$NAME-role \
  bound_service_account_names=vault-auth \
  bound_service_account_namespaces=vault-$NAME \
  policies=$NAME-policy \
  ttl=24h
```

Role pruefen:

```
kubectl exec -n vault vault-0 -- vault read auth/kubernetes/role/$NAME-role
```

Schritt 6: Deployment anlegen

Annotations im Ueberblick



Vault Annotations erklärt

agent-inject-secret im Detail



agent-inject-secret erklärt

Manifest erstellen — `$NAME` wird automatisch eingesetzt:

```
cat > 01-deployment.yml << EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    annotations:
      vault.hashicorp.com/agent-inject: "true"
      vault.hashicorp.com/role: "$NAME-role"
      vault.hashicorp.com/agent-inject-secret-config: "secret/data/$NAME/config"
      vault.hashicorp.com/agent-inject-template-config: |
        {{- with secret "secret/data/$NAME/config" -}}
        username={{ .Data.data.username }}
        password={{ .Data.data.password }}
        {{- end }}
    spec:
      serviceAccountName: vault-auth
      containers:
        - name: app
          image: nginx:alpine
EOF
```

```
kubectl apply -f . -n vault-$NAME
```

Schritt 7: Ergebnis pruefen

Pod-Status pruefen — `2/2` bedeutet: `app` + `vault-agent` Sidecar laufen:

```
kubectl get pods -n vault-$NAME
```

Erwartete Ausgabe:

NAME	READY	STATUS	RESTARTS	AGE
myapp-xxxxx	2/2	Running	0	10s

Injiziertes Secret lesen:

```
kubectl exec -n vault-$NAME deploy/myapp -c app -- cat /vault/secrets/config
```

Erwartete Ausgabe:

```
username=dbuser
password=supersecret123
```

Container-Struktur des Pods ansehen (Init Container + 2 regulaere Container):

```
kubectl describe pod -n vault-$NAME -l app=myapp | grep -A 2 "Init Containers:|Containers:"
```

Schritt 8: Secret aktualisieren (Bonus)

Das Passwort in Vault aendern:

```
kubectl exec -n vault vault-0 -- vault kv put secret/$NAME/config \
  username="dbuser" \
  password="neuespassword456"
```

Nach einiger Zeit (ca. 5 min) aktualisiert der vault-agent Sidecar die Datei automatisch im Pod:

```
kubectl exec -n vault-$NAME deploy/myapp -c app -- cat /vault/secrets/config
```

Aufraeumen

```
kubectl delete namespace vault-$NAME
```

Vault-Eintraege aufraeumen (optional):

```
kubectl exec -n vault vault-0 -- vault kv delete secret/$NAME/config
kubectl exec -n vault vault-0 -- vault policy delete $NAME-policy
kubectl exec -n vault vault-0 -- vault delete auth/kubernetes/role/$NAME-role
```

Zusammenfassung

Was	Wofuer
Mutating Webhook	Faengt jeden neuen Pod ab, fuegt Init-Container + Sidecar ein
vault-agent-init	Holt das Secret einmalig beim Pod-Start, legt Datei an
vault-agent	Laeuft als Sidecar, erneuert Token-Lease, schreibt Updates
ServiceAccount vault-auth	Beweist gegenueber Vault, wer der Pod ist (via JWT)
Vault Role	Verbindet SA + Namespace mit einer Policy
Vault Policy	Legt fest, welche Secrets gelesen werden duerfen
/vault/secrets/config	Datei im Pod — kein Kubernetes Secret Objekt

Kubernetes API-Objekte (Teil 2)

Kubernetes Praxis

Befehle in pod ausfuehren - Übung

```
kubectl run my-nginx --image=nginx:1.23

kubectl exec my-nginx -- ls -la
kubectl exec -it my-nginx -- bash
kubectl exec -it my-nginx -- sh

## in der shell
cat /etc/os-release
cd /var/log/nginx
ls -la
exit

## Logs ausgeben
kubectl logs my-nginx
```

Welche Pods mit Namen gehoeren zu einem Service

```
kubectl get svc svc-nginx -o wide
kubectl get pods -l web=my-nginx
```

Security

ServiceLinks nicht in env in Pod einbinden

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      ## Das ist hier -->
      enableServiceLinks: false
      containers:
        - name: mariadb-cont
          image: mariadb:10.11
          envFrom:
            - configMapRef:
                name: mariadb-configmap
```

Pod with unprivileged user

Schritt 1:

```
cd
mkdir -p manifests
cd manifests
mkdir -p unpriv
cd unpriv
```

```
nano pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-unprivileged
spec:
  securityContext:
    runAsUser: 1000 # Container läuft mit UID 1000 statt root
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

```
kubectl apply -f .
kubectl get pods
```

Schritt 2: Debuggen

```
## CrashLoopBackoff
kubectl get pods nginx-unprivileged
kubectl describe pods nginx-unprivileged
```

```
## permission denied identifiziert
kubectl logs nginx-unprivileged
```

Schritt 3: Lösung anderes image nehmen

```
## in pod.yaml
## Zeile image: nginx:1.25
## in -> image: nginxinc/nginx-unprivileged:1.25
## ändern
```

```
kubectl apply -f .
kubectl get pods
```

Pod ohne capabilities starten. Funktioniert !

Walkthrough (nginx)

```
cd
mkdir -p manifests/nocap
cd manifests/nocap
nano nocap-pod-nginx.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nocap-nginx
spec:
  containers:
    - name: web
      image: nginx
      securityContext:
        capabilities:
          drop:
            - all
```

```
kubectl apply -f .
kubectl get pods
```

Lösung capability setzen (achtung kein CAP davor)

```
apiVersion: v1
kind: Pod
metadata:
  name: nocap-nginx
spec:
  containers:
    - name: web
      image: nginx
      securityContext:
        capabilities:
          drop:
            - all
          add:
            - CHOWN
```

```
kubectl delete -f .
kubectl apply -f .
```

Walkthrough (nginxinc/nginx-unprivileged)

```
cd
mkdir -p manifests/nocap
cd manifests/nocap
nano nocap-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nocap-nginx
spec:
  containers:
    - name: web
      image: nginxinc/nginx-unprivileged:1.25
      securityContext:
        capabilities:
          drop:
            - all
```

```
kubectl apply -f .
kubectl get pods
```

Helm (Kubernetes Paketmanager)

Helm - Was kann Helm

- **Installieren und Deinstallieren** von Anwendungen in Kubernetes (`helm install / helm uninstall`)
- **Upgraden** von bestehenden Installationen (`helm upgrade`)
- **Rollbacks** durchführen, falls etwas schief läuft (`helm rollback`)
- **Anpassen** von Anwendungen durch Konfigurationswerte (`values.yaml`)
- **Veröffentlichen** eigener Charts (z. B. in einem Helm-Repository)

Helm Grundlagen

Wo kann ich Helm-Charts suchen ?

- Im Telefonbuch von helm <https://artifacthub.io/>

Komponenten

Chart

- beinhaltet Beschreibung und Komponenten

Chart - Bereitstellungsformen

- url
- .tgz (abkürzung tar.gz) - Format
- oder Verzeichnis

```
Wenn wir ein Chart installieren, wird eine Release erstellen
(parallel: image -> container, analog: chart -> release)
```

Installation

Was brauchen wir ?

- helm client muss installiert sein

Und sonst so ?

```
## Beispiel ubuntu
## snap install --classic helm

## Cluster auf das ich zugreifen kann und im client -> helm und kubectl
## Voraussetzung auf dem Client-Rechner (helm ist nichts als anderes als ein Client-Programm)
Ein lauffähiges kubectl auf dem lokalen System (welches sich mit dem Cluster verbinden.
-> saubere -> .kube/config

## Test
kubectl cluster-info
```

Helm Warum ?

```
Ein Paket für alle Komponenten
Einfaches Installieren, Updaten und deinstallieren
Konfigurations-Values-Files übergeben zum Konfigurieren
Feststehende Struktur
Versionierung (jedes Chart hat ein Version)
In meinem Kubernetes-Cluster kann ich sehen, welche Version des Charts/der Charts installiert wurde
Ein Chart für viele Kunden und für viele Umgebungen (Chart und passe das mit Konfigurationswerten an)
```

Helm Example

Prerequisites

- helm needs a config-file (kubeconfig) to know how to connect and credentials in there
- Good: helm (as well as kubectl) works as unprivileged user as well - Good for our setup
- install helm on ubuntu (client) as root: `snap install --classic helm`
 - this installs helm3
- Please only use: helm3. No server-side components needed (in cluster)
 - Get away from examples using helm2 (hint: `helm init`) - uses tiller

Simple Walkthrough (Example 0: Step 1)

```
## Repo hinzupfgen
helm repo add bitnami https://charts.bitnami.com/bitnami
## gecachte Informationen aktualisieren
helm repo update

helm search repo bitnami
## helm install release-name bitnami/mysql
```

Simple Walkthrough (Example 0: Step 2: for learning - pull)

```
helm pull bitnami/mysql
tar xvfz mysql*
```

Simple Walkthrough (Example 0: Step 3: install)

```
helm install my-mysql bitnami/mysql
## Chart runterziehen ohne installieren
## helm pull bitnami/mysql

## Release anzeigen zu lassen
helm list

## Status einer Release / Achtung, heisst nicht unbedingt nicht, dass pod läuft
helm status my-mysql

## weitere release installieren
## helm install neuer-release-name bitnami/mysql
```

Under the hood

```
## Helm speichert Informationen über die Releases in den Secrets
kubectl get secrets | grep helm
```

Example 1: - To get know the structure

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update
helm pull bitnami/mysql
tar xzvf mysql-9.0.0.tgz

## Show how the template would look like being sent to kube-api-server
helm template bitnami/mysql
```

Example 2: We will setup mysql without persistent storage (not helpful in production ;o)

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update

helm install my-mysql bitnami/mysql
```

Example 2 - continue - fehlerbehebung

```
helm uninstall my-mysql
## Install with persistentStorage disabled - Setting a specific value
helm install my-mysql --set primary.persistence.enabled=false bitnami/mysql

## just as notice
## helm uninstall my-mysql
```

Example 2b: using a values file

```
## mkdir helm-mysql
## cd helm-mysql
## vi values.yml
primary:
  persistence:
    enabled: false

helm uninstall my-mysql
helm install my-mysql bitnami/mysql -f values.yml
```

Example 3: Install wordpress

Example 3.1: Setting values with --set

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress \
  --set wordpressUsername=admin \
  --set wordpressPassword=password \
  --set mariadb.auth.rootPassword=secretpassword \
  bitnami/wordpress
```

Example 3.2: Setting values with values.yml file

```
cd
mkdir -p manifests
cd manifests
mkdir helm-wordpress
cd helm-wordpress
nano values.yml
```

```
## values.yml
wordpressUsername: admin
wordpressPassword: password
mariadb:
  auth:
    rootPassword: secretpassword
```

```
## helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress -f values.yml bitnami/wordpress
```

Referenced

- <https://github.com/bitnami/charts/tree/master/bitnami/mysql/#installing-the-chart>
- <https://helm.sh/docs/intro/quickstart/>

Installation, Upgrade, Uninstall helm-Chart exercise - simple (mariadb-cloudpirates)

Schritt 1: install mariadb von cloudpirates

```
## Mini-Step 1: Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.5.1 --dry-run=server
```

```
## Mini-Step 2: Installieren
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.5.1
```

```
## Geht das denn auch ?
kubectl get pods
## sehr gut in helm v4
helm status my-mariadb
```

Schritt 2: Exercise: Upgrade to new version

Schritt 2.1 Default values (auf terminal) ausfindig machen

```
## Recherchiere wie die Werte gesetzt werden (artifactory.io) oder verwende die folgenden Befehle:
helm show values oci://registry-1.docker.io/cloudpirates/mariadb
helm show values oci://registry-1.docker.io/cloudpirates/mariadb | less
```

Schritt 2.2 Upgrade und resources ändern

```
cd
mkdir -p mariadb-values
cd mariadb-values
mkdir prod
cd prod
```

```
nano values.yaml
```

```
resources:
  limits:
    memory: 300Mi
  requests:
    memory: 300Mi
    cpu: 100m
```

```
cd ..
```

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.5.3 --dry-run -f
prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.5.3 -f
prod/values.yaml
```

```
kubectl get pods
```

Umschauen

```
kubectl get pods
## Ab Version 4 (helm) sinnvoll
helm status my-mariadb
helm list
## alle helm charts anzeigen, die im gesamten Cluster installiert wurden
helm list -A
helm history my-mariadb
```

Umschauen get

```
## Wo speichert er Information, die er später mit helm get abrufen
kubectl get secrets
```

```
helm get values my-mariadb
helm get manifest my-mariadb
## Zeile ausgeben und 4 Zeilen danach und 4 Zeilen davor
helm get manifest my-mariadb | grep "300Mi" -A4 -B4
## alles was ich ausgeben kann an Daten aus secrets .
helm get all my-mariadb
```

```
## Hack COMPUTED VALUES anzeigen lassen
## Welche Werte (values) hat er zur Installation verwendet
helm get all my-mariadb | grep -i computed -A 200
## besser Variante von David
helm get all my-mariadb | sed -n '/COMPUTED/, /HOOKS/p'
```

Tip: values aus alter revision anzeigen

```
## Beispiel:
helm get values my-mariadb --revision 1
```

Schritt 3: Exercise: Upgrade to new version

Schritt 3.1. Upgrade und resources beibehalten

- Values wurden bereits im vorherigen Schritt angelegt

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.10.1 --dry-run=server
-f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.10.1 -f
prod/values.yaml
```

```
kubectl get pods
## kein neuer pod
```

Schritt 3.2 Fehlgeschlagene Installation, wie lösen ?

```
## Schlägt fehler, weil mit dem upgrade bestimmte Felder nicht überschrieben dürfen, die geändert wurden im Template
```

Lösung

- Deinstallieren (pvc bleibt erhalten auch beim Deinstallieren -> so macht das helm)
- Und wieder installieren in der neuen Version

```
## Frage, ist das pvc noch ?
kubectl get pvc
## Ja !
```

NAME	STATUS	VOLUME	CAPACITY
data-my-mariadb-0	Bound	pvc-575fd652-e0da-46ed-b917-a2c28629e164	8Gi
RWO	do-block-storage	<unset>	66m

```
## alte revisions behalten
helm uninstall my-mariadb --keep-history
kubectl get pvc
## auch nach der Deinstallation ist der pvc noch da
## Super !!
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --reset-values --version 0.10.1 -f
prod/values.yaml
```

```
kubectl get pods
helm get values my-mariadb
```

Änderung zwischen versionen (Warum trat der Fehler auf)

```
helm get manifest my-mariadb --revision 2 > rev2.yaml
helm get manifest my-mariadb --revision 3 > rev3.yaml
## schaut nach serviceaccount
diff rev2.yaml rev3.yaml
```

Uninstall

```
helm uninstall my-mariadb
## namespace wird nicht gelöscht
## händisch löschen
kubectl delete ns <dein-name>
## crd's werden auch nicht gelöscht
kubectl create ns <dein-name>
```

Problem: OutOfMemory (OOM-Killer) if container passes limit in memory

- if memory of container is bigger than limit an OOM-Killer will be triggered
- How to fix. Use memory limit in the application too!
 - <https://techcommunity.microsoft.com/blog/appsonazureblog/unleashing-javascript-applications-a-guide-to-boosting-memory-limits-in-node-js/4080857>

Installation, Upgrade, Uninstall helm-Chart exercise (wordpress-groundhog2k)

Schritt 1: install wordpress von groundhog2k

```
## Repo hinzufügen (einmalig)
helm repo add groundhog2k https://groundhog2k.github.io/helm-charts/
helm repo update
```

```
## Verzeichnisstruktur anlegen
cd
mkdir -p wordpress-values/prod
cd wordpress-values/prod
```

```
nano values.yaml
```

```
## Ingress aktivieren
ingress:
  enabled: true
  className: traefik
  hosts:
    - host: wordpress-<euernamenskuerzel>.appv1.do.t3isp.de
      paths:
        - path: /
          pathType: Prefix

## WordPress Storage (persistent)
storage:
  requestedSize: 10Gi
  accessModes:
    - ReadWriteOnce

## MariaDB Subchart
mariadb:
  enabled: true
  settings:
    rootPassword: geheim123
  userDatabase:
    name: wordpress
    user: wpuser
    password: wppass123
  storage:
    requestedSize: 8Gi
```

```
cd ..
```



```
## Mini-Step 1: Testen
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.3 --dry-run=client -f prod/values.yaml
```

```
## Mini-Step 2: Installieren
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.3 -f prod/values.yaml
```

```
## Geht das denn auch ?
kubectl get pods
kubectl get pvc
kubectl get ingress
```

```
## Im Browser
http://wordpress-<euernamenskuerzel>.appv1.do.t3isp.de
```

Schritt 2: Umschauen

```
kubectl get pods
kubectl get pvc
kubectl get ingress
helm status my-wordpress
helm list
## alle helm charts anzeigen, die im gesamten Cluster installiert wurden
helm list -A
helm history my-wordpress
```

Schritt 3: Umschauen get

```
## Wo speichert er Information, die er später mit helm get abrufen
kubectl get secrets
```

```
helm get values my-wordpress
helm get manifest my-wordpress
## Zeige alle Kinds an
helm get manifest my-wordpress | grep -i -A 4 kind
## Can I see all values use -> YES
## Look for COMPUTED VALUES in get all ->
helm get all my-wordpress
```

```
## Hack COMPUTED VALUES anzeigen lassen
## Welche Werte (values) hat er zur Installation verwendet
helm get all my-wordpress | grep -i computed -A 200
```

Schritt 4: Exercise: Upgrade to new version

Schritt 4.1 Default values (auf terminal) ausfindig machen

```
## Recherchiere wie die Werte gesetzt werden (artifacthub.io) oder verwende die folgenden Befehle:
helm show values groundhog2k/wordpress
helm show values groundhog2k/wordpress | less
```

Schritt 4.2 Upgrade und resources ändern

```
cd ~/wordpress-values/prod
nano values.yaml
```

Ergänze die resources:

```
## Resources für WordPress
resources:
  limits:
    memory: 512Mi
  requests:
    memory: 256Mi
    cpu: 100m

## Ingress aktivieren
ingress:
  enabled: true
  className: traefik
  hosts:
    - host: wordpress-<dein-namenskuerzel>.appv1.do.t3isp.de
      paths:
        - path: /
          pathType: Prefix
```

```
## WordPress Storage (persistent)
storage:
  requestedSize: 10Gi
  accessModes:
    - ReadWriteOnce

## MariaDB Subchart
mariadb:
  enabled: true
  settings:
    rootPassword: geheim123
  userDatabase:
    name: wordpress
    user: wpuser
    password: wppass123
  storage:
    requestedSize: 8Gi
  resources:
    limits:
      memory: 300Mi
    requests:
      memory: 200Mi
      cpu: 100m
```

```
cd ..
```

```
## Testen
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.4 --dry-run -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.4 -f prod/values.yaml
```

```
kubectl get pods
kubectl describe pods my-wordpress-0
helm list
helm history my-wordpress
helm get values my-wordpress
```

Schritt 4.3 Weiteres Update der Chart - Version

```
## Testen
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.5 --dry-run -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.5 -f prod/values.yaml
```

```
## Schlägt fehl, weil mit dem apply bestimmte Felder nicht überschrieben dürfen, die geändert wurden im Template
```

Lösung

- Deinstallieren (pvc bleibt erhalten auch beim Deinstallieren -> so macht das helm)
- Und wieder installieren in der neuen Version

```
## Frage, ist das pvc noch ?
kubectl get pvc
## Ja !
```

```
helm uninstall my-wordpress
kubectl get pvc
## auch nach der Deinstallation ist der pvc noch da
## Super !!
```

```
## Real Upgrade
helm upgrade --install my-wordpress groundhog2k/wordpress --reset-values --version 0.14.5 -f prod/values.yaml
```

```
kubectl get pods
```

Tipp: values aus alter revision anzeigen

```
## Beispiel:
helm get values my-wordpress --revision 1
```

Uninstall

```
helm uninstall my-wordpress
## namespace wird nicht gelöscht
## händisch löschen
kubectl delete ns app-<namenskuerzel>
## crd's werden auch nicht gelöscht
```

Problem: OutOfMemory (OOM-Killer) if container passes limit in memory

- if memory of container is bigger than limit an OOM-Killer will be triggered
- How to fix: Use memory limit in the application too!
 - PHP memory_limit in WordPress konfigurieren
 - <https://techcommunity.microsoft.com/blog/appsonazureblog/unleashing-javascript-applications-a-guide-to-boosting-memory-limits-in-node-js/4080857>

Hinweis: Externe Datenbank verwenden (optional)

Falls du eine externe MariaDB/MySQL verwenden möchtest statt dem integrierten Subchart:

```
nano values.yaml
```

```
mariadb:
  enabled: false

externalDatabase:
  name: wordpress
  user: wpuser
  password: wppass123
  host: my-external-db
```

Helm Exercise with nginx

Part 1: Chart erstellen

```
cd
mkdir -p charts
cd charts
## mit helm neues Chart erstellen
helm create beispiel-chart
```

Part 2: chart installieren

```
helm upgrade --install my-nginx beispiel-chart
```

Part 3: funktioniert es ?

```
kubectl get pods
helm list
```

Part 4: Spezielle Konfiguration

Part 4.1: Analyse

```
## Können wir die replicas und den type server ändern
## Entweder variante 1 ins Chart
less beispiel-chart/values.yaml
## mit hlem bordmitteln
helm show values beispiel-chart | less
```

Part 4.2. values.yaml (eigene Knfiguration) erstellen und anwenden

```
cd
mkdir helm-values
cd helm-values
mkdir beispiel-chart
cd beispiel-chart
nano values.yaml
```

```
## in der Datei values.yaml
replicaCount: 2
service:
  type: NodePort
```

```
cd
cd charts
helm upgrade --install my-nginx beispiel-chart --reset-values -f ../helm-values/beispiel-chart/values.yaml
kubectl get pods
```

```
## neue Revision
helm list
## hier NodePort auslesen
kubectl get svc my-nginx-beispiel-chart
kubectl get nodes -o wide
```

```
## Testen
curl http://<ip-aus-nodes>:<nodePort>
## z.B.
curl http://159.223.24.231:32465
```

Part 4.3 Explore

```
helm list
helm list -A (über alle namespaces hinweg)
```

```
## Zeige mir alles von der installierten Release
helm get all my-nginx
helm get values my-nginx
helm get manifest my-nginx
```

```
## chart von online
## für unser chart
cd
cd charts
helm show values beispiel-chart # latest version
```

Part 5 Uninstall nginx

```
## Achtung keine Deinstallation von CRD's, keine Deinstallation von PVC (Persistent Volume Claims), RBAC
helm uninstall my-nginx

## Überprüfung, ob Deinstallation erfolgt ist:
helm list
```

Helm Spickzettel

Hilfe

```
helm help
helm help <command>
helm help upgrade
```

Alle helm-releases anzeigen

```
## im eigenen Namespace
helm list
## in allen Namespaces
helm list -A
## für einen speziellen
helm -n kube-system list
```

Helm - Chart installieren

```
## Empfehlung mit namespace
## Repo hinzufügen für Client
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-nginx bitnami/nginx --version 19.0.1 --create-namespace --namespace=app-<namenskuerzel>
## besser upgrade --install --reset-values (immer alle Werte wie im aktuellen Installationsbefehl
helm upgrade --install --reset-values my-nginx bitnami/nginx --version 19.0.1 --create-namespace --namespace=app-<namenskuerzel>
```

Helm - prüfen (nach Installation)

```
helm status my-release
helm get values my-release
helm get manifest my-release
```

Helm - Suche

```
## welche Repos sind konfiguriert
helm repo list
helm search repo bitnami
helm search hub
```

Helm - template

```
## Rendern des Templates
helm repo add bitnami https://charts.bitnami.com/bitnami
helm template my-nginx bitnami/nginx
helm template bitnami/nginx
```

Helm Charts erstellen und analysieren

Eigenes Helm-Chart erstellen

Chart erstellen

```
cd
mkdir my-charts
cd my-charts

helm create my-app
```

Install helm - chart

```
## Variante 1:
helm -n my-app-<namenskuerzel> install meine-app my-app --create-namespace

## Variante 2:
cd my-app
helm -n my-app-<namenskuerzel> install meine-app . --create-namespace

helm -n my-app-<namenskuerzel> status meine-app
kubectl -n my-app-<namenskuerzel> get all
kubectl -n my-app-<namenskuerzel> get pods
```

Aufräumen

```
helm -n my-app-<namenskuerzel> uninstall meine-app
```

Chart zur Analyse runterladen und entpacken

```
cd
mkdir -p charts-download
cd charts-download

## Lädt die letzte version herunter
helm pull oci://registry-1.docker.io/cloudpirates/mariadb

## Lädt bestimmte chart-version runter
## helm pull oci://registry-1.docker.io/cloudpirates/mariadb --version 0.9.0
## evtl. entpacken wenn gewünscht
## tar xvf mariadb-12.1.6.tgz

## Schnelle Variante
helm pull oci://registry-1.docker.io/cloudpirates/mariadb --version 0.9.0 --untar
```

Eigenes Chart in eine Registry pushen (helm push)

Hintergrund

Seit Helm 3.8 werden Charts wie Container-Images in einer OCI-Registry abgelegt (Docker Hub, GitLab, Harbor, ...). Der Ablauf ist immer gleich:

Schritt	Befehl
Chart bauen	helm create/ eigenes Chart
Chart paketieren (.tgz)	helm package <chart-ordner>
An der Registry anmelden	helm registry login <registry>
Chart hochladen	helm push <chart>.tgz oci://<registry>/<pfad>
Chart nutzen	helm install ... oci://<registry>/<pfad>/<chart>

Fuer das Training laeuft eine gemeinsame Registry im Cluster (siehe [Trainer-Setup](#)). Jeder Teilnehmer pusht unter seinem eigenen Pfad <dein-name>/... , dadurch kommt sich niemand in die Quere.

Registry	registry.appv2.do.t3isp.de
----------	----------------------------

Benutzer	training
Passwort	helm-push-2026

Schritt 1: Chart erstellen und paketieren

```
cd
mkdir -p helm-push
cd helm-push
helm create hello-chart
helm package hello-chart
ls
```

Erwartete Ausgabe:

```
Successfully packaged chart and saved it to: /home/<dein-name>/helm-push/hello-chart-0.1.0.tgz
```

Schritt 2: Push ohne Anmeldung (soll fehlschlagen)

```
helm push hello-chart-0.1.0.tgz oci://registry.appv2.do.t3isp.de/<dein-name>
```

Erwarteter Fehler:

```
Error: failed to perform "Exists" on destination: HEAD "https://registry.appv2.do.t3isp.de/v2/<dein-name>/hello-chart/manifests/sha256:...": basic credential not found
```

Schritt 3: An der Registry anmelden

```
helm registry login registry.appv2.do.t3isp.de -u training
## Passwort: helm-push-2026
```

Erwartete Ausgabe:

```
Login Succeeded
```

Helm merkt sich die Anmeldung in `~/.config/helm/registry/config.json` .

Schritt 4: Chart pushen

```
helm push hello-chart-0.1.0.tgz oci://registry.appv2.do.t3isp.de/<dein-name>
```

Erwartete Ausgabe:

```
Pushed: registry.appv2.do.t3isp.de/<dein-name>/hello-chart:0.1.0
Digest: sha256:...
```

```
## Was liegt in der Registry ? (Registry-API, deshalb curl mit Benutzer)
curl -u training:helm-push-2026 https://registry.appv2.do.t3isp.de/v2/_catalog
curl -u training:helm-push-2026 https://registry.appv2.do.t3isp.de/v2/<dein-name>/hello-chart/tags/list
```

Schritt 5: Chart aus der Registry installieren

```
helm show chart oci://registry.appv2.do.t3isp.de/<dein-name>/hello-chart
helm -n helm-push-<dein-name> upgrade --install hello oci://registry.appv2.do.t3isp.de/<dein-name>/hello-chart --version 0.1.0 --create-namespace
helm -n helm-push-<dein-name> list
kubectl -n helm-push-<dein-name> get pods
```

Schritt 6: Neue Version pushen und upgraden

```
## Version in Chart.yaml hochsetzen: 0.1.0 -> 0.2.0
vi hello-chart/Chart.yaml
```

```
helm package hello-chart
helm push hello-chart-0.2.0.tgz oci://registry.appv2.do.t3isp.de/<dein-name>
curl -u training:helm-push-2026 https://registry.appv2.do.t3isp.de/v2/<dein-name>/hello-chart/tags/list
```

Erwartete Ausgabe:

```
{"name":"<dein-name>/hello-chart","tags":["0.2.0","0.1.0"]}
```

```
## Ohne --version nimmt helm automatisch die neueste Version
helm -n helm-push-<dein-name> upgrade --install hello oci://registry.appv2.do.t3isp.de/<dein-name>/hello-chart
helm -n helm-push-<dein-name> list
```

Erwartete Ausgabe: Spalte `CHART` zeigt `hello-chart-0.2.0` , `REVISION` ist `2`.

Aufraeumen

```
kubect1 delete namespace helm-push-<dein-name>
helm registry logout registry.appv2.do.t3isp.de
cd
rm -rf helm-push
```

Zusammenfassung

Befehl	Zweck
helm package <ordner>	Chart als .tgz paketieren
helm registry login <registry>	Anmelden (Token wird lokal gespeichert)
helm push <tgz> oci://<registry>/<pfad>	Chart hochladen
helm show chart oci://...	Chart-Infos aus der Registry anzeigen
helm pull oci://...	Chart herunterladen
helm install/upgrade ... oci://...	Chart direkt aus der Registry installieren
helm registry logout <registry>	Abmelden

Variante: Docker Hub

Mit einem eigenen Docker-Hub-Account geht es genauso. Das Chart ist dann oeffentlich sichtbar. Statt des Passworts am besten ein Access Token verwenden (Docker Hub: Account settings -> Personal access tokens).

```
helm registry login registry-1.docker.io -u <dockerhub-user>
helm push hello-chart-0.1.0.tgz oci://registry-1.docker.io/<dockerhub-user>
helm show chart oci://registry-1.docker.io/<dockerhub-user>/hello-chart
```

Wie starte ich am besten einfach - mit eigenem Projekt

Helm Spezial: Umgang mit Einrückungen

Whitespaces meistern mit "-"

Grundlagen

- In Helm (bzw. in Go-Templates) hast du verschiedene Möglichkeiten, den Umgang mit Whitespace (z. B. Leerzeichen, Zeilenumbrüche) zu steuern:
- `{{ ... }}` :
Standardvariante. Lässt den Whitespace außerhalb der geschweiften Klammern unverändert.
- `{{- ... }}` :
Entfernt den Whitespace links (vor) dem Ausdruck und AUCH die Zeilenumbrüche davor
- `{{ ... -}}` :
Entfernt den Whitespace rechts (nach) dem Ausdruck, aber AUCH Zeilenumbrüche
- `{{- ... -}}` :
Entfernt Whitespace sowohl links als auch rechts des Ausdrucks, aber AUCH Zeilenumbrüche

Exercise Whitespaces

Explanation

- `{{- ->` trim on left side / INCLUDING new lines
- `-}}` -> trim on right side / ALSO: new lines
- trim tabs, whitespaces a.s.o. (see ref)

Walkthrough

```
cd
mkdir -p helm-exercises
cd helm-exercises

## When ever we encounter error while parsing yaml, we can use comment !!!
helm create testenv
cd testenv/templates
rm -fR *.yaml
rm -fR tests

nano test.yaml

## "{{23 -}" < {"- 45}}"
```

```
helm template ..
helm template --debug ..
```

```
## now with new lines
nano test2.yaml
```

```
## {{23 -}}
newline here
```

```
helm template ..
helm template --debug ..
```

Reference:

- https://pkg.go.dev/text/template#hdr-Text_and_spaces

Named Templates

Prerequisites range-example

- helm-exercises/range chart exists

Step 0: start in project

```
cd
cd helm-exercises/range
```

Step 1: Put in file starting _helpers.tpl

```
cd templates
nano _helpers.tpl
```

- Das am Ende einfügen ->

```
{{/* Definiere ein named template namens "mychart.containerPort" */}}
{{- define "range.containerPort" -}}
- name: http
  containerPort: 80
  protocol: TCP
{{- end }}
```

Step 2: templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: web
          image: "nginx:latest"
          ports:
            {{ include "range.containerPort" . }}
```

```
helm template ..
```

```
## Problem, Zeile 1 o.k., nächste Zeile nicht richtig eingerückt
```



```

    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: web
          image: "nginx:latest"
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
+ln1@k8s-client: ~/helm-exercises/range/templates$ █

```

Step 3: Einrückung richtig setzen

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: web
          image: "nginx:latest"
          ports:
            {{- include "range.containerPort" . | nindent 12 }}

```

```
helm template ..
```

Helm Advanced

Helm Dependencies Exercise

Prepare: Create folder structure

```

cd
mkdir -p my-charts
helm create app
cd app

```

Exercise 1: Create chart with Dependency

```
nano Chart.yaml
```

```

## Add dependencies
dependencies:
  - name: redis
    version: "0.9.x"
    repository: "oci://registry-1.docker.io/cloudpirates"

```

```

## Das 1. Mal - dann wird Chart.lock angelegt
helm dependency update
ls -la Chart.lock
ls -la charts/

```

```

rm -fr charts
helm dependency build

```

```

helm dependency --help
### what is the difference

```

```
helm template .
```

Exercise 2: Create chart with condition

Schritt 1

```
nano Chart.yaml
```

```
## change or add dependency block
## adding condition
dependencies:
  - name: redis
    version: "0.9.x"
    repository: "oci://registry-1.docker.io/cloudpirates"
    condition: redis.enabled
```

```
helm dependency build
```

```
nano values.yaml
```

```
## unten anfügen
redis:
  enabled: false
```

```
helm template .
## oder wenn die release app in namespace installiert wurde
helm -n app-<euer-name> template app .
```

Schritt 2

```
## values-file anlegen
cd
mkdir -p helm-values
cd helm-values
mkdir app
cd app
```

```
nano values.yaml
```

```
redis:
  enabled: true
```

```
cd
cd my-charts
helm template app -f ../helm-values/app/values.yaml
helm template app -f ../helm-values/app/values.yaml | grep kind -A 2
```

Schritt 3: Installation update

```
helm -n app-<euer-name> upgrade --reset-values --install app app -f ../helm-values/app/values.yaml
```

```
helm -n app-<euer-name> status app
```

Schritt 4: Aufräumen

```
helm -n app-<euer-name> uninstall app
```

Helm - Fehleranalyse

Beispiel Cloudpirates - helm chart nginx

Test (mit aktuell letzter Version 0.1.10)

```
helm upgrade --install my-nginx oci://registry-1.docker.io/cloudpirates/nginx --reset-values --version 0.1.10
```

```
### wie sehen die logs aus
kubectl logs deployment/my-nginx
```

```
Permission denied port 80
```

Lauffähig mit (leider aktuell so nicht in der Doku)

```
cd
mkdir helm-values/nginx
cd helm-values/nginx
nano values.yaml
```

```
containerPorts:
- name: http
  containerPort: 8080
  protocol: TCP

serverConfig: |
server {
    listen 0.0.0.0:8080;
    root /usr/share/nginx/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ /index.html;
    }
}

livenessProbe:
  type: httpGet
  path: /

readinessProbe:
  type: httpGet
  path: /
```

```
helm upgrade --install my-nginx oci://registry-1.docker.io/cloudpirates/nginx --version 0.1.10 --reset-values -f values.yaml
```

```
kubectl exec -it deploy/my-nginx -- sh
```

```
## in der shell
id
```

```
/ $ id
uid=101/nginx) gid=101/nginx) groups=101/nginx)
/ $ ^C
```

```
## pod läuft nicht unter root
## Deshalb funktioniert ein Öffnen des Ports 80 beim Starten nicht
```

Gitlab ci/cd (auch mit helm)

Helm mit gitlab ci/cd ausrollen

Step 1: Create gitlab - repo and pipeline

1. Create new repo on gitlab
2. Click on pipeline Editor and creat .gitlab-ci.yml with Button

Step 2: Push your helm chart files to repo

- Now looks like this

📁 training-helm-chart-kubernetes-gitlab-ci-cd



Update .gitlab-ci.yml file

Jochen Metzger authored 8 minutes ago

Name	Last commit
📁 charts/my-app	initial release
📁 config	Add new file
🔥 .gitlab-ci.yml	Update .gitlab-ci.yml file

Step 3: Add your KUBECONFIG as Variable (type: File) to Variables

- https://gitlab.com/jmetzger/training-helm-chart-kubernetes-gitlab-ci-cd/-/settings/ci_cd#is-cicd-variables-settings

oohen Metzger / Training Helm Chart Kubernetes Gitlab Ci Cd / CI/CD Settings

☐ Maintainer

☒ Developer

Save changes

Project variables

Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked variable feature masks variable values, but is not a guaranteed method to prevent malicious users from accessing variables. [How can I make my](#)

CI/CD Variables </> 1

Key ↑	Value	Enviro
KUBECONFIG_SECRET 🔒 ACCESS TO KUBERNETES File	***** 🔒	All (de

> Pipeline trigger tokens

Trigger a pipeline for a branch or tag by generating a trigger token and using it with an API call. The token impersonates s

> Deploy freezes

Add a freeze period to prevent unintended releases during a period of time for a given environment. You must update the deploy freezes added here. [Learn more](#). Specify deploy freezes using [cron syntax](#).

> Job token permissions

Edit variable

Type

File

Environments

All (default)

Visibility

☒ Visible

Can be seen in job logs.

☐ Masked

Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet [regular expressions requirements](#).

☐ Masked and hidden

Masked in job logs, and can never be revealed in the CI/CD settings after the variable is saved.

Flags

☐ Protect variable

Export variable to pipelines running on protected branches and tags only.

☐ Expand variable reference

\$ will be treated as the start of a reference to another variable.

Description (optional)

ACCESS TO KUBERNETES

The description of the variable's value or usage.

Step 4: Create a pipeline for deployment

```
stages:
  # List of stages for jobs, and their order of execution
  - deploy

variables:
  APP_NAME: my-first-app

deploy:
```

```

stage: deploy
image:
  name: alpine/helm:3.2.1
## Important to unset entrypoint
entrypoint: [""]
script:
  - ls -la
  - cd; mkdir .kube; cd .kube; cat $KUBECONFIG_SECRET > config; ls -la;
  - cd $CI_PROJECT_DIR; helm upgrade ${APP_NAME} ./charts/my-app --install --namespace ${APP_NAME} --create-namespace -f
  ./config/values.yaml
rules:
  - if: $CI_COMMIT_BRANCH == 'master'
    when: always

```

Reference: Example Project (Public)

- <https://gitlab.com/jmetzger/training-helm-chart-kubernetes-gitlab-ci-cd>

Uebung: Unit Tests im Merge Request Dashboard

Hintergrund

GitLab kann Test-Ergebnisse direkt im **Merge Request Dashboard** anzeigen — ohne in die Job-Logs schauen zu muessen.

Dazu speichert der CI-Job einen JUnit-XML-Report als Artifact:

```

artifacts:
  when: always      # auch bei fehlgeschlagenen Tests hochladen!
  reports:
    junit: report.xml

```

GitLab vergleicht automatisch den Test-Status von Feature-Branch und Ziel-Branch und hebt neu fehlgeschlagene oder gelöste Tests hervor.

Funktioniert auf allen GitLab-Tiers (Free, Premium, Ultimate).

Wie weiss GitLab, dass es ein Unit-Test-Report ist?

Der entscheidende Schluessel ist `reports: junit:` — das ist kein frei wählbarer Name, sondern ein **fest definiertes GitLab-Keyword**.

Damit sagst du GitLab explizit:

```
"Diese Datei ist ein JUnit-XML-Report —
parse sie und zeige das Ergebnis im MR-Dashboard an."
```

Ohne `reports: junit:` wäre `report.xml` nur eine normale Datei zum Herunterladen — kein Test-Widget, kein Branch-Vergleich.

Das Format der Datei muss zum Keyword passen: fuer `junit:` erwartet GitLab valides JUnit-XML. `pytest --junitxml=report.xml` liefert genau das.

Weitere reports-Keywords

GitLab kennt noch andere solche Keywords — jedes aktiviert ein eigenes Widget im MR:

Keyword	Widget im MR	Tier
<code>reports: junit:</code>	Test summary (passed/failed/neu)	Free
<code>reports: coverage_report:</code>	Code-Coverage-Badge und Diff	Free
<code>reports: codequality:</code>	Code-Quality-Findings	Free
<code>reports: sast:</code>	Security-Findings (SAST)	Free*
<code>reports: dependency_scanning:</code>	Abhängigkeiten mit CVEs	Ultimate

*SAST-Report wird hochgeladen auf Free, aber das inline MR-Widget nur auf Ultimate.

Was ist im Demo-Repo drin?

```

calculator.py      <- Python-Funktionen (add, subtract, multiply, divide, is_even)
test_calculator.py <- 10 pytest-Tests
requirements.txt   <- pytest, pytest-cov
.gitlab-ci.yml     <- Pipeline mit JUnit- und Coverage-Artifact

```

Die komplette Pipeline auf einen Blick:

```

stages:
  - test

unit-tests:
  stage: test
  image: python:3.11-slim
  script:
    - pip install -r requirements.txt --quiet

```

```

- pytest test_calculator.py -v
  --junitxml=report.xml
  --cov=calculator --cov-report=term --cov-report=xml:coverage.xml
coverage: '/TOTAL.*\s+(\d+%)$/'
artifacts:
  when: always
  reports:
    junit: report.xml
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml

```

Zwei Reports gleichzeitig:

Report	Keyword	Was im MR erscheint
report.xml	reports: junit:	Test summary (passed/failed/neu)
coverage.xml	reports: coverage_report:	Coverage-Badge + Zeilen im Diff

Schritt 1: Auf GitLab einloggen

```

https://gitlab.com

Account:  training.tn<X>      (X = deine Teilnehmernummer, z.B. training.tn2)
Passwort: vom Trainer

```

Schritt 2: Demo-Repo forken

1. <https://gitlab.com/jmetzger/training-gitlab-ci-tests> aufrufen
2. Oben rechts: "Fork" klicken
3. Im Fork-Dialog:
 - Namespace: training.tn<X> (euer gemeinsamer Account)
 - Project name: training-ci-tests-<dein-name> (z.B. training-ci-tests-anna – eindeutig, da ihr den Account teilt!)
 - Visibility: Public
4. "Fork project" klicken

Schritt 3: Pipeline auf main-Branch manuell starten

1. Im geforkten Repo: Build -> Pipelines
2. Oben rechts: "Run pipeline" klicken
3. Branch: main, dann "Run pipeline" bestaetigen
4. Job "unit-tests" anklicken

Erwartete Ausgabe im Job-Log:

```

10 passed in 0.03s

Name           Stmts   Miss  Cover
-----
calculator.py    12      0   100%
-----
TOTAL            12      0   100%

```

Alle Tests gruen, Coverage 100% — der Ausgangszustand ist sauber.

Im MR (nach dem Forken, wenn eine Pipeline laeuft) erscheint ausserdem:

```
Coverage: 100%
```

Schritt 4: Feature-Branch mit Bug erstellen

Im Web-Editor (kein lokales Git noetig):

1. Im Repo links: "Code" -> "<> Repository"
2. Datei "calculator.py" anklicken
3. Oben rechts: "Edit" -> "Edit single file"

```
4. Zeile 15 aendern:

VORHER:
def is_even(n):
    return n % 2 == 0

NACHHER (Bug einbauen):
def is_even(n):
    return n % 2 != 0 # BUG

5. Unten bei "Commit changes":
- Commit message: "Add is_even feature - WIP"
- Haekchen bei "Create a new branch"
- Branch name: feature/is-even-<dein-name>
  (z.B. feature/is-even-anna)
- "Commit changes" klicken
```

Schritt 5: Merge Request erstellen

```
1. GitLab zeigt oben einen blauen Banner:
  "Create merge request"

2. Klick darauf

3. Im MR-Formular:
- Title: "Fix is_even - <dein-name>"
- Source branch: feature/is-even-<dein-name>
- Target branch: main

4. "Create merge request" klicken
```

Schritt 6: Test-Ergebnisse im MR Dashboard beobachten

Die Pipeline laeuft jetzt auf dem Feature-Branch. Sobald sie fertig ist (ca. 1-2 Minuten), erscheint im MR:

```
Test summary
2 newly failed tests

test_calculator > test_is_even_true
test_calculator > test_is_even_false
```

Erwartetes Verhalten: Pipeline-Status rot, 2 neu fehlgeschlagene Tests sichtbar.

Auf einen Test klicken zeigt den genauen Fehler:

```
assert False is True
+ where False = is_even(4)
```

Im "Changes"-Tab des MR sind die geaenderten Zeilen in `calculator.py` mit dem Coverage-Status annotiert — Zeile 16 ist rot markiert, da der Bug dazu fuehrt dass ein Code-Pfad nicht korrekt durchlaufen wird.

Schritt 7: Bug fixen

```
1. Im MR oben: "Code" -> "Open in Web IDE"
  (oder zurueck zur Datei: Code -> calculator.py -> Edit)

2. Zeile 15 zurueck auf:

def is_even(n):
    return n % 2 == 0

3. Commit message: "Fix is_even: correct comparison operator"

4. Commit auf denselben Feature-Branch
```

Schritt 8: Tests werden gruen

Pipeline laeuft erneut. Im MR erscheint jetzt:

```
Test summary
2 fixed tests

test_calculator > test_is_even_true    (fixed)
test_calculator > test_is_even_false   (fixed)
```

Pipeline-Status: gruen.

Schritt 9: Merge Request mergen

1. "Merge" klicken
2. "Delete source branch" anhaeken (optional)
3. Bestaetigen

Zusammenfassung

Schritt	Was im MR-Dashboard erscheint
Pipeline auf main	10 passed, Coverage 100%
Feature-Branch mit Bug	2 newly failed tests (rot)
Bug gefixt, neuer Commit	2 fixed tests (gruen), Coverage 100%
Alle Tests gruen	Pipeline gruen, Merge moeglich

Aufraeumen

Settings -> General -> Advanced -> Delete project

Exercise: Docker Image bauen und in Registry pushen

Ziel

- Ein Docker-Image mit GitLab CI/CD automatisch bauen
- Das Image in die GitLab Container Registry pushen

Schritt 1: Referenzprojekt importieren

```
## In GitLab einloggen, dann:
https://gitlab.com/projects/new#import_project

## Folgende URL als Import-Quelle angeben:
https://gitlab.com/jmetzger/training-build-test-ci-cd-gitlab.git

## Einen eigenen Projektnamen vergeben
## Visibility: public
```

Schritt 2: Pipeline anschauen

- Im Projekt: **Build** -> **Pipeline editor** öffnen
- Die Datei `.gitlab-ci.yml` enthält den Build-Job

Schritt 3: Pipeline ausführen

- Unter **Build** -> **Pipelines** -> **Run pipeline** -> Branch: **master** -> **Run pipeline**
- Die Pipeline beobachten (3 Jobs: build, test, code_quality)
- Nach erfolgreichem Lauf: **Deploy** -> **Container Registry** prüfen
- Das gebaute Image ist dort sichtbar (Tag = Commit-Hash)

Schritt 4: Eigene Änderung triggern

```
## Hinweis: Das Repo verwendet "master" als default Branch (nicht "main")

## Eine kleine Änderung an einer Datei vornehmen (z.B. index.js)
## und committen -> Pipeline startet automatisch
```

Referenz

- Beispielprojekt (public): <https://gitlab.com/jmetzger/training-build-test-ci-cd-gitlab>

Helpful plugins

Use shortnames for kubectl - commands

- <https://gist.github.com/doelopper/f4a9a211e74f8a2d44eb4afb21f0a38>

Kubernetes Debugging

Probleme über Logs identifiziert - z.B. non-root image

Schritt 1:

```
cd
mkdir -p manifests
cd manifests
mkdir -p unpriv
cd unpriv
```



```
nano pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-unprivileged
spec:
  securityContext:
    runAsUser: 1000 # Container läuft mit UID 1000 statt root
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

```
kubectl apply -f .
kubectl get pods
```

Schritt 2: Debuggen

```
## CrashLoopBackoff
kubectl get pods nginx-unprivileged
kubectl describe pods nginx-unprivileged
```

```
## permission denied identifiziert
kubectl logs nginx-unprivileged
```

Schritt 3: Lösung anderes image nehmen

```
## in pod.yaml
## Zeile image: nginx:1.25
## in -> image: nginxinc/nginx-unprivileged:1.25
## ändern
```

```
kubectl apply -f .
kubectl get pods
```

Übung: FE zu Backend Verbindungen debuggen mit kubectl debug und NetworkPolicy

Hintergrund

In produktiven Umgebungen laufen Container oft als minimale Images ohne Debug-Tools (curl, wget, nc). `kubectl debug` schleust einen ephemeral Container mit Debug-Tools in einen laufenden Pod ein — ohne den Pod neu starten zu muessen.

Ein **Timeout** beim Verbindungsversuch ist ein typisches Zeichen fuer eine blockierende NetworkPolicy:

Fehlerbild	Ursache	Diagnose-Befehl
Connection timed out	NetworkPolicy blockiert den Traffic	<code>kubectl describe networkpolicy</code>

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 20-debug-networkpolicy
cd 20-debug-networkpolicy
```

Namespace anlegen:

```
kubectl create namespace debug-<dein-name>
```

Schritt 2: Backend Deployment und Service anlegen

```
## vi 01-backend.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
```

```

template:
  metadata:
    labels:
      app: backend
  spec:
    containers:
      - name: backend
        image: nginx:alpine
        ports:
          - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80

```

```
kubectl apply -f . -n debug-<dein-name>
```

Schritt 3: Frontend Deployment und Service anlegen

Das Frontend läuft als minimales Python-Image (kein curl, wget, nc).

```

## vi 02-frontend.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: python:3.12-slim
          command: ["python", "-m", "http.server", "8080"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  selector:
    app: frontend
  ports:
    - port: 8080
      targetPort: 8080

```

```

kubectl apply -f . -n debug-<dein-name>
kubectl get pods -n debug-<dein-name>

```

Warten bis beide Pods laufen:

```

kubectl wait deployment frontend backend \
  --for=condition=Available -n debug-<dein-name> --timeout=60s

```

Schritt 4: Baseline — Verbindung funktioniert

Tools im Frontend-Pod prüfen — kein curl, wget, nc vorhanden:

```

FE_POD=$(kubectl get pod -n debug-<dein-name> -l app=frontend -o jsonpath='{.items[0].metadata.name}')
echo $FE_POD

```

```
kubectl exec -it $FE_POD -n debug-<dein-name> -- sh -c 'which curl; which wget; which nc'
```

Erwartete Ausgabe:

```
no curl
no wget
no nc
```

Mit `kubectrl debug` einen busybox-Container einschleusen und Verbindung testen:

```
kubectrl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

Im Debug-Container:

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: `<h1>Welcome to nginx!</h1>` — Verbindung funktioniert.

Problem 1: NetworkPolicy blockiert FE zu Backend

Schritt 5: NetworkPolicy fuer Backend anwenden

```
## vi 03-networkpolicy-backend.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: api-consumer
    ports:
    - protocol: TCP
      port: 80
```

```
kubectrl apply -f . -n debug-<dein-name>
```

Schritt 6: Verbindung testen — Timeout

```
kubectrl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwarteter Fehler:

```
wget: download timed out
```

Timeout statt `Connection refused` — Endpoints existieren, die NetworkPolicy blockiert.

Schritt 7: Diagnose — NetworkPolicy und Labels pruefen

```
kubectrl describe networkpolicy backend-policy -n debug-<dein-name>
kubectrl get pods -n debug-<dein-name> --show-labels
```

Diagnose: NetworkPolicy erlaubt nur `role=api-consumer`. Frontend-Pod hat dieses Label nicht.

Schritt 8: Fix — Label zum Frontend Deployment hinzufuegen

```
kubectrl patch deployment frontend -n debug-<dein-name> \
-p '{"spec":{"template":{"metadata":{"labels":{"role":"api-consumer"}}}}}'
```

```
kubectl get pods -n debug-<dein-name> -l role=api-consumer
```

Neuen Pod-Namen holen und erneut testen:

```
FE_POD=$(kubectl get pod -n debug-<dein-name> -l app=frontend,role=api-consumer \
-o jsonpath='{.items[0].metadata.name}')
```

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: <h1>Welcome to nginx!</h1> — Verbindung OK.

Problem 2: NetworkPolicy blockiert Backend zu Frontend

Schritt 9: NetworkPolicy fuer Frontend anwenden

```
## vi 04-networkpolicy-frontend.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: backend-consumer
    ports:
    - protocol: TCP
      port: 8080
```

```
kubectl apply -f . -n debug-<dein-name>
```

Schritt 10: Rueckweg vom Backend debuggen

```
BE_POD=$(kubectl get pod -n debug-<dein-name> -l app=backend -o jsonpath='{.items[0].metadata.name}')
echo $BE_POD
```

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwarteter Fehler:

```
wget: download timed out
```

Schritt 11: Diagnose und Fix

```
kubectl describe networkpolicy frontend-policy -n debug-<dein-name>
kubectl get pods -n debug-<dein-name> -l app=backend --show-labels
```

Diagnose: NetworkPolicy erlaubt nur `role=backend-consumer`. Backend-Pod hat dieses Label nicht.

```
kubectl patch deployment backend -n debug-<dein-name> \
-p '{"spec":{"template":{"metadata":{"labels":{"role":"backend-consumer"}}}}}'
```

```
kubectl get pods -n debug-<dein-name> -l role=backend-consumer
```

Schritt 12: Rueckweg erneut testen

```
BE_POD=$(kubectl get pod -n debug-<dein-name> -l app=backend,role=backend-consumer \
-o jsonpath='{.items[0].metadata.name}')

kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh

wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwartete Ausgabe:

```
<title>Directory listing for /</title>
```

Rueckweg funktioniert.

Aufraeumen

```
kubectl delete namespace debug-<dein-name>
```

Zusammenfassung

Problem	Fehlermeldung	Diagnose	Fix
NetworkPolicy FE -> Backend	timed out	kubectl describe networkpolicy + --show-labels	Label role=api-consumer am Frontend
NetworkPolicy Backend -> FE	timed out	kubectl describe networkpolicy + --show-labels	Label role=backend-consumer am Backend

Übung: Service-Verbindungsprobleme debuggen mit kubectl debug (ohne NetworkPolicy)

Hintergrund

Wenn Pods sich gegenseitig nicht erreichen koennen, gibt es zwei haeufige Ursachen - beide zeigen Connection refused , aber aus unterschiedlichen Gruenden:

Fehlerbild	Ursache	Erkennungsmerkmal
Connection refused	Falscher Service-Selector - keine Endpoints	kubectl get endpoints zeigt <none>
Connection refused	Falscher targetPort - Endpoint zeigt falsche Port	kubectl get endpoints zeigt Endpoint mit falscher Port

kubectl debug schleust einen ephemeral Container mit Debug-Tools in einen laufenden Pod ein - ohne den Pod neu starten zu muessen.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 21-debug-service
cd 21-debug-service
```

Schritt 2: Backend Deployment und Service anlegen

Achtung: Im Service steckt ein Fehler - den sollt ihr selbst finden.

```
nano 01-backend.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
        tier: backend
    spec:
      containers:
        - name: backend
```

```

        image: nginx:alpine
        ports:
          - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: backend-api
  ports:
    - port: 80
      targetPort: 80

```

```

kubectl create ns debug-<dein-name>
kubectl apply -f 01-backend.yml -n debug-<dein-name>

```

Schritt 3: Frontend Deployment und Service anlegen

Das Frontend laeuft als minimales Python-Image (kein curl, wget, nc) und startet einen einfachen HTTP-Server auf Port 8080.

Achtung: Auch im Frontend-Service steckt ein Fehler.

```
nano 02-frontend.yml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
        tier: frontend
    spec:
      containers:
        - name: frontend
          image: python:3.12-slim
          command: ["python", "-m", "http.server", "8080"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  selector:
    app: frontend-api
  ports:
    - port: 8080
      targetPort: 8080

```

```
kubectl apply -f 02-frontend.yml -n debug-<dein-name>
```

```

kubectl get pods -n debug-<dein-name>
kubectl get services -n debug-<dein-name>

```

Erwartete Ausgabe:

NAME	READY	STATUS	RESTARTS	AGE
backend-xxx	1/1	Running	0	30s
frontend-xxx	1/1	Running	0	20s

NAME	TYPE	CLUSTER-IP	PORT(S)
backend-svc	ClusterIP	10.x.x.x	80/TCP
frontend-svc	ClusterIP	10.x.x.x	8080/TCP

Problem 1: FE zu Backend - Falscher Selector, keine Endpoints

Schritt 4: kubectl debug - Verbindung testen

```
FE_POD=$(kubectl get pod -n debug-<dein-name> -l app=frontend -o jsonpath='{.items[0].metadata.name}')
echo $FE_POD
```

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

Im Debug-Container:

```
nslookup backend-svc
wget -qO- http://backend-svc --timeout=5
```

Erwartete Ausgabe:

```
Name:  backend-svc.debug-<dein-name>.svc.cluster.local
Address: 10.x.x.x

wget: can't connect to remote host (10.x.x.x): Connection refused
```

DNS loest auf - aber `Connection refused` . Kein Listener hinter dem Service.

```
exit
```

Schritt 5: Endpoints und Selector pruefen

```
kubectl get endpoints backend-svc -n debug-<dein-name>
kubectl describe service backend-svc -n debug-<dein-name> | grep -E 'Selector|Port|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=backend --show-labels
```

Diagnose: Service sucht `app=backend-api` , Pods haben `app=backend` .

Schritt 6: Fix - Selector korrigieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
-p '{"spec":{"selector":{"app":"backend"}}}'
```

```
kubectl get endpoints backend-svc -n debug-<dein-name>
```

Erneut testen:

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: `<h1>Welcome to nginx!</h1>` - Verbindung OK.

Problem 2: FE zu Backend - Falscher targetPort

Schritt 7: targetPort kaputt konfigurieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":80,"targetPort":8080}]}}'
```

Schritt 8: kubectl debug - Verbindung testen

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
```

Erwartete Ausgabe:

```
wget: can't connect to remote host (10.x.x.x): Connection refused
```

Wieder `Connection refused` - aber diesmal aus anderem Grund.

```
exit
```

Schritt 9: Diagnose - Endpoint vorhanden aber Port falsch

```
kubectl get endpoints backend-svc -n debug-<dein-name>
kubectl describe service backend-svc -n debug-<dein-name> | grep -E 'Port|Target|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=backend -o jsonpath='{.items[0].spec.containers[0].ports}'
```

Diagnose: TargetPort zeigt auf 8080, Container lauscht auf 80.

Schritt 10: Fix - targetPort korrigieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":80,"targetPort":80}]}}'
```

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: <h1>Welcome to nginx!</h1> - Verbindung OK.

Problem 3: Rueckweg Backend zu Frontend - Falscher Selector

Schritt 11: kubectl debug auf Backend-Pod - Verbindung zum Frontend testen

```
BE_POD=$(kubectl get pod -n debug-<dein-name> -l app=backend -o jsonpath='{.items[0].metadata.name}')
echo $BE_POD
```

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

Im Debug-Container:

```
nslookup frontend-svc
wget -qO- http://frontend-svc:8080 --timeout=5
```

Erwartete Ausgabe:

```
Name:   frontend-svc.debug-<dein-name>.svc.cluster.local
Address: 10.x.x.x

wget: can't connect to remote host (10.x.x.x): Connection refused

exit
```

Schritt 12: Endpoints und Selector pruefen

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
kubectl describe service frontend-svc -n debug-<dein-name> | grep -E 'Selector|Port|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=frontend --show-labels
```

Diagnose: Service sucht app=frontend-api , Pods haben app=frontend .

Schritt 13: Fix - Selector korrigieren

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"selector":{"app":"frontend"}}}'
```

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
```

Erneut testen:

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
```



```
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwartete Ausgabe: `<title>Directory listing for /</title>` - Verbindung OK.

Problem 4: Rueckweg Backend zu Frontend - Falscher targetPort

Schritt 14: targetPort kaputt konfigurieren

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":8080,"targetPort":9090}]}}'
```

Schritt 15: kubectl debug - Verbindung testen

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
```

Erwartete Ausgabe:

```
wget: can't connect to remote host (10.x.x.x): Connection refused
```

```
exit
```

Schritt 16: Diagnose und Fix

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
kubectl describe service frontend-svc -n debug-<dein-name> | grep -E 'Port|Target|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=frontend -o jsonpath='{.items[0].spec.containers[0].ports}'
```

Diagnose: TargetPort zeigt auf 9090, Container lauscht auf 8080.

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":8080,"targetPort":8080}]}}'
```

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwartete Ausgabe: `<title>Directory listing for /</title>` - Rueckweg OK.

Aufraeumen

```
kubectl delete namespace debug-<dein-name>
```

Zusammenfassung

Problem	Richtung	kubectl get endpoints	Diagnose	Fix
Falscher Selector	FE → Backend	<none>	Selector passt nicht zu Pod-Labels	Selector anpassen
Falscher targetPort	FE → Backend	Port falsch	TargetPort != ContainerPort	targetPort anpassen
Falscher Selector	Backend → FE	<none>	Selector passt nicht zu Pod-Labels	Selector anpassen
Falscher targetPort	Backend → FE	Port falsch	TargetPort != ContainerPort	targetPort anpassen

Merkhilfe: Endpoints leer → Selector-Problem. Endpoints vorhanden aber falsche Port → targetPort-Problem.

Übung: nginx als Nicht-Root-User startet nicht - CrashLoopBackOff mit describe/logs/exec debuggen

Hintergrund

Aus Sicherheitsgrunden sollen Container nicht als root laufen. Ueber den `securityContext` kann man Kubernetes anweisen, den Prozess im Container mit einer anderen User-ID zu starten - und mit `runAsNonRoot: true` sogar erzwingen, dass er niemals als root laeuft.

Feld	Bedeutung
runAsUser: 1000	Prozess laeuft mit UID 1000 statt root (UID 0)
runAsGroup: 1000	Prozess laeuft mit GID 1000
runAsNonRoot: true	Kubelet verweigert den Start, wenn der Prozess als UID 0 laufen wuerde

Das Problem: Viele Standard-Images (z.B. `nginx`) gehen davon aus, dass sie als root starten. Sie schreiben in Verzeichnisse, die root gehoeren, oder binden privilegierte Ports (< 1024). Setzt man dann per `securityContext` einen anderen User, crasht der Container - und man muss herausfinden, warum.

Genau das ist die Uebung: Ihr bekommt ein kaputtes Manifest und sollt den Fehler mit `kubectl describe` , `kubectl logs` und `kubectl exec` selbst finden und beheben.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 22-debug-nginx-nonroot
cd 22-debug-nginx-nonroot
```

Schritt 2: Deployment und Service anlegen

Achtung: Dieses Manifest funktioniert absichtlich nicht.

```
nano 01-deployment.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-nonroot
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-nonroot
  template:
    metadata:
      labels:
        app: nginx-nonroot
    spec:
      securityContext:
        runAsUser: 1000
        runAsGroup: 1000
        runAsNonRoot: true
      containers:
        - name: nginx
          image: nginx:1.30
          ports:
            - containerPort: 80

nano 02-service.yml

apiVersion: v1
kind: Service
metadata:
  name: nginx-nonroot
spec:
  selector:
    app: nginx-nonroot
  ports:
    - port: 80
      targetPort: 80

kubectl create ns debug-<dein-name>
kubectl apply -f . -n debug-<dein-name>
```

Schritt 3: Symptom beobachten

```
kubectl get pods -n debug-<dein-name>
```

Erwartete Ausgabe (nach ca. 1 Minute, der Status wechselt zwischen `Error` und `CrashLoopBackOff`):

NAME	READY	STATUS	RESTARTS	AGE
nginx-nonroot-8cf677d9b-x2g5f	0/1	CrashLoopBackOff	3 (41s ago)	62s

Der Service hat keine Endpoints, weil der Pod nie `Ready` wird:

```
kubectl get endpoints nginx-nonroot -n debug-<dein-name>

Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME           ENDPOINTS      AGE
nginx-nonroot          1m
```

Die Warnung koennt ihr ignorieren - entscheidend ist die leere Spalte `ENDPOINTS` (auf aelteren Clustern steht dort `<none>`).

Schritt 4: Aufgabe - Fehler selbst finden

Findet heraus, **warum** der Container abstuerzt, und bringt den Pod nach `Running` . Am Ende muss ein `curl` gegen den Service die nginx-Startseite liefern.

Eure Werkzeuge:

```
kubectl describe pod -n debug-<dein-name> -l app=nginx-nonroot
kubectl logs -n debug-<dein-name> deploy/nginx-nonroot
kubectl logs -n debug-<dein-name> deploy/nginx-nonroot --previous
kubectl exec -n debug-<dein-name> <pod> -- <befehl>
```

Zwei Loesungswege sind erlaubt:

- **Weg A:** Ein Image nehmen, das fuer Nicht-Root gebaut ist
- **Weg B:** Das Standard-Image `nginx:1.30` behalten und den Pod so anpassen, dass nginx als UID 1000 laufen kann

Versucht es zuerst ohne die Hinweise. Wenn ihr nicht weiterkommt, klappt die Hinweise nacheinander auf.

- ▶ Hinweis 1: Wo schaue ich zuerst?
- ▶ Hinweis 2: Was sagen die Logs?
- ▶ Hinweis 3: Der Container crasht sofort - wie komme ich mit exec rein?
- ▶ Hinweis 4: Ich habe `/var/cache/nginx` repariert - jetzt kommt der naechste Fehler
- ▶ Loesung Weg A: Image `nginxinc/nginx-unprivileged`
- ▶ Loesung Weg B: Standard-Image `nginx:1.30` behalten

Schritt 5: Loesung pruefen

Egal welcher Weg - so muss es am Ende aussehen:

```
kubectl get pods -n debug-<dein-name>
kubectl logs -n debug-<dein-name> deploy/nginx-nonroot | tail -3
kubectl get endpoints nginx-nonroot -n debug-<dein-name>
```

Erwartete Ausgabe:

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-nonroot-79fb6d876c-vq5bc      1/1     Running   0           30s

2026/09/25 09:32:19 [notice] 1#1: start worker processes
2026/09/25 09:32:19 [notice] 1#1: start worker process 22
2026/09/25 09:32:19 [notice] 1#1: start worker process 23

NAME           ENDPOINTS      AGE
nginx-nonroot  10.244.0.125:8080  5m
```

Und der Test gegen den Service aus einem Wegwerf-Pod heraus:

```
kubectl run curl-test --rm -i --restart=Never -n debug-<dein-name> \
--image=curlimages/curl:8.10.1 -- curl -s http://nginx-nonroot
```

Erwartete Ausgabe: die nginx-Startseite mit `<h1>Welcome to nginx!</h1>` .

Aufraeumen

```
kubectl delete namespace debug-<dein-name>
```

Zusammenfassung

Was ihr gesehen habt	Werkzeug	Erkenntnis
CrashLoopBackOff, Exit Code: 1	kubectl get pods, kubectl describe pod	Container startet und stirbt sofort - der Grund steht hier noch nicht
[emerg] mkdir() "/var/cache/nginx/client_temp" failed (13: Permission denied)	kubectl logs	nginx darf als UID 1000 nicht in root-Verzeichnisse schreiben
[emerg] open() "/run/nginx.pid" failed (13: Permission denied)	kubectl logs (nach dem ersten Fix)	Zweite Schicht: PID-Datei
Verzeichnisse gehoeren root mit 755	kubectl exec (Inspect-Pod mit sleep)	Ursache bestaetigt

Loesung	Aufwand	Wann sinnvoll
Weg A: nginxinc/nginx-unprivileged, Port 8080	3 Zeilen	Immer, wenn es ein fertiges Nicht-Root-Image gibt
Weg B: emptyDir fuer /var/cache/nginx und /var/run + ConfigMap mit listen 8080	ConfigMap + Volumes	Wenn das Image nicht getauscht werden darf oder es kein Nicht-Root-Image gibt

Merkhilfe: `describe` sagt **dass** es crasht, `logs` sagt **warum**, `exec` (notfalls mit `sleep` statt dem echten Prozess) zeigt euch den **Zustand im Container**.

Übung: Pod laeuft, Anwendung kaputt - Fehler nur in den Logs sichtbar (403/502)

Hintergrund

In der vorherigen Debugging-Uebung hat Kubernetes euch den Fehler quasi hinterhergetragen: `CrashLoopBackOff`, `Exit Code: 1`, Restarts - jeder sieht sofort, dass etwas nicht stimmt.

Der haeufigere Fall in der Praxis ist unangenehmer: Der Pod ist `Running`, `1/1 Ready`, `kubect1 describe` zeigt keine Warnung, der Service hat Endpoints - und trotzdem liefert die Anwendung Fehler. Kubernetes weiss nichts davon, weil der Prozess ja laeuft. Der einzige Ort, an dem der Fehler steht, sind die Logs der Anwendung.

Was Kubernetes sieht	Was der Benutzer sieht	Wo der Grund steht
Pod Running, 1/1 Ready	403 Forbidden, 502 Bad Gateway	nur in <code>kubect1 logs</code>

In dieser Uebung baut ihr eine kleine Website (nginx) mit einem `/api/`-Pfad, der an ein Backend weitergereicht wird. Beides ist kaputt - und keiner der beiden Fehler taucht in `kubect1 get` **oder** `kubect1 describe` auf.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 23-debug-fehler-nur-in-logs
cd 23-debug-fehler-nur-in-logs
```

Schritt 2: Backend anlegen

Das Backend ist ein minimaler Python-HTTP-Server, der auf Port 8080 ein Verzeichnis-Listing ausliefert.

Achtung: In diesem Manifest steckt einer der beiden Fehler.

```
nano 01-backend.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: python:3.12-slim
          command: ["python", "-m", "http.server", "8080"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
```

Schritt 3: Website-Inhalt und nginx-Konfiguration anlegen

Der Inhalt der Website kommt aus einer ConfigMap. Achtung: Auch hier steckt ein Fehler.

```
nano 02-web-content.yml
```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: web-content
data:
  index.htm: |
    <html>
    <body>
    <h1>Hallo vom Kubernetes-Training</h1>
    <p>Backend-Status: <a href="/api/">/api/</a></p>
    </body>
    </html>

```

Die nginx-Konfiguration liefert unter `/` den Inhalt aus und reicht `/api/` an das Backend weiter (Reverse Proxy). Diese Datei ist in Ordnung.

```
nano 03-web-nginx-conf.yml
```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: web-nginx-conf
data:
  default.conf: |
    server {
      listen      80;
      server_name localhost;

      location / {
        root    /usr/share/nginx/html;
        index  index.html;
      }

      location /api/ {
        proxy_pass http://backend-svc;
      }
    }

```

Schritt 4: Website-Deployment und Service anlegen

Dieses Manifest ist in Ordnung.

```
nano 04-web.yml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          volumeMounts:
            - name: content
              mountPath: /usr/share/nginx/html
            - name: conf
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf
      volumes:
        - name: content
          configMap:
            name: web-content
        - name: conf
          configMap:
            name: web-nginx-conf
---
apiVersion: v1

```

```
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

```
kubectl create ns logs-<dein-name>
kubectl apply -f . -n logs-<dein-name>
```

Schritt 5: Aus Kubernetes-Sicht ist alles in Ordnung

```
kubectl get pods -n logs-<dein-name>
kubectl get endpoints -n logs-<dein-name>
kubectl describe pod -n logs-<dein-name> -l app=web | grep -A10 "^Events:"
```

Erwartete Ausgabe:

NAME	READY	STATUS	RESTARTS	AGE
backend-6bfb7b6785-snlc6	1/1	Running	0	30s
web-7dd7bd48b9-gsjqm	1/1	Running	0	30s

NAME	ENDPOINTS	AGE
backend-svc	10.244.0.2:80	31s
web	10.244.0.54:80	30s

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	31s	default-scheduler	Successfully assigned ...
Normal	Pulled	31s	kubelet	Container image "nginx:1.27" already present on machine
Normal	Created	31s	kubelet	Container created
Normal	Started	31s	kubelet	Container started

Beide Pods `Running` , beide `Ready` , beide Services haben Endpoints, keine einzige Warnung. Kubernetes ist zufrieden.

Schritt 6: Der Benutzer ist nicht zufrieden

Testet die Website aus einem Wegwerf-Pod heraus:

```
kubectl run curl-test --rm -i --restart=Never -n logs-<dein-name> \
  --image=curlimages/curl:8.10.1 -- curl -s -w "%{http_code}\n" http://web/

kubectl run curl-test --rm -i --restart=Never -n logs-<dein-name> \
  --image=curlimages/curl:8.10.1 -- curl -s -w "%{http_code}\n" http://web/api/
```

Erwartete Ausgabe:

```
<html>
<head><title>403 Forbidden</title></head>
...
HTTP 403

<html>
<head><title>502 Bad Gateway</title></head>
...
HTTP 502
```

Die Startseite liefert `403 Forbidden` , der API-Pfad `502 Bad Gateway` .

Schritt 7: Aufgabe - beide Fehler finden

Findet beide Ursachen und behebt sie. Am Ende muss `curl http://web/` die Seite mit `Hallo vom Kubernetes-Training` liefern und `curl http://web/api/` das Verzeichnis-Listing des Backends (`HTTP 200`).

Euer Werkzeug ist diesmal vor allem `kubectl logs` . Nuetzliche Varianten:

```
kubectl logs -n logs-<dein-name> deploy/web
kubectl logs -n logs-<dein-name> deploy/web --tail=20
kubectl logs -n logs-<dein-name> deploy/web -f
kubectl logs -n logs-<dein-name> -l app=web
kubectl logs -n logs-<dein-name> deploy/web --since=5m
```

Tipp: Oeffnet ein zweites Terminal mit `kubectl logs -f` und schickt im ersten Terminal die `curl` -Anfragen ab. So seht ihr live, was nginx zu jeder Anfrage schreibt.

- Hinweis 1: Fehler 1 - was steht zum 403 in den Logs?
- Hinweis 2: Fehler 2 - was steht zum 502 in den Logs?
- Loesung

Schritt 8: Loesung pruefen

```
kubectl run curl-test --rm -i --restart=Never -n logs-<dein-name> \
  --image=curlimages/curl:8.10.1 -- curl -s -w "\nHTTP %{http_code}\n" http://web/

kubectl run curl-test --rm -i --restart=Never -n logs-<dein-name> \
  --image=curlimages/curl:8.10.1 -- curl -s -w "\nHTTP %{http_code}\n" http://web/api/
```

Erwartete Ausgabe:

```
<h1>Hallo vom Kubernetes-Training</h1>
...
HTTP 200

<title>Directory listing for /</title>
...
HTTP 200
```

Und in den Logs sehen jetzt beide Seiten gut aus - nginx mit 200 im Access-Log, und das Backend bekommt endlich Anfragen:

```
kubectl logs -n logs-<dein-name> deploy/web --tail=2
kubectl logs -n logs-<dein-name> deploy/backend

10.244.0.107 - - [25/Sep/2026:09:42:32 +0000] "GET / HTTP/1.1" 200 118 "-" "curl/8.10.1" "-"
10.244.0.103 - - [25/Sep/2026:09:42:39 +0000] "GET /api/ HTTP/1.1" 200 832 "-" "curl/8.10.1" "-"

10.244.0.54 - - [25/Sep/2026 09:42:39] "GET / HTTP/1.0" 200 -
```

Aufräumen

```
kubectl delete namespace logs-<dein-name>
```

Zusammenfassung

Symptom	kubectl get / describe	Log-Zeile	Ursache	Fix
403 Forbidden auf /	alles gruen	directory index of "/usr/share/nginx/html/" is forbidden	ConfigMap-Key index.htm statt index.html	Key umbenennen
502 Bad Gateway auf /api/	alles gruen	connect() failed (111: Connection refused) while connecting to upstream ... upstream: "http://<ClusterIP>:80/"	targetPort: 80, Container lauscht auf 8080	targetPort: 8080

Merkhilfe: Running heisst nur "der Prozess lebt", nicht "die Anwendung funktioniert". Wenn Kubernetes gruen ist und der Benutzer rot sieht, sind die Logs der einzige Zeuge. Und: Ohne Readiness-Probe faellt so ein Fehler Kubernetes nie auf - mit einer Probe auf / waere der Pod bei Fehler 1 gar nicht erst Ready geworden.

Übung: CrashLoopBackOff, aber die Logs sind sauber - OOMKilled (137) und Completed (0) ueber Exit Codes finden

Hintergrund

Bisher hat euch bei einem CrashLoopBackOff immer kubectl logs gerettet: Irgendwo stand ein [emerg] oder [ERROR], und damit war die Ursache klar.

Es gibt aber Faelle, in denen die Logs voellig unauffaellig sind - der Prozess hat gar keine Gelegenheit, einen Fehler zu schreiben, oder er ist aus seiner Sicht ganz normal fertig. Dann hilft nur der Blick auf **wie** der Container beendet wurde: Reason und Exit Code unter Last State in kubectl describe.

Exit Code	Bedeutung	Typische Ursache
0	Prozess hat sich selbst normal beendet	Prozess ist kein Vordergrund-Prozess (Daemon), Skript ist einfach fertig
1	Prozess hat sich mit Fehler beendet	Anwendungsfehler - steht in den Logs
137	Prozess wurde von aussen mit SIGKILL (9) getoetet: 128 + 9	OOMKilled (Memory-Limit) oder Liveness-Probe fehlgeschlagen
139	Segmentation Fault: 128 + 11	Bug im Programm oder in einer Bibliothek
143	Prozess wurde mit SIGTERM (15) beendet: 128 + 15	Normales Herunterfahren durch Kubernetes

Merkregel: Alles ueber 128 heisst "ein Signal hat den Prozess beendet" - Exit Code minus 128 ist die Signalnummer.

In dieser Uebung lauft ihr in zwei dieser Faelle. Beide Pods landen im CrashLoopBackOff, beide Logs sehen gesund aus.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
```

```
cd manifests
mkdir 24-debug-crashloop-exitcodes
cd 24-debug-crashloop-exitcodes
```

Schritt 2: Zwei Deployments anlegen

Beide Manifeste funktionieren absichtlich nicht.

Das erste ist eine kleine Python-Anwendung, die beim Start Daten in einen Cache im Arbeitsspeicher laedt.

```
nano 01-cache-app.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cache-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cache-app
  template:
    metadata:
      labels:
        app: cache-app
    spec:
      containers:
        - name: cache-app
          image: python:3.12-slim
          command: ["python", "-u", "-c"]
          args:
            - |
              import time
              print("cache-app startet, lade Daten in den Cache ...")
              cache = []
              for i in range(20):
                cache.append(bytearray(10 * 1024 * 1024))
                print(f"({i+1})*10 MB geladen")
                time.sleep(0.5)
              print("Cache geladen, bereit")
              while True:
                time.sleep(60)
          resources:
            requests:
              memory: "32Mi"
            limits:
              memory: "64Mi"
```

Das zweite ist ein nginx, bei dem jemand den Startbefehl explizit ins Manifest geschrieben hat.

```
nano 02-web.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          command: ["nginx"]
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
```



```
selector:
  app: web
ports:
- port: 80
  targetPort: 80
```

```
kubectl create ns crash-<dein-name>
kubectl apply -f . -n crash-<dein-name>
```

Schritt 3: Symptom beobachten

```
kubectl get pods -n crash-<dein-name>
```

Erwartete Ausgabe (nach ca. 1 Minute; die Status wechseln jeweils mit `CrashLoopBackOff` ab):

NAME	READY	STATUS	RESTARTS	AGE
cache-app-5d46df6f49-s5wh2	0/1	OOMKilled	3 (48s ago)	73s
web-55f497cf86-ntgh2	0/1	Completed	3 (60s ago)	73s

Schaut genau hin: Die `STATUS`-Spalte zeigt zwischendurch nicht nur `CrashLoopBackOff`, sondern auch `OOMKilled` bzw. `Completed`. Das ist bereits der erste Hinweis.

Schritt 4: Aufgabe - beide Pods nach Running bringen

Findet fuer beide Deployments heraus, warum der Container immer wieder beendet wird, und behebt es. Am Ende muss `cache-app` in den Logs `Cache` geladen, bereit melden und `curl http://web` die `nginx`-Startseite liefern.

Fangt mit den Logs an - und wundert euch nicht, wenn ihr dort nichts findet. Dann geht es weiter mit:

```
kubectl describe pod -n crash-<dein-name> <pod> | grep -A6 "Last State"
```

Praktischer Einzeiler, der `Reason` und `Exit Code` fuer alle Pods im Namespace zeigt:

```
kubectl get pods -n crash-<dein-name> -o custom-columns='NAME:.metadata.name,LAST-REASON:.status.containerStatuses[0].lastState.terminated.reason,EXIT:.status.containerStatuses[0].lastState.terminated.exitCode,RESTARTS:.status.containerStatuses[0].restartCount'
```

NAME	LAST-REASON	EXIT	RESTARTS
cache-app-5d46df6f49-s5wh2	OOMKilled	137	4
web-55f497cf86-ntgh2	Completed	0	4

► Hinweis 1: `cache-app` - die Logs hoeren einfach auf

► Hinweis 2: `web` - die Logs sehen perfekt aus

► Loesung

Schritt 5: Loesung pruefen

```
kubectl logs -n crash-<dein-name> deploy/cache-app | tail -2
kubectl logs -n crash-<dein-name> deploy/web | tail -2
```

```
200 MB geladen
Cache geladen, bereit
```

```
2026/09/25 10:04:46 [notice] 1#1: start worker process 7
2026/09/25 10:04:46 [notice] 1#1: start worker process 8
```

Bei `nginx` starten die `Worker` jetzt von `1#1` aus - `PID 1` ist der `nginx-Master` und bleibt am Leben.

```
kubectl run curl-test --rm -i --restart=Never -n crash-<dein-name> \
  --image=curlimages/curl:8.10.1 -- curl -s http://web
```

Erwartete Ausgabe: die `nginx`-Startseite mit `<h1>Welcome to nginx!</h1>`.

Wer einen `Metrics-Server` im Cluster hat, kann den echten Verbrauch der `cache-app` jetzt sehen:

```
kubectl top pod -n crash-<dein-name>
```

Aufraeumen

```
kubectl delete namespace crash-<dein-name>
```

Zusammenfassung

Pod	STATUS zwischen durch	Last State	Exit Code	Logs	Ursache	Fix
cache-app	OOMKilled	OOMKilled	137	brechen ohne Fehler ab	Memory-Limit <code>64Mi</code> zu klein fuer <code>200 MB Cache</code>	Limit auf <code>256Mi</code>

web	Completed	Completed	0	sehen normal aus	nginx daemonisiert, PID 1 beendet sich	nginx -g "daemon off;" oder command entfernen
-----	-----------	-----------	---	------------------	--	---

Merkhilfe: Sind die Logs sauber, frag nach dem Exit Code. `0` heisst "der Prozess ist freiwillig gegangen" - meist ein Vordergrund-Problem. `137` heisst "jemand hat ihn erschossen" - meist `OOMKilled`, sonst die Liveness-Probe (steht dann in den Events).

Kubernetes RBAC

Kubernetes RBAC - was darf Traefik

- Service Account wird in den Pod gehängt
- Und das was dieser ServiceAccount darf, darf der Pod auch über z.B. ein `kubectl` call machen

Weg der Berechtigung

```
Service Account -> Rolebinding/ClusterRolebinding -> ClusterRole/Role
```

Schritte zur Analyse

```
## wie ist das eigentlich beim ingress controller
## Welche ServiceAccount
kubectl -n ingress get sa traefik
## Wurde ClusterRole und Role verwendet - ClusterRole ist Serverweit
helm -n ingress get manifest traefik | grep -i -A 4 kind
## Das darf dieser Rolle
kubectl get clusterrole traefik-ingress -o yaml
## Rolle wird mit User verknüpft, dadurch darf der User das, was die Rolle darf
kubectl get clusterrolebinding traefik-ingress -o yaml
```

Weiter lernen

Lernumgebung

- <https://killercode.com/>

Kubernetes Doku - Bestimmte Tasks lernen

- <https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/>

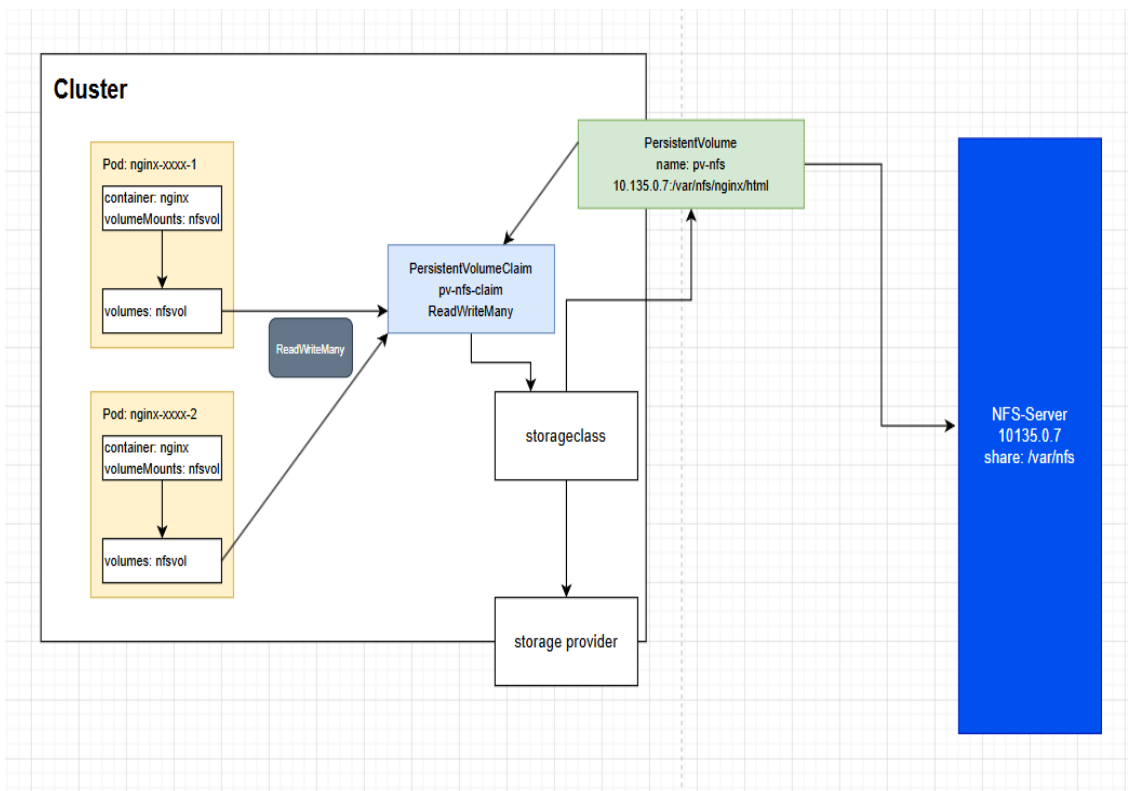
Kubernetes Videos mit Hands On

- <https://www.youtube.com/watch?v=16fgzklcF7Y>

Kubernetes Storage (CSI)

Überblick Persistent Volumes (CSI)

Grafik



Überblick

Warum CSI ?

- Each vendor can create his own driver for his storage

Vorteile ?

- Automatically create storage when required.
- Make storage available to containers wherever they're scheduled.
- Automatically delete the storage when no longer needed.

Wie war es vorher ?

Vendor needed to wait till his code was checked in in tree of kubernetes (in-tree)

Unterschied static vs. dynamisch

The main difference relies on the moment when you want to configure storage. For instance, if you need to pre-populate data in a volume, you choose static provisioning. Whereas, if you need to create volumes on demand, you go for dynamic provisioning.

Komponenten

Treiber

- Für jede Storage Class (Storage Provider) muss es einen Treiber geben

Storage Class

Liste der Treiber mit Features (CSI)

- <https://kubernetes-csi.github.io/docs/drivers.html>

Übung Persistent Storage

- Step 1 + 2: nur Trainer
- ab Step 3: Trainees

Requirements:

- Ein NFS-Server oder eine Storage mit NFS muss im Netz zur Verfügung stehen.

Step 1: Do the same with helm - chart

```

helm repo add csi-driver-nfs https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/charts
helm upgrade --install csi-driver-nfs csi-driver-nfs/csi-driver-nfs --namespace kube-system --version 4.13.4 --reset-values

```

Step 2: Storage Class

```
cd
mkdir -p manifests
cd manifests
mkdir csi-storage
cd csi-storage
nano 01-storageclass.yml

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-csi
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.135.0.14
  share: /var/nfs
reclaimPolicy: Retain
volumeBindingMode: Immediate

kubectl apply -f .
```

Step 3: Persistent Volume Claim

```
cd
mkdir -p manifests
cd manifests
mkdir csi
cd csi
nano 02-pvc.yaml

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-nfs-dynamic
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
  storageClassName: nfs-csi

kubectl apply -f .
kubectl get pvc
##
kubectl get pv
```

Step 4: Pod

```
nano 03-pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: nginx-nfs
spec:
  containers:
    - image: nginx:1.23
      name: nginx-nfs
      command:
        - "/bin/bash"
        - "-c"
        - set -euo pipefail; while true; do echo $(date) >> /mnt/nfs/outfile; sleep 1; done
      volumeMounts:
        - name: persistent-storage
          mountPath: "/mnt/nfs"
          readOnly: false
  volumes:
    - name: persistent-storage
      persistentVolumeClaim:
        claimName: pvc-nfs-dynamic

kubectl apply -f .
kubectl get pods
```

```
kubectl describe pods nginx-nfs
```

Step 5: Testing

```
kubectl exec -it nginx-nfs -- bash
```

```
cd /mnt/nfs
ls -la
## outfile
head /mnt/nfs/outfile
tail -f /mnt/nfs/outfile
```

```
CTRL+C
exit
```

Step 6: Destroy

```
kubectl delete -f 03-pod.yaml
```

```
### Verify in nfs - trainer !!
```

Step 7: Recreate

```
kubectl apply -f 03-pod.yaml
```

```
kubectl exec -it nginx-nfs -- bash
```

```
## is old data here ?
head /mnt/nfs/outfile
##
tail -f /mnt/nfs/outfile
```

```
CTRL + C
exit
```

Step 8: Cleanup

```
kubectl delete -f .
```

Reference:

- <https://rudimartinsen.com/2024/01/09/nfs-csi-driver-kubernetes/>

Beispiel mariadb

- How to persistently use mariadb with a storage class / driver nfs.csi.

Step 1: Treiber installieren

- <https://github.com/kubernetes-csi/csi-driver-nfs/blob/master/docs/install-csi-driver-v4.6.0.md>

```
curl -skSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.6.0/deploy/install-driver.sh | bash -s v4.6.0 --
```

Step 2: Storage Class

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-csi
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.135.0.18
  share: /var/nfs
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - nfsvers=3
```

Step 3: PVC, Configmap, Deployment

```
mkdir -p manifests
cd manifests
mkdir mariadb-csi
cd mariadb-csi
```

```
nano 01-pvc.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-nfs-dynamic-mariadb
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
  storageClassName: nfs-csi
```

```
kubectl apply -f .
```

```
nano 02-configmap.yml
```

```
### 02-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
  name: mariadb-configmap
data:
  # als Wertepaare
  MARIADB_ROOT_PASSWORD: 11abc432
```

```
nano 03-deployment.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:10.11
          envFrom:
            - configMapRef:
                name: mariadb-configmap
          volumeMounts:
            - name: persistent-storage
              mountPath: "/var/lib/mysql"
              readOnly: false
          volumes:
            - name: persistent-storage
              persistentVolumeClaim:
                claimName: pvc-nfs-dynamic-mariadb
```

```
kubectl apply -f .
```

```
kubectl describe po mariadb-deployment-euer-pod
```

Kubernetes Installation

k3s installation

Exercise

Schritt 1: windows store ubuntu installieren und ausführen

Schritt 2: Installation von k3s

Siehe:

<https://docs.k3s.io/quick-start>

```
## in den root benutzer wechseln
sudo su -
## passwort eingeben

curl -sL https://get.k3s.io | sh -

## ca. 2 Minuten warten
## Läuft es ?
systemctl status k3s
## evtl. noch die config-datei kopieren, falls kubectl cluster-info nicht funktioniert
mkdir -p ~/.kube; cp -a /etc/rancher/k3s/k3s.yaml ~/.kube/config
kubectl cluster-info
```

Erster Test

```
kubectl run nginx --image=nginx:1.27
kubectl get pods
kubectl get nodes -o wide
```

Abschalten wenn nicht verwendet

```
systemctl stop k3s
## k3s automatischer start beim booten ausschalten
systemctl disable k3s

## Wenn ihr ihn verwendet wollt
systemctl start k3s
```

Kubernetes Monitoring

Prometheus Monitoring Server (Overview)

What does it do ?

- It monitors your system by collecting data
- Data is pulled from your system by defined endpoints (http) from your cluster
- To provide data on your system, a lot of exporters are available, that
 - collect the data and provide it in Prometheus

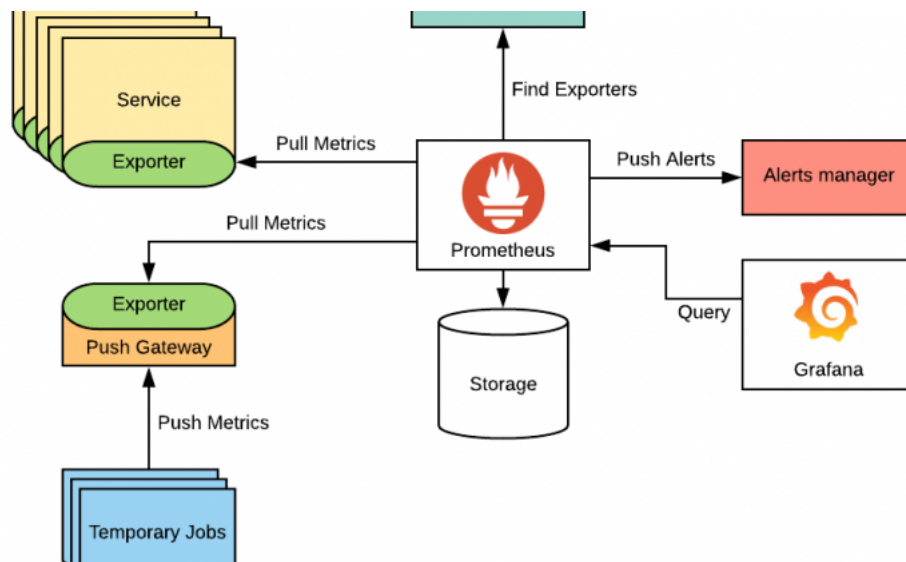
Technical

- Prometheus has a TDB (Time Series Database) and is good as storing time series with data
- Prometheus includes a local on-disk time series database, but also optionally integrates with remote storage systems.
- Prometheus's local time series database stores data in a custom, highly efficient format on local storage.
- Ref: <https://prometheus.io/docs/prometheus/latest/storage/>

What are time series ?

- A time series is a sequence of data points that occur in successive order over some period of time.
- Beispiel:
 - Du willst die täglichen Schlusspreise für eine Aktie für ein Jahr dokumentieren
 - Damit willst Du weitere Analysen machen
 - Du würdest das Paar Datum/Preis dann in der Datumsreihenfolge sortieren und so ausgeben
 - Dies wäre eine "time series"

Komponenten von Prometheus



Quelle: <https://www.devopsschool.com/>

Prometheus Server

1. Retrieval (Sammeln)
 - Data Retrieval Worker
 - pull metrics data
2. Storage
 - Time Series Database (TDB)
 - stores metrics data
3. HTTP Server
 - Accepts PromQL - Queries (e.g. from Grafana)
 - accept queries

Grafana ?

- Grafana wird meist verwendet um die grafische Auswertung zu machen.
- Mit Grafana kann ich einfach Dashboards verwenden
- Ich kann sehr leicht festlegen (Durch Data Sources), wo meine Daten herkommen

Prometheus Metriktypen (Counter, Gauge, Histogram, Summary)

Welche gibt es ?

- Counter
- Gauge
- Histogram
- Summary

In der Prometheus-Dokumentation werden sie auch explizit als metric types bezeichnet. Wenn du also über „eine Gauge“ sprichst, meinst du korrekt: „Eine Metrik vom Typ Gauge“

1. Counter

Ein Counter ist ein **ständig wachsender Wert**. Er beginnt bei 0 und kann **nur steigen** (außer bei einem Reset, z. B. Pod-Restart).

Beispiel:

- `http_requests_total` - zählt, wie viele HTTP-Anfragen es gab
- `errors_total` - zählt Fehlermeldungen

+ Nur hochzählend! Für „Rate“-Abfragen ideal.

Typischer PromQL-Ausdruck:

```
rate(http_requests_total[5m])
```

Zeigt, wie viele Anfragen pro Sekunde in den letzten 5 Minuten kamen.

2. Gauge

Ein Gauge ist ein **aktueller Messwert**, der **steigen und sinken** kann - wie ein Thermometer.

Beispiel:

- `memory_usage_bytes` - aktueller Speicherverbrauch
- `cpu_temperature` - aktuelle CPU-Temperatur

Ideal für Zustände wie Auslastung, offene Verbindungen etc.

PromQL:

```
memory_usage_bytes
```

zeigt den letzten bekannten Wert.

3. Histogram

Ein Histogram misst **Verteilungen von Werten**, z. B. Antwortzeiten. Es zählt, **wie viele Ereignisse in bestimmte Wertebereiche ("Buckets")** fallen.

Beispiel:

- `http_request_duration_seconds_bucket`

Diese Metrik ist gekoppelt mit:

- `_count` (Gesamtanzahl)
- `_sum` (Summe aller Werte)

Sehr nützlich für Latenzen und Antwortzeitverteilungen.

PromQL-Beispiel (90. Perzentil über 5 Minuten):

```
histogram_quantile(0.9, rate(http_request_duration_seconds_bucket[5m]))
```

4. Summary (ähnlich wie Histogram, aber clientseitig berechnet)

Ein Summary enthält direkt **Perzentile**, allerdings:

- weniger aggregierbar über mehrere Instanzen
- erzeugt mehr Metriken
- eher selten verwendet in modernen Setups

Beispiel:

- `http_request_duration_seconds{quantile="0.9"}`

⚠ Für verteilte Systeme nicht gut skalierbar → lieber Histogram verwenden!

Zusammenfassung als Tabelle:

Typ	Eigenschaften	Beispiel	Ideal für...
Counter	nur steigend	<code>http_requests_total</code>	Events, Fehler, Anfragen
Gauge	auf- und absteigend	<code>memory_usage_bytes</code>	Zustände, Nutzung
Histogram	Buckets + Summe/Count	<code>*_bucket, *_sum, *_count</code>	Latenzverteilung, SLA-Analyse
Summary	Clientseitige Perzentile	<code>*_quantile</code>	Einfache Latenzmetriken

Achtung: Bitte kein Prometheus-Agent verwenden

Was dann ?

- Nehmt den Operator von Prometheus

Warum ?

- Coole Objekte wie PodMonitor, ServiceMonitor, PrometheusRules funktionieren nicht !
- Dann musst du nämlich die alten ScrapeConfigs verwenden (IHHHHH !)

Prometheus / Grafana Stack installieren

Hinweis: Es gibt eine neuere Anleitung mit Ingress, TLS und BasicAuth-Schutz fuer Prometheus: [install-with-helm-ingress.md](#) Diese Variante (mit port-forward) eignet sich zum schnellen Ausprobieren.

- using the kube-prometheus-stack (recommended !: includes important metrics)

Step 1: Prepare values-file

```
cd
mkdir -p manifests
cd manifests
mkdir -p monitoring
cd monitoring
```

```
vi values.yml
```

```

fullnameOverride: prometheus

alertmanager:
  fullnameOverride: alertmanager

grafana:
  fullnameOverride: grafana

kube-state-metrics:
  fullnameOverride: kube-state-metrics

prometheus-node-exporter:
  fullnameOverride: node-exporter

```

Step 2: Install with helm

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack -f values.yml --namespace monitoring --create-namespace --version 61.3.1

```

Step 3: Connect to prometheus from the outside world

Step 3.1: Start proxy to connect (to on Linux Client)

```

## this is shown in the helm information
helm -n monitoring get notes prometheus

## Get pod that runs prometheus
kubectl -n monitoring get service
kubectl -n monitoring port-forward svc/prometheus-prometheus 9090 &

```

Step 3.2: Start a tunnel in (from) your local-system to the server

```

ssh -L 9090:localhost:9090 tln1@164.92.129.7

```

Step 3.3: Open prometheus in your local browser

```

## in browser
http://localhost:9090

```

Step 4: Connect to the grafana from the outside world

Step 4.1: Start proxy to connect

```

## Do the port forwarding
## Adjust your pods here
kubectl -n monitoring get pods | grep grafana
kubectl -n monitoring port-forward grafana-56b45d8bd9-bp899 3000 &

```

Step 4.2: Start a tunnel in (from) your local-system to the server

```

ssh -L 3000:localhost:3000 tln1@164.92.129.7

```

References:

- <https://github.com/prometheus-community/helm-charts/blob/main/charts/kube-prometheus-stack/README.md>
- <https://artifacthub.io/packages/helm/prometheus-community/prometheus>

Prometheus / Grafana mit Ingress, Letsencrypt und BasicAuth (inkl. Alertmanager)

Hinweis: Der Alertmanager-Teil (Schritt 6) ist aus dem Q3-Advanced-Training uebernommen und an unser DOKS-Setup angepasst (kein MetallLB, kein Wildcard-DNS-Script). Komplett live auf einem laufenden Trainings-Cluster (kube-prometheus-stack 86.3.1) nachgetestet - Prometheus (401/302), Alertmanager (401/200) und Grafana (302 Login-Redirect) funktionieren wie beschrieben.

Voraussetzungen

- Traefik installiert (Namespace `ingress`)
- **Traefik-CRDs vorhanden** - nicht selbstverstaendlich! Pruefen mit `kubectl get crd | grep traefik.io`. Fehlen sie (kein `middlewares.traefik.io` in der Liste), muss Traefik entweder mit `crds.enabled: true` (Default bei einer sauberen Neuinstallation) neu ausgerollt, oder die CRDs separat nachinstalliert werden - additiv, ohne den laufenden Traefik-Pod anzufassen:

```

kubectl apply -f https://raw.githubusercontent.com/traefik/traefik-helm-chart/v40.3.0/traefik/crds/traefik.io_middlewares.yaml

```

(Versionsnummer an die installierte Traefik-Chart-Version anpassen, siehe `helm -n ingress list`)

- `cert-manager` installiert + `ClusterIssuer letsencrypt-prod` vorhanden (aus Uebung: <https://letsencrypt-ingress-traefik>)
- `htpasswd` installiert: `apt install apache2-utils`

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests/monitoring
cd manifests/monitoring
```

Schritt 2: values.yml erstellen

```
vi values.yml

fullnameOverride: prometheus

alertmanager:
  fullnameOverride: alertmanager

grafana:
  fullnameOverride: grafana
  adminPassword: DEIN-PASSWORT
  ingress:
    enabled: true
    ingressClassName: traefik
    hosts:
      - grafana.<dein-name>.do.t3isp.de
    annotations:
      cert-manager.io/cluster-issuer: "letsencrypt-prod"
    tls:
      - secretName: grafana-tls
        hosts:
          - grafana.<dein-name>.do.t3isp.de

kube-state-metrics:
  fullnameOverride: kube-state-metrics

prometheus-node-exporter:
  fullnameOverride: node-exporter
```

Schritt 3: Prometheus-Stack installieren

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack \
  -f values.yml --namespace monitoring --create-namespace --version 86.3.1
kubectl -n monitoring get pods
```

Schritt 4: BasicAuth Secret fuer Prometheus erstellen

```
kubectl create secret generic prometheus-basic-auth \
  --from-literal=users="$(htpasswd -nb admin DEIN-PASSWORT)" \
  -n monitoring
```

Schritt 5: Traefik Middleware + Prometheus Ingress

```
vi prometheus-ingress.yml

apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: prometheus-auth
  namespace: monitoring
spec:
  basicAuth:
    secret: prometheus-basic-auth
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: prometheus-ingress
  namespace: monitoring
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    traefik.ingress.kubernetes.io/router.middlewares: monitoring-prometheus-auth@kubernetescd
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - prometheus.<dein-name>.do.t3isp.de
```

```

    secretName: prometheus-tls
  rules:
  - host: prometheus.<dein-name>.do.t3isp.de
    http:
      paths:
      - path: /
        pathType: Prefix
      backend:
        service:
          name: prometheus-prometheus
          port:
            number: 9090

```

```
kubectl apply -f prometheus-ingress.yml -n monitoring
```

Schritt 6: Alertmanager Ingress (gleiche Middleware wiederverwenden)

Die `Middleware` aus Schritt 5 ist nicht an einen Service gebunden - sie lässt sich 1:1 auch am Alertmanager-Ingress referenzieren.

Erst den tatsächlichen Service-Namen prüfen (hängt vom `fullnameOverride` in der `values.yml` ab):

```
kubectl -n monitoring get svc | grep alertmanager
```

```
vi alertmanager-ingress.yml
```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: alertmanager-ingress
  namespace: monitoring
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    traefik.ingress.kubernetes.io/router.middlewares: monitoring-prometheus-auth@kubernetescrd
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - alertmanager.<dein-name>.do.t3isp.de
      secretName: alertmanager-tls
  rules:
  - host: alertmanager.<dein-name>.do.t3isp.de
    http:
      paths:
      - path: /
        pathType: Prefix
      backend:
        service:
          name: prometheus-alertmanager # Name kommt vom TOP-LEVEL fullnameOverride, nicht vom alertmanager.fullnameOverride
          port:
            number: 9093

```

```
kubectl apply -f alertmanager-ingress.yml -n monitoring
```

Der kube-prometheus-stack legt automatisch eine `Watchdog` -Alert an, die dauerhaft feuert - guter Beweis dafür, dass die Alerting-Pipeline lebt:



Alertmanager mit der ständig aktiven Watchdog-Alert

Schritt 7: Zertifikate prüfen

```

## Alle drei Zertifikate müssen READY=True sein
kubectl -n monitoring get cert

```

Schritt 8: Credentials nachschlagen (falls vergessen)

```

## Grafana-Passwort aus dem Kubernetes Secret auslesen:
kubectl -n monitoring get secret grafana -o jsonpath="{.data.admin-password}" | base64 -d
echo ""

```

```

## Prometheus BasicAuth: Benutzername ist immer "admin",
## Passwort ist das, das du in Schritt 4 mit htpasswd gesetzt hast.
## Zur Erinnerung: es steht auch in deiner values.yml unter adminPassword

```

Schritt 10: Testen

```
## Ohne Credentials -> 401 (Zugang verweigert)
curl -s -o /dev/null -w "%{http_code}" https://prometheus.<dein-name>.do.t3isp.de

## Mit Credentials -> 200 (Zugang erlaubt)
curl -u admin:DEIN-PASSWORT -s -o /dev/null -w "%{http_code}" https://prometheus.<dein-name>.do.t3isp.de
```

Ohne Auth kommt ein sauberes 401 direkt von der Traefik-Middleware (nicht von Prometheus selbst):

 Prometheus ohne Basic-Auth: 401 Unauthorized von Traefik

```
## Alertmanager genauso testen:
curl -s -o /dev/null -w "%{http_code}" https://alertmanager.<dein-name>.do.t3isp.de
curl -u admin:DEIN-PASSWORT -s -o /dev/null -w "%{http_code}" https://alertmanager.<dein-name>.do.t3isp.de
```

```
## Grafana im Browser aufrufen:
https://grafana.<dein-name>.do.t3isp.de
## Login: admin / DEIN-PASSWORT
```

 Grafana Login

Hintergrund: Warum BasicAuth fuer Prometheus?

Grafana hat einen eigenen Login (Benutzerverwaltung, Rollen, Sessions). Prometheus hat kein eingebautes Authentication-System. Traefik loest das ueber eine `Middleware` -- kein extra Pod noetig.

Das Muster: Secret (htpasswd) -> Middleware CRD -> Ingress-Annotation

Aufraeumen

```
kubectl delete namespace monitoring
```

Referenzen

- <https://github.com/prometheus-community/helm-charts/blob/main/charts/kube-prometheus-stack/README.md>
- <https://doc.traefik.io/traefik/middlewares/http/basicauth/>

Uebung: Prometheus UI und PromQL

Hintergrund

Prometheus sammelt Metriken per Pull-Modell: alle 15 Sekunden ruft es bei jedem bekannten Target den Endpunkt `/metrics` ab und speichert die Werte in seiner Time-Series-Datenbank (TSDB).

```
App/Service
└─ /metrics (HTTP)
    │
    │ Prometheus scraped alle 15s
    ▼
Prometheus TSDB
    │
    │ PromQL-Query
    ▼
Grafana / Browser-UI
```

In dieser Uebung schauen wir uns diesen Weg von innen an: rohe Metriken → Prometheus Targets → PromQL-Abfragen.

Voraussetzung

Der kube-prometheus-stack ist installiert mit Ingress und BasicAuth (siehe install-with-helm-ingress.md).

Prometheus im Browser aufrufen:

```
https://prometheus.<dein-name>.do.t3isp.de
## Login: admin / DEIN-PASSWORT
```

Schritt 1: Targets - was scrapt Prometheus?

Im Prometheus-Browser-UI oben auf **Status** → **Targets** klicken.

Hier sieht man alle Endpoints, die Prometheus aktiv abfragt:

- **State UP** (gruen) = Prometheus erreicht den Endpoint und bekommt Metriken
- **State DOWN** (rot) = Endpoint nicht erreichbar

Wichtige Targets im kube-prometheus-stack:

Target	Was es liefert
node-exporter	CPU, RAM, Disk, Netzwerk pro Node

kube-state-metrics	Zustand von Pods, Deployments, ReplicaSets
kubelet / cAdvisor	Container-Ressourcen (CPU/RAM pro Container)
prometheus selbst	Interne Prometheus-Metriken
alertmanager	Alertmanager-Metriken

Frage zum Nachdenken: Wie weiss Prometheus, welche Pods es scrapen soll? → Antwort: ServiceMonitor CRDs (dazu spaeter mehr)

 Prometheus Target health: alle Targets UP

Schritt 2: /metrics direkt ansehen

Prometheus zieht Metriken von HTTP-Endpunkten im Raw-Format. Wir koennen uns diese Rohdaten direkt mit einem curl-Pod anschauen:

```
kubectl run metrics-check -it --rm --image=curlimages/curl \
  --restart=Never -n monitoring -- \
  curl -s http://node-exporter.monitoring.svc.cluster.local:9100/metrics \
  | head -40
```

Hier sieht man das Rohformat:

```
## HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
## TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
node_cpu_seconds_total{cpu="0",mode="system"} 234.56
node_cpu_seconds_total{cpu="0",mode="user"} 567.89
```

Jede Zeile hat das Format:

```
metrik_name{label1="wert1", label2="wert2"} zahlenwert
```

Genau diese Zeilen empfaengt Prometheus bei jedem Scrape-Intervall.

Schritt 3: Erste PromQL-Abfragen im Prometheus UI

Im Prometheus UI den Tab **Graph** oeffnen.

Im Suchfeld die folgenden Abfragen einzeln eingeben und auf **Execute** klicken.

3.1 Welche Targets sind erreichbar?

```
up
```

Ergebnis: Eine 1 = erreichbar, eine 0 = nicht erreichbar. Labels zeigen welcher Job und welcher Instance-Endpoint gemeint ist.

3.2 Alle laufenden Pods im Cluster

```
kube_pod_info
```

Jede Zeile steht fuer einen Pod. Labels enthalten Namespace, Pod-Name, Node.

3.3 Wieviele Pods laufen pro Namespace?

```
count by (namespace) (kube_pod_info)
```

count by (namespace) gruppiert die Ergebnisse und zaehlt.

3.4 CPU-Verbrauch pro Container (letzte 5 Minuten)

```
rate(container_cpu_usage_seconds_total{container!=""}[5m])
```

rate() berechnet den Durchschnitt pro Sekunde ueber 5 Minuten. Container mit leerem Namen sind Infrastruktur-Container, deshalb der Filter.

3.5 RAM-Verbrauch pro Container in MB

```
container_memory_usage_bytes{container!=""} / 1024 / 1024
```

Ergebnis direkt in Megabyte.

3.6 Wieviele Replicas eines Deployments sind verfuegbar?

```
kube_deployment_status_replicas_available
```

Sieht man sofort, wenn ein Deployment nicht alle Replicas hat.

3.7 Pods die neu gestartet wurden (Restarts)

```
kube_pod_container_status_restarts_total > 0
```

Zeigt alle Container, die mindestens einmal neu gestartet wurden. Sehr nuetzlich fuer CrashLoopBackOff-Diagnose.

Schritt 4: Graph-Ansicht verwenden

Eine Abfrage eingeben, dann auf den Tab **Graph** (neben "Table") klicken:

```
rate(container_cpu_usage_seconds_total{container!=""}[5m])
```

Zeitraum oben rechts einstellen (z.B. **15m** fuer die letzten 15 Minuten).

Der Graph zeigt den zeitlichen Verlauf - genau diese Daten fragt Grafana per PromQL ab und stellt sie als Dashboard dar.



Prometheus: PromQL-Query mit Graph-Ansicht

Schritt 5: Labels verstehen

Labels sind das Herzstück von Prometheus. Jede Metrik kann beliebig viele Labels haben, nach denen gefiltert und gruppiert werden kann.

Gezielt nach einem Namespace filtern:

```
kube_pod_info{namespace="monitoring"}
```

Nach mehreren Werten filtern (OR mit Regex):

```
kube_pod_info{namespace=~"monitoring|default"}
```

Label aus dem Ergebnis herauslassen:

```
sum by (node) (kube_pod_info)
```

Zaehlt Pods pro Node (alles ausser Node-Label wird zusammengefasst).

PromQL Spickzettel

Anwendungsfall	Abfrage
Alle Targets erreichbar?	up
CPU-Rate pro Container	<code>rate(container_cpu_usage_seconds_total{container!=""}[5m])</code>
RAM pro Container (MB)	<code>container_memory_usage_bytes{container!=""} / 1024 / 1024</code>
Pods pro Namespace	<code>count by (namespace) (kube_pod_info)</code>
Deployment-Replicas OK?	<code>kube_deployment_status_replicas_available</code>
Container-Restarts	<code>kube_pod_container_status_restarts_total > 0</code>
Node-Auslastung CPU %	<code>100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)</code>
Freier RAM pro Node (GB)	<code>node_memory_MemAvailable_bytes / 1024 / 1024 / 1024</code>
HTTP-Requests pro Sekunde	<code>rate(http_requests_total[5m])</code>

Die letzte Zeile (`http_requests_total`) erscheint nur, wenn eine eigene App diese Metrik exposed - dazu spaeter mehr in der Uebung zu Custom Metriken.

→ [Uebung: Custom Metriken mit eigener Demo-App](#)

Naechster Schritt: Grafana

Dieselben PromQL-Abfragen aus dieser Uebung kann man direkt in Grafana als Dashboard-Panel verwenden:

```
https://grafana.<dein-name>.do.t3isp.de
## Login: admin / DEIN-PASSWORT
```

Connections → **Data Sources** zeigt, dass Grafana bereits mit Prometheus verbunden ist. Unter **Dashboards** sind fertige Kubernetes-Dashboards bereits importiert.

PromQL: rate()

Kurzbeschreibung

- `rate()` ist eine **Funktion in PromQL** – genauer gesagt eine sogenannte **Range-Vector-Funktion**.

Details zu `rate()`

- Typ:** Funktion
- Input:** Ein **Range Vector** (also z. B. `metric[5m]`)
- Zweck:** Berechnet die **durchschnittliche Änderungsrate pro Sekunde** eines Counters über das angegebene Zeitfenster.

Beispiel:

```
rate(http_requests_total[5m])
```

→ `http_requests_total[5m]` ist der Range Vector: alle Werte dieser Zeitreihe der letzten 5 Minuten. → `rate()` berechnet aus diesen Werten die durchschnittliche Steigerung pro Sekunde (unter Berücksichtigung von Counter-Resets).

Kurz gesagt:

✓ `rate()` ist eine eingebaute **PromQL-Funktion** ✓ funktioniert **nur mit Range Vectors**, ✓ und ist **für Counter-Metriken gedacht**.

Stell dir vor:

Du hast einen ****Zähler** (metrics type: counter) **, der **jedes Mal um 1 hochzählt**, wenn jemand einen Muffin isst. Du schaust alle paar Minuten auf diesen Zähler.

Beispiel:

Du schaust **alle Minute** auf den Zähler und schreibst die Zahl auf:

Uhrzeit	Gesehene Zahl auf dem Zähler
12:00	100 Muffins gegessen
12:01	110
12:02	125
12:03	135
12:04	150
12:05	160

?Frage:

Wie viele Muffins wurden **in den letzten 5 Minuten** gegessen, und **wie viele pro Sekunde**?

Schritt für Schritt:

1. Vergleiche den Anfang und das Ende:

- Am Anfang (12:00): 100 Muffins
- Am Ende (12:05): 160 Muffins

2. Wieviel Unterschied?

- $160 - 100 = 60$ Muffins in 5 Minuten

3. Wie viele Sekunden sind das?

- 5 Minuten = **300 Sekunden**

4. Wie viele pro Sekunde?

- $60 \div 300 = 0,2$ Muffins pro Sekunde

Das ist genau das, was `rate()` macht:

```
rate(muffin_eaten_total[5m]) = 0.2
```

Er schaut:

- Wie stark hat sich die Zahl in den letzten 5 Minuten verändert?
- Teilt das durch die Zeit (300 Sekunden)
- Als Ergebnis bekomme ich die durchschnittliche Änderung pro Sekunde (in den letzten 5 Minuten)

Wichtig:

- Er **zählt nicht nur den letzten Wert**.
- Er **vergleicht Anfang und Ende**.
- Er **berechnet eine Durchschnittsgeschwindigkeit** (wie schnell gegessen wurde).

So wie ein Tacho im Auto zeigt, wie schnell du gerade fährst, zeigt `rate()` wie schnell der Counter gewachsen ist – **pro Sekunde im Durchschnitt**.

PromQL: irate()

✂ `irate()` : Sofortiger (instantaner) Anstieg

- **Verwendet nur die letzten zwei Datenpunkte** im Intervall.
- Gibt die „momentane“ Rate zum Zeitpunkt des Queries zurück.
- Ideal für **Alerts**, bei denen eine **schnelle Reaktion** auf Lastspitzen nötig ist.

```
irate(http_requests_total[5m])
```

Vergleich:

Funktion	Typ	Verwendet Samples	Verwendung
<code>rate()</code>	Durchschnitt	Alle im Intervall	Graphen, Trends
<code>irate()</code>	Sofortwert	Nur die letzten 2	Alarme, Peak-Erkennung

Beispiel in der Praxis:

- Du hast ein Scrape-Intervall von 15 s.
- Dann nutzt `rate(http_requests_total[1m])` **4 Messpunkte**.
- `irate(http_requests_total[1m])` nutzt **nur die letzten beiden**.

PromQL: `rate()` vs. `irate()`

Vergleich

- `irate` nimmt nur die letzten beiden Werte
- `rate` bezieht alle Werte mit ein.

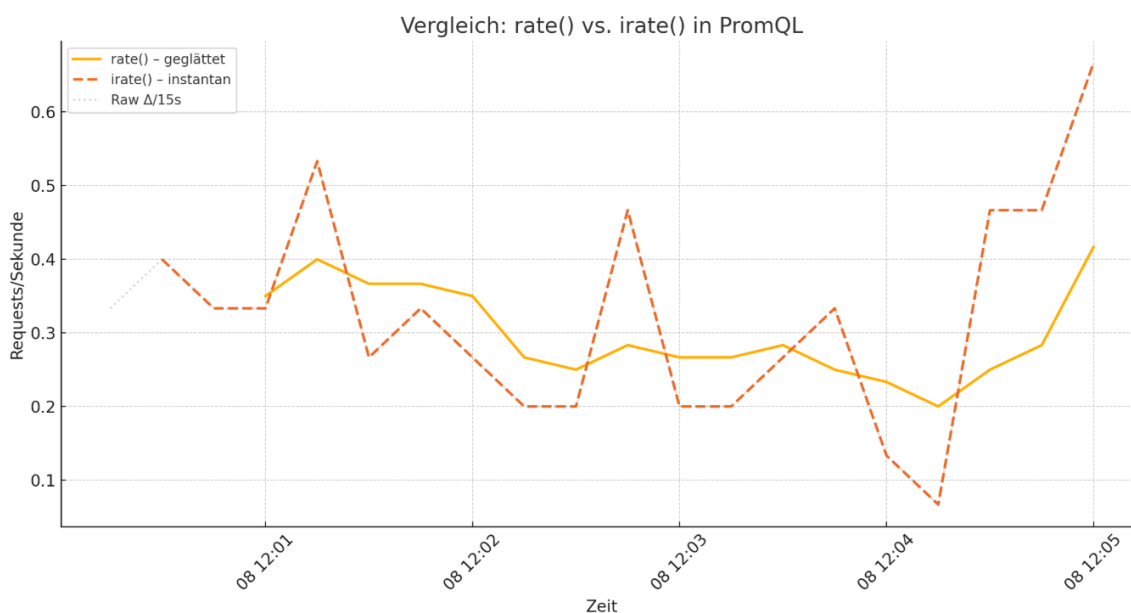
Wann ist `irate()` sinnvoll (für) ?

- Alarme, bei denen du schnell auf Spikes reagieren willst.
- Wenn du den aktuellsten Ausschlag brauchst (z. B. Lastspitzen, Traffic-Peaks).

Wann ist `rate()` sinnvoll

- Dashboards, bei denen Stabilität und Trends im Vordergrund stehen.
- Beispiel: Traffic, CPU-Auslastung, Request-Rate über längere Zeiträume.

Vergleich: `Rate()` Vs. `Irate()` In PromQL



Im Diagramm siehst du deutlich:

- `rate()` (**blau**) ist geglättet und zeigt einen stabileren Verlauf – gut für Trends.
- `irate()` (**orange gestrichelt**) reagiert stärker auf kurzfristige Schwankungen – gut für Alarme.
- Die graue Linie zeigt den tatsächlichen Rohwertwechsel pro Sekunde.

Uebung: Custom Metriken mit eigener Demo-App

Hintergrund

Kubernetes-Standard-Metriken (CPU, RAM, Restarts) kennen wir bereits. In dieser Uebung deployen wir eine eigene App, die **fachliche Metriken** bereitstellt - zum Beispiel aktive Nutzer oder verarbeitete Bestellungen.

Prometheus scrapst diese Metriken genauso wie alle anderen - der Unterschied ist nur, dass wir sie selbst in der App definieren.

Die Demo-App

Der vollständige Source-Code liegt im Repository:

- [app.py](#) - Flask-App mit drei Metriken
- [Dockerfile](#) - wie das Image gebaut wird

- [requirements.txt](#) - Python-Abhaengigkeiten

Das fertige Image: `dockertrainereu/k8s-prometheus-demo:latest`

Metriken der App (Port 8080)

Metrik	Typ	Beschreibung
http_requests_total	Counter	Anzahl HTTP-Requests, nach Methode und Endpoint
active_users	Gauge	Aktuell aktive Nutzer (simuliert, 10-200)
orders_processed_total	Counter	Verarbeitete Bestellungen, nach Status (success/error)

Endpoints der App:

Endpoint	Beschreibung
/	Startseite
/buy	Loest eine Bestellung aus (erhoet orders_processed_total)
/metrics	Prometheus-Metriken im Textformat (Port 8080)

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests/custom-metriken
cd manifests/custom-metriken
```

Schritt 2: Deployment

```
vi 01-deployment.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: demo-app
          image: dockertrainereu/k8s-prometheus-demo:latest
          ports:
            - name: metrics
              containerPort: 8080
```

Schritt 3: Service

```
vi 02-service.yml

apiVersion: v1
kind: Service
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  selector:
    app: demo-app
  ports:
    - name: metrics
      port: 8080
      targetPort: 8080
```

Schritt 4: ServiceMonitor

Der ServiceMonitor sagt Prometheus: "Scrape alle Services mit diesem Label."

```
vi 03-servicemonitor.yml
```

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: demo-app
  namespace: monitoring
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: demo-app
  namespaceSelector:
    matchNames:
      - default
  endpoints:
    - port: metrics
      interval: 15s
      path: /metrics
```

Schritt 5: Alles anwenden

```
kubectl apply -f 01-deployment.yml -f 02-service.yml
kubectl apply -f 03-servicemonitor.yml -n monitoring
```

```
kubectl get pods
kubectl get svc
kubectl -n monitoring get servicemonitor demo-app
```

Schritt 6: Metriken direkt pruefen

Bevor wir Prometheus befragen, pruefen wir ob die App selbst Metriken liefert:

```
kubectl run metrics-check -it --rm --image=curlimages/curl \
  --restart=Never -- \
  curl -s http://demo-app.default.svc.cluster.local:8080/metrics
```

Erwartete Ausgabe (Auszug):

```
## HELP http_requests_total Total number of HTTP requests
## TYPE http_requests_total counter
http_requests_total{endpoint="/metrics",method="GET"} 1.0
## HELP active_users Number of currently active users in the system
## TYPE active_users gauge
active_users 42.0
## HELP orders_processed_total Total number of processed orders
## TYPE orders_processed_total counter
orders_processed_total{status="success"} 3.0
```

Schritt 7: In Prometheus pruefen

```
https://prometheus.<dein-name>.do.t3isp.de
```

Unter **Status** → **Targets** sollte nach ca. 30 Sekunden ein neues Target `demo-app` mit State **UP** erscheinen.

Im **Graph**-Tab diese Abfragen ausprobieren:

```
active_users
```

```
rate(http_requests_total[5m])
```

```
rate(orders_processed_total{status="success"}[5m])
```

Schritt 8: Last erzeugen und beobachten

In einem zweiten Terminal etwas Traffic erzeugen:

```
kubectl run load -it --rm --image=curlimages/curl --restart=Never -- \
  sh -c 'for i in $(seq 1 20); do curl -s http://demo-app.default.svc.cluster.local:8080/buy; done'
```

Dann in Prometheus erneut abfragen:

```
orders_processed_total
```

Der Counter sollte gestiegen sein.

Schritt 9: In Grafana visualisieren

```
https://grafana.<dein-name>.do.t3isp.de
```

1. Links **Dashboards** → **New** → **New Dashboard**
2. **Add visualization**
3. Datasource: **Prometheus**
4. Metric: `active_users` eingeben
5. **Run queries** → Graph erscheint
6. Titel: "Aktive Nutzer" → **Apply**

Ein zweites Panel mit `rate(orders_processed_total{status="success"}[5m])` zeigt die Bestellrate pro Sekunde.

Aufraeumen

```
kubect1 delete -f 01-deployment.yml -f 02-service.yml
kubect1 delete -f 03-servicemonitor.yml -n monitoring
```

Demo-App Source Code: app.py

```
from prometheus_client import Counter, Gauge, generate_latest, CONTENT_TYPE_LATEST
import random
import threading
import time
```

App laeuft auf Port 8080

Metriken werden bereitgestellt unter: http://:8080/metrics

Prometheus scrapt genau diesen Endpoint alle 15s (konfiguriert per ServiceMonitor)

```
app = Flask(name)

http_requests_total = Counter('http_requests_total', 'Total number of HTTP requests', ['method', 'endpoint'])

active_users_gauge = Gauge('active_users', 'Number of currently active users in the system')

orders_processed_total = Counter('orders_processed_total', 'Total number of processed orders', ['status'])

def simulate_background_activity():
    while True:
        active_users_gauge.set(random.randint(10, 200))
        if random.random() > 0.2:
            orders_processed_total.labels(status='success').inc()
        else:
            orders_processed_total.labels(status='error').inc()
        time.sleep(5)

@app.route("/")
def index():
    http_requests_total.labels(method='GET', endpoint='/').inc()
    return '
```

Demo App

Metriken: </metrics>

```
,

@app.route('/buy')
def buy():
    http_requests_total.labels(method='GET', endpoint='/buy').inc()
    orders_processed_total.labels(status='success').inc()
    return 'Order placed!'

@app.route('/metrics')
def metrics():
    http_requests_total.labels(method='GET', endpoint='/metrics').inc()
    return Response(generate_latest(),
                    mimetype=CONTENT_TYPE_LATEST)

if __name__ == '__main__':
    t = threading.Thread(target=simulate_background_activity, daemon=True)
    t.start()
    app.run(host='0.0.0.0', port=8080)
```

Demo-App Source Code: Dockerfile

```
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 8080

CMD ["python", "app.py"]
```

Demo-App Source Code: requirements.txt

```
prometheus-client==0.20.0
```

Vergleich: Splunk vs. CheckMK vs. Prometheus/Grafana

Tool-Vergleich auf einen Blick



Monitoring Tools — Domain-Abdeckung

Logs Metriken Infra-Checks SIEM / Security APM

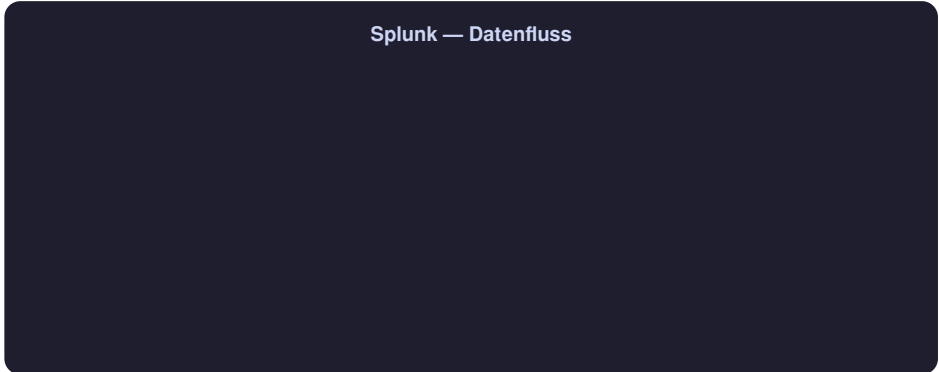
Splunk Enterprise Kernkompetenz moeglich — Kernkompetenz ja*

Prometheus + Grafana mit Loki Kernkompetenz begrenzt — Tempo

CheckMK — ja Kernkompetenz — —

Kernkompetenz moeglich / teilweise nicht vorgesehen * Splunk APM = zugekauftes Produkt (SignalFx)

Splunk-Architektur



Datenquellen

Kubernetes Logs App-Server Firewall / Network Windows / AD Universal Forwarder (Agent auf Host) Indexer komprimiert + indiziert Daten Search Head SPL-Queries Web UI Ausgabe

Dashboards Alerts SIEM / SOC

SPL = Search Processing Language | SOC = Security Operations Center

Welches Tool fuer welche Frage?



Splunk im Detail

Was Splunk kann

Log-Aggregation — alle Logs zentral sammeln und in Echtzeit durchsuchen:

```
index=kubernetes namespace=production | stats count by pod_name | sort -count
```

SIEM (Security Information and Event Management):

- Korreliert Events aus verschiedenen Quellen (Firewall, AD, Kubernetes, App)
- Erkennt Angriffsmuster automatisch (Brute-Force, Lateral Movement)
- Compliance-Reporting (PCI-DSS, SOC2, ISO 27001)
- Incident Response — Timeline eines Angriffs rekonstruieren

Splunk heute — mehr als Logs:

- Splunk APM (Application Performance Monitoring, zugekauft via SignalFx)
- Splunk Infrastructure Monitoring
- Splunk Synthetic Monitoring

Open-Source-Alternative: Grafana Loki

Fuer reines Log-Management in Kubernetes ist **Grafana Loki** oft ausreichend:

- Kostenlos, direkt in Grafana integriert
- Gleiche Idee wie Splunk (Logs sammeln + durchsuchen)
- Kein SIEM, aber fuer Entwickler-Use-Cases ideal

Kosten

Splunk lizenziert nach **Datenvolumen pro Tag** — in grossen Umgebungen einer der teuersten Posten im Monitoring-Budget.

Einsatzbereich	Empfehlung
Kubernetes Dev/Staging	Grafana Loki (kostenlos)
Kubernetes Production	Grafana Loki oder Splunk
Enterprise + Compliance	Splunk
Security / SOC	Splunk

Kubernetes Monitoring (checkmk)

Checkmk Server mit Docker einrichten - Script (umfangreich) mit letsencrypt und ssl

Prerequisites

- Mindestens 8 GB Arbeitsspeicher

Walkthrough

- Umgesetzt mit digitalocean
- Für normale Verwendung ohne digitalocean einfach diese Bereich auskommentieren

```
#!/bin/bash
set -e

## Startzeit für Dauer-Berechnung
START_TIME=$(date +%s)

## =====
## WICHTIG - Vor Deployment manuell setzen!
## =====
DO_TOKEN="ENTER_YOUR_DO_TOKEN"
CMK_PASSWORD="ENTER_YOUR_CHECKMK_ADMIN_PASS_HERE"
## =====

## =====
## NUTZER und ssh konfigurieren
## =====

groupadd sshadmin
USERS="11trainingdo"
echo $USERS
for USER in $USERS
do
    echo "Adding user $USER"
    useradd -s /bin/bash --create-home $USER
    usermod -aG sshadmin $USER
    echo "$USER:YOUR_PASSWORD_HERE" | chpasswd
done
```

```

## We can sudo with 11trainingdo
usermod -aG sudo 11trainingdo

## 20.04 and 22.04 this will be in the subfolder
if [ -f /etc/ssh/sshd_config.d/50-cloud-init.conf ]
then
    sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config.d/50-cloud-init.conf
fi

## seen this in ubuntu 24.04 important here
if [ -f /etc/ssh/sshd_config.d/60-cloudimg-settings.conf ]
then
    sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config.d/60-cloudimg-settings.conf
fi

### both is needed
sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config

usermod -aG sshadmin root

## TBD - Delete AllowUsers Entries with sed
## otherwise we cannot login by group
echo "AllowGroups sshadmin" >> /etc/ssh/sshd_config

## Looks like it takes a while till ssh is running
date > /var/log/training_reload_ssh
systemctl restart ssh

## =====
## CONFIGURATION
## =====
BASE_DOMAIN="do.t3isp.de"
EMAIL="j.metzger@t3company.de"

## Dynamisch aus Hostname (wird in DO GUI gesetzt)
SUBDOMAIN=$(hostname -s)
DOMAIN="${SUBDOMAIN}.${BASE_DOMAIN}"

CMK_VERSION="2.4.0p20"
NGINX_VERSION="1.27-alpine"
CERTBOT_VERSION="v3.0.1"

INSTALL_DIR="/root/checkmk"
STATUS_FILE="/root/install-status.txt"

## Farben
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

log_info() { echo -e "${GREEN}[INFO]${NC} $1"; }
log_warn() { echo -e "${YELLOW}[WARN]${NC} $1"; }
log_error() { echo -e "${RED}[ERROR]${NC} $1"; }

update_status() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> ${STATUS_FILE}
}

cleanup_on_error() {
    log_error "Installation fehlgeschlagen. Räume auf..."
    update_status "[FAILED] Installation fehlgeschlagen"
    cd ${INSTALL_DIR} 2>/dev/null && docker-compose down 2>/dev/null || true
    exit 1
}

trap cleanup_on_error ERR

## Status-Datei initialisieren
echo "=== Checkmk Installation Status ===" > ${STATUS_FILE}
update_status "[START] Installation gestartet für ${DOMAIN}"

echo "===== "
echo "|      Checkmk Installation mit SSL - docker-compose only      |"
echo "===== "
echo ""
echo "Hostname: $(hostname)"

```

```

echo "Subdomain: ${SUBDOMAIN}"
echo "Domain: ${DOMAIN}"
echo ""

## =====
## PRE-FLIGHT CHECKS
## =====

log_info "Starte Pre-Flight Checks..."
update_status "[RUNNING] Pre-Flight Checks"

## DO Token Check
if [ -z "${DO_TOKEN}" ] || [ "${DO_TOKEN}" = "<dein-digitalocean-api-token>" ]; then
    log_error "DO_TOKEN nicht gesetzt!"
    echo "Bitte DO_TOKEN im Script setzen!"
    update_status "[FAILED] DO_TOKEN nicht gesetzt"
    exit 1
fi

## CMK Password Check
if [ -z "${CMK_PASSWORD}" ] || [ "${CMK_PASSWORD}" = "<sicheres-passwort-fuer-checkmk>" ]; then
    log_error "CMK_PASSWORD nicht gesetzt!"
    echo "Bitte CMK_PASSWORD im Script setzen!"
    update_status "[FAILED] CMK_PASSWORD nicht gesetzt"
    exit 1
fi

## Root Check
if [ "$EUID" -ne 0 ]; then
    log_error "Script muss als root ausgeführt werden!"
    exit 1
fi

update_status "[OK] Pre-Flight Checks bestanden"

## =====
## PAKETE INSTALLIEREN
## =====

log_info "Installiere Pakete..."
update_status "[RUNNING] Pakete installieren"
export DEBIAN_FRONTEND=noninteractive
apt-get update -qq
apt-get install -y -qq docker.io docker-compose curl wget dnsutils ufw > /dev/null 2>&1

systemctl enable docker > /dev/null 2>&1
systemctl start docker
update_status "[OK] Pakete installiert"

## =====
## DOCTL INSTALLIEREN
## =====

log_info "Installiere doctl..."
update_status "[RUNNING] doctl installieren"
cd /tmp
wget -q https://github.com/digitalocean/doctl/releases/download/v1.104.0/doctl-1.104.0-linux-amd64.tar.gz
tar xf doctl-1.104.0-linux-amd64.tar.gz > /dev/null 2>&1
mv doctl /usr/local/bin/
chmod +x /usr/local/bin/doctl
rm -f doctl-*.tar.gz

## on cloud-init we need to set home
export HOME=/root

doctl auth init -t ${DO_TOKEN} > /dev/null 2>&1

if ! doctl compute domain list > /dev/null 2>&1; then
    log_error "DigitalOcean API Token ungültig oder keine DNS-Berechtigung!"
    update_status "[FAILED] DO API Token ungültig oder keine DNS-Berechtigung"
    exit 1
fi
log_info "DigitalOcean DNS-Zugriff validiert ✓"
update_status "[OK] doctl installiert und DNS-Zugriff validiert"

## =====
## IP & DNS
## =====

```



```

log_info "Ermittle Droplet IP..."
update_status "[RUNNING] IP und DNS konfigurieren"
DROPLET_IP=$(curl -s http://169.254.169.254/metadata/v1/interfaces/public/0/ipv4/address)

if [ -z "$DROPLET_IP" ]; then
    log_error "Konnte Droplet IP nicht ermitteln!"
    update_status "[FAILED] Droplet IP nicht ermittelt"
    exit 1
fi
log_info "Droplet IP: ${DROPLET_IP}"

## Domain Check
if ! doctl compute domain list --format Domain --no-header | grep -q "^${BASE_DOMAIN}$"; then
    log_error "Domain ${BASE_DOMAIN} nicht in DigitalOcean!"
    update_status "[FAILED] Base Domain nicht gefunden"
    exit 1
fi

## A-Record erstellen/aktualisieren
log_info "Erstelle/Aktualisiere A-Record für ${SUBDOMAIN}..."
EXISTING_RECORD=$(doctl compute domain records list ${BASE_DOMAIN} --format Name,Type --no-header | grep "^${SUBDOMAIN}
[[:space:]]" | grep "A" || true)

if [ -n "$EXISTING_RECORD" ]; then
    RECORD_ID=$(doctl compute domain records list ${BASE_DOMAIN} --format ID,Name,Type --no-header | grep "${SUBDOMAIN}
[[:space:]]" | grep "A" | awk '{print $1}')
    doctl compute domain records update ${BASE_DOMAIN} --record-id ${RECORD_ID} --record-data ${DROPLET_IP} > /dev/null 2>&1
    log_info "A-Record aktualisiert"
else
    doctl compute domain records create ${BASE_DOMAIN} \
        --record-type A \
        --record-name ${SUBDOMAIN} \
        --record-data ${DROPLET_IP} \
        --record-ttl 300 > /dev/null 2>&1
    log_info "A-Record erstellt"
fi

## DNS Propagation Check - prüft Google DNS und DO Nameserver
log_info "Warte auf DNS Propagation (max. 5 Minuten)..."
DNS_RESOLVED=false
for i in {1..60}; do
    # Prüfe zuerst Google DNS
    RESOLVED_IP=$(dig +short ${DOMAIN} @8.8.8.8 | grep -E '^[0-9.]+$' | head -n1)

    if [ "$RESOLVED_IP" = "$DROPLET_IP" ]; then
        DNS_RESOLVED=true
        log_info "DNS aufgelöst (Google): ${DOMAIN} → ${DROPLET_IP} ✓"
        break
    fi

    # Nach 2 Minuten: akzeptiere auch DO Nameserver (Let's Encrypt funktioniert damit)
    if [ $i -ge 24 ]; then
        RESOLVED_IP_DO=$(dig +short ${DOMAIN} @ns1.digitalocean.com | grep -E '^[0-9.]+$' | head -n1)
        if [ "$RESOLVED_IP_DO" = "$DROPLET_IP" ]; then
            DNS_RESOLVED=true
            log_info "DNS aufgelöst (DO NS): ${DOMAIN} → ${DROPLET_IP} ✓"
            log_warn "Google DNS noch nicht propagiert - fahre trotzdem fort"
            break
        fi
    fi

    if [ $((i % 10)) -eq 0 ]; then
        log_warn "Warte... (${i*5})/300 Sekunden"
    fi
    sleep 5
done

if [ "$DNS_RESOLVED" = false ]; then
    log_error "DNS Propagation fehlgeschlagen!"
    update_status "[FAILED] DNS Propagation fehlgeschlagen"
    exit 1
fi
update_status "[OK] DNS konfiguriert: ${DOMAIN} → ${DROPLET_IP}"

## =====
## FIREWALL
## =====

```

```

log_info "Konfiguriere Firewall..."
update_status "[RUNNING] Firewall konfigurieren"
if command -v ufw > /dev/null 2>&1; then
    # WICHTIG: Regeln ZUERST hinzufügen, DANN aktivieren!
    ufw allow 22/tcp > /dev/null 2>&1
    ufw allow 80/tcp > /dev/null 2>&1
    ufw allow 443/tcp > /dev/null 2>&1
    ufw allow 8000/tcp > /dev/null 2>&1 # Checkmk Agent Registration
    ufw --force enable > /dev/null 2>&1
    log_info "Firewall konfiguriert ✓"
fi
update_status "[OK] Firewall konfiguriert (22, 80, 443, 8000)"

## =====
## VERZEICHNISSE ERSTELLEN
## =====

log_info "Erstelle Verzeichnisstruktur..."
update_status "[RUNNING] Verzeichnisse erstellen"
rm -rf ${INSTALL_DIR} 2>/dev/null || true
mkdir -p ${INSTALL_DIR}/{nginx/html,certbot/conf,certbot/www}
cd ${INSTALL_DIR}
update_status "[OK] Verzeichnisse erstellt"

## =====
## DOCKER-COMPOSE ERSTELLEN
## =====

log_info "Erstelle docker-compose.yml..."
update_status "[RUNNING] docker-compose.yml erstellen"
cat > docker-compose.yml << EOF
version: '3.8'

services:
  checkmk:
    image: checkmk/check-mk-cloud:${CMK_VERSION}
    container_name: monitoring
    restart: always
    environment:
      - CMK_PASSWORD=${CMK_PASSWORD}
      - TZ=Europe/Berlin
    volumes:
      - monitoring:/omd/sites
    tmpfs:
      - /opt/omd/sites/cmik/tmp:uid=1000,gid=1000
    ports:
      - "127.0.0.1:5000:5000"
      - "8000:8000"
    networks:
      - checkmk

  nginx:
    image: nginx:${NGINX_VERSION}
    container_name: nginx
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/html:/etc/nginx/html:ro
      - ./certbot/conf:/etc/letsencrypt:ro
      - ./certbot/www:/var/www/certbot:ro
    networks:
      - checkmk
    depends_on:
      - checkmk

  certbot:
    image: certbot/certbot:${CERTBOT_VERSION}
    container_name: certbot
    volumes:
      - ./certbot/conf:/etc/letsencrypt
      - ./certbot/www:/var/www/certbot
    entrypoint: "/bin/sh -c 'trap exit TERM; while :; do certbot renew --quiet; sleep 12h & wait \${$!}; done;'"

networks:

```

```

checkmk:
  driver: bridge

volumes:
  monitoring:
EOF
update_status "[OK] docker-compose.yml erstellt"

## =====
## CUSTOM 404 ERROR PAGE
## =====

log_info "Erstelle Custom 404 Error Page..."
cat > nginx/html/404.html << 'HTML_EOF'
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>404 - Seite nicht gefunden</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
      background: linear-gradient(135deg, #1a5f2a 0%, #2d8a3e 100%);
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      color: #333;
    }
    .container {
      background: white;
      border-radius: 12px;
      box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
      padding: 3rem 2rem;
      max-width: 500px;
      text-align: center;
    }
    .error-code {
      font-size: 8rem;
      font-weight: bold;
      background: linear-gradient(135deg, #1a5f2a 0%, #2d8a3e 100%);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      background-clip: text;
      line-height: 1;
      margin-bottom: 1rem;
    }
    h1 { font-size: 1.5rem; color: #333; margin-bottom: 1rem; }
    p { color: #666; line-height: 1.6; margin-bottom: 2rem; }
    .btn {
      display: inline-block;
      background: linear-gradient(135deg, #1a5f2a 0%, #2d8a3e 100%);
      color: white;
      padding: 0.75rem 2rem;
      border-radius: 6px;
      text-decoration: none;
      font-weight: 500;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="error-code">404</div>
    <h1>Seite nicht gefunden</h1>
    <p>Die angeforderte Seite existiert nicht.</p>
    <a href="/" class="btn">Zur Startseite</a>
  </div>
</body>
</html>
HTML_EOF

## =====
## PHASE 1: HTTP-ONLY NGINX CONFIG
## =====

log_info "Erstelle HTTP-only Nginx Config (Phase 1)..."

```

```

update_status "[RUNNING] Nginx HTTP Config erstellen"
cat > nginx/nginx.conf << NGINX_EOF
server {
    listen 80;
    server_name ${DOMAIN};

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    error_page 404 /404.html;
    location = /404.html {
        root /etc/nginx/html;
        internal;
    }

    location / {
        proxy_pass http://checkmk:5000/;
        proxy_buffering off;
        proxy_http_version 1.1;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $http_connection;
        proxy_set_header Host $host;
        proxy_intercept_errors on;
    }
}
NGINX_EOF
update_status "[OK] Nginx HTTP Config erstellt"

## =====
## CONTAINER STARTEN (HTTP ONLY)
## =====

log_info "Starte Container (HTTP only)..."
update_status "[RUNNING] Container starten"
docker-compose up -d checkmk nginx

log_info "Warte auf Container-Start (60 Sekunden für Checkmk)..."
sleep 60

## Container Health Check
if ! docker ps | grep -q monitoring; then
    log_error "Checkmk Container läuft nicht!"
    docker logs monitoring
    update_status "[FAILED] Checkmk Container läuft nicht"
    exit 1
fi

if ! docker ps | grep -q nginx; then
    log_error "nginx läuft nicht!"
    docker logs nginx
    update_status "[FAILED] Nginx Container läuft nicht"
    exit 1
fi

log_info "Alle Container laufen ✓"
update_status "[OK] Container gestartet"

## HTTP Test
log_info "Teste HTTP Verbindung..."
sleep 5
if curl -s -o /dev/null -w "%{http_code}" http://localhost/ | grep -q "200\|302"; then
    log_info "HTTP funktioniert ✓"
else
    log_warn "HTTP Test fehlgeschlagen - fahre trotzdem fort"
fi

## =====
## SSL ZERTIFIKAT HOLEN
## =====

log_info "Fordere SSL Zertifikat an..."
update_status "[RUNNING] SSL Zertifikat anfordern"
if docker run --rm \
    -v ${INSTALL_DIR}/certbot/conf:/etc/letsencrypt \
    -v ${INSTALL_DIR}/certbot/www:/var/www/certbot \

```

```

certbot/certbot:${CERTBOT_VERSION} certonly \
--webroot \
--webroot-path=/var/www/certbot \
--email ${EMAIL} \
--agree-tos \
--no-eff-email \
-d ${DOMAIN}; then
log_info "SSL Zertifikat erfolgreich erstellt ✓"
else
log_error "SSL Zertifikat Erstellung fehlgeschlagen!"
update_status "[FAILED] SSL Zertifikat fehlgeschlagen"
exit 1
fi

## Zertifikat Check
if [ ! -f "certbot/conf/live/${DOMAIN}/fullchain.pem" ]; then
log_error "Zertifikat nicht gefunden!"
ls -la certbot/conf/live/
update_status "[FAILED] Zertifikat nicht gefunden"
exit 1
fi

log_info "Zertifikat validiert ✓"
update_status "[OK] SSL Zertifikat erstellt"

## =====
## PHASE 2: HTTPS NGINX CONFIG
## =====

log_info "Update zu HTTPS Nginx Config (Phase 2)..."
update_status "[RUNNING] Nginx HTTPS Config erstellen"
cat > nginx/nginx.conf << NGINX_EOF
server {
    listen 80;
    server_name ${DOMAIN};

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://\$host\$request_uri;
    }
}

server {
    listen 443 ssl http2;
    server_name ${DOMAIN};

    ssl_certificate /etc/letsencrypt/live/${DOMAIN}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/${DOMAIN}/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    error_page 404 /404.html;
    location = /404.html {
        root /etc/nginx/html;
        internal;
    }

    location / {
        proxy_pass http://checkmk:5000/;
        proxy_buffering off;
        proxy_http_version 1.1;
        proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto \$scheme;
        proxy_set_header Upgrade \$http_upgrade;
        proxy_set_header Connection \$http_connection;
        proxy_set_header Host \$host;
        proxy_intercept_errors on;
    }
}
NGINX_EOF

```

```

# Nginx restart
log_info "Restart Nginx mit SSL Config..."
sleep 3
docker-compose restart nginx
update_status "[OK] Nginx HTTPS Config aktiviert"

## Certbot Renewal Service starten
log_info "Starte Certbot Renewal Service..."
docker-compose up -d certbot

## =====
## FINAL CHECKS
## =====

log_info "Führe finale Tests durch..."
update_status "[RUNNING] Finale Tests"
sleep 5

HTTP_CODE=$(curl -k -s -o /dev/null -w "%{http_code}" https://${DOMAIN}/ | echo "000")
if [[ "$HTTP_CODE" =~ ^(200|302)$ ]]; then
    log_info "HTTPS funktioniert ✓"
    update_status "[OK] HTTPS funktioniert"
else
    log_warn "HTTPS Status: ${HTTP_CODE} - bitte manuell prüfen"
    update_status "[WARN] HTTPS Status: ${HTTP_CODE}"
fi

## =====
## CREDENTIALS AUSGABE
## =====

cat > credentials.txt << EOF

=====
CHECKMK INSTALLATION - CREDENTIALS
=====

URL:             https://${DOMAIN}/cmk/
Droplet IP: ${DROPLET_IP}

ADMIN LOGIN:
Username: cmkadmin
Password: ${CMK_PASSWORD}

WORKFLOW:
1. Browser Öffnen: https://${DOMAIN}/cmk/
2. Login mit cmkadmin / ${CMK_PASSWORD}
3. Hosts hinzufügen und monitoren

DOCKER BEFEHLE:
Status:   cd ${INSTALL_DIR} && docker-compose ps
Logs:     docker-compose logs -f monitoring
Restart:  docker-compose restart
Stop:     docker-compose down
Start:    docker-compose up -d

SSL ERNEUERUNG:
Automatisch alle 12h via Certbot Container
Manuell: docker-compose run --rm certbot renew

INSTALLATION:
Verzeichnis: ${INSTALL_DIR}
Nginx:       ${INSTALL_DIR}/nginx/nginx.conf

AGENT REGISTRATION:
Port 8000 ist offen für Agent-Registrierung

=====
EOF

chmod 600 credentials.txt

echo ""
echo "===== "
echo "|| ✓ INSTALLATION ERFOLGREICH ABGESCHLOSSEN ||"
echo "===== "
echo ""
cat credentials.txt
echo ""

```

```
log_info "Credentials gespeichert in: ${INSTALL_DIR}/credentials.txt"

## Dauer berechnen
END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))
MINUTES=$((DURATION / 60))
SECONDS=$((DURATION % 60))

log_info "Installation abgeschlossen in ${MINUTES} Minuten und ${SECONDS} Sekunden!"

update_status "[DONE] Installation erfolgreich abgeschlossen"
update_status "Dauer: ${MINUTES} Minuten ${SECONDS} Sekunden"
update_status "URL: https://${DOMAIN}/cmk/"
update_status "User: cmkadmin"
```

Checkmk in Kubernetes einrichten (checkmk raw)

Hintergrund

CheckMK kann Kubernetes-Cluster ueber die Kubernetes API monitoren. Die Integration besteht aus:

Komponente	Funktion	Deployment
Cluster Collector	Sammelt Metriken aus dem Cluster (CPU, RAM, Netzwerk)	Helm Chart in K8s
Node Collector	Laeuft auf jedem Node, sammelt Ressourcen-Auslastung	DaemonSet via Helm
Kubernetes Special Agent	Fragt K8s API ab (Pods, Services, Deployments, etc.)	Laeuft auf CheckMK Server
Piggyback Hosts	Virtuelle Hosts fuer K8s Objekte in CheckMK	Manuell in CheckMK RAW

Wichtig: CheckMK RAW erfordert manuelle Erstellung der Piggyback-Hosts (in kommerziellen Editionen automatisch).

Diese Anleitung verwendet:

- **Ingress (Traefik) mit HTTPS/TLS** statt NodePort
- **Let's Encrypt** für automatische SSL-Zertifikate via cert-manager
- **ClusterIP Service** für internen Zugriff

Voraussetzungen

- Zugang zum Kubernetes Cluster mit kubectl
- CheckMK RAW Site: <https://checkmk-tln<X>.do.t3isp.de/> (X = Teilnehmer-Nummer)
- Helm installiert
- **cert-manager installiert** (siehe [ingress/https-letsencrypt-ingress-traefik.md](https://ingress-https-letsencrypt-ingress-traefik.md))
- **ClusterIssuer letsencrypt-prod konfiguriert**
- **Traefik Ingress Controller** installiert und funktionsfähig

Schritt 1: Helm Repository hinzufuegen

```
helm repo add checkmk-chart https://checkmk.github.io/checkmk_kube_agent
helm repo update
```

Verfuegbare Versionen anzeigen:

```
helm search repo checkmk-chart --versions | head -10
```

Schritt 2: Cluster Collector deployen

Standard-Konfiguration anzeigen:

```
helm show values checkmk-chart/checkmk > /tmp/checkmk-values.yaml
```

Helm Chart installieren:

```
helm upgrade --install checkmk checkmk-chart/checkmk \
  --namespace checkmk-monitoring \
  --create-namespace \
  --version 1.9.0 \
  --reset-values
```

Erklärung der Flags:

- **--create-namespace** : Erstellt den Namespace automatisch (kein `kubectl create namespace` nötig)
- **--version 1.8.0** : Verwendet spezifische Chart-Version (reproduzierbar)
- **--reset-values** : Stellt sicher, dass keine alten Values übernommen werden

Schritt 3: Deployment pruefen

```
kubectl get pods -n checkmk-monitoring
kubectl get svc -n checkmk-monitoring
```

```
kubectl get daemonset -n checkmk-monitoring
```

Erwartete Pods:

- `checkmk-cluster-collector-*` - 1 Pod
- `checkmk-node-collector-*` - 1 Pod pro Node (DaemonSet)

Schritt 4: Ingress-Objekt erstellen

Ingress mit TLS/HTTPS via Traefik und Let's Encrypt:

```
cd
mkdir -p manifests
cd manifests
mkdir -p checkmk
cd checkmk
```

```
nano ingress.yaml
```

```
## vi ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: checkmk-collector-ingress
  namespace: checkmk-monitoring
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - checkmk-collector.tln<X>.do.t3isp.de
    secretName: checkmk-collector-tls
  rules:
  - host: "checkmk-collector.tln<X>.do.t3isp.de"
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: checkmk-cluster-collector
            port:
              number: 8080
```

Wichtig: Ersetze `<X>` mit deiner Teilnehmer-Nummer!

Ingress erstellen:

```
kubectl apply -f ingress.yaml
```

Ingress pruefen:

```
kubectl get ingress -n checkmk-monitoring
kubectl describe ingress checkmk-collector-ingress -n checkmk-monitoring
```

Zertifikat pruefen:

```
kubectl get certificate -n checkmk-monitoring
kubectl get secret checkmk-collector-tls -n checkmk-monitoring
```

Voraussetzung: cert-manager muss installiert sein und der ClusterIssuer `letsencrypt-prod` muss existieren (siehe [ingress/https-letsencrypt-ingress-traefik.md](#)).

Schritt 5: Service Account Token extrahieren

Der ServiceAccount wurde automatisch vom Helm Chart erstellt. Token extrahieren:

```
## Secret-Name finden
kubectl get secrets -n checkmk-monitoring | grep checkmk

## Token extrahieren und dekodieren
kubectl get secret checkmk-checkmk -n checkmk-monitoring -o jsonpath='{.data.token}' | base64 --decode > sa-token
```

Wichtig: Token speichern - wird fuer CheckMK benoetigt!

Alternative (wenn kein Secret existiert):

```
## Token direkt vom ServiceAccount holen
kubectl create token checkmk -n checkmk-monitoring --duration=87600h
```


Schritt 6: CA-Zertifikat extrahieren

```
## weil 1. Cluster digitalocean, index 0
kubectl config view --raw -o jsonpath='{.clusters[1].cluster.certificate-authority-data}' | base64 --decode > k8s-ca.crt
```

CA-Zertifikat anzeigen:

```
cat k8s-ca.crt
## und prüfen ob es das richtige ist:
## muss kubernetes heissen
openssl x509 -in k8s-ca.crt -issuer -noout
```

Schritt 7: Cluster Collector Endpoint ermitteln

Ingress-URL verwenden:

```
kubectl get ingress -n checkmk-monitoring
```

Der Cluster Collector Endpoint ist:

```
https://checkmk-collector.tln<X>.do.t3isp.de
```

Wichtig:

- Ersetze <X> mit deiner Teilnehmer-Nummer
- HTTPS wird durch Let's Encrypt bereitgestellt
- Der Endpoint ist von aussen erreichbar

Testen (sollte Metriken zurueckgeben):

```
curl https://checkmk-collector.tln<X>.do.t3isp.de/openmetrics
```

Schritt 8: CheckMK konfigurieren - Token speichern

1. Oeffne CheckMK: `https://checkmk-tln<X>.do.t3isp.de/`
2. Gehe zu **Setup > General > Passwords**
3. Klicke **Add password**
4. Konfiguration:
 - **Unique ID:** `k8s-token`
 - **Title:** `Kubernetes Service Account Token`
 - **Password:** Token aus Schritt 5 einfüegen
5. **Save**

Schritt 9: CA-Zertifikat in CheckMK importieren

1. **Setup > Global settings > Site management**
2. Suche nach "**Trusted certificate authorities for SSL**"
3. Fuege den Inhalt von `/tmp/k8s-ca.crt` hinzu
4. **Save**

Schritt 10: Piggyback Host erstellen

1. **Setup > Hosts > Add host**
2. Konfiguration:
 - **Hostname:** `k8s-cluster-<dein-name>` (z.B. `k8s-cluster-jmetzger`)
 - **IP address family:** **No IP** (wichtig!)

API integrations if configured, else Checkmk agent ▼

- **Monitoring agents:**

- Labels hinzufuegen:

- **Key:** `cmk/kubernetes`
- **Value:** `yes`

3. **Save & go to service configuration** (NICHT "Save & run service discovery")

Wichtig: Der Host bekommt keine IP, da er nur Piggyback-Daten empfaengt!

Schritt 11: Kubernetes Special Agent konfigurieren

1. **Setup > Agents > VM, cloud, container > Kubernetes**
2. Klicke **Add rule**
3. **Kubernetes cluster configuration:**
 - **Cluster name:** `k8s-cluster-<dein-name>` (oder eigener Name)
 - **Token:** Waehle `k8s-token` aus Dropdown
 - **API server endpoint:** `https://<DEINE-K8S-API-IP>:6443`

- **SSL certificate verification:** Enabled (mit importiertem CA-Cert)

4. Collector: Enrich With Usage Data

- **Cluster collector endpoint:** `https://checkmk-collector-t1n<X>.do.t3isp.de`

5. Kubernetes API:

- **Object selection:** Waehle gewuenschte Objekte:
 - ✓ Pods
 - ✓ Nodes
 - ✓ Deployments
 - ✓ DaemonSets
 - ✓ StatefulSets
 - ✓ Services
 - ✓ Namespaces
 - ✓ CronJobs (falls benoetigt)

6. Namespace filtering:

- **Include namespaces:** Leer lassen fuer alle Namespaces
- **Exclude namespaces:** `kube-public,kube-node-lease` (optional)

Hinweise:

- `kube-system` wird inkludiert um System-Komponenten zu monitoren. `kube-public` und `kube-node-lease` sind meist leer bzw. nicht relevant fuer Monitoring.
- **Filter-Logik:** Include wird zuerst angewendet (leer = alle), danach Exclude. **Exclude hat immer Vorrang** - ein Namespace in beiden Listen wird ausgeschlossen.

7. Explicit hosts:

- Waehle `k8s-cluster-<dein-name>` (Host aus Schritt 10)

8. Save

Schritt 12: Aenderungen aktivieren

1. Oben rechts auf "**1 change**" (oder mehr) klicken
2. **Activate on selected sites**
3. Warten bis Aktivierung abgeschlossen

Schritt 13: Service Discovery durchfuehren

1. **Setup > Hosts**
2. Suche Host `k8s-cluster-<dein-name>`
3. Klicke auf Host
4. **Run service discovery**
5. **Accept all**
6. **Activate changes**

Erwartete Services:

- `Kubernetes Cluster CPU resources`
- `Kubernetes Cluster Memory resources`
- `Kubernetes Node <node-name>`
- Weitere Services je nach Objekt-Auswahl

Schritt 14: Piggyback Hosts fuer K8s Objekte erstellen (CheckMK RAW)

In CheckMK RAW werden Hosts fuer Kubernetes-Objekte NICHT automatisch erstellt. Manuelle Erstellung:

```
## Auf dem CheckMK Server (falls SSH-Zugang)
## Site-Kontext wechseln
OMD[site]> cmk-piggyback list orphans
```

Alternativ in CheckMK GUI:

1. **Setup > Hosts > Add host**
2. Fuer jedes K8s Objekt (Pod, Deployment, etc.) einen Host erstellen:
 - **Hostname:** Exakter Name aus Piggyback-Daten (z.B. `pod_nginx-deployment-xyz`)
 - **IP address family:** No IP
 - **Label:** `cmk/kubernetes: yes`
3. Service Discovery durchfuehren

Tipp fuer CheckMK RAW: Beginne mit wichtigen Objekten (Nodes, Deployments), nicht alle Pods einzeln.

Schritt 15: Periodic Service Discovery konfigurieren (optional)

Automatische Discovery fuer neue Services:

1. **Setup > Periodic service discovery**
2. **Add rule**
3. **Conditions:**
 - **Host labels:** `cmk/kubernetes:yes`

4. **Service discovery:**
 - **Mode:** "Add unmonitored services and new host labels"
 - **Interval:** 15 Minuten (kuerzer als Standard)
5. **Save**

Schritt 16: Monitoring testen

Services pruefen:

1. **Monitor > All hosts**
2. Filter: `cmk/kubernetes:yes`
3. Services pruefen - Status sollte OK sein

Dashboards (nur in kommerziellen Editionen):

- CheckMK RAW hat keine vordefinierten K8s Dashboards
- Manuell Views erstellen moeglich

Troubleshooting

Problem: Keine Services gefunden

Loesung:

```
## Piggyback-Daten pruefen (auf CheckMK Server)
OMD[site]> cmk-piggyback list
OMD[site]> cmk-piggyback show <hostname>
```

Problem: Connection refused zum Cluster Collector

Loesung:

```
## Ingress pruefen
kubectl get ingress -n checkmk-monitoring
kubectl describe ingress checkmk-collector-ingress -n checkmk-monitoring

## Service pruefen
kubectl get svc -n checkmk-monitoring

## Testen von lokalem Rechner
curl https://checkmk-collector-tln<X>.app.do.t3isp.de/openmetrics

## Zertifikat pruefen (sollte Ready: True sein)
kubectl get certificate -n checkmk-monitoring
```

Problem: Unauthorized bei API-Zugriff

Loesung:

```
## Token-Gultigkeit pruefen
kubectl get secrets -n checkmk-monitoring

## Neues Token erstellen
kubectl create token checkmk -n checkmk-monitoring --duration=87600h
```

Problem: Piggyback Hosts erstellen in RAW zu aufwaendig

Loesung:

- Fokus auf wichtige Hosts (Nodes, kritische Deployments)
- Script zur automatischen Host-Erstellung schreiben
- Oder: Upgrade zu kommerzieller Edition erwägen

Aufraeumen

Helm Release entfernen:

```
helm uninstall checkmk -n checkmk-monitoring
```

Namespace loeschen:

```
kubectl delete namespace checkmk-monitoring
```

In CheckMK:

1. Hosts loeschen: **Setup > Hosts**
2. Rule loeschen: **Setup > Agents > VM, cloud, container > Kubernetes**
3. Token loeschen: **Setup > General > Passwords**
4. **Activate changes**

Zusammenfassung

Komponente	Status

Cluster Collector	Deployed via Helm (ClusterIP Service)
Ingress	Traefik mit TLS/Let's Encrypt
Node Collector	DaemonSet auf allen Nodes
K8s Special Agent	Konfiguriert in CheckMK
Piggyback Host	Manuell erstellt (RAW)
Service Discovery	Durchgefuehrt
Monitoring	Aktiv via HTTPS

CheckMK RAW Besonderheiten:

- ✓ Vollstaendige K8s API Integration
- ✓ Cluster Collector fuer Metriken
- ✗ Keine automatischen Piggyback Hosts
- ✗ Keine vorgefertigten Dashboards
- Manueller Aufwand hoeher als kommerzielle Editionen

Weiterfuehrende Informationen

Offizielle Dokumentation: https://docs.checkmk.com/latest/en/monitoring_kubernetes.html

Checkmk in Kubernetes einrichten (checkmk enterprise/cloud edition)

CheckMK Edition Enterprise / Cloud - Kubernetes Monitoring einrichten

- Wir verwenden im Training die Edition Cloud (weil diese eine 30-Tage Trial-Version bietet)

Hintergrund

CheckMK kann Kubernetes-Cluster ueber die Kubernetes API monitoren. Die Integration besteht aus:

Komponente	Funktion	Deployment
Cluster Collector	Sammelt Metriken aus dem Cluster (CPU, RAM, Netzwerk)	Helm Chart in K8s
Node Collector	Laeuft auf jedem Node, sammelt Ressourcen-Auslastung	DaemonSet via Helm
Kubernetes Special Agent	Fragt K8s API ab (Pods, Services, Deployments, etc.)	Laeuft auf CheckMK Server
Piggyback Hosts	Virtuelle Hosts fuer K8s Objekte in CheckMK	Manuell in CheckMK RAW

Wichtig: CheckMK RAW erfordert manuelle Erstellung der Piggyback-Hosts (in kommerziellen Editionen (Enterprise und Cloud) automatisch).

Diese Anleitung verwendet:

- **Ingress (Traefik) mit HTTPS/TLS** statt NodePort
- **Let's Encrypt** fuer automatische SSL-Zertifikate via cert-manager
- **ClusterIP Service** fuer internen Zugriff

Voraussetzungen

- Zugang zum Kubernetes Cluster mit kubectl
- CheckMK RAW Site: <https://tln<X>.do.t3isp.de/> (X = Teilnehmer-Nummer)
- Helm installiert
- **cert-manager installiert** (siehe `ingress/https-letsencrypt-ingress-traefik.md`)
- **ClusterIssuer letsencrypt-prod konfiguriert**
- **Traefik Ingress Controller** installiert und funktionsfähig

Schritt 1: Helm Repository hinzufuegen

```
helm repo add checkmk-chart https://checkmk.github.io/checkmk_kube_agent
helm repo update
```

Verfuegbare Versionen anzeigen:

```
helm search repo checkmk-chart --versions | head -10
```

Schritt 2: Cluster Collector deployen (mit Ingress)

Standard-Konfiguration anzeigen:

```
helm show values checkmk-chart/checkmk > ~/checkmk-values.yaml
```

Values für ingress setzen

```
cd
mkdir -p helm-charts/checkmk
cd helm-charts/checkmk
nano values.yaml
```

Wichtig: Ersetze <X> mit deiner Teilnehmer-Nummer!

```
clusterCollector:
  ingress:
    enabled: true
    className: traefik
    annotations:
      cert-manager.io/cluster-issuer: "letsencrypt-prod"
    hosts:
      - host: checkmk-collector.tln<X>.do.t3isp.de
        paths:
          - path: /
            pathType: Prefix
  tls:
    - hosts:
      - checkmk-collector.tln<X>.do.t3isp.de
      secretName: checkmk-collector-tls
```

Helm Chart installieren:

```
helm upgrade --install checkmk checkmk-chart/checkmk \
  --namespace checkmk-monitoring \
  --create-namespace \
  --version 1.11.0 \
  --reset-values \
  -f values.yaml
```

Erklärung der Flags:

- `--create-namespace` : Erstellt den Namespace automatisch (kein `kubectl create namespace` nötig)
- `--version 1.11.0` : Verwendet spezifische Chart-Version (reproduzierbar)
- `--reset-values` : Stellt sicher, dass keine alten Values übernommen werden
- `-f values.yaml` : Konfigurationswerte aus values.yaml verwenden (für ingress.yaml)

Schritt 3: Deployment prüfen

```
kubectl get pods -n checkmk-monitoring
kubectl get svc -n checkmk-monitoring
kubectl get daemonset -n checkmk-monitoring
```

Erwartete Pods:

- `checkmk-cluster-collector-*` - 1 Pod
- `checkmk-node-collector-*` - 1 Pod pro Node (DaemonSet)

Schritt 4: Zertifikat prüfen

Zertifikat prüfen:

```
kubectl -n checkmk-monitoring get ingress
kubectl get certificate -n checkmk-monitoring
kubectl get secret checkmk-collector-tls -n checkmk-monitoring
```

Voraussetzung: cert-manager muss installiert sein und der ClusterIssuer `letsencrypt-prod` muss existieren (siehe [ingress/https-letsencrypt-ingress-traefik.md](#)).

Schritt 5: Service Account Token extrahieren

Der ServiceAccount wurde automatisch vom Helm Chart erstellt. Token extrahieren:

```
## Secret-Name finden
kubectl get secrets -n checkmk-monitoring | grep checkmk

## Token extrahieren und dekodieren
kubectl get secret checkmk-checkmk -n checkmk-monitoring -o jsonpath='{.data.token}' | base64 --decode > sa-token
cat sa-token
```

Wichtig: Token speichern - wird fuer CheckMK benoetigt!

Schritt 6: CA-Zertifikat extrahieren

```
## Testen, ob ich so ein Zertifikat sehe
kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' | base64 --decode
```

```
tln1@client-bka:~$ kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' | base64 --decode
```

-----BEGIN CERTIFICATE-----

```
MIIDBTCCAe2gAwIBAgIIEMu/Xm7FNewdQYJKoZIhvcNAQELBQAwFTETMBEGA1UE
AxMKa3ViZXJuZXRlczAeFw0yNjAyMTAxNzIzMDRaFw0zNjAyMDgxNzI4MDRaMBUx
EzARBgNVBAMTCmt1YmVybmV0ZXMwggeiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEK
AoIBAQDFgKpomLYHMxLDLBWXQc/vYNoLH9NAoYd1FBwCKJjQ1NMTQod2wUcSPhTw
ogEC5MdvNjIbJLKIXjiVCbAmUTYxIpjsEFP8wrxDESVoG3TEK0I53Vfzc4pN8l0I
i6bqWmUGxLfOZe7aN/HJiupvlG/oaKf5KrnIpxI6fF1lpNvNeW9elNQwRubh9+2N
x/G+6purVVFbfZ3VlMAqhfkawRubYJgyxYIDNz6aemcrysJTkre2f/G+a4x5JvB
0bgZS1wclaymntQyKpOWNSjVQFAYthsjJWKBi/t24xeTx+mD1o7T08ZTas8f3iQT
XJZp4lStqKsfzcPpBP0qpY6BGdprAgMBAAGjWTBXMA4GA1UdDwEB/wQEAwICpDAP
BgNVHRMBAf8EBTADAQH/MB0GA1UdDgQWBBSntCSEQ2vHWlIRCMaGQ8fefQJDTAV
BgNVHREEDjAMGgprdwJlcm5ldGZvMA0GCSqGSIb3DQEBCwUAA4IBAQ8Bdvg83ue2Y
dYFCg4UY28FLTguS5uqf+x179jmbfkJX86iBws3xOp/1uaRJCHWW8RNJ4iwKqLtf
2g0GPuaPGmAHFRINj4p5jfoqVBjDeo8S63nb9Y0j37Z4Zz1AnTh1V2NUo/cYPxlh
ChmBVpRN93qrYbHM3Asp3AUXj1CYFdq9ARyag2UjJ/KmVHkvJuT0nns64tDrK11N
KTM1Ua9vf0YkWye0wgzaDnFMoY/c+ggP02QcI2GCHQ8g4FTX8C87ofmq0c20tjrg
OPSJbc5YBEICvg7n0FC6n80rAj5KRyH7KGrICUWpbaSAhpigB7olLPhlsYdlEmrr
Grx8nDkGdb61
```

-----END CERTIFICATE-----

```
## Abspeichern
kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' | base64 --decode > k8s-ca.crt
```

CA-Zertifikat anzeigen:

```
cat k8s-ca.crt
## und prüfen ob es das richtige ist:
## muss kubernetes heissen
openssl x509 -in k8s-ca.crt -issuer -noout
```

Schritt 7: Cluster Collector Endpoint ermitteln

Ingress-URL verwenden:

```
kubectl get ingress -n checkmk-monitoring
```

Der Cluster Collector Endpoint ist:

```
https://checkmk-collector.tln<X>.do.t3isp.de
```

Wichtig:

- Ersetze <X> mit deiner Teilnehmer-Nummer
- HTTPS wird durch Let's Encrypt bereitgestellt
- Der Endpoint ist von aussen erreichbar

Testen (sollte Not authenticated zurueckgeben):

```
curl https://checkmk-collector.tln<X>.do.t3isp.de
```

```
{"detail": "Not authenticated"}
```

Schritt 8: CheckMK konfigurieren - Token speichern

1. Öffne CheckMK: <https://tln<X>.do.t3isp.de/>

- ## 7. Activate on selected sites

Schritt 9: CA-Zertifikat in CheckMK importieren

- ## 7. Activate on selected sites

Schritt 10: Piggyback Host erstellen

- ### 5. Activate on selected sites

Wichtig: Der Host bekommt keine IP, da er nur Piggyback-Daten empfängt!

Schritt 11: Ordner für Dynamic Host Management einrichten

- ### 5. Activate on selected sites

Schritt 12: Dynamic Host Management einrichten

- ## 2. Unique ID und Title eintragen

General properties

Unique ID (required)

k8s-connection

Title (required)

k8s connection

Comment

Documentation URL

Configuration activation

☐ Do not apply this configuration

Site

cmk - Local site cmk

3. Bei **Create Hosts in** den gerade erstellten Ordner aus 11 angeben

Connection properties

Connector type Piggyback data ⓘ

☒ Restrict source hosts ⓘ

Sync interval (required)

0 d 0 h 1 min 0 s

Piggyback creation options (required)

⌵ X Create hosts in

k8s-cluster-jmetzger-data ▾

4. Hosts attributes können wir so lassen (es wird nur piggyback data erstellt)
5. Setze einen Haken bei

Delete vanished hosts

☒ Automatically delete hosts without piggyback data

6. (Bei Restrict Source (ganz oben), den erstellten piggyback host auswählen (aus 10.) z.B. k8s-cluster-jmetzger

Piggyback data ⓘ

☒ Restrict source hosts ⓘ

k8s-cluster-jmetzger

7. Service Discovery -> Discover Services During Creation -> Ankreuzen (falls nicht bereits aktiv)

Service discovery

☒ Discover services during creation

8. Save
9. 1 Change anklicken
10. Activate on selected site

Schritt 13: Kubernetes Special Agent konfigurieren

1. Setup > Agents > VM, cloud, container > Kubernetes
2. Klicke **Add rule**
3. Kubernetes cluster configuration:
 - **Cluster name:** k8s-cluster-<dein-name> (oder eigener Name)
 - **Token:** Wähle k8s-token aus Dropdown (erst from Password Store)

Token

From password store ▲ Kub

API Explicit

From password store

https://<control-plane-ip>:443

- **API server endpoint:** https://<DEINE-K8S-API-IP>:6443
 - **SSL certificate verification:** Enabled (mit importiertem CA-Cert)
5. **Collector: Enrich With Usage Data**
 - **Cluster collector endpoint:** https://checkmk-collector.tln<X>.do.t3isp.de (Achtung https vorner ist wichtig)
 6. **Kubernetes API:**
 - **Object selection:** Wähle gewünschte Objekte:
 - ☒ Pods
 - ☒ Nodes
 - ☒ Deployments
 - ☒ DaemonSets
 - ☒ StatefulSets

- ✓ Services
- ✓ Namespaces
- ✓ CronJobs (falls benoetigt)

7. Explicit hosts:

- Waehle `k8s-cluster-<dein-name>` (Host aus Schritt 10)

8. Save

9. Oben rechts auf "1 change" (oder mehr) klicken

10. Activate on selected sites

11. Warten bis Aktivierung abgeschlossen

Schritt 14: Service Discovery durchfuehren

1. Setup > Hosts

2. Suche Host `k8s-cluster-<dein-name>`

3. Klicke auf Host



4. Run service discovery (Das ist eines der Symbole - gelber Kasten)

Erwartete Services:

- `Kubernetes Cluster CPU resources`
- `Kubernetes Cluster Memory resources`
- `Kubernetes Node <node-name>`
- Weitere Services je nach Objekt-Auswahl

5. Nach dem finden der Services dieses übernehmen

Schritt 15: Periodic Service Discovery konfigurieren (optional)

Automatische Discovery fuer neue Services:

1. Setup > Periodic service discovery

2. Add rule

3. Conditions:

- **Host labels:** `cmk/kubernetes:yes`

4. Service discovery:

- **Mode:** "Add unmonitored services and new host labels"
- **Interval:** 15 Minuten (kuerzer als Standard)

5. Rescan

6. Save

Schritt 16: Monitoring testen

Services pruefen:

1. Monitor > All hosts

2. Filter: `cmk/kubernetes:yes`

3. Services pruefen - Status sollte OK sein

Dashboards (nur in kommerziellen Editionen):

- CheckMK RAW hat keine vordefinierten K8s Dashboards
- Manuell Views erstellen moeglich

Troubleshooting

Problem: Keine Services gefunden

Loesung:

```
## Piggyback-Daten pruefen (auf CheckMK Server)
OMD[site]> cmk-piggyback list
OMD[site]> cmk-piggyback show <hostname>
```

Problem: Connection refused zum Cluster Collector

Loesung:

```
## Ingress pruefen
kubectl get ingress -n checkmk-monitoring
kubectl describe ingress checkmk-collector-ingress -n checkmk-monitoring

## Service pruefen
kubectl get svc -n checkmk-monitoring
```

```
## Testen von lokalem Rechner
curl https://checkmk-collector-tln<X>.app.do.t3isp.de/openmetrics

## Zertifikat pruefen (sollte Ready: True sein)
kubectl get certificate -n checkmk-monitoring
```

Problem: Unauthorized bei API-Zugriff

Loesung:

```
## Token-Gueltigkeit pruefen
kubectl get secrets -n checkmk-monitoring

## Neues Token erstellen
kubectl create token checkmk -n checkmk-monitoring --duration=87600h
```

Aufraeumen

Helm Release entfernen:

```
helm uninstall checkmk -n checkmk-monitoring
```

Namespace loeschen:

```
kubectl delete namespace checkmk-monitoring
```

In CheckMK:

1. Hosts loeschen: **Setup > Hosts**
2. Rule loeschen: **Setup > Agents > VM, cloud, container > Kubernetes**
3. Token loeschen: **Setup > General > Passwords**
4. **Activate changes**

Zusammenfassung

Komponente	Status
Cluster Collector	Deployed via Helm (ClusterIP Service)
Ingress	Traefik mit TLS/Let's Encrypt
Node Collector	DaemonSet auf allen Nodes
K8s Special Agent	Konfiguriert in CheckMK
Piggyback Host	Manuell erstellt (RAW)
Service Discovery	Durchgefuehrt
Monitoring	Aktiv via HTTPS

Weiterfuehrende Informationen

Offizielle Dokumentation: https://docs.checkmk.com/latest/en/monitoring_kubernetes.html

Checkmk Raw vs. Enterprise in Bezug auf Kubernetes

Prerequisites

- Use at least version 2.1 of checkmk

Spoiler Alert

- Kubernetes läßt sich in checkmk nur gut monitoren mit der checkmk Enterprise - Version

Was hat die checkmk Enterprise gegenüber der Raw-Version besonderes für Kubernetes

- Spezielle Kubernetes Dashboard, bereits fix und fertig. (Ansonsten sehr viel Arbeit nachzubauen)
- Noch wichtiger: Dynamic Host Management

Dynamic Host Management

- Neues Pods werden automatisch erkannt (auch Services, Deployments usw.)
- Ressourcen, die nicht mehr da sind, werden automatisch rausgelöscht.

Alternative Dynamic Host Management: Ich müsste das händisch machen bzw. selbst scripten

- Ich müsste rest-api call ausführen, um dies über die api von check zu setzen.
- Neue Hosts müßte ich discovern, alte rauslöschen (automatisch)

Alternative: Dashboards

- Dashboards selber bauen
- Grafana mit checkmk - Daten (Aus meiner Sicht wenig praktikabel, dann entweder ganz Grafana + Prometheus oder checkmk Enterprise)

Kubernetes Dashboards

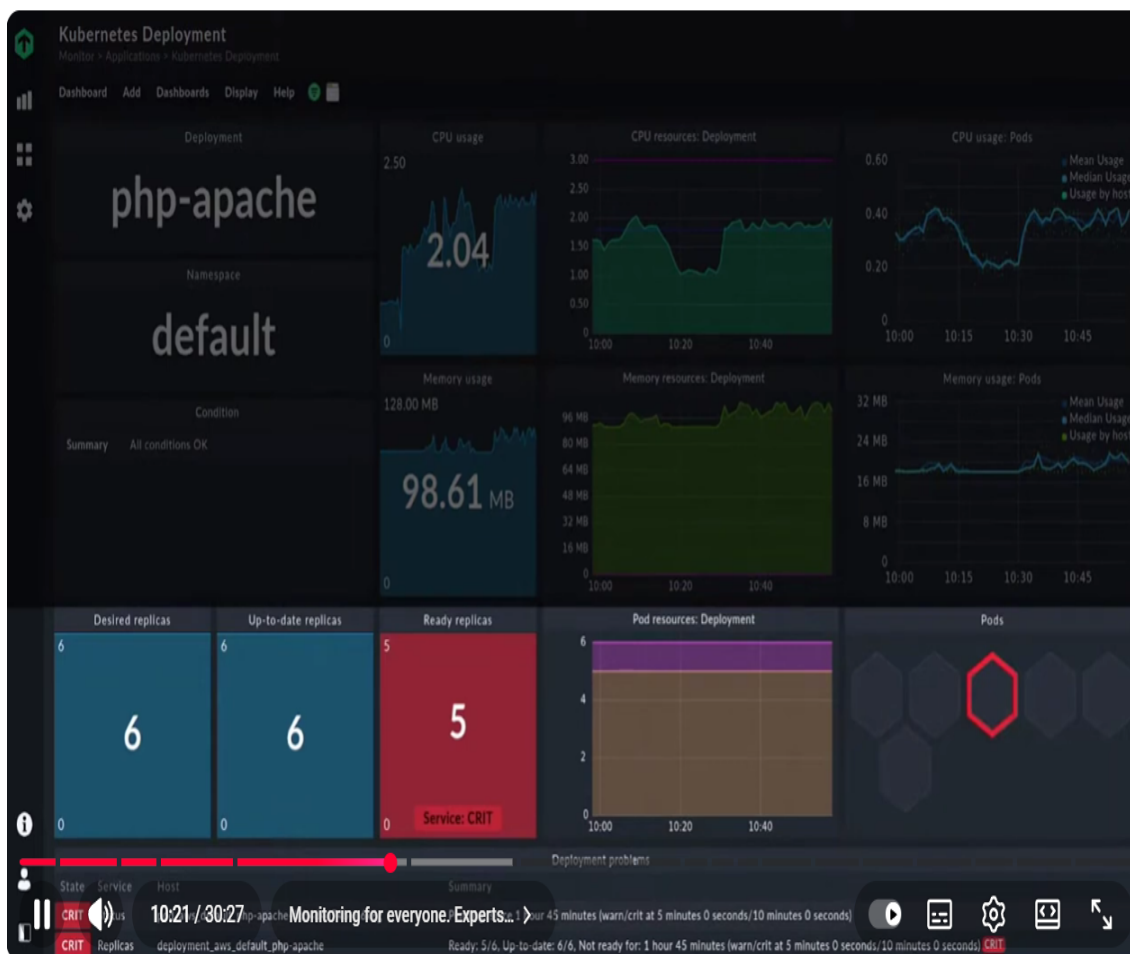
- These dashboards are only available in Checkmk Enterprise (so not in community version: CheckMk Raw)

- But, you could also create them yourself (not recommended)
 - Ref: <https://docs.checkmk.com/latest/en/dashboards.html>

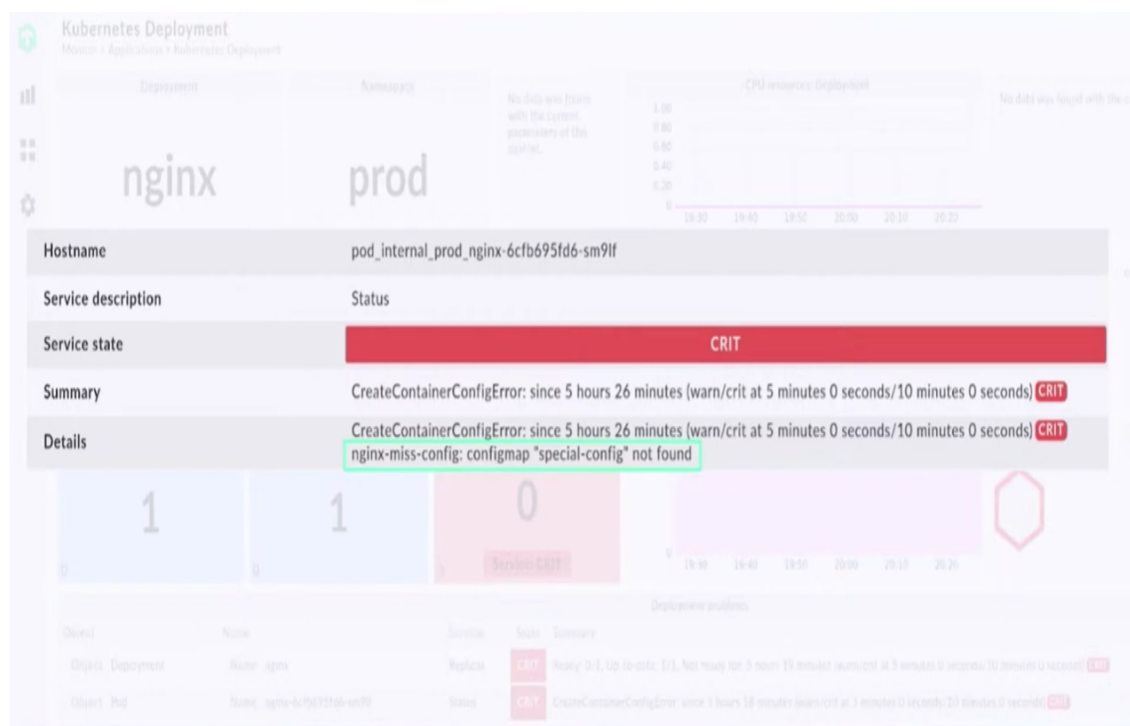
Cluster Dashboard



Deployment Dashboard



Detail



References:

- [Checkmk about new Kubernetes Integration \(since 2022\)](#).

Prometheus-Metriken in Checkmk integrieren

Warum integrieren?

Checkmk und Prometheus haben unterschiedliche Staerken und ergaenzen sich:

	Checkmk	Prometheus
Kernstaerke	Infrastructure-Checks, Service-Status, Alerting	Metriken, Time-Series, Dashboards
Kubernetes	Host/Service-Inventur, Piggyback-Objekte	CPU/Memory/Request-Rate, HPA
Alerting	Zentrale Notification-Engine	Alertmanager (eher technical)

Teil 1: Prometheus-Metriken in Checkmk einbinden

Moeglichkeit 1: Checkmk Prometheus Special Agent (empfohlen)

Checkmk bringt einen eingebauten Special Agent mit, der direkt gegen die Prometheus HTTP-API scraped und daraus native Checkmk-Checks erzeugt.

```
Setup -> Special Agents -> Prometheus -> Add rule
```

Parameter:

```
URL:          http://prometheus:9090
Auth:         optional (Basic/Bearer)
PromQL-Queries:
- Service name: "pod_cpu_usage"
  PromQL:      rate(container_cpu_usage_seconds_total[5m])
  Unit:        1/s
  Levels:      warn=0.8, crit=0.9
```

Intern ruft Checkmk folgende Prometheus-Endpunkte ab:

```
GET /api/v1/query?query=<promql>
GET /api/v1/targets
```

Checkmk erzeugt aus jeder PromQL-Query einen eigenen Check-Service mit OK/WARN/CRIT-Status, Performance-Daten (Graphen) und Einbindung in Checkmk-Notifications.

Geeignet fuer: Bestehende Prometheus-Infrastruktur, bei der Checkmk als zentrale Alerting-Plattform genutzt werden soll.

Moeglichkeit 2: OpenMetrics direkt scrapen (ohne Prometheus-Server)

Checkmk kann Prometheus-Exposition-Format direkt von Exportern lesen — ohne Prometheus-Server als Mittelstufe.

```
Setup -> Services -> OpenMetrics Exporter -> Add rule
```

```
URL:          http://node-exporter:9100/metrics
Metriken:
- node_cpu_seconds_total
- node_memory_MemAvailable_bytes
```

Vorteil: Kein Prometheus noetig — Checkmk scraped Exporter direkt.

Nachteil: Kein PromQL, keine Aggregation ueber mehrere Targets.

Moeglichkeit 3: Prometheus Alertmanager -> Checkmk Event Console

Prometheus-Alerts werden in die Checkmk Event Console weitergeleitet — Checkmk uebernimmt dann Notification und Eskalation.

```
alertmanager.yml:

receivers:
- name: checkmk
  webhook_configs:
  - url: http://checkmk/cm/api/1.0/domain-types/event_console/actions/send_event/invoke
    http_config:
      bearer_token: <checkmk-api-token>
```

Checkmk-Seite:

```
Setup -> Event Console -> Rules -> New rule
Match: source "alertmanager"
Action: Create alert / Notification
```

Geeignet fuer: Teams, die Prometheus-Alerting behalten wollen, aber Checkmk fuer zentrale Notifications (PagerDuty, Mail, Slack) nutzen.

Moeglichkeit 4: Checkmk Enterprise — natives Kubernetes-Monitoring

In Checkmk Enterprise/Cloud gibt es keinen Umweg ueber Prometheus — Checkmk liest direkt aus der Kubernetes-API und einem eigenen Cluster Collector:

```
Kubernetes API (:6443)          -> Checkmk Special Agent
Cluster Collector (/openmetrics) -> Checkmk Special Agent
```

Der Cluster Collector (Helm-Deployment im Cluster) stellt einen OpenMetrics-Endpoint bereit, den Checkmk selbst scraped. Kein Prometheus involviert.

```
helm repo add checkmk https://checkmk.github.io/checkmk_kube_agent
helm install checkmk-cluster-collector checkmk/checkmk \
  --set clusterCollector.enabled=true \
  --set nodeCollector.enabled=true
```

Teil 2: Anwendungsseitige Metriken einbinden

Fuer Metriken aus eigenen Applikationen, die nicht ueber den Cluster Collector kommen, gibt es folgende Wege:

Weg 1: OpenMetrics Special Agent — Checkmk scraped /metrics direkt

Wenn die App einen Prometheus-Exposition-Endpoint hat (/metrics), kann Checkmk diesen ohne Prometheus-Server direkt abfragen.

Setup -> Services -> Add rule -> OpenMetrics Exporter

```
URL:          http://<service-name>.<namespace>:8080/metrics
Metriken:
- http_requests_total
- orders_processed_total
- active_users
Scrape-Interval: 60s
```

In Kubernetes erreicht Checkmk den App-Service ueber den cluster-internen DNS oder einen Ingress.

Einschraenkung: Keine PromQL-Aggregation — Checkmk verarbeitet die Metriken 1:1.

Weg 2: Checkmk REST API — App pusht Metriken aktiv

Die App oder ein Sidecar/Adapter pusht Metriken via Checkmk REST API (Push-Modell statt Pull).

```
curl -X PUT \
  "https://checkmk/cm/api/1.0/domain-types/metric/collections/all" \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{
    "host_name": "my-app-pod",
    "service_name": "orders_processed",
    "metrics": [
      { "name": "orders_total", "value": 4200, "unit": "count" }
    ]
  }'
```

Geeignet fuer: Apps ohne Pull-Endpoint, z.B. nach einem Batch-Job oder einer asynchronen Verarbeitung.

Weg 3: Custom Active Check Plugin — Checkmk fragt App-API ab

Ein eigenes Plugin auf dem Checkmk-Server fragt die App direkt ab (REST-API, proprietäeres Format) und gibt das Ergebnis als Checkmk-Check aus.

/omd/sites/<site>/lib/nagios/plugins/check_myapp_metrics :

```
#!/usr/bin/env python3
import requests, sys

resp = requests.get("http://my-app:8080/api/metrics")
data = resp.json()

orders = data["orders_per_minute"]
if orders < 10:
    print(f"CRIT - Orders/min: {orders} | orders={orders}")
    sys.exit(2)
elif orders < 50:
    print(f"WARN - Orders/min: {orders} | orders={orders}")
    sys.exit(1)
else:
    print(f"OK - Orders/min: {orders} | orders={orders}")
    sys.exit(0)
```

```
Setup -> Integrations -> Monitoring Agents -> Add rule -> Active checks (Custom)
Command: check_myapp_metrics --host $HOSTNAME$
```

Geeignet fuer: Apps mit eigenem API-Format oder komplexer Logik (mehrere Endpunkte kombinieren, Schwellwerte berechnen).

Weg 4: Piggyback — CronJob im Cluster erzeugt Checkmk-Daten

Ein Kubernetes CronJob sammelt Metriken aus mehreren Pods/Apps und schickt sie als Piggyback-Sektion an Checkmk. Jeder Pod erscheint als eigener virtueller Host.

```
## CronJob-Script: Metriken -> Checkmk-Piggyback-Format -> REST API push
piggyback_data = ""
<<<my-app-pod-1>>>
<<<local>>>
0 orders_processed orders=4200;1000;500 Orders processed: 4200
<<<>>>
""
requests.put("https://checkmk/.../piggyback", data=piggyback_data, ...)
```

Geeignet fuer: Dynamische Pod-Landschaften, wo fuer jeden Pod ein eigener virtueller Host in Checkmk erscheinen soll.

Empfehlung je nach Szenario

Architektur-Entscheidung

Szenario A: Nur Checkmk (Enterprise)

```
K8s API + Cluster Collector -> Checkmk Special Agent
-> Checkmk hat Inventory, Checks, Notifications
-> Prometheus nicht noetig
```

Szenario B: Nur Prometheus/Grafana

```
kube-state-metrics + node-exporter -> Prometheus -> Grafana
-> Optimal fuer Dashboards, HPA, Custom-Metriken
-> Kein zentrales IT-Monitoring
```

Szenario C: Hybrid (empfohlen fuer Enterprise-Umgebungen)

```
Prometheus/Grafana -> Metriken, Dashboards, HPA
Checkmk -> Service-Status, IT-Monitoring, Notifications
Alertmanager -> leitet kritische Alerts -> Checkmk Event Console
```

Welches Tool fuer welche Frage?

Frage	Tool
Ist mein Pod/Service up?	Checkmk
Wie ist die Request-Rate?	Prometheus
CPU/Memory-Trend ueber 30 Tage?	Prometheus / Grafana
Wer bekommt den Alert um 3 Uhr?	Checkmk (Notification Routing)
HPA skalieren nach Custom-Metrik?	Prometheus (KEDA/HPA)

Anwendungsmetriken: Welcher Weg?

Situation	Empfehlung
App hat <code>/metrics</code> (Prometheus-Format)	Weg 1 — OpenMetrics Special Agent
App hat eigene REST-API	Weg 3 — Custom Active Check Plugin
App hat kein Pull-Endpoint (Batch, Jobs)	Weg 2 — REST API Push nach Job-Ende
Viele dynamische Pods, einzeln sichtbar	Weg 4 — Piggyback via CronJob
Prometheus bereits vorhanden	Prometheus Special Agent mit PromQL (Teil 1, Moeglichkeit 1)

Uebung: Kubernetes API Health als HTTP Active Check

Hintergrund

Jeder Kubernetes-Cluster stellt drei Health-Endpunkte bereit, die **ohne Authentifizierung** erreichbar sind:

Endpunkt	Inhalt	Antwort
<code>/healthz</code>	Allgemeiner Cluster-Health	ok
<code>/livez</code>	API-Server am Leben?	ok
<code>/readyz</code>	API-Server bereit?	ok

Diese Endpunkte sind in **jedem vanilla Kubernetes-Cluster** immer vorhanden – kein Prometheus, kein extra Deployment noetig.

Ziel: Den `/healthz` -Endpoint als **HTTP Active Check** in Checkmk einrichten. Checkmk misst dann die Antwortzeit als Metrik und schlaegt Alarm, wenn der Endpoint nicht mehr erreichbar ist.

Active Check vs. Passive Check:

Typ	Wo laeuft er?	Typische Plugins
Passive Check	Auf dem ueberwachten Host	CPU, RAM, Disk
Active Check	Auf dem Checkmk-Server	check_httpv2, check_tcp

Schritt 1: API-Server URL ermitteln

```
kubectl cluster-info
```

Erwartete Ausgabe:

```
Kubernetes control plane is running at https://<uuid>.k8s.ondigitalocean.com
```

Die URL notieren (ohne `/` am Ende) – sie wird in Schritt 3 benoetigt.

Schritt 2: Endpunkt manuell testen

Vom Checkmk-Server aus (z.B. via SSH oder im Docker-Container):

```
curl -sk https://<uuid>.k8s.ondigitalocean.com/healthz
```

Erwartete Ausgabe:

```
ok
```

HTTP 200 + `ok` = API-Server gesund. Der Parameter `-k` ist noetig, weil DigitalOcean Managed Kubernetes ein eigenes CA-Zertifikat verwendet.

Schritt 3: Active Check in Checkmk einrichten (GUI)

- 1. **Setup** aufrufen → oben in der Suchleiste `HTT`P eingetippen
- 2. **"Check HTTP web service"** auswaehlen
- 3. **"Add rule"** klicken
- 4. Einstellungen befuellen:

Abschnitt "General properties":

- Description: `Kubernetes API Health Check`

Abschnitt "Check HTTP web service" → "HTTP web service endpoints to monitor" → Add entry :

- **Service name → Name:** `Kubernetes API Health`
- **Service name → Prefix:** Use `"HTTP(S)"` as service name prefix (Standard)
- **URL:** `https://<uuid>.k8s.ondigitalocean.com/healthz`

Weiter unten im gleichen Entry-Bereich oder unter **"Standard settings for all endpoints"**:

- **Certificate validity:** `Ignore certificate` auswaehlen

Optional (fuer Inhaltspruefung):

- **Search for strings → Search in body:** `ok` eintragen

5. Abschnitt "Conditions":

- **Explicit hosts:** `k8s-cluster-<dein-name>` eintragen (z.B. `k8s-cluster-tln1` fuer Teilnehmer 1)

- 6. **Save** klicken

Schritt 4: Changes aktivieren und Service Discovery

```
## Im Checkmk-Menue oben rechts:
## "Activate pending changes" klicken (gelbes Banner)
```

Danach unter **Monitoring** → **Hosts** → **k8s-cluster-**:

- **"Service discovery"** aufrufen
- Den neuen Service `HTTPS Kubernetes API Health` auf **"Monitor"** setzen
- Erneut **"Activate pending changes"**

Ergebnis

Nach der Aktivierung erscheint der Service in der Host-Ansicht:

```
HTTPS Kubernetes API Health   OK   Version: HTTP/2.0, Status: 200 OK
```


Die **Metriken** des Services:

Metrik	Beispielwert	Bedeutung
response_time	0.031s	Gesamte Antwortzeit
time_http_headers	0.031s	Zeit bis Header empfangen
time_http_body	0.0001s	Zeit fuer Body-Uebertragung
response_size	2 B	Groesse der Antwort (ok = 2 Bytes)

Auf den Service klicken → **"Service metrics"** zeigt die Antwortzeit als Zeitreihen-Graph.

Schwellenwerte konfigurieren (optional)

In der gleichen Regel unter **"Standard settings for all endpoints"**:

- **Response time** → Warn: z.B. 1.0 s
- **Response time** → Crit: z.B. 5.0 s

Was passiert bei einem Fehler?

Situation	Checkmk-Status
HTTP 200 + Body ok	OK (gruen)
HTTP 200 + Body passt nicht	CRIT (rot)
HTTP != 200	CRIT (rot)
Timeout / nicht erreichbar	CRIT (rot)

Aufraeumen

Die Regel wieder loeschen:

Setup → **Check HTTP web service** → Regel loeschen → Activate changes

Kubernetes Perfomance

Heap analyse

```
4. Heap memory analyse

## JVM-Beispiel: Direkt im Container
kubectl exec -it <pod> -- jmap -heap <pid>
kubectl exec -it <pod> -- jstat -gc <pid> 1000

## Oder Metriken exportieren
## - JMX Exporter für JVM
## - Prometheus + Grafana für Visualisierung
## - /actuator/metrics (Spring Boot)

## zu viel speicher
kubectl top pods

## OOM killer
kubectl describe po <pod-name>

## Crashed
## OOMKilled Pods finden
kubectl get pods -A -o json | \
jq '.items[] | select(.status.containerStatuses[].lastState.terminated.reason=="OOMKilled")'

## CPU-Throttling Events
kubectl get events -A --field-selector reason=FailedScheduling

## Pods ohne Limits (gefährlich!)
kubectl get pods -A -o json | \
jq '.items[] | select(.spec.containers[].resources.limits==null)'
```

Kubernetes QoS / HealthChecks / Live / Readiness

Quality of Service - evict pods

Die Class wird auf Basis der Limits und Requests der Container vergeben

```
Request: Definiert wieviel ein Container mindestens braucht (CPU,memory)
Limit: Definiert, was ein Container maximal braucht.
```

```
in spec.containers.resources
kubectl explain pod.spec.containers.resources
```

Art der Typen:

- Guaranteed
- Burstable
- BestEffort

Wie werden die Pods evicted

- Das wird in der folgenden Reihenfolge gemacht: Zu erst alle BestEffort, dann burstable und zum Schluss Guaranteed

Guaranteed

```
Type: Guaranteed:
https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod/#create-a-pod-that-gets-assigned-a-qos-class-of-guaranteed

set when limit equals request
(request: das braucht er,
limit: das braucht er maximal)

Garantied ist die höchste Stufe und diese werden bei fehlenden Ressourcen
als letztes "evicted"
```

Guaranteed Exercise

```
cd
mkdir -p manifests
cd manifests
mkdir qos
cd qos
nano 01-pod.yaml
```

```
apiVersion: v1

kind: Pod
metadata:
  name: qos-demo
spec:
  containers:
  - name: qos-demo-ctr
    image: nginx
    resources:
      limits:
        memory: "200Mi"
        cpu: "700m"

      requests:
        memory: "200Mi"
        cpu: "700m"
```

```
kubectl apply -f .
kubectl describe po qos-demo
```

Risiko Guaranteed

- Limit: CPU: Diese wird maximal zur Verfügung gestellt
- Limit: Memory: Wenn die Anwendung das Limit überschreitet, greift der OOM-Killer (Out of Memory Killer)
- Wenn Limit Memory: Dann auch dafür sorgen, dass das laufende Programme selbst auch eine Speichergrenze
 - Java-Programm ohne Speichergrenze oder zu hoher Speichergrenze

Burstable

- At least one Container in the Pod has a memory or CPU request or limit

```
cd
mkdir -p manifests
cd manifests
mkdir burstable
cd burstable
```

```
nano 01-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
```

```

  name: qos-burstable
spec:
  containers:
  - name: qos-demo-2-ctr
    image: nginx
    resources:
      limits:
        memory: "200Mi"
      requests:
        memory: "100Mi"

```

```

kubectl apply -f .
kubectl describe po qos-burstable

```

BestEffort

- gar keine Limits und Requests gesetzt (bitte nicht machen)

LiveNess/Readiness - Probe / HealthChecks

Übung 1: Liveness (command)

```

What does it do ?

* At the beginning pod is ready (first 30 seconds)
* Check will be done after 5 seconds of pod being startet
* Check will be done periodically every 5 seconds and will check
  * for /tmp/healthy
  * if file is there will return: 0
  * if file is not there will return: 1
* After 30 seconds container will be killed
* After 35 seconds container will be restarted

```

```

cd
mkdir -p manifests/probes
cd manifests/probes
nano 01-pod-liveness-command.yml

```

```

apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-exec
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/busybox
    args:
    - /bin/sh
    - -c
    - touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600
  livenessProbe:
    exec:
      command:
      - cat
      - /tmp/healthy
    initialDelaySeconds: 5
    periodSeconds: 5

```

```

## apply and test
kubectl apply -f .
kubectl describe -l test=liveness pods | grep -A 40 Events
sleep 30
kubectl describe -l test=liveness pods | grep -A 40 Events
sleep 5
kubectl describe -l test=liveness pods | grep -A 40 Events

```

```

## cleanup
kubectl delete -f .

```

Übung 2: Liveness Probe (HTTP)

```
## Step 0: Understanding Prerequisite:
This is how this image works:
## after 10 seconds it returns code 500
http.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
    duration := time.Now().Sub(started)
    if duration.Seconds() > 10 {
        w.WriteHeader(500)
        w.Write([]byte(fmt.Sprintf("error: %v", duration.Seconds())))
    } else {
        w.WriteHeader(200)
        w.Write([]byte("ok"))
    }
})
```

```
## Step 1: Pod - manifest
## vi 02-pod-liveness-http.yml
## status-code >=200 and < 400 o.k.
## else failure
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/liveness
    args:
    - /server
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
        httpHeaders:
        - name: Custom-Header
          value: Awesome
      initialDelaySeconds: 3
      periodSeconds: 3
```

```
## Step 2: apply and test
kubectl apply -f 02-pod-liveness-http.yml
## after 10 seconds port should have been started
sleep 10
kubectl describe pod liveness-http
```

Reference:

- <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

Uebung: Readiness Probe mit HTTP

Hintergrund

Die Readiness Probe entscheidet, ob ein Pod **Traffic empfangen darf**.

Solange die Probe fehlschlaegt, entfernt Kubernetes den Pod aus den Service-Endpoints.

Probe	Was passiert bei Fehler?	Pod wird neu gestartet?
Readiness	Pod bekommt keinen Traffic	Nein
Liveness	Pod wird neu gestartet	Ja

In dieser Uebung prueft die Readiness Probe per HTTP ob die Datei `/ready` im nginx-Webroot existiert. Ist sie nicht da → 404 → Pod NotReady → kein Traffic.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 03c-readiness-probe
cd 03c-readiness-probe
```

Schritt 2: Deployment erstellen

```
nano 01-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-readiness
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-readiness
  template:
    metadata:
      labels:
        app: nginx-readiness
    spec:
      securityContext:
        fsGroup: 101
      containers:
        - name: nginx
          image: nginxinc/nginx-unprivileged:1.28
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 2
          volumeMounts:
            - name: webroot
              mountPath: /usr/share/nginx/html
      volumes:
        - name: webroot
          emptyDir: {}
```

Schritt 3: Service erstellen

```
nano 02-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-readiness-svc
spec:
  selector:
    app: nginx-readiness
  ports:
    - port: 80
      targetPort: 8080
```

Schritt 4: Deployen

```
kubectl create namespace readiness-<dein-name>
kubectl apply -f . -n readiness-<dein-name>
```

Schritt 5: Pods beobachten — alle sind 0/1

```
kubectl get pods -n readiness-<dein-name>
```

Erwartete Ausgabe:

NAME	READY	STATUS	RESTARTS	AGE
nginx-readiness-95cc7554d-4kvsg	0/1	Running	0	12s
nginx-readiness-95cc7554d-89zr7	0/1	Running	0	12s
nginx-readiness-95cc7554d-8mrf7	0/1	Running	0	12s

Die Pods laufen, aber sind **nicht ready** — die HTTP-Probe bekommt noch 404.

Schritt 6: Probe-Fehler ansehen

```
kubectl describe pod -l app=nginx-readiness -n readiness-<dein-name>
```

In den Events steht:

```
Warning   Unhealthy   kubelet   Readiness probe failed: HTTP probe failed with statuscode: 404
```

Und in den Conditions:

```
Ready           False
ContainersReady False
```

Schritt 7: Service hat keine Endpoints

```
kubectl get endpoints nginx-readiness-svc -n readiness-<dein-name>
```

Erwartete Ausgabe:

```
NAME                ENDPOINTS   AGE
nginx-readiness-svc                30s
```

Kein Pod bekommt Traffic — das ist das Wirkprinzip der Readiness Probe.

Schritt 8: Einen Pod "bereit machen"

Einen Pod-Namen merken:

```
kubectl get pods -n readiness-<dein-name>
```

Die Datei `/ready` im Webroot anlegen — nginx antwortet jetzt mit 200:

```
kubectl exec -n readiness-<dein-name> <pod-name> -- sh -c 'echo ok > /usr/share/nginx/html/ready'
```

Nach ~10 Sekunden prueft die Probe erneut:

```
kubectl get pods -n readiness-<dein-name>
```

Erwartete Ausgabe:

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-readiness-95cc7554d-4kvsg    1/1     Running   0           45s
nginx-readiness-95cc7554d-89zr7    0/1     Running   0           45s
nginx-readiness-95cc7554d-8mrf7    0/1     Running   0           45s
```

Und der Service hat jetzt genau einen Endpoint:

```
kubectl get endpoints nginx-readiness-svc -n readiness-<dein-name>
```

```
NAME                ENDPOINTS                     AGE
nginx-readiness-svc  10.108.0.240:8080            46s
```

Schritt 9: Alle Pods ready machen

Fuer die anderen zwei Pods ebenfalls die Datei anlegen:

```
kubectl exec -n readiness-<dein-name> <pod-name-2> -- sh -c 'echo ok > /usr/share/nginx/html/ready'
kubectl exec -n readiness-<dein-name> <pod-name-3> -- sh -c 'echo ok > /usr/share/nginx/html/ready'
```

```
kubectl get pods -n readiness-<dein-name>
kubectl get endpoints nginx-readiness-svc -n readiness-<dein-name>
```

Erwartete Ausgabe:

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-readiness-95cc7554d-4kvsg    1/1     Running   0           70s
nginx-readiness-95cc7554d-89zr7    1/1     Running   0           70s
nginx-readiness-95cc7554d-8mrf7    1/1     Running   0           70s

NAME                ENDPOINTS                                     AGE
nginx-readiness-svc  10.108.0.22:8080,10.108.0.240:8080,10.108.1.48:8080  71s
```

Alle 3 Pods sind ready — alle 3 sind im Service.

Schritt 10: Pod waehrend Betrieb "krank machen"

Datei aus einem laufenden Pod entfernen:

```
kubectl exec -n readiness-<dein-name> <pod-name> -- rm /usr/share/nginx/html/ready
```

Nach ~10 Sekunden:

```
kubectl get pods -n readiness-<dein-name>
kubectl get endpoints nginx-readiness-svc -n readiness-<dein-name>
```

Was passiert:

NAME	READY	STATUS	RESTARTS	AGE	
nginx-readiness-95cc7554d-4kvsg	0/1	Running	0	90s	<- NotReady
nginx-readiness-95cc7554d-89zr7	1/1	Running	0	90s	
nginx-readiness-95cc7554d-8mrf7	1/1	Running	0	90s	

NAME	ENDPOINTS	AGE
nginx-readiness-svc	10.108.0.22:8080,10.108.1.48:8080	91s

Wichtig: RESTARTS: 0 — der Pod wird **nicht neu gestartet**.

Er bekommt nur keinen Traffic mehr, laeuft aber weiter.

Aufraeumen

```
kubectl delete namespace readiness-<dein-name>
```

Zusammenfassung

Was passiert	Ergebnis
Probe schlaegt fehl (404)	Pod = NotReady, kein Endpoint im Service
Probe erfolgreich (200)	Pod = Ready, Endpoint im Service
Probe schlaegt wieder fehl	Pod aus Endpoints entfernt, kein Neustart
Service	Routet Traffic nur zu Ready-Pods

Die Readiness Probe schuetzt andere Pods und Nutzer davor, Anfragen an einen Pod zu senden, der noch nicht (oder nicht mehr) bereit ist sie zu verarbeiten.

Taints / Tolerations

Taints

Taints schliessen auf einer Node alle Pods aus, die nicht bestimmte tolerations haben:

Möglichkeiten:

- o Sie werden nicht gescheduled - NoSchedule
- o Sie werden nicht executed - NoExecute
- o Sie werden möglichst nicht gescheduled. - PreferNoSchedule

Tolerations

Tolerations werden auf Pod-Ebene vergeben:
tolerations:

Ein Pod kann (wenn es auf einem Node taints gibt), nur gescheduled bzw. ausgeführt werden, wenn er die Labels hat, die auch als Taints auf dem Node vergeben sind.

Walkthrough

Step 1: Cordon the other nodes - scheduling will not be possible there

```
## Cordon nodes n11 and n111
## You will see a taint here
kubectl cordon n11
kubectl cordon n111
kubectl describe n111 | grep -i taint
```

Step 2: Set taint on first node

```
kubectl taint nodes n1 gpu=true:NoSchedule
```

Step 3

```
cd
mkdir -p manifests
cd manifests
```

```
mkdir tainttest
cd tainttest
nano 01-no-tolerations.yml
```

```
##vi 01-no-tolerations.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-test-no-tol
  labels:
    env: test-env
spec:
  containers:
  - name: nginx
    image: nginx:1.21
```

```
kubectl apply -f .
kubectl get po nginx-test-no-tol
kubectl get describe nginx-test-no-tol
```

Step 4:

```
## vi 02-nginx-test-wrong-tol.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-test-wrong-tol
  labels:
    env: test-env
spec:
  containers:
  - name: nginx
    image: nginx:latest
  tolerations:
  - key: "cpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
```

```
kubectl apply -f .
kubectl get po nginx-test-wrong-tol
kubectl describe po nginx-test-wrong-tol
```

Step 5:

```
## vi 03-good-tolerations.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-test-good-tol
  labels:
    env: test-env
spec:
  containers:
  - name: nginx
    image: nginx:latest
  tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
```

```
kubectl apply -f .
kubectl get po nginx-test-good-tol
kubectl describe po nginx-test-good-tol
```

Taints rausnehmen

```
kubectl taint nodes n1 gpu:true:NoSchedule-
```

uncordon other nodes

```
kubectl uncordon n11
kubectl uncordon n111
```


References

- [Doku Kubernetes Taints and Tolerations](#)
- <https://blog.kubecost.com/blog/kubernetes-taints/>

Installation mit microk8s

Schritt 1: auf 3 Maschinen mit Ubuntu 24.04LTS

Walkthrough

```
sudo snap install microk8s --classic
## Important enable dns // otherwise not dns lookup is possible
microk8s enable dns
microk8s status

## Execute kubectl commands like so
microk8s kubectl
microk8s kubectl cluster-info

## Make it easier with an alias
echo "alias kubectl='microk8s kubectl'" >> ~/.bashrc
source ~/.bashrc
kubectl
```

Working with snaps

```
snap info microk8s
```

Ref:

- <https://microk8s.io/docs/setting-snap-channel>

Schritt 2: cluster - node2 + node3 einbinden - master ist node 1

Walkthrough

```
## auf master (jeweils für jedes node neu ausführen)
microk8s add-node

## dann auf jeweiligem node vorigen Befehl der ausgegeben wurde ausführen
## Kann mehr als 60 sekunden dauern ! Geduld...Geduld..Geduld
##z.B. -> ACHTUNG evtl. IP ändern
microk8s join 10.128.63.86:25000/567a21bdfc9a64738ef4b3286b2b8a69
```

Auf einem Node addon aktivieren z.B. ingress

```
gucken, ob es auf dem anderen node auch aktiv ist.
```

Add Node only as Worker-Node

```
microk8s join 10.135.0.15:25000/5857843e774c2ebe368e14e8b95bdf80/9bf3ceb70a58 --worker
Contacting cluster at 10.135.0.15

root@n41:~# microk8s status
This MicroK8s deployment is acting as a node in a cluster.
Please use the master node.
```

Ref:

- <https://microk8s.io/docs/high-availability>

Schritt 3: Remote Verbindung einrichten

Walkthrough

```
## auf master (jeweils für jedes node neu ausführen)
microk8s add-node

## dann auf jeweiligem node vorigen Befehl der ausgegeben wurde ausführen
## Kann mehr als 60 sekunden dauern ! Geduld...Geduld..Geduld
##z.B. -> ACHTUNG evtl. IP ändern
microk8s join 10.128.63.86:25000/567a21bdfc9a64738ef4b3286b2b8a69
```

Auf einem Node addon aktivieren z.B. ingress

```
gucken, ob es auf dem anderen node auch aktiv ist.
```

Add Node only as Worker-Node

```
microk8s join 10.135.0.15:25000/5857843e774c2ebe368e14e8b95bdf80/9bf3ceb70a58 --worker
Contacting cluster at 10.135.0.15

root@n41:~# microk8s status
This MicroK8s deployment is acting as a node in a cluster.
Please use the master node.
```

Ref:

- <https://microk8s.io/docs/high-availability>

Installation mit kubeadm

Schritt für Schritt mit kubeadm

Version

- Ubuntu 20.04 LTS
- 1 control plane und 3 worker nodes

Done for you (for 4 Servers)

- Servers are setup:
 - ssh-running
 - kubeadm, kubelet, kubectl installed
 - containerd - runtime installed
- Installed on all nodes (with cloud-init)

```
#!/bin/bash

groupadd sshadmin
USERS="mysupersecretuser"
SUDO_USER="mysupersecretuser"
PASS="yoursupersecretpass"
for USER in $USERS
do
    echo "Adding user $USER"
    useradd -s /bin/bash --create-home $USER
    usermod -aG sshadmin $USER
    echo "$USER:$PASS" | chpasswd
done

## We can sudo with $SUDO_USER
usermod -aG sudo $SUDO_USER

## 20.04 and 22.04 this will be in the subfolder
if [ -f /etc/ssh/sshd_config.d/50-cloud-init.conf ]
then
    sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config.d/50-cloud-init.conf
fi

### both is needed
sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config

usermod -aG sshadmin root

## TBD - Delete AllowUsers Entries with sed
## otherwise we cannot login by group

echo "AllowGroups sshadmin" >> /etc/ssh/sshd_config
systemctl reload sshd

## Now let us do some generic setup
echo "Installing kubeadm kubelet kubectl"

#### A lot of stuff needs to be done here
#### https://www.linuxtechy.com/install-kubernetes-on-ubuntu-22-04/

## 1. no swap please
swapoff -a
sudo sed -i 's/ swap / s/^(.*)$/#\1/g' /etc/fstab

## 2. Loading necessary modules
echo "overlay" >> /etc/modules-load.d/containerd.conf
echo "br_netfilter" >> /etc/modules-load.d/containerd.conf
```

```

modprobe overlay
modprobe br_netfilter

## 3. necessary kernel settings
echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.d/kubernetes.conf
sysctl --system

## 4. Update the meta-information
apt-get -y update

## 5. Installing container runtime
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.add-apt-repository
"deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"      apt-get install -y containerd.io

## 6. Configure containerd
containerd config default > /etc/containerd/config.toml
sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml
systemctl restart containerd
systemctl enable containerd

## 7. Add Kubernetes Repository for Kubernetes
mkdir -m 755 /etc/apt/keyrings
apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/$K8S_VERSION/deb/Release.key | gpg --dearmor -o /etc/apt/keyrings/echo "deb [signed-
by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/$K8S_VERSI
# 8. Install kubect1 kubeadm kubect1
apt-get -y update
apt-get install -y kubelet kubeadm kubect1
apt-mark hold -y kubelet kubeadm kubect1

## 9. Install helm
snap install helm --classic

## Installing nfs-common
apt-get -y install nfs-common

```

Prerequisites

- 4 Servers setup and reachable through ssh.
- user: 11trainingdo
- pass: PLEASE ask your instructor

```

## Important - Servers are not reachable through
## Domain !! Only IP.
controlplane.tln<nr>.t3isp.de
worker1.tln<nr>.do.t3isp.de
worker2.tln<nr>.do.t3isp.de
worker3.tln<nr>.do.t3isp.de

```

Step 1: Setup controlnode (login through ssh)

```

## This CIDR is the recommendation for calico
## Other CNI's might be different
CLUSTER_CIDR="192.168.0.0/16"

kubeadm init --pod-network-cidr=$CLUSTER_CIDR && \
mkdir -p /root/.kube && \
cp -i /etc/kubernetes/admin.conf /root/.kube/config && \
chown $(id -u):$(id -g) /root/.kube/config && \
cp -i /root/.kube/config /tmp/config.kubeadm && \
chmod o+r /tmp/config.kubeadm

## Copy output of join (needed for workers)
## e.g.
kubeadm join 159.89.99.35:6443 --token rpylp0.rdphpzbavdyx3llz \
--discovery-token-ca-cert-hash sha256:05d42f2c051a974a27577270e09c77602eeec85523b1815378b815b64cb99932

```

Step 2: Setup worker1 - node (login through ssh)

```

## use join command from Step 1:
kubeadm join 159.89.99.35:6443 --token rpylp0.rdphpzbavdyx3llz \
--discovery-token-ca-cert-hash sha256:05d42f2c051a974a27577270e09c77602eeec85523b1815378b815b64cb99932

```

Step 3: Setup worker2 - node (login through ssh)

```
## use join command from Step 1:
kubeadm join 159.89.99.35:6443 --token rpylp0.rdphpzbavdyx3llz \
  --discovery-token-ca-cert-hash sha256:05d42f2c051a974a27577270e09c77602eeec85523b1815378b815b64cb99932
```

Step 4: Setup worker3 - node (login through ssh)

```
## use join command from Step 1:
kubeadm join 159.89.99.35:6443 --token rpylp0.rdphpzbavdyx3llz \
  --discovery-token-ca-cert-hash sha256:05d42f2c051a974a27577270e09c77602eeec85523b1815378b815b64cb99932
```

Step 5: CNI-Setup (calico) on controlNode (login through ssh)

```
kubect1 get nodes
```

```
## Output
```

```
root@controlplane:~# kubect1 get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
controlplane	NotReady	control-plane	6m27s	v1.28.6
worker1	NotReady	<none>	3m18s	v1.28.6
worker2	NotReady	<none>	2m10s	v1.28.6
worker3	NotReady	<none>	60s	v1.28.6

```
## Installing calico CNI
```

```
kubect1 create -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.2/manifests/tigera-operator.yaml
kubect1 create -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.2/manifests/custom-resources.yaml
kubect1 get ns
kubect1 -n calico-system get all
kubect1 -n calico-system get pods -o wide -w
```

```
## After if all pods are up and running -> CTRL + C
```

```
kubect1 -n calico-system get pods -o wide
## all nodes should be ready now
kubect1 get nodes -o wide
```

```
## Output
```

```
root@controlplane:~# kubect1 get nodes
```

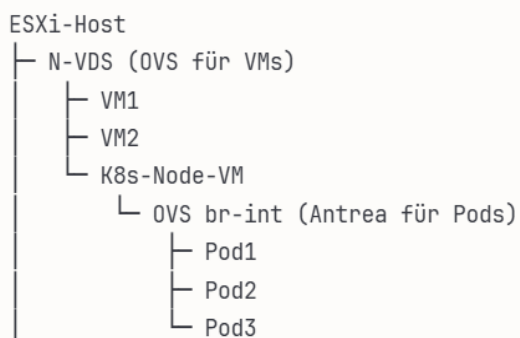
NAME	STATUS	ROLES	AGE	VERSION
controlplane	Ready	control-plane	14m	v1.28.6
worker1	Ready	<none>	11m	v1.28.6
worker2	Ready	<none>	10m	v1.28.6
worker3	Ready	<none>	9m9s	v1.28.6

Do it with ansible

- <https://spacelift.io/blog/ansible-kubernetes>

Kubernetes Netzwerk (Antrea / NSX)

Netzwerkspezialität bei NSX mit Antrea



Besonderheit Netzwerk

- Dann kannst direkt von anderen VM's in ESXi in einen Pod reinrouten
- Eine VM kann einen Pod im Kubernetes Cluster über sein Pod-IP direkt erreichen

Konsequenz des Netzwerkaufbau

- Range für Pods (IP-Range) -> Cluster CIDR, darf nicht schon nsx verwendet worden sein

Tipps & Tricks

Pods bleiben im terminate-mode stehen

Ein Pod hängt in Terminierung meist aus folgenden Gründen:

Häufigste Ursachen:

1. Graceful Shutdown Timeout

- Container reagiert nicht auf SIGTERM
- Nach `terminationGracePeriodSeconds` (default 30s) wird SIGKILL gesendet
- Check: `kubectl describe pod <pod>` → Events

2. Finalizers blockieren

```
kubectl get pod <pod> -o yaml | grep finalizers -A 5
```

Manuelle Entfernung (Notfall):

```
kubectl patch pod <pod> -p '{"metadata":{"finalizers":null}}'
```

3. PreStop Hook hängt

- Zählt zur Grace Period
- Hook-Fehler blockiert Terminierung

4. Volume Unmount Problem

- Node kann Volume nicht freigeben
- Oft bei NFS/CSI-Storage

5. Node NotReady

- Kubelet antwortet nicht
- Pod bleibt in Terminating bis Node zurück oder nach ~5min force-deleted

Quick Fix:

```
## Force delete (wenn nichts anderes hilft)
kubectl delete pod <pod> --grace-period=0 --force
```

Debug-Befehle:

```
kubectl get pod <pod> -o yaml
kubectl describe pod <pod>
kubectl logs <pod> --previous
```

Was sind finalizer

- Finalizers sind Strings im metadata.finalizers-Array eines Pods, die verhindern, dass Kubernetes das Objekt vollständig löscht, bis alle Finalizer entfernt wurden.

```
Bei kubectl delete pod setzt Kubernetes nur metadata.deletionTimestamp
Pod geht in Status Terminating
Controller/Operator mit zuständigem Finalizer führt Cleanup aus
Controller entfernt seinen Finalizer aus dem Array
Erst wenn Array leer → vollständige Löschung
```

Probleme mit Volumes

VolumeAttachment-Objekt (separates K8s-Objekt):

```
apiVersion: storage.k8s.io/v1
kind: VolumeAttachment
metadata:
  finalizers:
  - external-attacher/csi-driver-name # <-- Hier ist der Finalizer
spec:
  attacher: csi-driver
  nodeName: worker-1
  source:
    persistentVolumeName: pvc-xyz
```

Pod-Objekt hat normalerweise **keine Volume-Finalizer**:

```
apiVersion: v1
kind: Pod
```

```
metadata:
  finalizers: [] # Meist leer oder mit anderen Finalizern
```

Warum Pods trotzdem bei Volumes hängen:

1. **Kubelet wartet** auf erfolgreiches Volume-Unmount (nicht Finalizer-basiert)
2. **VolumeAttachment-Finalizer** verhindert Detach vom Node
3. **Node nicht erreichbar** → Kubelet kann nicht unmounten
4. **CSI-Driver-Probleme** → external-attacher kann Finalizer nicht clearen

Typisches Problem-Szenario:

```
## Pod bleibt Terminating
kubectl get pod -o yaml
## deletionTimestamp gesetzt, aber keine Volume-Finalizer

## VolumeAttachment existiert noch
kubectl get volumeattachment
## Hat Finalizer vom CSI-Driver
```

Force-Delete hilft hier nicht beim Volume-Problem, sondern man muss das VolumeAttachment-Objekt oder den Node-Status adressieren.

Datenbanken

MariaDB Operator

- <https://github.com/mariadb-operator/mariadb-operator>

HA mit dem Postgres Operator -> cloudnativePG umsetzen

Vorteile:

- Unter der Schirmherrschaft der Linux Foundation
- Alle Komponenten auch für HA sind bereits nativ implementiert

Installation über Helm-Chart

- Raussuchen aus artifacthub.io

Wie bauen wir jetzt ein HA-Cluster

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-ha
spec:
  instances: 3
  storage:
    size: 10Gi
  postgresql:
    parameters:
      shared_buffers: "256MB"
      max_connections: "100"
  backup:
    barmanObjectStore:
      destinationPath: "s3://pg-backups/"
```

- eine Instanz ist RW
- die beiden anderen Instanzen sind ro (das wird über die Service geschlüsselt)

Mit Synchronisierung zumindest auf eine Replica

- Stichwort: minSyncReplicas (damit ist erst erfolgreich geschrieben, wenn das erfüllt ist und dann wir das auch erst an den Client gemeldet)

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-ha
  namespace: postgres
spec:
  instances: 3

  # Synchronre Replikation - kein Datenverlust bei Failover
  minSyncReplicas: 1
  maxSyncReplicas: 2

  storage:
    size: 10Gi
    storageClass: do-block-storage
```

```
postgresql:
  parameters:
    shared_buffers: "256MB"
    max_connections: "100"
    wal_level: "replica"

# Pods auf verschiedene Nodes verteilen
affinity:
  topologyKey: kubernetes.io/hostname
```

Was ist das für eine Lösung

- Active / Standby

Graph-Datenbank neo4j installieren mit helm

Community Version mit normalen Helm-Chart installieren

- <https://artifacthub.io/packages/helm/neo4j-helm-charts/neo4j>

Für den Zugriff von aussen

- Reverse Proxy installieren [Chart im Internet](#)

```
## ingress-values.yaml
reverseProxy:
  serviceName: "my-neo4j-admin" # Name des Neo4j-Admin-Service
ingress:
  enabled: true
  className: traefik
  tls:
    enabled: true
    config:
      - secretName: neo4j-tls
        hosts:
          - neo4j.example.com
```

```
helm install neo4j-rp neo4j/neo4j-reverse-proxy -f ingress-values.yaml
```

Enterprise Version

- Abweichende Werte in den values, aber ansonsten wie community

```
## enterprise-values.yaml
neo4j:
  name: my-neo4j
  password: "sicheres-passwort"
  edition: "enterprise"
  acceptLicenseAgreement: "yes" # oder "eval" für Evaluierung
  resources:
    cpu: "2"
    memory: "4Gi"

volumes:
  data:
    mode: defaultStorageClass
    defaultStorageClass:
      requests:
        storage: 20Gi
```

Graph-Datenbank neo4j Cluster mit Operator installieren

- <https://github.com/neo4j-partners/neo4j-kubernetes-operator/blob/main/examples/clusters/multi-server-cluster.yaml>

Podman

Podman vs. Docker

Was ist Podman?

Podman (kurz für: *Pod Manager*) ist eine Open-Source-Container-Engine, die als Alternative zu Docker entwickelt wurde.

Warum Podman?

Hier die wichtigsten Gründe, warum Podman gerne verwendet wird:

Vorteil	Erklärung
Daemonless	Podman benötigt keinen zentralen Hintergrunddienst (Daemon), sondern läuft als normaler Benutzerprozess. Dadurch weniger Angriffsfläche und flexibler.

Rootless Support	Container können ohne Root-Rechte gestartet werden, was Sicherheitsvorteile bringt.
Docker-kompatibel	Podman kann Dockerfiles bauen und verwendet dieselben Container Images. Auch <code>podman</code> CLI ist zu <code>docker</code> CLI fast identisch (<code>alias docker=podman</code> geht oft problemlos).
Pods-Unterstützung	Inspiziert von Kubernetes: Mehrere Container in einem gemeinsamen Netzwerk-Namespace (Pod). Praktisch für lokale Kubernetes-ähnliche Setups.
Bessere Integration für Systemd	Einfaches Erstellen von Systemd-Units aus Containern (<code>podman generate systemd</code>). Ideal für Serverdienste ohne externes Orchestration-Tool.
Kein Root-Daemon	Keine ständigen Root-Rechte nötig, weniger Sicherheitsrisiko durch kompromittierte Daemons.
Red Hat / Fedora / CentOS bevorzugen Podman	Dort wird Podman inzwischen oft als Standardlösung ausgeliefert.

Typische Einsatzbereiche

- Entwicklung: wie Docker
- Serverbetrieb: Container als Systemdienste
- Kubernetes-nahes Testing (Pods)
- Rootless Deployment auf Servern ohne Container-Daemon

Beispiel: Container starten

```
podman run -d -p 8080:80 nginx
```

Fast wie bei Docker.

Podman vs Docker kurz gesagt:

	Docker	Podman
Daemon	ja	nein
Rootless	begrenzt	ja
Kubernetes-nah	weniger	stärker
Systemd-Integration	wenig	stark
Kompatibilität	verbreitet	Docker-kompatibel

Kubernetes Installation mit Proxmox und kubespray

Schritt 1: virtuellen Maschine deployen

- 4GB (control nodes eher 8GB) - minimaler Arbeitsspeicher
- Debian als Betriebssystem
- minimale Installation
- ssh-server installiert (openssh-server)
- sudo benutzer ohne Passwort für privilege Escalation (z.B. admin darf als root arbeiten)

Info 1.1 Netzwerk

- Alle virtuellen Maschinen im gleichen Netzwerk (kein VLAN)
- Kubernetes mit eigenem VLAN
- Alternativ neuerdings: mit SDN (Performance beobachten !)

Schritt 2: maschine für ansible deployen/nutzen

- private/public key erstellen und den public auf die maschinen aus Schritt 1 verteilen
- ansible und git installieren
- kubespray clonen oder docker image verwenden (dann braucht man kein ansible installieren)
- `apt update -y`; `apt install docker.io -y`
- Inventory rauskopieren und anpassen

```
[all]
## Master/Control Plane Node
kube-master ansible_host=192.168.1.10
## Worker Nodes
kube-worker1 ansible_host=192.168.1.11
kube-worker2 ansible_host=192.168.1.12

[kube_control_plane]
kube-master

[etcd]
kube-master

[kube_node]
```



```
kube-worker1
kube-worker2
```

```
## evtl config anpassen vornehmen in den group vars
## z.B.
https://github.com/kubernetes-sigs/kubespray/blob/master/inventory/sample/group_vars/k8s_cluster/k8s-cluster.yml
```

```
ansible-playbook -i inventory/mycluster/ cluster.yml -b -v \
--private-key=~/.ssh/private_key
```

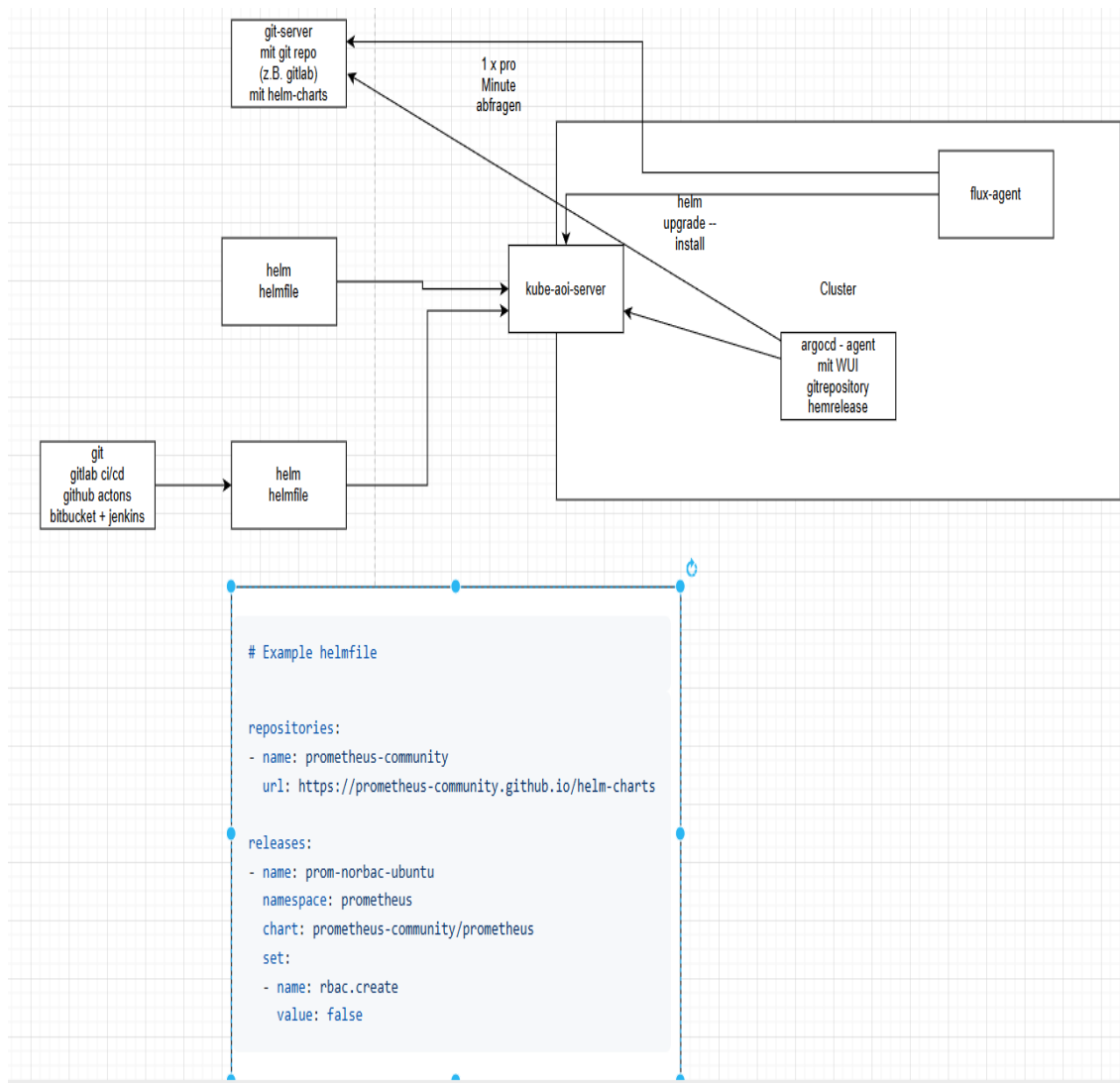
```
## zum hostsystem verbinden und die kubeconfig
## in der Regel
cd /home/<user-mit-dem-ich-installiert-habe>/.kube
cat config
```

Kubernetes Workloads

Welche Wege gibt es Kubernetes Workloads auszurollen

- gitlab ci/cd
- github actions
- bitbucket + jenkins
- ArgoCD
- Flux
- helm/helmfile

Grafik



Kubernetes Infrastructure (Performance)

Performance of etcd for setup

```
etcdctl check perf --load="s"
(s steht für small:

https://andreaskaris.github.io/blog/openshift/etcd_perf/
The performance check's workload model. Accepted workloads: s (small), m (medium), l (large), xl (xLarge)
```

Kubernetes - Überblick

Liste wichtiger/sinnvoller Client-Tools

- <https://github.com/jmetzger/training-kubernetes-einfuehrung/blob/main/tools/liste-client-tools.md>

Kubernetes - Hochverfügbarkeit

Strategien für Hochverfügbarkeit

Kernprinzip: Control Plane benötigt RTT < 10ms zwischen etcd-Nodes für stabilen Quorum-Betrieb. Workloads können über höhere Latenzen hinweg verteilt werden. (Fall 1 - Fall 3)

Legende

RTO

- Recovery Time Objective
- Die maximale akzeptable Zeit, bis ein System nach einem Ausfall wieder verfügbar sein muss.

RPO

- Recovery Point Objective (häufig zusammen genannt)
- Der maximale akzeptable Datenverlust, gemessen in Zeit.

FALL 1: Single-Cluster in einem Rechenzentrum

Ausgangssituation

- 3 Control Plane Nodes (stacked etcd)
- 1+n Worker Nodes
- Alle Nodes im selben RZ

Sinnvoll wenn:

- RTO: 1-5 Minuten (Wie lange bis wieder verfügbar)
- RPO: Nahe Null (bei korrekter Storage-Konfiguration)
- Budget: Gering bis mittel
- Keine regulatorischen Anforderungen für Geo-Redundanz
- Primäres Ziel: Schutz vor Hardware-/Software-Ausfällen

HA-Strategie Control Plane

etcd-Cluster (3 Nodes)

```
## Unbedingt odd number: 3 oder 5 nodes
## 3 Nodes tolerieren 1 Ausfall
## 5 Nodes tolerieren 2 Ausfälle
```

- Separate Hosts für etcd (empfohlen) oder stacked etcd
- Separate Disks mit niedriger Latenz (NVMe SSD)
- Monitoring: etcd-Metriken (Leader Elections, DB Size)
- Backup-Strategie: Regelmäßige etcd-Snapshots

API Server Load Balancing

- Redundante kube-apiserver auf allen Control Plane Nodes
- Virtual IP ((z.B kube-vip) und/oder Hardware Load Balancer vor API Servers
- Tools: kube-vip, HAProxy, keepalived

Scheduler & Controller Manager

- Laufen auf allen Control Plane Nodes
- Leader Election über Leases in kube-system
- Automatisches Failover bei Node-Ausfall

HA-Strategie Workload

```
## In diesem Fall Kann-Option
## Wenn eine Node wegbricht, wird ohnehin auf eine andere Node verteilt
```

Pod-Replikation

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3 # Minimum für HA
  template:
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: web-app
```

- maxSkew = 1: Unterschied der einzelnen Pods auf Nodes darf maximal 1 sein, d.h. z.B. node1: 1, node2: 2,
- topologyKey: Verteilung über Nodes

PodDisruptionBudgets (PDB)

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-app-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web-app
```

- Es müssen insgesamt mindestds noch 2 pods laufen

Anti-Affinity Rules

- Pods auf unterschiedliche Worker Nodes verteilen
- `requiredDuringSchedulingIgnoredDuringExecution` für strikte Trennung

Storage HA

Lokale Storage-Lösungen:

- Rook-Ceph: Distributed Storage mit Replikation
- Longhorn: Leichtgewichtige Alternative (schlechte Performance)
- OpenEBS: Verschiedene Storage Engines (nur readwriteonce)

Externe Storage:

- NFS mit HA (z.B. über DRBD)
- iSCSI mit Multipathing (nur readwriteonce)
- Cloud-Provider CSI-Driver (bei Managed Kubernetes)

Backup:

- Velero für Cluster-Backups
- Regelmäßige Application-Level Backups
- Externe Backup-Location (S3-kompatibel)

Netzwerk HA

CNI-Redundanz:

- Calico, Cilium oder Flannel mit HA-Konfiguration
- Redundante Netzwerk-Interfaces auf Nodes

Load Balancer:

- MetallLB für Bare-Metal (Layer 2 oder BGP-Mode)
- Cloud-Provider Load Balancer (bei Managed Services)

Ingress HA:

```
## Nginx Ingress mit mindestens 2 Replicas
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --set controller.replicaCount=3 \
  --set controller.service.externalTrafficPolicy=Local
```

Single Points of Failure (SPOFs)

Komponente	Risiko	Mitigation

Load Balancer vor API Server	Hoch	kube-vip mit VRRP oder redundante Hardware-LB
Storage Backend	Hoch	Distributed Storage (Ceph, Longhorn)
Gesamtes RZ	Hoch	Nur durch Multi-RZ-Setup adressierbar

Tools & Komponenten

- **kube-vip:** VIP für API Server (Layer 2 oder BGP)
- **HAProxy/keepalived:** Alternative zu kube-vip
- **MetalLB:** Load Balancer für Bare-Metal
- **Velero:** Backup und Restore
- **Prometheus + Alertmanager:** Monitoring
- **etcdctl:** etcd-Management und Backups

Vor-/Nachteile

Vorteile:

- Einfachste HA-Lösung
- Niedrige Latenz innerhalb Cluster
- Geringste Komplexität
- Moderate Kosten

Nachteile:

- Kein Schutz vor RZ-Ausfall
- Limitierte geografische Redundanz

Kostenabschätzung

- **Relativ:** Basis (1x)
- **Absolute Kosten:** 3 Control Plane + 3+ Worker Nodes + Storage

RTO/RPO

- **RTO:** 1-5 Minuten (bei automatischem Pod-Failover)
- **RPO:** < 1 Minute (bei korrekter Storage-Replikation)

Quellen

- [Kubernetes HA Topologies](#)
- [etcd Administration Guide](#)
- [kube-vip Documentation](#)

FALL 2: Multi-Zone Cluster (RTT < 10ms)

Ausgangssituation

- 3-5 Control Plane Nodes über Availability Zones verteilt
- Worker Nodes in allen Zones
- RTT zwischen Zones: < 10ms (typisch < 2ms innerhalb Region)
- Cloud-Provider oder Metro-RZ mit dedizierten Zonen

Sinnvoll wenn:

- RTO: < 1 Minute
- RPO: Nahe Null
- Budget: Mittel
- Cloud-Native Deployment (AWS, GCP, Azure)
- Schutz gegen Zone-Ausfall erforderlich
- Keine strengen Latenzanforderungen zwischen Zones

HA-Strategie Control Plane

etcd über Zones verteilt

- **Kritisch:** RTT muss < 10ms bleiben für etcd-Quorum
- Mindestens 3 Zones mit jeweils 1 Control Plane Node
- Bei 5 Nodes: 2-2-1 oder 2-1-2 Distribution

Latenz-Monitoring:

```
## etcd Performance-Check
etcdctl check perf --load="s"
## Warn bei > 10ms Backend Commit Duration
```

HA-Strategie Workload

Topology Spread Constraints

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 6
  template:
    spec:
```

```

topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app: web-app
- maxSkew: 2
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway

```

Zone-Aware PodDisruptionBudgets

```

apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 4 # Bei 6 Replicas über 3 Zones
  selector:
    matchLabels:
      app: web-app
  unhealthyPodEvictionPolicy: AlwaysAllow

```

Storage HA

Zone-Aware Storage Classes

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: zone-redundant
provisioner: kubernetes.io/aws-ebs # Beispiel AWS
parameters:
  type: gp3
  iops: "3000"
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
  - key: topology.kubernetes.io/zone
    values:
      - eu-central-1a
      - eu-central-1b
      - eu-central-1c

```

Replizierte Storage-Lösungen:

- Rook-Ceph mit Zone-Awareness
- Portworx mit Zone-Replication
- Cloud-Provider replizierte Volumes

Cloud-Provider Integration

AWS:

- EKS mit Multi-AZ Control Plane (Managed)
- EBS Multi-Attach für Shared Storage
- ELB/ALB automatisch über Zones verteilt

GCP:

- GKE Regional Clusters (Multi-Zonal Control Plane)
- Regional Persistent Disks
- Cloud Load Balancer über Zones

Azure:

- AKS mit Availability Zones
- Zone-Redundant Storage (ZRS)
- Azure Load Balancer Standard (Zone-Redundant)

Tools & Komponenten

- **Cloud-Provider CCM:** Automatische Zone-Awareness
- **CSI-Driver:** Zone-Aware Storage Provisioning
- **Cluster Autoscaler:** Zone-Aware Scaling
- **Velero:** Multi-Zone Backups

Vor-/Nachteile

Vorteile:

- Schutz gegen Zone-Ausfall
- Automatisches Failover

- Native Cloud-Integration
- Transparenz für Workloads

Nachteile:

- Höhere Cloud-Kosten (Cross-Zone Traffic)
- Potenzielle Latenz zwischen Zones
- Abhängigkeit von Cloud-Provider

Kostenabschätzung

- **Relativ:** 1.5-2x (wegen Cross-Zone Traffic)
- **Zusatzkosten:** Data Transfer zwischen Zones (~0.01-0.02 €/GB)

RTO/RPO

- **RTO:** < 1 Minute (automatisches Failover)
- **RPO:** 0 (synchrone Replikation)

Quellen

- [Kubernetes Zone-Aware Scheduling](#)
- [AWS EKS Best Practices](#)
- [GCP GKE Multi-Zonal Clusters](#)

FALL 3: Stretched Cluster über RZ (RTT < 10ms)

Ausgangssituation

- 2-3 physisch getrennte Rechenzentren
- RTT zwischen RZ: < 10ms (Metro-Cluster Szenario)
- Dedizierte Glasfaser-Verbindung zwischen RZ
- Geografische Distanz: typisch < 50-100 km

Sinnvoll wenn:

- RTO: < 1 Minute
- RPO: Nahe Null
- Budget: Hoch
- Regulatorische Anforderungen für Geo-Redundanz
- Synchrone Daten-Replikation erforderlich
- Sehr niedrige Latenz zwischen RZ verfügbar

HA-Strategie Control Plane

Stretched etcd-Cluster mit Witness

```
RZ1: 2 etcd Nodes
RZ2: 2 etcd Nodes
RZ3 (optional): 1 Witness Node

Total: 5 Nodes für 2-Node Ausfall-Toleranz
```

Kritische Anforderungen:

- RTT zwischen allen etcd-Nodes: < 10ms
- Stabile, dedizierte Netzwerk-Verbindung
- Monitoring für Network Partitions

Split-Brain Prevention:

- Odd number of etcd nodes
- Optional: Witness-Node in drittem Standort
- Fencing-Mechanismen auf Storage-Level

HA-Strategie Workload

Site-Aware Scheduling ``yaml apiVersion: v1 kind: Node metadata: labels: topology.kubernetes.io/region: eu-central topology.kubernetes.io/zone: fra1 # RZ1 site: primary

apiVersion: apps/v1 kind: Deployment spec: replicas: 6 template: spec: topologySpreadConstraints: - maxSkew: 1 topologyKey: site whenUnsatisfiable: DoNotSchedule

```
#### Storage-Replikation zwischen RZ

**Synchrone Replikation:**
- **Rook-Ceph** mit stretched cluster mode
  - Min. 5 OSD Nodes (2-2-1 über Sites)
  - Replicated Pools mit site-awareness

- **Portworx** mit DR-License
  - Synchronous Replication zwischen Sites
  - Automatic Failover

- **DRBD** für Block-Storage
  - Kernel-Level Replication
```

- Dual-Primary Mode möglich

Netzwerk-Design

Anforderungen:

- Dedizierte Layer 2/3 Verbindung zwischen RZ
- BGP-Routing für Pod-Network
- Redundante Uplinks (LACP)

CNI-Empfehlungen:

- **Calico** mit BGP zwischen Sites
- **Cilium** mit Cluster Mesh (single cluster mode)

Risiken & Mitigationen

Risiko	Impact	Mitigation
----- ----- -----		
Network Partition	Kritisch	Witness Node, Fencing
Latenz-Spikes	Hoch	SLA für Interconnect, Monitoring
Split-Brain Storage	Kritisch	Quorum-basierte Systeme
Asynchroner etcd	Kritisch	Automatisches Health-Check, Rollback

Tools & Komponenten

- **Rook-Ceph**: Distributed Storage
- **Portworx**: Enterprise Storage mit DR
- **DRBD**: Block-Level Replication
- **Cilium/Calico**: Advanced Networking
- **Disaster Recovery Tools**: Velero mit Multi-Site Backups

Vor-/Nachteile

Vorteile:

- Transparente RZ-Redundanz
- Ein logischer Cluster
- Niedrige RTT für Workloads
- Automatisches Failover

Nachteile:

- Sehr hohe Komplexität
- Abhängigkeit von stabiler, niedriger Latenz
- Risiko für Split-Brain
- Hohe Kosten für Interconnect
- Schwierige Fehlerdiagnose

Kostenabschätzung

- **Relativ:** 3-4x (Infrastructure, Interconnect, Storage-Lizenz)
- **Zusatzkosten:** Dedizierte Glasfaser, HA-Storage-Lizenzen

RTO/RPO

- **RTO:** < 1 Minute (bei korrekter Konfiguration)
- **RPO:** 0 (synchrone Replikation)

Warnung

⚠ ****Stretched Clusters sind komplex und fehleranfällig.**** Nur bei zwingenden Business-Anforderungen und entsprechender Expertise empfohlen. Multi-Cluster-Setups (Fall 4) sind oft die bessere Wahl.

Quellen

- [etcd Latency Requirements] (<https://etcd.io/docs/v3.5/op-guide/hardware/>)
- [Ceph Stretched Cluster Mode] (<https://docs.ceph.com/en/latest/rados/operations/stretch-mode/>)
- [Portworx Disaster Recovery] (<https://docs.portworx.com/portworx-enterprise/operations/operate-kubernetes/disaster-recovery>)

FALL 4: Multi-Cluster (RTT > 10ms oder unabhängige RZ)

Ausgangssituation

- Separate Kubernetes-Cluster pro RZ/Region
- RTT zwischen RZ: > 10ms (typisch 30-200ms)
- Geografisch verteilte Standorte
- Jeder Cluster ist unabhängig lauffähig

Sinnvoll wenn:

- RTO: 1-10 Minuten (manuell) oder < 1 Minute (automatisch)
- RPO: Sekunden bis Minuten (je nach Replikation)
- Budget: Mittel bis hoch
- Geo-Redundanz über weite Distanzen
- Disaster Recovery Anforderung

```

- Latenz-Optimierung für User (Edge-Deployment)
- Regulatorische Anforderungen (Data Residency)

#### Multi-Cluster Architektur-Patterns

#### 4.1 Active-Passive (Cold Standby)

**Setup:**
- Primary Cluster: Alle Workloads aktiv
- Secondary Cluster: Bereit, aber idle
- DNS-Failover zu Secondary bei Primary-Ausfall

**Komponenten:**
- **GitOps**: ArgoCD oder Flux auf beiden Clustern
- **Backup**: Velero für Disaster Recovery
- **DNS**: External-DNS oder GSLB (Global Server Load Balancing. Eine DNS-basierte Load-Balancing-Lösung für geografisch verteilte Systeme/Cluster).

**Vor-/Nachteile:**
- ✓ Einfachste Multi-Cluster Lösung
- ✓ Niedrige Laufkosten
- ✗ RTO: 5-15 Minuten (manuelle Aktivierung)
- ✗ Secondary-Cluster ungenutzt

#### 4.2 Active-Active (Hot Standby)

**Setup:**
- Traffic auf beide Cluster verteilt (z.B. 50/50 oder Geo-basiert)
- Automatisches Failover bei Cluster-Ausfall

**Komponenten:**
- **Global Load Balancer**: AWS Route53, Cloudflare, F5 GTM
- **Service Mesh**: Istio Multi-Cluster oder Linkerd
- **Data Replication**: Anwendungsspezifisch oder DB-Level

**Traffic Distribution:**
```yaml
Beispiel: External-DNS mit GeoDNS
apiVersion: v1
kind: Service
metadata:
 name: web-app
 annotations:
 external-dns.alpha.kubernetes.io/hostname: app.example.com
 external-dns.alpha.kubernetes.io/aws-geolocation-routing-policy: "eu-central-1"
spec:
 type: LoadBalancer

```

#### Vor-/Nachteile:

- ✓ RTO: < 1 Minute (automatisch)
- ✓ Optimale Resource-Nutzung
- ✓ Geo-Latenz-Optimierung
- ✗ Hohe Komplexität
- ✗ Daten-Konsistenz-Herausforderungen

#### Multi-Cluster Networking

##### Submariner

- Submariner ist ein Open-Source-Projekt, das sichere IP-Tunnels zwischen Kubernetes-Clustern erstellt und das Netzwerk im Wesentlichen "abflacht", sodass Pods und Services in verschiedenen Clustern direkt kommunizieren können
- Open-Source Multi-Cluster Networking
- Direct Pod-to-Pod Communication
- Cross-Cluster Service Discovery

```

Installation
subctl deploy-broker --kubeconfig cluster1-config
subctl join --kubeconfig cluster1-config broker-info.subm
subctl join --kubeconfig cluster2-config broker-info.subm

Service Export
subctl export service web-app -n production

```

#### Eigenschaften:

- IPsec/WireGuard Tunnel zwischen Clustern
- Automatisches Route-Advertisement
- Unterstützt unterschiedliche CNIs



## Cilium Cluster Mesh

### Setup:

```
Cluster 1
cilium clustermesh enable
cilium clustermesh connect --destination-context cluster2

Shared Service
kubectl annotate service web-app io.cilium/shared-service="true"
```

### Eigenschaften:

- Native Pod-to-Pod Communication ohne Tunnel
- Service Affinity (bevorzuge lokale Pods)
- eBPF-basiert, sehr performant

## Multi-Cluster GitOps

### ArgoCD Multi-Cluster

#### Setup:

```
Cluster registrieren
argocd cluster add cluster1-context
argocd cluster add cluster2-context

ApplicationSet für beide Cluster
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
 name: web-app
spec:
 generators:
 - clusters:
 selector:
 matchLabels:
 env: production
 template:
 spec:
 source:
 repoURL: https://github.com/org/repo
 path: apps/web-app
 destination:
 name: '{{name}}'
 namespace: production
```

### Strategien:

- **Pull-Model:** Jeder Cluster hat eigenes ArgoCD
- **Push-Model:** Zentrales ArgoCD verwaltet alle Cluster

### Flux Multi-Cluster

```
Flux Configuration per Cluster
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
 name: apps
spec:
 interval: 5m
 path: ./clusters/cluster1
 prune: true
 sourceRef:
 kind: GitRepository
 name: fleet-infra
```

## Daten-Replikation Strategien

### Anwendungsebene

- Application-Managed Replication (z.B. Kafka Multi-DC)
- Event Sourcing mit Cross-Cluster Event Streams
- CQRS mit regionalen Read-Replicas

### Datenbank-Replikation

#### PostgreSQL:

```
CrunchyData Postgres Operator mit Standby
apiVersion: postgres-operator.crunchydata.com/v1beta1
kind: PostgresCluster
spec:
```

```

instances:
- name: instance1
 replicas: 3
 dataVolumeClaimSpec:
 accessModes:
 - ReadWriteOnce
 patroni:
 dynamicConfiguration:
 postgresql:
 parameters:
 wal_level: logical
 standby:
 enabled: true
 repoName: repo1

```

#### MariaDB:

- (Galera Cluster für synchrone Multi-Master) - performance Probleme bei zu hoher RTT

#### NoSQL:

- MongoDB Replica Sets über Regions
- Cassandra Multi-DC Replication
- Redis Sentinel mit Cross-DC Replication

#### Global Load Balancing (GSLB)

##### Cloudflare Load Balancer

```

{
 "name": "app.example.com",
 "default_pools": ["eu-central-pool", "us-east-pool"],
 "region_pools": {
 "WEUR": ["eu-central-pool"],
 "EEU": ["eu-central-pool"],
 "NAM": ["us-east-pool"]
 },
 "steering_policy": "geo"
}

```

#### AWS Route53

```

Terraform Beispiel
resource "aws_route53_record" "app" {
 zone_id = var.zone_id
 name = "app.example.com"
 type = "A"

 geolocation_routing_policy {
 continent = "EU"
 }

 alias {
 name = aws_lb.eu_cluster.dns_name
 zone_id = aws_lb.eu_cluster.zone_id
 evaluate_target_health = true
 }
}

```

#### Multi-Cluster Monitoring

##### Prometheus Federation

```

Zentrale Prometheus-Instanz
scrape_configs:
- job_name: 'federate-cluster1'
 honor_labels: true
 metrics_path: '/federate'
 params:
 'match[]':
 - '{job="kubernetes-pods"}'
 static_configs:
 - targets:
 - 'prometheus-cluster1.monitoring:9090'
 labels:
 cluster: cluster1

```

#### Thanos für Multi-Cluster

- Zentrale Object Storage für Metriken
- Global Query Layer
- Cross-Cluster Alerting

Grafana Multi-Cluster Dashboards

- Cluster-Selector per Variable
- Aggregierte Ansichten über Cluster
- Geo-Maps für Traffic-Distribution

Failover-Strategien

DNS-basiert

```
External-DNS mit Healthcheck
apiVersion: v1
kind: Service
metadata:
 annotations:
 external-dns.alpha.kubernetes.io/hostname: app.example.com
 external-dns.alpha.kubernetes.io/set-identifier: cluster1
 external-dns.alpha.kubernetes.io/aws-weight: "100"
 external-dns.alpha.kubernetes.io/aws-health-check-id: <id>
```

Eigenschaften:

- RTO: TTL-abhängig (typisch 60-300s)
- Einfach zu implementieren
- Keine zusätzliche Infrastruktur

Application-Level Failover

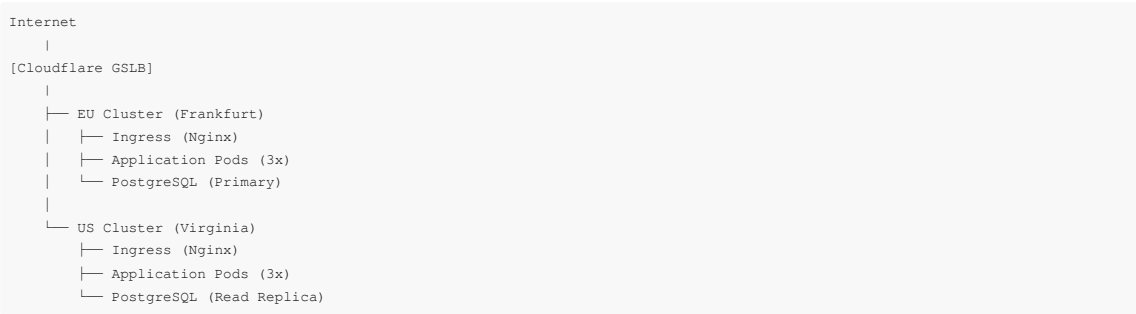
- Circuit Breaker in Service Mesh
- Retry-Logic mit Fallback-Cluster
- Client-Side Load Balancing

Tools & Komponenten Übersicht

Kategorie	Tool	Zweck
Cluster Management	Rancher	Multi-Cluster UI, Management
	KubeFed (deprecated)	Cluster Federation (Legacy)
Networking	Submariner	Pod-to-Pod Communication
	Cilium Cluster Mesh	eBPF Multi-Cluster
GitOps	ArgoCD	Multi-Cluster Deployments
	Flux	GitOps Engine
Load Balancing	Cloudflare	GSLB, DDoS Protection
	F5 GTM	Enterprise GSLB
	AWS Route53	Geo-Routing, Healthchecks
Monitoring	Thanos	Multi-Cluster Prometheus
	Grafana	Unified Dashboards
Disaster Recovery	Velero	Backup/Restore
	Kasten K10	Enterprise Backup

Implementierungs-Beispiel: 2-Cluster Active-Active

Architektur:



Deployment:

1. Basis-Setup:

```
Beide Cluster mit ArgoCD verbinden
argocd cluster add eu-cluster
argocd cluster add us-cluster
```

## 2. ApplicationSet:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
 name: web-app
spec:
 generators:
 - clusters:
 selector:
 matchLabels:
 env: production
 template:
 spec:
 project: default
 source:
 repoURL: https://github.com/org/apps
 path: web-app/overlays/{{values.cluster}}
 targetRevision: main
 destination:
 name: '{{name}}'
 namespace: production
```

## 3. Datenbank-Replikation (PostgreSQL):

```
Primary Cluster (EU)
apiVersion: postgres-operator.crunchydata.com/v1beta1
kind: PostgresCluster
metadata:
 name: web-db
spec:
 postgresVersion: 15
 instances:
 - name: instance1
 replicas: 2

Replica Cluster (US) - Logical Replication
Subscription zu EU Cluster
```

## 4. GSLB (Cloudflare):

```
Cloudflare Load Balancer mit Geo-Steering
curl -X POST "https://api.cloudflare.com/client/v4/zones/<zone>/load_balancers" \
-H "Authorization: Bearer <token>" \
-d '{
 "name": "app.example.com",
 "default_pools": ["eu-pool", "us-pool"],
 "region_pools": {
 "WEUR": ["eu-pool"],
 "NAM": ["us-pool"]
 },
 "steering_policy": "geo"
}'
```

### Vor-/Nachteile

#### Vorteile:

- Echte Geo-Redundanz (RZ-unabhängig)
- Keine etcd-Latenz-Limitierung
- Unabhängige Cluster-Updates
- Flexible Failover-Strategien
- Data Residency Compliance möglich

#### Nachteile:

- Höchste Komplexität
- Daten-Konsistenz Herausforderungen
- Höhere Betriebskosten
- Mehr Tooling erforderlich
- Manueller Aufwand für Synchronisation

**Kostenabschätzung**

- **Relativ:** 2-3x pro Cluster (Minimum 2 Cluster)
- **Zusatzkosten:** GSLB, Service Mesh, GitOps-Tooling

**RTO/RPO**

Strategie	RTO	RPO
Active-Passive (manuell)	5-15 min	1-5 min
Active-Passive (auto)	1-5 min	1-5 min
Active-Active	< 1 min	Sekunden-Minuten

**Quellen**

- [Kubernetes Multi-Cluster Networking](#)
- [Submariner Documentation](#)
- [Istio Multi-Cluster](#)
- [ArgoCD ApplicationSet](#)
- [Flux Multi-Cluster](#)

**FALL 5: Hybrid/Edge Scenarios**

**Ausgangssituation**

- Kombination aus Cloud und On-Premise
- Edge-Locations mit eingeschränkter Connectivity
- IoT/Industrial Use Cases
- Retail Stores, Remote Sites

**Sinnvoll wenn:**

- Data Sovereignty erforderlich (On-Premise)
- Latenz-kritische Workloads am Edge
- Offline-Fähigkeit erforderlich
- Schrittweise Cloud-Migration
- Budget: Variabel

**Lightweight Kubernetes Distributionen**

**K3s**

```
Master Installation
curl -sfl https://get.k3s.io | sh -

Agent (Edge) Installation
curl -sfl https://get.k3s.io | K3S_URL=https://master:6443 \
 K3S_TOKEN=<token> sh -
```

**Eigenschaften:**

- Minimaler Footprint (<512 MB RAM)
- SQLite statt etcd (für single-master)
- Integrierter Load Balancer (ServiceLB)
- Ideal für Edge-Deployments

**MicroK8s**

```
Installation
snap install microk8s --classic

HA-Cluster
microk8s add-node
microk8s join <master-ip>:<port>/<token>
```

**Eigenschaften:**

- Snap-basiert, Auto-Updates
- Add-Ons für DNS, Storage, Ingress
- Gut für Desktop/Dev-Environments

**KubeEdge**

```
Cloud-Seite
keadm init --advertise-address=<cloud-ip>

Edge-Seite
keadm join --cloudcore-ipport=<cloud-ip>:10000 \
 --token=<token>
```

**Eigenschaften:**

- Speziell für IoT/Edge
- Offline-Autonomie
- Lightweight Edge-Runtime
- Cloud-Edge Message Bus

#### Hybrid-Cluster Management

##### Rancher

- Zentrale UI für alle Cluster (Cloud + On-Prem)
- Multi-Cluster App Catalog
- RBAC und Policy Management
- Monitoring über Cluster hinweg

##### VMware Tanzu

- Enterprise Kubernetes für Hybrid Cloud
- Integration mit vSphere
- Consistent Operations überall

#### Connectivity-Patterns

##### VPN-Mesh:

```

Cloud Cluster
|
[WireGuard VPN]
|
├── Branch Office 1 (K3s)
├── Branch Office 2 (K3s)
└── Edge Device (KubeEdge)

```

##### Hub-and-Spoke:

- Zentraler Hub-Cluster (Cloud)
- Spoke-Cluster (Edge/Branch)
- Submariner für Pod-Connectivity

#### Edge-Specific Patterns

##### Edge-Autonomy:

```

Application läuft lokal, synchronisiert bei Connectivity
apiVersion: apps/v1
kind: Deployment
metadata:
 name: edge-app
 annotations:
 edge.kubernetes.io/offline-capable: "true"
spec:
 replicas: 1
 template:
 spec:
 nodeSelector:
 edge-location: retail-store-42

```

##### Data Sync:

- Lokale Datenerfassung am Edge
- Batch-Upload bei Connectivity
- Conflict Resolution bei Sync

#### Tools & Komponenten

- **K3s**: Lightweight Kubernetes
- **MicroK8s**: Snap-based Kubernetes
- **KubeEdge**: Cloud-Edge Framework
- **Rancher**: Multi-Cluster Management
- **Submariner**: Hybrid Networking
- **Flux/ArgoCD**: GitOps auch für Edge

##### Vor-/Nachteile

###### Vorteile:

- Flexible Deployment-Optionen
- On-Premise Data Residency
- Niedrige Latenz am Edge
- Offline-Fähigkeit

###### Nachteile:

- Management-Komplexität
- Heterogene Infrastruktur
- Connectivity-Abhängigkeit
- Update-Management schwierig

Kostenabschätzung

- **Cloud:** 1-2x Standard-Cluster
- **Edge:** Hardware + K3s (minimal)
- **Gesamt:** Stark use-case abhängig

RTO/RPO

- **Cloud-Edge:** Abhängig von Connectivity
- **Edge-Autonomie:** RTO < 1min (lokal), RPO variabel

Quellen

- [K3s Documentation](#)
- [MicroK8s Documentation](#)
- [KubeEdge Documentation](#)
- [Rancher Documentation](#)

Entscheidungsmatrix

Kriterium	Fall 1	Fall 2	Fall 3	Fall 4	Fall 5
RZ-Anzahl	1	1 (Multi-Zone)	2-3	2+	Hybrid
RTT Control Plane	< 1ms	< 10ms	< 10ms	> 10ms	Variabel
RTO	1-5 min	< 1 min	< 1 min	1-10 min	Variabel
RPO	< 1 min	0	0	Sek-Min	Variabel
Komplexität	Niedrig	Mittel	Sehr Hoch	Hoch	Hoch
Kosten (relativ)	1x	1.5-2x	3-4x	2-3x/Cluster	Variabel
Geo-Redundanz	✗	✓ (Zones)	✓ (RZ)	✓✓	✓
Empfehlung	Standard	Cloud-Native	⚠ Nur bei Zwang	✓ Empfohlen	Use-Case

Tooling Comparison

GitOps: ArgoCD vs Flux

Feature	ArgoCD	Flux
UI	✓ Web UI	✗ CLI only (FluxCD UI 3rd party)
Multi-Cluster	✓ Native	✓ Multi-tenancy
RBAC	✓ Integriert	⚠ Kubernetes RBAC
Sync Waves	✓ Hooks	✓ Dependencies
Helm	✓ Native	✓ HelmRelease CRD
Kustomize	✓	✓
Image Updates	⚠ Image Updater	✓ Image Automation
Resource Usage	Höher	Niedriger

**Empfehlung:** ArgoCD für Enterprise mit UI-Bedarf, Flux für GitOps-Puristen

Zusammenfassung & Empfehlungen

Quick Decision Tree

```
Start
|
├─ Ein RZ ausreichend?
| └─ JA → Fall 1 (Single-Cluster)
|
├─ Cloud-Provider mit Zones?
| └─ JA → Fall 2 (Multi-Zone)
|
├─ RTT < 10ms zwischen RZ?
| └─ JA → ⚠ Fall 3 (Stretched) - nur bei Zwang!
| └─ NEIN → Fall 4 (Multi-Cluster) ✓
|
└─ Edge/Hybrid?
 └─ JA → Fall 5 (Hybrid/Edge)
```

Pragmatische Empfehlungen

1. **Start Simple:** Fall 1 (Single-Cluster) ist für 80% ausreichend
2. **Cloud-Native:** Fall 2 (Multi-Zone) bei Cloud-Deployment

3. **Avoid Stretched Clusters:** Fall 3 nur bei regulatorischen Zwängen
4. **Go Multi-Cluster:** Fall 4 für echte Geo-Redundanz
5. **Edge wenn nötig:** Fall 5 nur für spezifische Use Cases

#### Typische Fehler vermeiden

✗ **Stretched Cluster bei hoher Latenz** → Split-Brain Risiko  
✔ **Multi-Cluster stattdessen**

✗ **Keine PodDisruptionBudgets** → Outage bei Node-Drain  
✔ **PDBs für alle kritischen Apps**

✗ **Single Replica für kritische Services** → No HA  
✔ **Min. 3 Replicas + Anti-Affinity**

✗ **Keine Backup-Tests** → DR funktioniert nicht  
✔ **Regelmäßige Restore-Drills**

✗ **etcd ohne Monitoring** → Unerkannte Performance-Issues  
✔ **etcd-Metriken + Alerts**

---

#### Weiterführende Ressourcen

##### Offizielle Dokumentation

- [Kubernetes Production Best Practices](#)

##### Tools & Projekte

- [Awesome Kubernetes](#)
- [Kubernetes Failure Stories](#)

---

## Kubernetes - Authentication

### oidc mit kubectl

#### Voraussetzung (allgemein)

- IDP-Provider muss vorhanden sein

#### Einrichtung auf dem Kubernetes Server (so -> seit 1.30 (GA 1.32))

##### Auf dem Server-Host (Control Plane)

```
Erstellen als
/etc/kubernetes/auth-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: AuthenticationConfiguration
jwt:
- issuer:
 url: https://provider1.example.com
 audiences:
 - kubernetes
 claimMappings:
 username:
 claim: email
 groups:
 claim: groups

- issuer:
 url: https://provider2.example.com
 audiences:
 - k8s-prod
 claimMappings:
 username:
 claim: sub
 groups:
 claim: roles

API - Flag setzen
Diese Datei liegt bereits vor und muss angepasst werden
Static Pod
/etc/kubernetes/manifests/kube-apiserver.yaml
spec:
 containers:
 - command:
 - kube-apiserver
 - --authentication-config=/etc/kubernetes/auth-config.yaml
 volumeMounts:
 - name: auth-config
 mountPath: /etc/kubernetes/auth-config.yaml
 readOnly: true
```



```
volumes:
- name: auth-config
 hostPath:
 path: /etc/kubernetes/auth-config.yaml
```

- Es handelt sich um einen static - pod, wenn die Datei geändert wurde wird der Pod automatisch neu erstellt

### Voraussetzung (Client-Seite)

- krew (Plugin Manager für kubectl muss installiert sein)

```
Linux/macOS
(
 set -x; cd "$(mktemp -d)" &&
 OS="$(uname | tr '[:upper:]' '[:lower:]')" &&
 ARCH="$(uname -m | sed -e 's/x86_64/amd64/' -e 's/\(arm\)\(64\)\{0,1\}/\1\2/' -e 's/aarch64/arm64/')" &&
 KREW="krew-${OS}_${ARCH}" &&
 curl -fsSLO "https://github.com/kubernetes-sigs/krew/releases/latest/download/${KREW}.tar.gz" &&
 tar zxvf "${KREW}.tar.gz" &&
 ./"${KREW}" install krew
)

PATH erweitern
export PATH="${KREW_ROOT:-$HOME/.krew}/bin:$PATH"
```

### Aufsetzen auf dem Client

```
OIDC-Plugin installieren (falls noch nicht vorhanden)
kubectl krew install oidc-login

Kubeconfig anpassen
kubectl config set-credentials oidc-user \
 --exec-api-version=client.authentication.k8s.io/v1 \
 --exec-command=kubectl \
 --exec-arg=oidc-login \
 --exec-arg=get-token \
 --exec-arg=--oidc-issuer-url=https://dein-idp.example.com \
 --exec-arg=--oidc-client-id=kubernetes \
 --exec-arg=--oidc-client-secret=SECRET
```

```
Das führt zu folgender kubeconfig
users:
- name: oidc-user
 user:
 exec:
 apiVersion: client.authentication.k8s.io/v1
 command: kubectl
 args:
 - oidc-login
 - get-token
 - --oidc-issuer-url=https://keycloak.example.com/realms/myrealm
 - --oidc-client-id=kubernetes
 - --oidc-client-secret=SECRET # optional
```

```
neuen context erstellen
Cluster-Eintrag muss vorhanden sein
kubectl config set-context oidc-context \
 --cluster=dein-cluster \
 --user=oidc-user
```

```
Context verwenden
kubectl config use-context oidc-context
```

```
jetzt kannst du ganz normal Befehle verwenden
z. B.
kubectl get pods
```

### Wie funktioniert das ganze ?

```
kubectl get pods
```

#### 2. kubectl prüft Kubeconfig

- Findet OIDC-Config (exec-Plugin: kubectl oidc-login )
- **Kein Token vorhanden** → startet Auth-Flow

#### 3. Browser-Login

- Plugin öffnet Browser automatisch

- Du loggst dich bei deinem OIDC-Provider ein (z.B. Keycloak, Google)
- Provider gibt **ID-Token** (JWT) zurück
- Token wird **lokal gecacht** (~/.kube/cache/oidc-login/)

#### 4. kubectl → API-Server

```
Authorization: Bearer eyJhbGc... (JWT-Token)
```

#### Wie ist ein jwt aufgebaut ?

```
3 Teile
1. Header
2. Payload
3. Signature
```

#### Wie sieht der Payload aus ?

```
{
 "iss": "https://provider1.example.com",
 "aud": "kubernetes",
 "email": "jochen@example.com",
 "groups": ["admins", "developers"]
}
```

#### traefik authentication mit oidc

##### 1. OAuth2-Proxy Deployment (wird für die Authentifizierung verwendet)

- In go geschrieben
- Besser: Mit helm - Chart ausrollen

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: oauth2-proxy
 namespace: traefik
spec:
 replicas: 1
 selector:
 matchLabels:
 app: oauth2-proxy
 template:
 metadata:
 labels:
 app: oauth2-proxy
 spec:
 containers:
 - name: oauth2-proxy
 image: quay.io/oauth2-proxy/oauth2-proxy:latest
 args:
 - --provider=oidc
 - --oidc-issuer-url=https://keycloak.example.com/realms/myrealm
 - --client-id=traefik
 - --client-secret=your-secret-here
 - --cookie-secret=random-32-byte-string
 - --email-domain=*
 - --upstream=static://200
 - --http-address=0.0.0.0:4180
 - --redirect-url=https://auth.example.com/oauth2/callback
 ports:
 - containerPort: 4180

apiVersion: v1
kind: Service
metadata:
 name: oauth2-proxy
 namespace: traefik
spec:
 selector:
 app: oauth2-proxy
 ports:
 - port: 4180
 targetPort: 4180
```

##### 2. Traefik Middleware

```

apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
 name: oidc-auth
 namespace: default
spec:
 forwardAuth:
 address: http://oauth2-proxy.traefik.svc.cluster.local:4180
 authResponseHeaders:
 - X-Auth-Request-User
 - X-Auth-Request-Email
 - X-Auth-Request-Access-Token

```

### 3. IngressRoute für OAuth2-Proxy selbst

```

apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata:
 name: oauth2-proxy
 namespace: default
spec:
 entryPoints:
 - websecure
 routes:
 - match: Host(`auth.example.com`)
 kind: Rule
 services:
 - name: oauth2-proxy
 port: 4180
 tls:
 certResolver: letsencrypt

```

### 4. Geschützte App mit Middleware

```

apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata:
 name: protected-app
 namespace: default
spec:
 entryPoints:
 - websecure
 routes:
 - match: Host(`app.example.com`)
 kind: Rule
 middlewares:
 - name: oidc-auth
 namespace: default
 services:
 - name: my-app
 port: 80
 tls:
 certResolver: letsencrypt

```

### Keycloak Client Setup

In Keycloak (oder anderem OIDC Provider):

- **Client ID:** traefik
- **Valid Redirect URIs:** <https://auth.example.com/oauth2/callback>
- **Access Type:** confidential
- **Standard Flow:** enabled

Cookie-Secret generieren:

```
python -c 'import os,base64; print(base64.urlsafe_b64encode(os.urandom(32)).decode())'
```

### Alternativ: Statt eigenen Pod als auth2-middleware möglich direkt in Traefik

- über die plugins beim Installieren des Helm-Charts
- <https://artifacthub.io/packages/helm/traefik/traefik>
- Plugin-Catalog: <https://artifacthub.io/packages/helm/traefik/traefik>
- Man kann z.B. verwenden:
  - <https://plugins.traefik.io/plugins/6613338ea28c508f411a44d5/traefik-oidc>

```
experimental.plugins
```

## Kubernetes Deployment - Internals

### Strategy when creating and terminating pods in Deployment - RollingUpdate - maxSurge, maxUnavailability

#### Ausgangssituation

- replicas auf 8 eingestellt im Deployment
- Änderung auf image, dass es nicht gibt

#### Welche Werte spielen hier eine Rolle

- `deployment.spec.strategy.rollingUpdate.maxUnavailability`
- `deployment.spec.strategy.rollingUpdate.maxSurge`

#### Wie sind die Standardwerte

- `maxSurge`: 25%
- `maxUnavailability`: 25%

```
strategy:
 rollingUpdate:
 maxSurge: 25%
 maxUnavailable: 25%
 type: RollingUpdate
```

#### Ablauf. RollingUpdate

##### Runde 1: Rolling Update legt los

- Ausgangszustand: 8 pods laufen
- Herausfinden, wieviel Pods im alten Replicaset sofort terminiert werden (`maxUnavailability`)
  - Beispiel: 8 Replicas (25% Unavailability) = 2 Pods -> dürfen sofort terminiert werden
- Herausfinden, wieviele Pods im neuen Replicaset dazu gestartet werden dürfen von
  - Insgesamt dürfen replicas: 8 + 25% gestartet werden, d.h. gesamt 10
  - Aktuell laufen 8, also noch 2

##### Ende von Runde 1:

- 6 Pods im alten Replicaset
- 2 Pods im neuen Replicaset

##### Runde 1: Nächste Runde

- Ausgangszustand: 6 Pods laufen (von 8 replicas)
- Wieviel dürfen gestoppt werden im alten Replicaset (replicas 8, davon 25% -> 2)
  - Also insgesamt müssen im 6 laufen
  - Es können keinen weiteren terminiert werden
- Wieviel dürfen im neuen Replicaset jetzt noch gestartet werden
  - Insgesamt dürfen replicas: 8 + 25% gestartet werden, d.h. gesamt 10
  - Insgesamt laufen im alten Replicaset 6 und im neuen Replicaset 2, also dürfen noch 2 gestartet werden

#### Referenz:

- <https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/deployment-v1/>

```
strategy.rollingUpdate.maxUnavailability:
- scaled down to pods immediately when the rolling update
```

## Kubernetes Ingress (Eingehender Traffic ins Cluster)

### Wann LoadBalancer, wann Ingress

#### Vorteile Load-Balancer

- Ich brauche keine https Termination
- Es ist mir egal wieviel IP-Adresse ich verbrauche (weil 1 (eine!) IP pro Service)
- kein http-Dienst sondern tcp stream or grpc)

#### Nachteile sind

- Keine Routing von Pfaden (d.h. <http://meine-domain.de> und <http://meine-domain.de/backend> gehen beide zum gleich Service)
- Kein Auswertung von HTTP-Headern
- Sehr schlechte Stickyness - Features (maximal bindung über die IP)

#### Vorteile von Ingress (gut für http und https)

- Routing von Pfaden (d.h. <http://meine-domain.de> und <http://meine-domain.de/backend> gehen beide zu unterschiedlichen Services)
- Auswertung von HTTP-Headern
- Anbieter mit sehr guter Session Stickyness
- Nur eine IP für alle Dienste (die IP, die Ingress selbst benötigt, um erreichbar zu sein)

- HTTPS Termination

## Kubernetes Ingress (Nginx - deprecated)

### Ingress Controller auf Digitalocean (doks) mit helm installieren

#### Basics

- Das Verfahren funktioniert auch so auf anderen Plattformen, wenn helm verwendet wird und noch kein IngressController vorhanden
- Ist kein IngressController vorhanden, werden die Ingress-Objekte zwar angelegt, es funktioniert aber nicht.

#### Prerequisites

- kubectl muss eingerichtet sein
- helm

#### Walkthrough Simple (Setup Ingress Controller)

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm upgrade --install nginx-ingress ingress-nginx/ingress-nginx --namespace ingress --create-namespace --version 4.13.2
```

```
See when the external ip comes available
kubectl -n ingress get pods
kubectl -n ingress get svc
```

```
Output
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
SELECTOR
nginx-ingress-ingress-nginx-controller LoadBalancer 10.245.78.34 157.245.20.222 80:31588/TCP,443:30704/TCP 4m39s
app.kubernetes.io/component=controller,app.kubernetes.io/instance=nginx-ingress,app.kubernetes.io/name=ingress-nginx
```

```
Now setup wildcard - domain for training purpose
*.lab.t3isp.de A 157.245.20.222
```

#### Walkthrough (Setup Ingress Controller)

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm show values ingress-nginx/ingress-nginx
```

```
vi values.yml
controller:
 publishService:
 enabled: true
```

```
It will be setup with type loadbalancer - so waiting to retrieve an ip from the external loadbalancer
This will take a little.
helm install nginx-ingress ingress-nginx/ingress-nginx --namespace ingress --create-namespace -f values.yml
```

```
See when the external ip comes available
kubectl -n ingress get all
kubectl --namespace ingress get services -o wide -w nginx-ingress-ingress-nginx-controller
```

```
Output
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
SELECTOR
nginx-ingress-ingress-nginx-controller LoadBalancer 10.245.78.34 157.245.20.222 80:31588/TCP,443:30704/TCP 4m39s
app.kubernetes.io/component=controller,app.kubernetes.io/instance=nginx-ingress,app.kubernetes.io/name=ingress-nginx
```

```
Now setup wildcard - domain for training purpose
*.app1.t3isp.de A 157.245.20.222
```

#### Documentation for default ingress nginx

- <https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/configmap/>

#### Beispiel Ingress

#### Prerequisites

```
Ingress Controller muss aktiviert sein
microk8s enable ingress
```

#### Walkthrough

```
mkdir apple-banana-ingress
```

```
apple.yml
vi apple.yml
kind: Pod
apiVersion: v1
metadata:
 name: apple-app
 labels:
 app: apple
spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple"

kind: Service
apiVersion: v1
metadata:
 name: apple-service
spec:
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f apple.yml
```

```
banana
vi banana.yml
kind: Pod
apiVersion: v1
metadata:
 name: banana-app
 labels:
 app: banana
spec:
 containers:
 - name: banana-app
 image: hashicorp/http-echo
 args:
 - "-text=banana"

kind: Service
apiVersion: v1
metadata:
 name: banana-service
spec:
 selector:
 app: banana
 ports:
 - port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f banana.yml
```

```
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - http:
 paths:
 - path: /apple
 backend:
```

```
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80
```

```
ingress
kubectl apply -f ingress.yml
kubectl get ing
```

## Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Find the problem

```
Hints

1. Which resources does our version of kubectl support
Can we find Ingress as "Kind" here.
kubectl api-resources

2. Let's see, how the configuration works
kubectl explain --api-version=networking.k8s.io/v1 ingress.spec.rules.http.paths.backend.service

now we can adjust our config
```

## Solution

```
in kubernetes 1.22.2 - ingress.yml needs to be modified like so.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - http:
 paths:
 - path: /apple
 pathType: Prefix
 backend:
 service:
 name: apple-service
 port:
 number: 80
 - path: /banana
 pathType: Prefix
 backend:
 service:
 name: banana-service
 port:
 number: 80
```

## Beispiel mit Hostnamen

### Step 1: Walkthrough

```
cd
cd manifests
mkdir abi
cd abi
nano apple.yml
```

```
apple.yml
vi apple.yml
kind: Pod
apiVersion: v1
metadata:
 name: apple-app
 labels:
 app: apple
spec:
 containers:
```

```

- name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple-<euer-name>"

kind: Service
apiVersion: v1
metadata:
 name: apple-service
spec:
 type: ClusterIP
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image

```

```
kubectl apply -f apple.yml
```

```
nano banana.yml
```

```

banana
vi banana.yml
kind: Pod
apiVersion: v1
metadata:
 name: banana-app
 labels:
 app: banana
spec:
 containers:
 - name: banana-app
 image: hashicorp/http-echo
 args:
 - "-text=banana-<euer-name>"

kind: Service
apiVersion: v1
metadata:
 name: banana-service
spec:
 type: ClusterIP
 selector:
 app: banana
 ports:
 - port: 80
 targetPort: 5678 # Default port for image

```

```
kubectl apply -f banana.yml
```

## Step 2: Testing connection by podIP and Service

```

kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox

```

```

/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>

```

## Step 3: Walkthrough

```
nano ingress.yml
```

```

Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:

```



```

ingressClassName: nginx
rules:
- host: "<euername>.lab1.t3isp.de"
 http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80

```

```

ingress
kubectl apply -f ingress.yml

```

## Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Find the problem

### Schritt 1: api-version ändern

```

Welche Landkarten gibt es ?
kubectl api-versions
Auf welcher Landkarte ist Ingress jetzt
kubectl explain ingress

```

```

GROUP: networking.k8s.io
KIND: Ingress
VERSION: v1

```

```

ingress ändern ingress.yml
von
apiVersion: extensions/v1beta1
in
apiVersion: networking.k8s.io/v1

```

### Schritt 2: Fehler Eigenschaften beheben

```

Error from server (BadRequest): error when creating "ingress.yml": Ingress in version "v1" cannot be handled as a Ingress: strict decoding error: unknown field "spec.rules[0].http.paths[0].backend.serviceName", unknown field "spec.rules[0].http.paths[0].backend.servicePort", unknown field "spec.rules[0].http.paths[1].backend.serviceName", unknown field "spec.rules[0].http.paths[1].backend.servicePort"

```

```

Problem serviceName beheben
Was gibt es stattdessen
-> es gibt service aber keine serviceName
kubectl explain ingress.spec.rules.http.paths.backend
-> es gibt service.name
kubectl explain ingress.spec.rules.http.paths.backend.

```

```

Korrektur 2x in ingress.yaml: Inkl. servicePort (neu: service.port.number)
vorher
backend:
serviceName: apple-service
servicePort: 80
jetzt:
service:
name: apple-service
port:
number: 80

```

```

kubectl apply -f .

```

### Schritt 3: pathType ergänzen

```
* spec.rules[0].http.paths[0].pathType: Required value: pathType must be specified
* spec.rules[0].http.paths[1].pathType: Required value: pathType must be specified
```

- Es wird festgelegt wie der Pfad ausgewertet

```
Wir müssen pathType auf der 1. Unterebene von paths einfügen
Entweder exact oder prefix
```

```
http:
 paths:
 - path: /apple
 pathType: Exact
 backend:
 service:
```

```
kubectl apply -f .
```

#### Schritt 4: Testen aufrufen

```
curl http://<euername>.lab1.t3isp.de/apple
curl http://<euername>.lab1.t3isp.de/apple/ # Sollte nicht funktioniert
curl http://<euername>.lab1.t3isp.de/banana
curl http://<euername>.lab1.t3isp.de/banana/
curl http://<euername>.lab1.t3isp.de/banana/something
```

Das kann man auch so im Browser eingeben

#### Solution

```
nano ingress.yml
```

```
in kubernetes 1.22.2 - ingress.yml needs to be modified like so.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - host: "<euername>.lab1.t3isp.de"
 http:
 paths:
 - path: /apple
 pathType: Prefix
 backend:
 service:
 name: apple-service
 port:
 number: 80
 - path: /banana
 pathType: Prefix
 backend:
 service:
 name: banana-service
 port:
 number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
mit describe herausfinden, ob er die services gefunden
kubectl describe ingress example-ingress
```

#### Beispiel Deployment mit Ingress und Hostnamen

##### Step 1: Walkthrough

```
cd
cd manifests
mkdir abi
cd abi
```

```
nano apple-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: apple-app
 labels:
 app: apple
spec:
 replicas: 1
 selector:
 matchLabels:
 app: apple
 template:
 metadata:
 labels:
 app: apple
 spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple-<euer-name>"
```

```
nano apple-svc.yml
```

```
kind: Service
apiVersion: v1
metadata:
 name: apple-service
spec:
 type: ClusterIP
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

```
nano banana-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: banana-app
 labels:
 app: banana
spec:
 replicas: 1
 selector:
 matchLabels:
 app: banana
 template:
 metadata:
 labels:
 app: banana
 spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=banana-<euer-name>"
```

```
nano banana-svc.yml
```

```
kind: Service
apiVersion: v1
metadata:
 name: banana-service
spec:
 type: ClusterIP
 selector:
 app: banana
```

```
ports:
 - port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

## Step 2: Testing connection by podIP and Service

```
kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox
```

```
/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>
```

## Step 3: Walkthrough

```
nano ingress.yml
```

```
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - host: "<euername>.lab.t3isp.de"
 http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80
```

```
ingress
kubectl apply -f ingress.yml
```

## Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Step 4: Find the problem

### Fix 4.1: Fehler: no matches kind "Ingress" in version "extensions/v1beta1"

```
Gibt es diese Landkarte überhaupt
kubectl api-versions
auf welcher Landkarte/Gruppe befindet sich Ingress jetzt
kubectl explain ingress
-> jetzt auf networking.k8s.io/v1
```

```
nano ingress.yaml
```

```
auf apiVersion: extensions/v1beta1
wird -> networking.k8s.io/v1
```

```
kubectl apply -f .
```

### Fix 4.2: Bad Request unkown field ServiceName / ServicePort

```
was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
```

```
nano ingress.yml
```

```
Wir ersetzen
serviceName: apple-service
durch:
service:
name: apple-service

das gleiche für banana
```

```
kubectl apply -f .
```

#### Fix 4.3. BadRequest unknown field servicePort

```
was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service.port
number
kubectl explain ingress.spec.rules.http.paths.backend.service.port
```

```
neue Variante sieht so aus
backend:
 service:
 name: apple-service
 port:
 number: 80
das gleich für banana-service
```

```
kubectl apply -f .
```

#### Fix 4.4. pathType must be specified

```
Was macht das ?
kubectl explain ingress.spec.rules.http.paths.pathType
```

```
paths:
- path: /apple
 pathType: Prefix
 backend:
 service:
 name: apple-service
 port:
 number: 80
- path: /banana
 pathType: Exact
 backend:
 service:
 name: banana-service
 port:
 number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
```

#### Step 5: Testing

```
mit describe herausfinden, ob er die services gefunden
kubectl describe ingress example-ingress
```

```
Im Browser auf:
hier euer Name
http://jochen.lab.t3isp.de/apple
http://jochen.lab.t3isp.de/apple/
http://jochen.lab.t3isp.de/apple/foo
http://jochen.lab.t3isp.de/banana
geht nicht
http://jochen.lab.t3isp.de/banana/nix
```

#### Achtung: Ingress mit Helm - annotations

##### Welcher wird verwendet, angeben:

Damit das Ingress Objekt welcher Controller verwendet werden soll, muss dieser angegeben werden:

```
kubernetes.io/ingress.class: nginx
```

```

Als ganzes Object:
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
 kubernetes.io/ingress.class: nginx
spec:
 rules:
 - http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80

```

#### Ref:

- <https://www.digitalocean.com/community/tutorials/how-to-set-up-an-nginx-ingress-on-digitalocean-kubernetes-using-helm>

### Permanente Weiterleitung mit Ingress

#### Example

```

redirect.yml
apiVersion: v1
kind: Namespace
metadata:
 name: my-namespace

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 annotations:
 nginx.ingress.kubernetes.io/permanent-redirect: https://www.google.de
 nginx.ingress.kubernetes.io/permanent-redirect-code: "308"
 name: destination-home
 namespace: my-namespace
spec:
 rules:
 - http:
 paths:
 - backend:
 service:
 name: http-svc
 port:
 number: 80
 path: /source
 pathType: ImplementationSpecific

```

```

eine node mit ip-adresse aufrufen
curl -I http://41.12.45.21/source
HTTP/1.1 308
Permanent Redirect

```

#### Umbauen zu google ;o)

This annotation allows to return a permanent redirect instead of sending data to the upstream. For example `nginx.ingress.kubernetes.io/permanent-redirect: https://www.google.com` would redirect everything to Google.

#### Refs:

- <https://github.com/kubernetes/ingress-nginx/blob/main/docs/user-guide/nginx-configuration/annotations.md#permanent-redirect>
- 

### ServiceMesh

#### Why a ServiceMesh ?

## What is a service mesh ?

```
A service mesh is an infrastructure layer
that gives applications capabilities
like zero-trust security, observability,
and advanced traffic management, without code changes.
```

### Advantages / Features

1. Observability & monitoring
2. Traffic management
3. Resilience & Reliability
4. Security
5. Service Discovery

### Observability & monitoring

- Service mesh offers:
  - valuable insights into the communication between services
  - effective monitoring to help in troubleshooting application errors.

### Traffic management

- Service mesh offers:
  - intelligent request distribution
  - load balancing,
  - support for canary deployments.
  - These capabilities enhance resource utilization and enable efficient traffic management

### Resilience & Reliability

- By handling retries, timeouts, and failures,
  - service mesh contributes to the overall stability and resilience of services
  - reducing the impact of potential disruptions.

### Security

- Service mesh enforces security policies, and handles authentication, authorization, and encryption
  - ensuring secure communication between services and eventually, strengthening the overall security posture of the application.

### Service Discovery

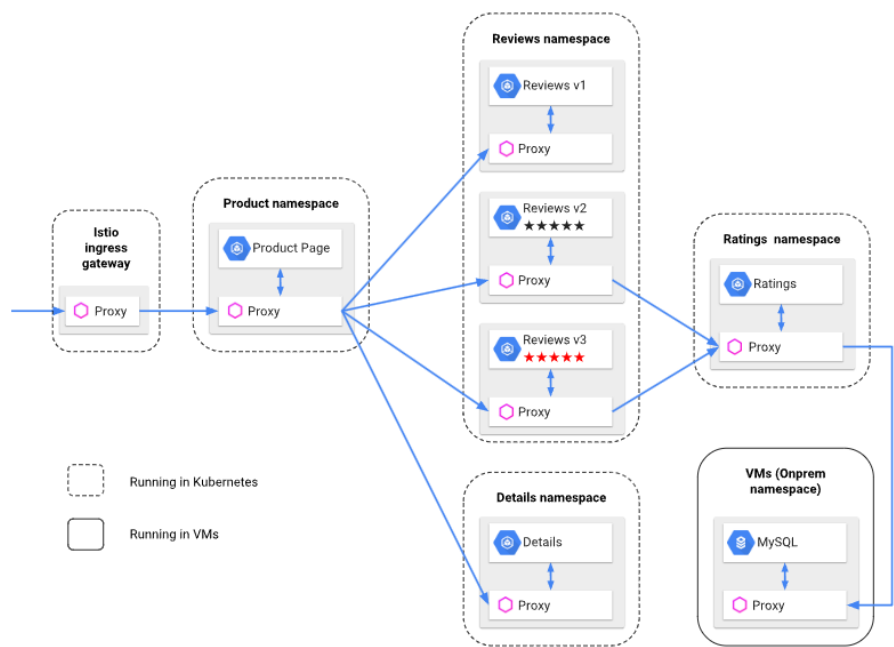
- With service discovery features, service mesh can simplify the process of locating and routing services dynamically
- adapting to system changes seamlessly. This enables easier management and interaction between services.

### Overall benefits

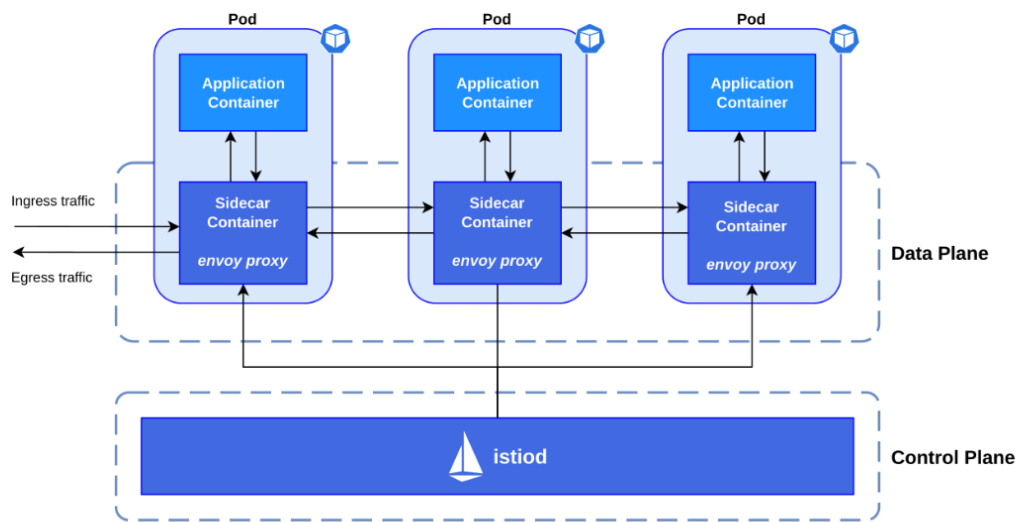
```
Microservices communication:
Adopting a service mesh can simplify the implementation of a microservices architecture by abstracting away infrastructure complexities.
It provides a standardized approach to manage and orchestrate communication within the microservices ecosystem.
```

## How does a ServiceMeshs work? (example istio

# Overview



## Istio control plane and data plane



• Source: kubebyexample.com

istio vs. ingress



## Klassisch Ingress (Kubernetes)

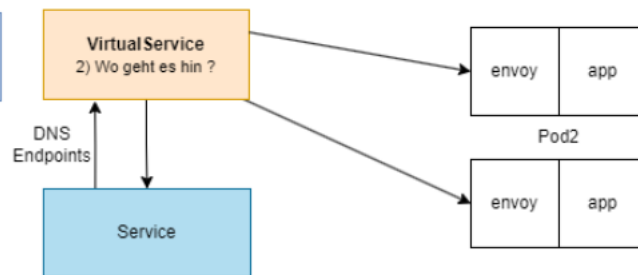
IngressController  
Nginx  
(Die Software  
aka nginx)

Ingress (Config)  
1) Was nehme ich an ?  
2) Wo geht es hin ?

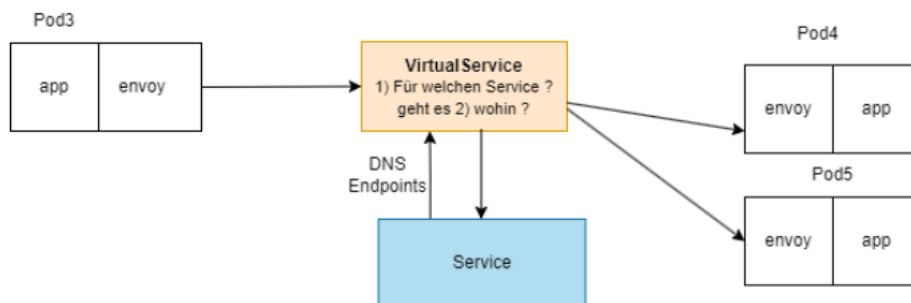
## Istio (Traffic in das Cluster hinein)

ingressgateway  
Nginx  
(Die Software  
aka envoy-proxy)

Gateway (config)  
1) Was nehme ich an ?  
(für Host / Namespace)  
-> in das Cluster hinein <-

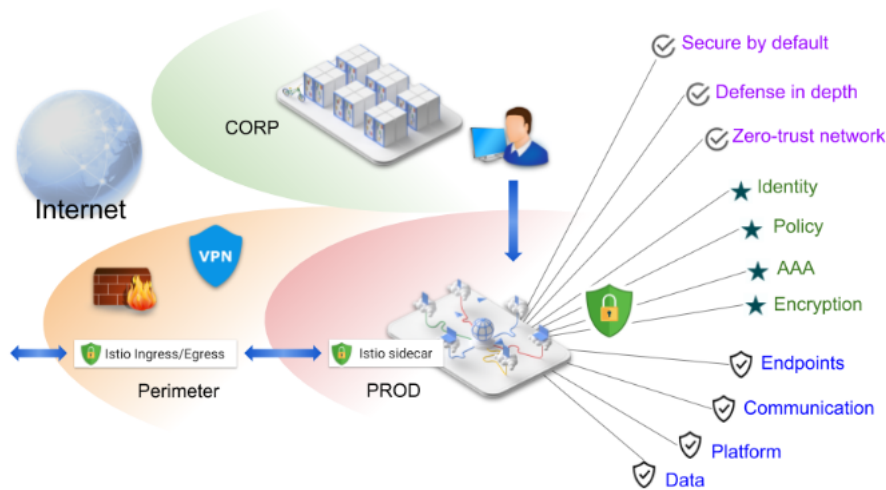


## Istio (Innerhalb des Clusters)



istio security features

Overview



Security overview

#### Security needs of microservices

- To defend against man-in-the-middle attacks, they need traffic encryption.
- To provide flexible service access control, they need mutual TLS and fine-grained access policies.
- To determine who did what at what time, they need auditing tools.

#### Implementation of security

The Istio security features provide strong identity, powerful policy, transparent TLS encryption, and authentication, authorization and audit (AAA) tools to protect your services and data. The goals of Istio security are:

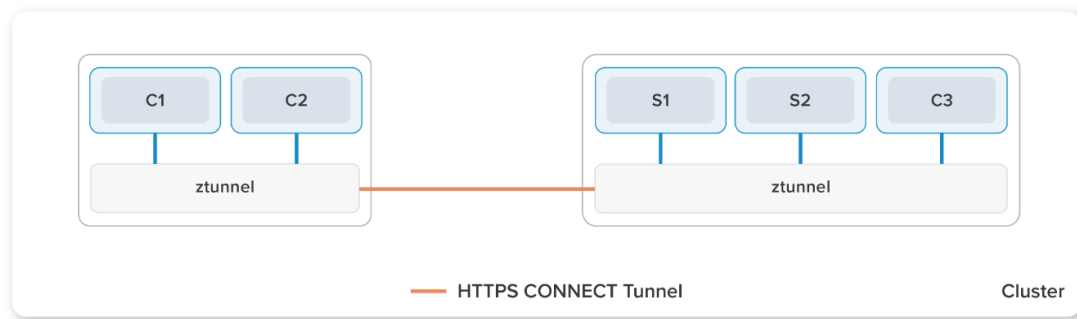
- Security by default: no changes needed to application code and infrastructure
- Defense in depth: integrate with existing security systems to provide multiple layers of defense
- Zero-trust network: build security solutions on distrusted networks

#### istio-service mesh - ambient mode

##### Light: Only Layer 4 per node (ztunnel)

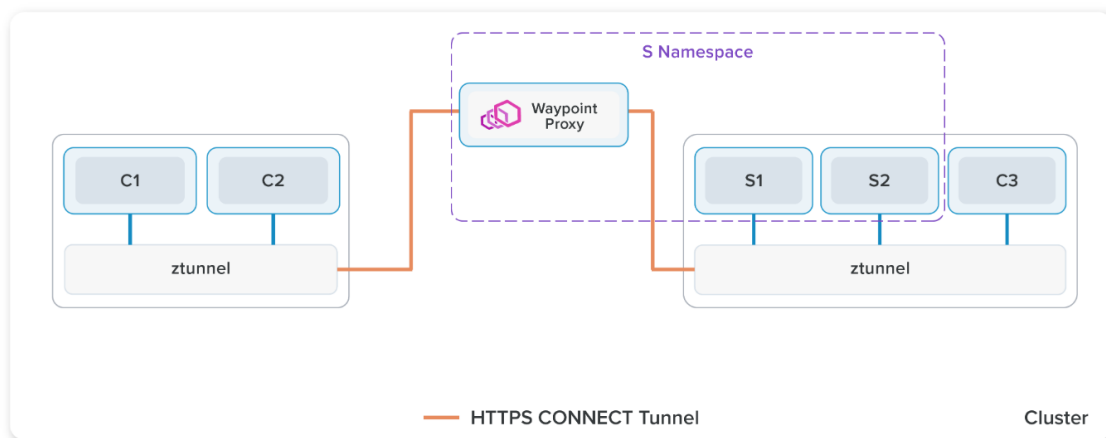
- No sidecar (envoy-proxy) per Pod, but one ztunnel agent per Node (Layer 4)
- Enables security features (mtls, traffic encryption)

Like so:



#### Full fledged: Layer 4 (ztunnel) per Node & Layer 7 per Namespace

- One waypoint - proxy is rolled out per Namespace, which connects to the ztunnel agents



When additional features are needed, ambient mesh deploys waypoint proxies, which ztunnels connect through for policy enforcement

#### Features in "fully-fledged" - ambient - mode

Application deployment use case	Ambient mode configuration
Zero Trust networking via mutual-TLS, encrypted and tunneled data transport of client application traffic, L4 authorization, L4 telemetry	ztunnel only (default)
As above, plus advanced Istio traffic management features (including L7 authorization, telemetry and VirtualService routing)	ztunnel and waypoint proxies

#### Advantages:

- Less overhead
- One can start step-by-step moving towards a mesh (Layer 4 firstly and if wanted Layer 7 for specific namespaces)
- Old pattern: sidecar and new pattern: ambient can live side by side.

#### Performance comparison - baseline, sidecar, ambient

- <https://livewyer.io/blog/2024/06/06/comparison-of-service-meshes-part-two/>
- <https://github.com/livewyer-gps/poc-servicemesh2024/blob/main/docs/test-report-istio.md>

## Kubernetes Ingress

### Ingress HA-Proxy Sticky Session

It easy to setup session stickyness

- <https://www.haproxy.com/documentation/kubernetes-ingress/ingress-tutorials/load-balancing/>

Nginx Ingress Session Stickyness

Yes, **session stickiness (affinity)** via **cookie-based stickiness does work** with the **open-source NGINX Ingress Controller**.

Here's what you need to know to get it working:

✔ How to Enable Sticky Sessions with Open Source NGINX Ingress

The open-source NGINX Ingress Controller supports sticky sessions using **annotations** and **cookies**.

Example Ingress YAML:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: my-app-ingress
 annotations:
 nginx.ingress.kubernetes.io/affinity: "cookie"
 nginx.ingress.kubernetes.io/session-cookie-name: "route"
 nginx.ingress.kubernetes.io/session-cookie-hash: "sha1"
spec:
 ingressClassName: nginx
 rules:
 - host: myapp.example.com
 http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: my-app-service
 port:
 number: 80
```

Explanation of the Annotations

- `nginx.ingress.kubernetes.io/affinity: "cookie"`  
→ Enables cookie-based affinity.
- `nginx.ingress.kubernetes.io/session-cookie-name: "route"`  
→ Names the session cookie (optional, default is `INGRESSCOOKIE` ).
- `nginx.ingress.kubernetes.io/session-cookie-hash: "sha1"`  
→ Defines how the cookie value is hashed. Can be `md5` , `sha1` , `index` , etc.

You can also optionally define:

```
nginx.ingress.kubernetes.io/session-cookie-path: "/"
nginx.ingress.kubernetes.io/session-cookie-expires: "172800" # in seconds
nginx.ingress.kubernetes.io/session-cookie-max-age: "172800"
```

Important Notes

- This works **only with the open-source NGINX ingress controller**.  
(The annotations won't apply to controllers like Traefik, HAProxy, or cloud-specific ones like AWS ALB.)
- You must ensure that **all replicas of the app can handle the traffic** if they receive the same cookie.
- This stickiness is **not persistent across restarts** unless the session cookie is preserved client-side (and the pod labels/service don't change unpredictably).

Bonus Tip: Enable Ingress Logging (Optional but Useful)

To troubleshoot or verify stickiness:

```
controller:
 config:
 enable-access-log: "true"
 log-format-upstream: "$request_id $remote_addr to $upstream_addr via $cookie_route"
```

Want a ready-made Helm values file or `kubect1` manifest for this?

Flow Overview: Cookie-Based Stickiness in NGINX Ingress

```
[Client Browser]
 ||
[NGINX Ingress Controller (Ingress Pod)]
 ||
[Kubernetes Service (ClusterIP)]
 ||
[App Pod A / B / C (Behind the Service)]
```

### What Actually Happens (Step-by-Step)

#### 1. Client makes a request

- Let's say the client hits `myapp.example.com`.

#### 2. Ingress Controller (NGINX) receives the request

- NGINX is exposed via a LoadBalancer, NodePort, or IngressClass.
- It parses the Ingress resource and applies sticky session rules based on annotations.

#### 3. First Request: No Cookie

- No session cookie is present, so:
  - NGINX picks a backend pod **randomly** via the `Kubernetes Service`.
  - It sets a sticky cookie on the response, e.g.:

```
Set-Cookie: route=backend1; Path=/; HttpOnly
```

#### 4. Subsequent Requests: Cookie Present

- On later requests, the client sends back:

```
Cookie: route=backend1
```

- NGINX uses this cookie value to route to the **same backend pod**.

### So Where Does the `Kubernetes Service` Come In?

The `Kubernetes Service` is used **internally by NGINX** to **proxy requests to pods**.

NGINX configuration looks like this (simplified):

```
upstream myapp-service {
 sticky cookie route;
 server 10.0.1.2:8080; # Pod A
 server 10.0.1.3:8080; # Pod B
}
```

- These IPs are discovered **via the `Kubernetes Service`** using Endpoints or EndpointSlices.
- NGINX tracks these pods and their IPs automatically (via a sync controller loop).

### ✓ Who Makes Routing Decisions?

- NGINX Ingress Controller** makes the routing decision **based on the cookie value**, not `Kubernetes`.
- `Kubernetes Service` is just a **source of backend pod IPs**, not the router in this case.

### https mit ingressController und Letsencrypt

#### Schritt 1: cert-manager installieren

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
--namespace cert-manager --create-namespace \
--version v1.19.1 \
--set installCRDs=true
```

- Ref: <https://artifacthub.io/packages/helm/cert-manager/cert-manager>

#### Schritt 2: Create ClusterIssuer (gets certificates from Letsencrypt)

```
cd
mkdir -p manifests/cert-manager
cd manifests/cert-manager
nano cluster-issuer.yaml
```

```
cluster-issuer.yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
 name: letsencrypt-prod
```

```
spec:
 acme:
 server: https://acme-v02.api.letsencrypt.org/directory
 email: your-email@example.com
 privateKeySecretRef:
 name: letsencrypt-prod
 solvers:
 - http01:
 ingress:
 class: nginx
```

```
kubectl apply -f .
Should be True
kubectl get clusterissuer
```

### Schritt 3: Ingress-Objekt mit TLS erstellen

```
tls-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 cert-manager.io/cluster-issuer: "letsencrypt-prod"
 nginx.ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 tls:
 - hosts:
 - test.devopslearnwith.us
 secretName: example-tls
 rules:
 - host: test.devopslearnwith.us
 http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: example-service
 port:
 number: 80
```

### Schritt 4: Herausfinden, ob Zertifikate erstellt werden

```
kubectl describe certificate example-tls
kubectl get cert
```

Schritt 5: Testen

#### Ref:

- <https://hbayraktar.medium.com/installing-cert-manager-and-nginx-ingress-with-lets-encrypt-on-kubernetes-fe0dff4b1924>

## Kubernetes Pod Termination

### LifeCycle Termination

- <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-termination>

### preStopHook

- <https://www.datree.io/resources/kubernetes-guide-graceful-shutdown-with-lifecycle-prestop-hook>

### How to wait till a pod gets terminated

When stopping (terminating) a pod in Kubernetes, there **are hooks available**, but they're limited to the **termination lifecycle**. If you're thinking of stopping a pod *without* killing it and still triggering hooks, that's trickier (more on that below). Here's a full breakdown of what's available:

#### ✔ Kubernetes Hooks for Pod Termination

##### 1. preStop Hook

Executed **before the container is terminated** (but *after* the SIGTERM signal is sent).

#### Example:

```
lifecycle:
 preStop:
```

```
exec:
 command: ["/bin/sh", "-c", "echo pre-stop hook running; sleep 10"]
```

- You can use this to delay shutdown or do cleanup.
- Note: Kubernetes still sends `SIGTERM` as normal; `preStop` doesn't block that, but the container stays up for the duration of the hook.

## 2. `terminationGracePeriodSeconds`

Specifies how long Kubernetes will wait before force-killing the container with `SIGKILL`.

```
spec:
 terminationGracePeriodSeconds: 30
```

- Combine this with `preStop` to allow your container to finish in-flight work or deregister from a service.

If you tell me **what kind of "stop" you're envisioning**, I can help tailor a hook-like workaround. For example:

→ "I want to stop background work when CPU is high" or

→ "Pause processing until a Kafka topic is ready"—totally different tactics.

## Kubernetes Security

### Best practices security pods

#### 5. Security / Best practice pods

##### 5.1. Pods

###### 1) Use Readiness / Liveness check

Not we really security, but to have a stable system

###### 2) Use Non-Root Images

(is not allowed in OpenShift anyways)

###### 3) SecurityContext: Restrict the Features in the pod/container as much as possible

Essentially covered by Default SCC's:

<https://docs.openshift.com/container-platform/4.18/authentication/managing-security-context-constraints.html>

Essentially use the v2 versions.

Question will Always be: Do I really Need this for this post

(e.g. HostNetwork). Is there are better/safer way to achieve this

### Best practices in general

#### 6. Security (other stuff)

##### 6.1. Be sure upgrade your system and use the newest versions (OS / OpenShift)

##### 6.2. Setup Firewall rules, for the cluster components. (OpenShift) -

[https://docs.openshift.com/container-platform/4.16/installing/install\\_config/configuring-firewall.html](https://docs.openshift.com/container-platform/4.16/installing/install_config/configuring-firewall.html)

##### 6.3. Do not install any components, that you do not Need (with helm)

##### 6.4. Always download Images instead of using them locally.

I think it also has to do with auth. When set to always, the pod will pull the image from the registry, hence it has to do auth and have valid credentials to actually get the image.

If the image is already in the node, and let's say permission has been removed to access that image for that node in the registry, a pod could still be created since the image is already there.

-> Wie sicherstellen, dass das gesetzt ist ?

OPA Gateway

#### 6.5. Scan all your Images before using them

##### 6.5.1. In development

##### 6.5.2. CI / CD Pipeline

##### 6.5.3 Registry (when uploading them)

#### 6.6. Restrict ssh Access

(no ssh-access to cluster nodes please !)

#### 6.7. Use NetworkPolicies

```
https://docs.openshift.com/container-platform/4.12/networking/network_policy/about-network-policy.html
-> BUT: Use the specific Network Policies of your CNI
```

```
Images in kubernetes von privatem Repo verwenden
```

```
* Zugriff auf registries mit authentifizierung
```

```
Exercise
```

```
mkdir -p manifests cd manifests mkdir private-repo cd private-repo
```

```
kubectl create secret docker-registry regcred --docker-server=registry.do.t3isp.de
--docker-username=11trainingdo --docker-password= --dry-run=client -o yaml > 01-secret.yaml
```

```
kubectl create secret generic mariadb-secret --from-literal=MARIADB_ROOT_PASSWORD=11abc432 --dry-run=client -o yaml > 02-secret.yml
```

```
nano 02-pod.yaml
```

```
apiVersion: v1 kind: Pod metadata: name: private-reg spec: containers:
```

- name: private-reg-container image: registry.do.t3isp.de/mariadb:11.4.5 envFrom:
  - secretRef: name: mariadb-secret

```
imagePullSecrets:
```

- name: regcred

```
kubectl apply -f . kubectl get pods -o wide private-reg kubectl describe pods private-reg
```

## Kubernetes Monitoring/Security

### Überwachung, ob Images veraltet sind, direkt in Kubernetes

- Can also update images (i would always go towards gitlab ci/cd doing this)
- Kann z.B. über slack benachrichtigen

### Setup (Achtung ungetestet)

```
Korrekte Struktur für aktuelle Keel Version
helmProvider:
 enabled: true

Korrekte Notification-Konfiguration
notification:
 slack:
 enabled: true
 token: "xoxb-YOUR-TOKEN"
 channel: "#updates"

Korrekte Approval-Konfiguration
approvals:
 enabled: true

Korrekte Trigger-Konfiguration
triggers:
 poll:
 enabled: true
 pubsub:
 enabled: false
```

```
helm repo add keel https://charts.keel.sh
helm repo update
```

```
2. Installation mit der values.yaml
helm install keel keel/keel \
 --namespace keel \
```



```
--create-namespace \
-f values.yaml
```

## Helm (IDE - Support)

### Kubernetes-Plugin IntelliJ

- <https://www.jetbrains.com/help/idea/kubernetes.html>

### IntelliJ - Helm Support Through Kubernetes Plugin

- <https://blog.jetbrains.com/idea/2018/10/intellij-idea-2018-3-helm-support/>

## Helm - Charts entwickeln

### Unser erstes Helm Chart erstellen

#### Chart erstellen

```
cd
mkdir my-charts
cd my-charts
```

```
helm create my-app
```

#### Install helm - chart

```
Variante 1:
helm -n my-app-<namenskuerzel> install meine-app my-app --create-namespace
```

```
Variante 2:
cd my-app
helm -n my-app-<namenskuerzel> install meine-app . --create-namespace
```

```
helm -n my-app-<namenskuerzel> status meine-app
kubectl -n my-app-<namenskuerzel> get all
kubectl -n my-app-<namenskuerzel> get pods
```

#### Aufräumen

```
helm -n my-app-<namenskuerzel> uninstall meine-app
```

### Wie starte ich am besten - Übung

#### Exercise

```
cd
mkdir -p my-charts
cd my-charts
helm create simple-chart
```

```
Alles Weg was wir nicht brauchen
cd simple-chart
rm values.yaml
cd templates
rm -f *.yaml
rm -fR tests
echo "Ausgabe nach Install" > NOTES.txt
```

```
nano deploy.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
spec:
 selector:
 matchLabels:
 app: nginx
 replicas: 8 # tells deployment to run 8 pods matching the template
 template:
 metadata:
 labels:
 app: nginx
 spec:
 containers:
```

```
- name: nginx
 image: nginx:1.26
 ports:
 - containerPort: 80
```

```
aus dem templates ordner raus
cd ..
aus dem chart raus
cd ..
```

```
Installieren
helm -n my-simple-app-<namenskuerzel> upgrade --install my-simple-app simple-chart --create-namespace
kubectl -n my-simple-app-<namenskuerzel> get all
```

## Exercise Phase 2: Um Replicas erweitern

```
cd simple-chart
nano values.yaml
```

```
deployment:
 replicas: 5
```

```
cd templates
nano deploy.yaml
```

```
aus der Zeile:
replicas: 9
wird ->
replicas: {{ .Values.deployment.replicas }}
```

```
â Gehen aus dem Chart raus
cd ..
cd ..
helm template simple-chart
helm -n my-simple-app-<namenskuerzel> upgrade --install my-simple-app simple-chart --create-namespace
kubectl -n my-simple-app-<namenskuerzel> get pods
```

```
nano simple-app-values.yaml
```

```
deployment:
 replicas: 2
```

```
helm -n my-simple-app-<namenskuerzel> upgrade --install my-simple-app simple-chart -f simple-app-values.yaml
kubectl -n my-simple-app-<namenskuerzel> get pods
```

## Helm und Kustomize kombinieren

### Helm und Kustomize kombinieren

#### Übersicht

Die Kombination von Helm und Kustomize bietet die Flexibilität von Kustomize mit der Paketierung und Versionierung von Helm. Dies ist besonders nützlich für komplexe Deployments, die environment-spezifische Anpassungen benötigen.

#### Helm Post-Rendering mit Kustomize

##### Grundlegendes Konzept

Helm kann nach dem Template-Rendering einen Post-Renderer aufrufen. Hier kann Kustomize die gerenderten Manifeste weiter anpassen.

##### Workflow

1. Helm rendert Templates basierend auf Values
2. Kustomize modifiziert die gerenderten Manifeste
3. Finale Manifeste werden deployed

#### Übung

##### Schritt 1: Arbeitsverzeichnis erstellen

```
Erstelle Arbeitsverzeichnis
cd
mkdir helm-kustomize-demo
cd helm-kustomize-demo
```

##### Schritt 2: Helm Chart erstellen

```
Erstelle ein neues Helm Chart
helm create my-chart
```

### Schritt 3: Kustomize-Verzeichnis erstellen

```
Erstelle Kustomize-Verzeichnis
mkdir kustomize
cd kustomize
```

### Schritt 4: Post-Renderer Script erstellen

```
Erstelle das Post-Renderer Script
cat > kustomize-post-renderer.sh << 'EOF'
#!/bin/bash
Wechsle ins kustomize Verzeichnis
cd "$(dirname "$0")"
Speichere Helm Output als base.yaml
cat <&0 > base.yaml
Führe kustomize build aus
kubectl kustomize .
EOF

Script ausführbar machen
chmod +x kustomize-post-renderer.sh
```

### Schritt 5: Patches-Verzeichnis erstellen

```
Erstelle patches Verzeichnis
mkdir -p patches
```

### Schritt 6: Deployment Patch erstellen

```
Erstelle deployment-patch.yaml
cat > patches/deployment-patch.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app-my-chart
spec:
 template:
 spec:
 containers:
 - name: my-chart
 resources:
 requests:
 memory: "80Mi"
 cpu: "300m"
 limits:
 memory: "80Mi"
 cpu: "300m"
EOF
```

### Schritt 7: Kustomization.yaml erstellen

```
Erstelle kustomization.yaml
cat > kustomization.yaml << 'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- base.yaml

patches:
- path: patches/deployment-patch.yaml

images:
- name: nginx
 newTag: "1.21"
EOF
```

### Schritt 8: Zurück ins Hauptverzeichnis

```
Gehe zurück ins Hauptverzeichnis
cd ..
```

#### Schritt 9: Verzeichnisstruktur prüfen

```
Prüfe die Verzeichnisstruktur
tree .
Ergebnis sollte sein:
.
├── kustomize/
| ├── kustomization.yaml
| ├── kustomize-post-renderer.sh
| └── patches/
| └── deployment-patch.yaml
└── my-chart/
├── Chart.yaml
├── charts/
├── templates/
└── values.yaml
```

#### Schritt 10: Deployment testen

```
Teste das Setup mit dry-run
helm upgrade --install -n my-kapp-<namenskuerzel> my-app ./my-chart --post-renderer ./kustomize/kustomize-post-renderer.sh --dry-run --debug --create-namespace
```

#### Schritt 11: Deployment ausführen

```
Führe das Deployment aus
helm upgrade --install -n my-kapp-<namenskuerzel> my-app ./my-chart --post-renderer ./kustomize/kustomize-post-renderer.sh --create-namespace

helm -n my-kapp-<namenskuerzel> list
helm -n my-kapp-<namenskuerzel> get manifest my-app
```

#### Schritt 12: Deployment prüfen

```
Prüfe das Deployment
kubectl -n my-kapp-<namenskuerzel> get pods
kubectl -n my-kapp-<namenskuerzel> describe deployment my-app-my-chart
```

### Environment-spezifische Anpassungen

#### Ordnerstruktur

```
helm-kustomize/
├── chart/
│ ├── Chart.yaml
│ ├── values.yaml
│ └── templates/
├── environments/
│ ├── dev/
│ │ ├── kustomization.yaml
│ │ └── patches/
│ ├── staging/
│ │ ├── kustomization.yaml
│ │ └── patches/
│ └── prod/
│ ├── kustomization.yaml
│ └── patches/
└── scripts/
 └── deploy.sh
```

#### Development Environment

```
environments/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base.yaml

patches:
```

```
- path: patches/dev-resources.yaml
```

```
replicas:
```

```
- name: my-app
 count: 1
```

```
commonLabels:
```

```
 environment: dev
```

## Production Environment

```
environments/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
resources:
```

```
- ../../base.yaml
```

```
patches:
```

```
- path: patches/prod-resources.yaml
- path: patches/prod-security.yaml
```

```
replicas:
```

```
- name: my-app
 count: 3
```

```
commonLabels:
```

```
 environment: prod
```

## Best Practices

### 2. Testing

```
Dry-run für Testing
helm template my-app ./chart --values values-dev.yaml | \
 kubectl kustomize environments/dev | \
 kubectl apply --dry-run=client -f -
```

## LoadBalancer on Premise (metallb)

### Metallb

#### General

- Supports bgp and arp
- Divided into controller, speaker

#### Installation Ways

- helm
- manifests

#### Step 1: install metallb

```
Just to show some basics
Page from metallb says that digitalocean is not really supported well
So we will not install the speaker .
```

```
helm repo add metallb https://metallb.github.io/metallb
```

```
Eventually disabling speaker
vi values.yaml
```

```
helm install metallb metallb/metallb --namespace=metallb-system --create-namespace --version 0.14.8
```

#### Step 2: addresspool und Propagation-type (config)

```
cd
mkdir -p manifests
cd manifests
mkdir lb
cd lb
nano 01-addresspool.yaml
```

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
```

```
metadata:
 name: first-pool
 namespace: metallb-system
spec:
 addresses:
 # we will use our external ip here
 - 134.209.231.154-134.209.231.154
 # both notations are possible
 - 157.230.113.124/32
```

```
kubectl apply -f .
```

```
nano 02-advertisement.yml
```

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
 name: example
 namespace: metallb-system
```

```
kubectl apply -f .
```

#### Schritt 4: Test ob i get an external ip

```
nano 03-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-nginx
spec:
 selector:
 matchLabels:
 run: web-nginx
 replicas: 3
 template:
 metadata:
 labels:
 run: web-nginx
 spec:
 containers:
 - name: cont-nginx
 image: nginx
 ports:
 - containerPort: 80
```

```
nano 04-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
 name: svc-nginx
 labels:
 svc: nginx
spec:
 type: LoadBalancer
 ports:
 - port: 80
 protocol: TCP
 selector:
 run: web-nginx
```

```
kubectl apply -f .
kubectl get pods
kubectl get svc
```

```
auf dem client
curl http://<ip aus get svc>
```

```
kubectl delete -f 03-deploy.yml 04-service.yml
```

#### Schritt 5: Referenz:

- <https://metallb.io/installation/#installation-with-helm>

## Kubernetes Verlässlichkeit erreichen

### Keine 2 pods auf gleichem Node - PodAntiAffinity

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 3
 selector:
 matchLabels:
 app: my-app
 template:
 metadata:
 labels:
 app: my-app
 spec:
 affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchLabels:
 app: my-app
 topologyKey: kubernetes.io/hostname
 containers:
 - name: my-app
 image: nginx:latest
 ports:
 - containerPort: 80
```

## Metrics-Server / Größe Cluster

### Metrics-Server mit helm installieren

#### Warum ? Was macht er ?

```
Der Metrics-Server sammelt Informationen von den einzelnen Nodes und Pods
Er bietet mit

kubectl top pods
kubectl top nodes

ein einfaches Interface, um einen ersten Eindruck über die Auslastung zu bekommen.
```

#### Walktrough

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm -n kube-system upgrade --install metrics-server metrics-server/metrics-server --version 3.13.0
```

```
Es dauert jetzt einen Moment bis dieser aktiv ist auch nach der Installation
Auf dem Client
kubectl top nodes
kubectl top pods
```

#### Kubernetes

- <https://kubernetes-sigs.github.io/metrics-server/>
- `kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml`

#### Speichernutzung und CPU berechnen für Anwendungen

- <https://learnk8s.io/kubernetes-node-size>

## Kubernetes -> High Availability Cluster (multi-data center)

### High Availability multiple data-centers

#### What needs to be there ?

- etcd in einer ungeraden Zahl an Rechenzentrum in ungerade Zahl, d.h. z.B. 1 etcd in 3 Rechenzentren = 3 etcd (ungerade)
  - etcd kann auch in einem Rechenzentrum in der Cloud sein, in der nur etcd läuft und keine anderen Komponenten (Schiedsrichter)
- Control Plane in jedem Rechenzentrum, was verwendet wird (nicht im "Schiedsrichter" - Rechenzentrum)
- HA für ControlPlane (entweder HA Proxy oder kube-vip (ist dann LoadBalancer im Cluster kubernetes-native))

#### Ausblick für Kubernetes Applikationen selbst

- Pods ausgerollt über (Deployment etc.) dürfen auch nicht nur in einem Rechenzentrum (pod anti affinity)

- IngressController in jedem Rechenzentrum (einfachste Variante : DaemonSet)

## PodAntiAffinity für Hochverfügbarkeit

### PodAffinity

#### Exercise

```
Deployment 1
apiVersion: apps/v1
kind: Deployment
metadata:
 name: app-a
spec:
 replicas: 1
 selector:
 matchLabels:
 app: app-a
 template:
 metadata:
 labels:
 app: app-a
 spec:
 containers:
 - name: app-a
 image: nginx
```

```
Deployment 2
apiVersion: apps/v1
kind: Deployment
metadata:
 name: app-b
spec:
 replicas: 1
 selector:
 matchLabels:
 app: app-b
 template:
 metadata:
 labels:
 app: app-b
 spec:
 affinity:
 podAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - app-a
 topologyKey: "kubernetes.io/hostname"
 containers:
 - name: app-b
 image: nginx
```

## Kubernetes -> etcd

### etcd - cleaning of events

## Who cleans up events in etc, kubernetes api server ?

### Overview

No, the **Kubernetes API server** does not directly clean up events in etcd.

#### Who Cleans Up Events in etcd?

- **The Kubernetes controller manager** is responsible for cleaning up expired events, **not the API server**.
- The **event garbage collection** process runs periodically to remove events whose **TTL (Time-to-Live)** has expired.
- The **default TTL is 1 hour**, but this can be configured using the `--event-ttl` flag in the API server settings.

#### How Does Event Cleanup Work?

1. **Event is Created** → Stored in `etcd`.
2. **TTL Countdown Begins** → Typically 1 hour unless configured otherwise.
3. **Event Expires** → Becomes eligible for deletion.
4. **Garbage Collection (Controller Manager)** → Detects expired events and removes them from `etcd`.

#### Key Takeaway:

- The API server **stores and serves** event data from `etcd`, but it **does not handle cleanup**.



```
- The controller manager is responsible for event garbage collection after TTL expiration.
```

```
etcd in multi-data-center setup
```

```
Wieviele ?
```

- \* Ungerade Zahl an etcd - Nodes (3,5,7 max. 7)
- \* Ausreichend in der Regel 3
- \* Wenn man möchte, dass 2 ausfallen können dann 5

```
Besonderheiten bei der Nutzung der etc.
```

- \* Schnelle Platte, idealerweise ssd .
- \* Begrenzung des Key->Values Stores auf 2,1 GB (standardmäßig)

```
Besonderheiten multi-data-center Setup
```

- \* Ursprünglich nicht dafür entwickelt.
- \* Sowohl ungerade Zahl an etcd-Nodes als auch ungerade Zahl an Rechenzentren.
- \* Ideal ist ein RTT (round trip time) von 10 ms / (maximal 100ms / 1.5 => ca. 66,6 ms)

```
etcd
```

- \* Tuning: <https://etcd.io/docs/v3.4/tuning/>
- \* Maintenance: <https://etcd.io/docs/v3.5/op-guide/maintenance/>

```
Kubernetes Storage
```

```
Praxis. Beispiel (Dev/Ops)
```

```
Create new server and install nfs-server
```

## on Ubuntu 20.04LTS

```
apt install nfs-kernel-server systemctl status nfs-server
```

```
vi /etc/exports
```

## adjust ip's of kubernetes master and nodes

### kmaster

```
/var/nfs/ 192.168.56.101(rw,sync,no_root_squash,no_subtree_check)
```

### knodel

```
/var/nfs/ 192.168.56.103(rw,sync,no_root_squash,no_subtree_check)
```

### knodel 2

```
/var/nfs/ 192.168.56.105(rw,sync,no_root_squash,no_subtree_check)
```

```
exportfs -av
```

```
On all nodes (needed for production)
```

```
apt install nfs-common
```

```
On all nodes (only for testing) (Version 1)
```

**Please do this on all servers (if you have access by ssh)**

**find out, if connection to nfs works !**

### for testing

```
mkdir /mnt/nfs
```

## 192.168.56.106 is our nfs-server

```
mount -t nfs 192.168.56.106:/var/nfs /mnt/nfs ls -la /mnt/nfs umount /mnt/nfs
```

```
Setup PersistentVolume and PersistentVolumeClaim in cluster

Schritt 1:
```

cd cd manifests mkdir -p nfs; cd nfs nano 01-pv.yml

apiVersion: v1 kind: PersistentVolume metadata:

## any PV name

name: pv-nfs-tln labels: volume: nfs-data-volume-tln spec: capacity: # storage size storage: 1Gi accessModes: # ReadWriteMany(RW from multi nodes), ReadWriteOnce(RW from a node), ReadOnlyMany(R from multi nodes) - ReadWriteMany persistentVolumeReclaimPolicy: # retain even if pods terminate Retain nfs: # NFS server's definition path: /var/nfs/tln/nginx server: 10.135.0.7 readOnly: false storageClassName: ""

kubect! apply -f 01-pv.yml

```
Schritt 2:
```

nano 02-pvc.yml

## vi 02-pvc.yml

## now we want to claim space

apiVersion: v1 kind: PersistentVolumeClaim metadata: name: pv-nfs-claim-tln spec: storageClassName: "" volumeName: pv-nfs-tln accessModes:

- ReadWriteMany resources: requests: storage: 1Gi

kubect! apply -f 02-pvc.yml

```
Schritt 3:
```

nano 03-deploy.yml

## deployment including mount

## vi 03-deploy.yml

apiVersion: apps/v1 kind: Deployment metadata: name: nginx-deployment spec: selector: matchLabels: app: nginx replicas: 4 # tells deployment to run 4 pods matching the template template: metadata: labels: app: nginx spec:

```
containers:
- name: nginx
 image: nginx:latest
 ports:
 - containerPort: 80

volumeMounts:
- name: nfsvol
 mountPath: "/usr/share/nginx/html"

volumes:
- name: nfsvol
 persistentVolumeClaim:
 claimName: pv-nfs-claim-tln<nr>
```

kubect! apply -f 03-deploy.yml

nano 04-service.yml

## now testing it with a service

### cat 04-service.yml

```
apiVersion: v1 kind: Service metadata: name: service-nginx labels: run: svc-my-nginx spec: type: NodePort ports:
```

- port: 80 protocol: TCP selector: app: nginx

kubectl apply -f 04-service.yml

```
Schritt 4
```

## connect to the container and add index.html - data

kubectl exec -it deploy/nginx-deployment -- bash

### in container

echo "hello dear friend" > /usr/share/nginx/html/index.html exit

### get external ip

kubectl get nodes -o wide

### now try to connect

kubectl get svc

### connect with ip and port

kubectl run -it --rm curlly --image=curlimages/curl -- /bin/sh

### curl http://

### exit

### oder alternative von extern (Browser) auf Client

http://30154 (Node Port) - ext-ip -> kubectl get nodes -o wide

### now destroy deployment

kubectl delete -f 03-deploy.yml

### Try again - no connection

kubectl run -it --rm curlly --image=curlimages/curl -- /bin/sh

### curl http://

### exit

```
Schritt 5
```

## now start deployment again

kubectl apply -f 03-deploy.yml

### and try connection again

kubectl run -it --rm curlly --image=curlimages/curl -- /bin/sh

### curl http://: # port -> > 30000

### exit

```
Kubernetes Netzwerk

Kubernetes Netzwerke Übersicht

Show us

![[pod to pod across nodes](https://www.inovex.de/wp-content/uploads/2020/05/Pod-to-Pod-Networking.png)]

Die Magie des Pause Containers

![[Overview Kubernetes Networking](https://www.inovex.de/wp-content/uploads/2020/05/Container-to-Container-Networking_3_neu-400x412.png)]

CNI

* Common Network Interface
* Feste Definition, wie Container mit Netzwerk-Bibliotheken kommunizieren

Docker - Container oder andere

* Container wird hochgefahren -> über CNI -> zieht Netzwerk - IP hoch.
* Container wird runtergefahren -> über CNI -> Netzwerk - IP wird released

Welche gibt es ?

* Flannel
* Canal
* Calico
* Cilium

Flannel

Overlay - Netzwerk

* virtuelles Netzwerk was sich oben drüber und eigentlich auf Netzwerkebene nicht existiert
* VXLAN

Vorteile

* Guter einfacher Einstieg
* reduziert auf eine Binary flanneld

Nachteile

* keine Firewall - Policies möglich
* keine klassischen Netzwerk-Tools zum Debuggen möglich.

Canal

General

* Auch ein Overlay - Netzwerk
* Unterstützt auch policies

Calico

Generell

* klassische Netzwerk (BGP)

Vorteile gegenüber Flannel

* Policy über Kubernetes Object (NetworkPolicies)

Vorteile

* ISTIO integrierbar (Mesh - Netz)
* Performance etwas besser als Flannel (weil keine Encapsulation)

Referenz

* https://projectcalico.docs.tigera.io/security/calico-network-policy
```

```
Cilium

Generell

microk8s Vergleich

* https://microk8s.io/compare
```

snap.microk8s.daemon-flannel Flannel is a CNI which gives a subnet to each host for use with container runtimes.

Flannel runs if ha-cluster is not enabled. If ha-cluster is enabled, calico is run instead.

The flannel daemon is started using the arguments in \${SNAP\_DATA}/args/flannel. For more information on the configuration, see the flannel documentation.

```
DNS - Resolution - Services

Exercise
```

kubect! run podtest --rm -ti --image busybox

```
Example with svc-nginx
```

## in sh

wget -O - <http://svc-nginx> wget -O - <http://svc-nginx.jochen> wget -O - <http://svc-nginx.jochen.svc> wget -O - <http://svc-nginx.jochen.svc.cluster.local>

```
How to find the FQDN (Full qualified domain name)
```

## in busybox (clusterIP)

Schritt 1: Service-IP ausfindig machen

wget -O - <http://svc-nginx>

**z.B. 10.109.24.227**

Schritt 2: nslookup mit dieser Service-IP

nslookup 10.109.24.227

## Ausgabe

**name = svc-nginx.jochen.svc.cluster.local**

```
Kubernetes Firewall / Cilium Calico

Um was geht es ?

* Wir wollen Firewall-Regeln mit Kubernetes machen (NetworkPolicy)
* Firewall in Kubernetes -> Network Policies

Gruppe mit eigenem cluster
```

= nix z.B. policy-demo => policy-demo

```
Gruppe mit einem einzigen Cluster
```

= Teilnehmernummer

z.B. policy-demo => policy-demo1

```
Walkthrough
```

## Schritt 1:

kubect! create ns policy-demo kubect! create deployment --namespace=policy-demo nginx --image=nginx kubect! expose --namespace=policy-demo deployment nginx --port=80

## lassen einen 2. pod laufen mit dem auf den nginx zugreifen

```
kubectl run --namespace=policy-demo access --rm -ti --image busybox -- /bin/sh
```

### innerhalb der shell

```
wget -q nginx -O -
```

```
Schritt 2: Policy festlegen, dass kein Ingress Traffic erlaubt ist
```

```
cd cd manifests mkdir network cd network nano 01-policy.yml
```

### Deny Regel

```
kind: NetworkPolicy apiVersion: networking.k8s.io/v1 metadata: name: default-deny namespace: policy-demo spec: podSelector: matchLabels: {}
```

```
kubectl apply -f 01-policy.yml
```

## lassen einen 2. pod laufen mit dem auf den nginx zugreifen

```
kubectl run --namespace=policy-demo access --rm -ti --image busybox -- /bin/sh
```

### innerhalb der shell

### kein Zugriff möglich

```
wget -O - nginx
```

```
Schritt 3: Zugriff erlauben von pods mit dem Label run=access
```

```
cd cd manifests cd network nano 02-allow.yml
```

### Schritt 3:

#### 02-allow.yml

```
kind: NetworkPolicy apiVersion: networking.k8s.io/v1 metadata: name: access-nginx namespace: policy-demo spec: podSelector: matchLabels: app: nginx ingress: - from: - podSelector: matchLabels: run: access
```

```
kubectl apply -f 02-allow.yml
```

## lassen einen 2. pod laufen mit dem auf den nginx zugreifen

### pod hat durch run -> access automatisch das label run:access zugewiesen

```
kubectl run --namespace=policy-demo access --rm -ti --image busybox -- /bin/sh
```

### innerhalb der shell

```
wget -q nginx -O -
```

```
kubectl run --namespace=policy-demo no-access --rm -ti --image busybox -- /bin/sh
```

## in der shell

```
wget -q nginx -O -
```

```
kubectl delete ns policy-demo
```

```
Ref:

* https://projectcalico.docs.tigera.io/security/tutorials/kubernetes-policy-basic
* https://kubernetes.io/docs/concepts/services-networking/network-policies/
* https://docs.cilium.io/en/latest/security/policy/language/#http

Sammlung istio/mesh

Schaubild

! [istio Schaubild] (https://istio.io/latest/docs/examples/virtual-machines/vm-bookinfo.svg)

Istio
```

## Visualization

### with kiali (included in istio)

<https://istio.io/latest/docs/tasks/observability/kiali/kiali-graph.png>

## Example

<https://istio.io/latest/docs/examples/bookinfo/>

The sidecars are injected in all pods within the namespace by labeling the namespace like so: `kubectl label namespace default istio-injection=enabled`

## Gateway (like Ingress in vanilla Kubernetes)

```
kubectl label namespace default istio-injection=enabled
```

```
istio tls

* https://istio.io/latest/docs/ops/configuration/traffic-management/tls-configuration/

istio - the next generation without sidecar

* https://istio.io/latest/blog/2022/introducing-ambient-mesh/

Kubernetes NetworkPolicy (Firewall)

Kubernetes Network Policy Beispiel

Schritt 1: Deployment und Service erstellen
```

```
KURZ=jm kubectl create ns policy-demo-$KURZ
```

```
cd mkdir -p manifests cd manifests mkdir -p np cd np
```

```
nano 01-deployment.yml
```

## nano 01-deployment.yml

```
apiVersion: apps/v1 kind: Deployment metadata: name: nginx-deployment spec: selector: matchLabels: app: nginx replicas: 1 template: metadata: labels: app: nginx spec: containers: - name: nginx image: nginx:1.23 ports: - containerPort: 80
```

```
kubectl -n policy-demo-$KURZ apply -f .
```

```
nano 02-service.yaml
```

### nano 02-service.yaml

apiVersion: v1 kind: Service metadata: name: nginx spec: type: ClusterIP # Default Wert ports:

- port: 80 protocol: TCP selector: app: nginx

```
kubectl -n policy-demo-$KURZ apply -f .
```

```
Schritt 2: Zugriff testen ohne Regeln
```

### lassen einen 2. pod laufen mit dem auf den nginx zugreifen

```
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

### innerhalb der shell

```
wget -q nginx -O -
```

### Optional: Pod anzeigen in 2. ssh-session zu jump-host

```
kubectl -n policy-demo-$KURZ get pods --show-labels
```

```
Schritt 3: Policy festlegen, dass kein Zugriff erlaubt ist.
```

```
nano 03-default-deny.yaml
```

### nano 03-default-deny.yaml

### Schritt 2: Policy festlegen, dass kein Ingress-Traffic erlaubt

### in diesem namespace: policy-demo-\$KURZ

kind: NetworkPolicy apiVersion: networking.k8s.io/v1 metadata: name: default-deny spec: podSelector: matchLabels: {}

```
kubectl -n policy-demo-$KURZ apply -f .
```

```
Schritt 3.5: Verbindung mit deny all Regeln testen
```

```
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

### innerhalb der shell

```
wget -q nginx -O -
```

```
Schritt 4: Zugriff erlauben von pods mit dem Label run=access (alle mit run gestarteten pods mit namen access haben dieses label per default)
```

```
nano 04-access-nginx.yaml
```



## nano 04-access-nginx.yaml

```
apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: access-nginx spec: podSelector: matchLabels: app: nginx ingress: - from: - podSelector: matchLabels: run: access
```

```
kubectl -n policy-demo-$KURZ apply -f .
```

```
Schritt 5: Testen (zugriff sollte funktionieren)
```

## lassen einen 2. pod laufen mit dem auf den nginx zugreifen

### pod hat durch run -> access automatisch das label run:access zugewiesen

```
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

## innerhalb der shell

```
wget -q nginx -O -
```

```
Schritt 6: Pod mit label run=no-access - da sollte es nicht gehen
```

```
kubectl run --namespace=policy-demo-$KURZ no-access --rm -ti --image busybox
```

## in der shell

```
wget -q nginx -O -
```

```
Schritt 7: Aufräumen
```

```
kubectl delete ns policy-demo-$KURZ
```

```
Ref:
```

```
* https://projectcalico.docs.tigera.io/security/tutorials/kubernetes-policy-basic
```

```
Übung: NetworkPolicy - Pod-Traffic absichern
```

```
Hintergrund
```

Ohne NetworkPolicy kann **jeder Pod mit jedem anderen Pod** im Cluster kommunizieren – egal in welchem Namespace. Das ist der unsichere Standardzustand.

NetworkPolicies wirken wie eine Firewall auf Pod-Ebene:

Konzept	Beschreibung
`podSelector`	Fuer welche Pods gilt diese Policy
`policyTypes`	`Ingress`, `Egress` oder beides
`ingress.from`	Wer darf eingehenden Traffic schicken
`egress.to`	Wohin darf ausgehender Traffic gehen
Leere Policy	Blockiert alles (Default-Deny)
Keine Policy	Alles erlaubt

**Wichtig:** NetworkPolicies werden vom CNI-Plugin durchgesetzt.  
Unser Cluster verwendet **Calico** – Standard-NetworkPolicies funktionieren hier vollstaendig. Mehr dazu am Ende der Uebung.

```

```

```
Schritt 1: Vorbereitung
```

```
cd mkdir -p manifests cd manifests mkdir 21-networkpolicy cd 21-networkpolicy
```

```
Namespace anlegen:
```

```
kubectl create namespace networkpolicy-
```

```

```

```
Schritt 2: Ausgangslage – kein Schutz
```

```
Frontend- und Backend-Pod anlegen:
```

**vi 00-baseline.yml**

**apiVersion: v1 kind: Pod metadata: name: backend labels: app: backend spec: containers: - name: app  
image: nginx:1.27 ports: - containerPort: 80**

**apiVersion: v1 kind: Service metadata: name: backend spec: selector: app: backend ports: - port: 80  
targetPort: 80**

apiVersion: v1 kind: Pod metadata: name: frontend labels: app: frontend spec: containers:

- name: web image: nginx:1.27 ports:
  - containerPort: 80

```
kubectl apply -f . -n networkpolicy- kubectl get pod,svc -n networkpolicy-
```

```
Warten bis beide Pods laufen:
```

```
kubectl wait pod frontend backend --for=condition=Ready -n networkpolicy- --timeout=60s
```

```
Frontend erreicht Backend – kein Problem:
```

```
kubectl exec -n networkpolicy- frontend -- curl -s http://backend | grep title
```

```
Erwartete Ausgabe: nginx-Titelzeile – Verbindung funktioniert ungehindert.
```

```

```

```
Schritt 3: Default-Deny – alles blockieren
```

```
Die erste und wichtigste Policy: keinen Traffic erlauben, den wir nicht
explizit freigegeben haben.
```

**vi 01-default-deny.yml**

apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: default-deny-all spec: podSelector: {} policyTypes:

- Ingress
- Egress

```
`podSelector: {}` bedeutet: gilt fuer **alle** Pods im Namespace.
```

```
kubectl apply -f . -n networkpolicy-
```

```
Verbindung testen:
```

```
kubectl exec -n networkpolicy- frontend -- curl -s --max-time 5 http://backend
```

```
Erwarteter Fehler:
```

```
curl: (28) Connection timed out after 5000 milliseconds
```

```
Auch DNS ist jetzt geblockt:
```

```
kubectl exec -n networkpolicy- frontend -- curl -s --max-time 5 http://example.com
```

```
Erwarteter Fehler: Timeout – kein Egress, kein DNS.

Schritt 4: DNS freigeben

Ohne DNS koennen Pods keine Servicenamen auflösen. Wir erlauben
Egress zu kube-dns im `kube-system`-Namespace.
```

## vi 02-allow-dns.yml

apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: allow-dns spec: podSelector: {} policyTypes:

- Egress egress:
  - to:
    - namespaceSelector: matchLabels: kubernetes.io/metadata.name: kube-system
- ports:
- protocol: UDP port: 53
  - protocol: TCP port: 53

```
kubect! apply -f .-n networkpolicy-
```

```
DNS testen:
```

```
kubect! exec -n networkpolicy- frontend -- getent hosts backend
```

```
Erwartete Ausgabe: IP-Adresse wird aufgelöst (z.B. `10.96.x.x backend.default.svc.cluster.local`).
```

```
HTTP zu Backend schlaegt aber noch fehl:
```

```
kubect! exec -n networkpolicy- frontend -- curl -s --max-time 5 http://backend
```

```
Erwarteter Fehler: Timeout – HTTP-Traffic noch gesperrt.

Schritt 5: Frontend darf Backend auf Port 80 erreichen
```

## vi 03-frontend-to-backend.yml

**apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: frontend-egress-to-backend spec:**  
**podSelector: matchLabels: app: frontend policyTypes: - Egress egress: - to: - podSelector: matchLabels:**  
**app: backend ports: - protocol: TCP port: 80**

apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: backend-ingress-from-frontend spec: podSelector: matchLabels: app: backend policyTypes:

- Ingress ingress:
  - from:
    - podSelector: matchLabels: app: frontend
- ports:
- protocol: TCP port: 80

```
kubect! apply -f .-n networkpolicy-
```

```
Verbindung testen:
```

```
kubect! exec -n networkpolicy- frontend -- curl -s http://backend | grep title
```

```
Erwartete Ausgabe: nginx-Titelzeile – Verbindung funktioniert wieder.

Schritt 6: Isolation verifizieren
```

```
Externer Traffic bleibt gesperrt (Egress-Deny greift):
```

```
kubect! exec -n networkpolicy- frontend -- curl -s --max-time 5 http://www.google.de
```

```
Erwarteter Fehler: Timeout.
```

```
Backend kann Frontend nicht erreichen (kein Egress vom Backend freigegeben):
```

```
kubect! exec -n networkpolicy- backend -- curl -s --max-time 5 http://frontend
```

```
Erwarteter Fehler: Timeout.
```

```
Alle aktiven NetworkPolicies anzeigen:
```

```
kubect! get networkpolicy -n networkpolicy-
```

```
Erwartete Ausgabe:
```

```
NAME POD-SELECTOR AGE allow-dns ... backend-ingress-from-frontend app=backend ... default-deny-all ... frontend-egress-to-backend app=frontend ...
```

```

```

```
Grenzen von Standard-NetworkPolicies
```

```
Standard-NetworkPolicies koennen nur nach IP, Port und Pod/namespace-Label
filtern – keine Domainnamen, keine L7-Regeln.
```

Anforderung	Standard-NP	Calico CRD
Pod-zu-Pod nach Label	ja	ja
Namespace-Isolation	ja	ja
Default-Deny cluster-weit	nein	ja ( <code>`GlobalNetworkPolicy`</code> )
Egress nach Domainname (FQDN)	nein	ja ( <code>`NetworkSet`</code> + FQDN)
HTTP-Methoden / Pfade (L7)	nein	ja (mit Istio/Cilium)

Wer Calico als CNI hat, kann mit ``GlobalNetworkPolicy`` eine **cluster-weite** Default-Deny-Policy setzen – ohne dass jeder Namespace seine eigene anlegen muss:

## Beispiel: Calico GlobalNetworkPolicy (kein Standard-Kubernetes!)

apiVersion: projectcalico.org/v3 kind: GlobalNetworkPolicy metadata: name: default-deny spec: selector: all() types:

- Ingress
- Egress

```
Referenz: https://docs.tigera.io/calico/latest/network-policy/networkpolicy-get-started
```

```

```

```
Aufräumen
```

```
kubect! delete namespace networkpolicy-
```

```

```

```
Zusammenfassung
```

Policy	Zweck
<code>`default-deny-all`</code>	Alles sperren – Ausgangsbasis
<code>`allow-dns`</code>	DNS fuer alle Pods freigeben
<code>`frontend-egress-to-backend`</code>	Frontend darf Backend ansprechen
<code>`backend-ingress-from-frontend`</code>	Backend nimmt nur vom Frontend an

Die Kombination aus Default-Deny + gezielten Freigaben ist das **Zero-Trust-Prinzip** auf Netzwerkebene.

```
Referenzen
```

\* <https://kubernetes.io/docs/concepts/services-networking/network-policies/>

```
* https://docs.tigera.io/calico/latest/network-policy/

Übung: FE zu Backend Verbindungen debuggen mit kubectl debug und NetworkPolicy

Hintergrund

In produktiven Umgebungen laufen Container oft als minimale Images ohne Debug-Tools
(curl, wget, nc). `kubectl debug` schleust einen ephemeral Container mit Debug-Tools
in einen laufenden Pod ein – ohne den Pod neu starten zu muessen.

Ein **Timeout** beim Verbindungsversuch ist ein typisches Zeichen fuer eine blockierende
NetworkPolicy:

| Fehlerbild | Ursache | Diagnose-Befehl |
|-----|-----|-----|
| `Connection timed out` | NetworkPolicy blockiert den Traffic | `kubectl describe networkpolicy` |

Schritt 1: Vorbereitung
```

```
cd mkdir -p manifests cd manifests mkdir 20-debug-networkpolicy cd 20-debug-networkpolicy
```

```
Namespace anlegen:
```

```
kubectl create namespace debug-
```

```

Schritt 2: Backend Deployment und Service anlegen
```

## vi 01-backend.yml

**apiVersion: apps/v1 kind: Deployment metadata: name: backend spec: replicas: 1 selector: matchLabels:**  
**app: backend template: metadata: labels: app: backend spec: containers: - name: backend image:**  
**nginx:alpine ports: - containerPort: 80**

```
apiVersion: v1 kind: Service metadata: name: backend-svc spec: selector: app: backend ports:
```

- port: 80 targetPort: 80

```
kubectl apply -f . -n debug-
```

```

Schritt 3: Frontend Deployment und Service anlegen

Das Frontend laeuft als minimales Python-Image (kein curl, wget, nc).
```

## vi 02-frontend.yml

**apiVersion: apps/v1 kind: Deployment metadata: name: frontend spec: replicas: 1 selector: matchLabels:**  
**app: frontend template: metadata: labels: app: frontend spec: containers: - name: frontend image:**  
**python:3.12-slim command: ["python", "-m", "http.server", "8080"] ports: - containerPort: 8080**

```
apiVersion: v1 kind: Service metadata: name: frontend-svc spec: selector: app: frontend ports:
```

- port: 8080 targetPort: 8080

```
kubectl apply -f . -n debug- kubectl get pods -n debug-
```

```
Warten bis beide Pods laufen:
```

```
kubectl wait deployment frontend backend
--for=condition=Available -n debug- --timeout=60s
```

```

```

```
Schritt 4: Baseline – Verbindung funktioniert
```

```
Tools im Frontend-Pod pruefen – kein curl, wget, nc vorhanden:
```

```
FE_POD=$(kubectl get pod -n debug- -l app=frontend -o jsonpath='{.items[0].metadata.name}') echo $FE_POD
```

```
kubectl exec -it $FE_POD -n debug- -- sh -c 'which curl; which wget; which nc'
```

```
Erwartete Ausgabe:
```

```
no curl no wget no nc
```

```
Mit `kubectl debug` einen busybox-Container einschleusen und Verbindung testen:
```

```
kubectl debug -it $FE_POD -n debug-
--image=busybox:1.36
--target=frontend
--profile=general
-- sh
```

```
Im Debug-Container:
```

```
wget -qO- http://backend-svc --timeout=5 exit
```

```
Erwartete Ausgabe: `<h1>Welcome to nginx!</h1>` – Verbindung funktioniert.
```

```

```

```
Problem 1: NetworkPolicy blockiert FE zu Backend
```

```
Schritt 5: NetworkPolicy fuer Backend anwenden
```

## vi 03-networkpolicy-backend.yml

```
apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: backend-policy spec: podSelector: matchLabels: app: backend policyTypes:
```

- Ingress ingress:
- from:
  - podSelector: matchLabels: role: api-consumer
- ports:
  - protocol: TCP port: 80

```
kubectl apply -f . -n debug-
```

```
Schritt 6: Verbindung testen – Timeout
```

```
kubectl debug -it $FE_POD -n debug-
--image=busybox:1.36
--target=frontend
--profile=general
-- sh
```

```
wget -qO- http://backend-svc --timeout=5 exit
```

```
Erwarteter Fehler:
```

```
wget: download timed out
```

```
Timeout statt `Connection refused` – Endpoints existieren, die NetworkPolicy blockiert.
```

```
Schritt 7: Diagnose – NetworkPolicy und Labels pruefen
```

```
kubect! describe networkpolicy backend-policy -n debug- kubect! get pods -n debug- --show-labels
```

```
Diagnose: NetworkPolicy erlaubt nur `role=api-consumer`. Frontend-Pod hat dieses Label nicht.
```

```
Schritt 8: Fix – Label zum Frontend Deployment hinzufuegen
```

```
kubect! patch deployment frontend -n debug-
-p '{"spec":{"template":{"metadata":{"labels":{"role":"api-consumer"}}}}}'
```

```
kubect! get pods -n debug- -l role=api-consumer
```

```
Neuen Pod-Namen holen und erneut testen:
```

```
FE_POD=$(kubect! get pod -n debug- -l app=frontend,role=api-consumer
-o jsonpath='{.items[0].metadata.name}')
```

```
kubect! debug -it $FE_POD -n debug-
--image=busybox:1.36
--target=frontend
--profile=general
-- sh
```

```
wget -qO- http://backend-svc --timeout=5 exit
```

```
Erwartete Ausgabe: `<h1>Welcome to nginx!</h1>` – Verbindung OK.
```

```

```

```
Problem 2: NetworkPolicy blockiert Backend zu Frontend
```

```
Schritt 9: NetworkPolicy fuer Frontend anwenden
```

## vi 04-networkpolicy-frontend.yml

```
apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: frontend-policy spec: podSelector: matchLabels: app: frontend policyTypes:
```

- Ingress ingress:
- from:
  - podSelector: matchLabels: role: backend-consumer
- ports:
  - protocol: TCP port: 8080

```
kubect! apply -f . -n debug-
```

```
Schritt 10: Rueckweg vom Backend debuggen
```

```
BE_POD=$(kubect! get pod -n debug- -l app=backend -o jsonpath='{.items[0].metadata.name}') echo $BE_POD
```

```
kubect! debug -it $BE_POD -n debug-
--image=busybox:1.36
--target=backend
--profile=general
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5 exit
```

```
Erwarteter Fehler:
```

wget: download timed out

```
Schritt 11: Diagnose und Fix
```

kubectl describe networkpolicy frontend-policy -n debug- kubectl get pods -n debug- -l app=backend --show-labels

```
Diagnose: NetworkPolicy erlaubt nur `role=backend-consumer`. Backend-Pod hat dieses Label nicht.
```

kubectl patch deployment backend -n debug-  
-p '{"spec":{"template":{"metadata":{"labels":{"role":"backend-consumer"}}}}}'

kubectl get pods -n debug- -l role=backend-consumer

```
Schritt 12: Rueckweg erneut testen
```

BE\_POD=\$(kubectl get pod -n debug- -l app=backend,role=backend-consumer  
-o jsonpath='{.items[0].metadata.name}')

kubectl debug -it \$BE\_POD -n debug-  
--image=busybox:1.36  
--target=backend  
--profile=general  
-- sh

wget -qO- <http://frontend-svc:8080> --timeout=5 exit

```
Erwartete Ausgabe:
```

...

Rueckweg funktioniert.

Aufraeumen

```
kubectl delete namespace debug-<dein-name>
```

Zusammenfassung

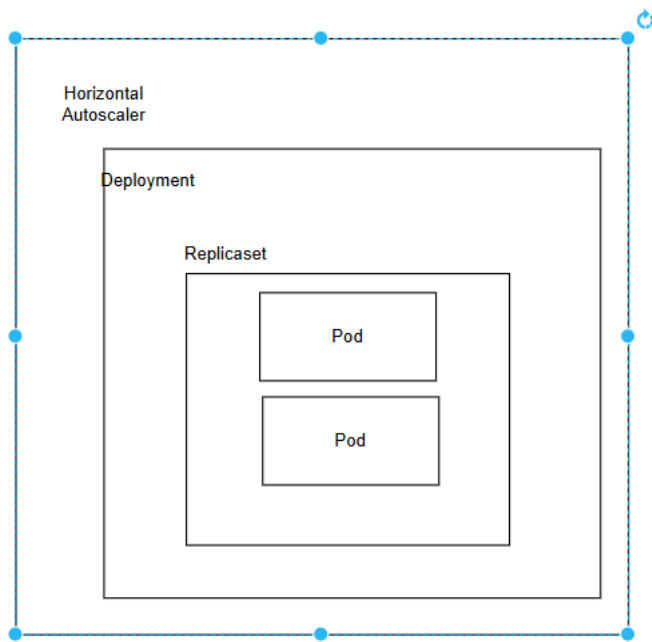
Problem	Fehlermeldung	Diagnose	Fix
NetworkPolicy FE -> Backend	timed out	kubectl describe networkpolicy + --show-labels	Label role=api-consumer am Frontend
NetworkPolicy Backend -> FE	timed out	kubectl describe networkpolicy + --show-labels	Label role=backend-consumer am Backend

Kubernetes Autoscaling

Kubernetes Autoscaling

Overview





**Example: newest version with autoscaling/v2 used to be hpa/v1**

#### Prerequisites

- Metrics-Server needs to be running

```
Test with
kubectl top pods
```

```
Install with helm chart
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm upgrade --install metrics-server metrics-server/metrics-server --version 3.13.0 --create-namespace --namespace=metrics-server --reset-values
```

```
after that it will be available in kube-system namespace as pod
kubectl -n metrics-server get pods
```

#### Step 1: deploy app

```
cd
mkdir -p manifests
cd manifests
mkdir hpa
cd hpa
nano 01-deploy.yaml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: hello
spec:
 replicas: 3
 selector:
 matchLabels:
 app: hello
 template:
 metadata:
 labels:
 app: hello
 spec:
 containers:
 - name: hello
 image: k8s.gcr.io/hpa-example
```

```

 resources:
 requests:
 cpu: 100m

kind: Service
apiVersion: v1
metadata:
 name: hello
spec:
 selector:
 app: hello
 ports:
 - port: 80
 targetPort: 80

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: hello
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: hello
 minReplicas: 2
 maxReplicas: 20
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 80

```

```
kubectl apply -f .
```

## Step 2: Load Generator

```
nano 02-loadgenerator.yml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: load-generator
 labels:
 app: load-generator
spec:
 replicas: 100
 selector:
 matchLabels:
 app: load-generator
 template:
 metadata:
 name: load-generator
 labels:
 app: load-generator
 spec:
 containers:
 - name: load-generator
 image: busybox
 command:
 - /bin/sh
 - -c
 - "while true; do wget -q -O- http://hello; done"

```

```
kubectl apply -f .
```

## Step 3: Zurücklehnen und genießen

```
watch kubectl get pods -l app=hello
```

```

2.Session aufmachen und ..
watch kubectl get nodes

```

## Downscaling

- Downscaling will happen after 5 minutes o

```
Adjust down to 1 minute
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: hello
spec:
 # change to 60 secs here
 behavior:
 scaleDown:
 stabilizationWindowSeconds: 60
 # end of behaviour change
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: hello
 minReplicas: 2
 maxReplicas: 20
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 80
```

For scaling down the stabilization window is 300 seconds (or the value of the --horizontal-pod-autoscaler-downscale-stabilization flag if provided)

#### Reference

- <https://docs.digitiocean.com/tutorials/cluster-autoscaling-ca-hpa/>
- <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/#autoscaling-on-more-specific-metrics>
- <https://medium.com/expedia-group-tech/autoscaling-in-kubernetes-why-doesnt-the-horizontal-pod-autoscaler-work-for-me-5f0094694054>

## Autoscaling

#### Example:

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
 name: busybox-1
spec:
 scaleTargetRef:
 kind: Deployment
 name: busybox-1
 minReplicas: 3
 maxReplicas: 4
 targetCPUUtilizationPercentage: 80
```

#### Reference

- <https://medium.com/expedia-group-tech/autoscaling-in-kubernetes-why-doesnt-the-horizontal-pod-autoscaler-work-for-me-5f0094694054>

## Kubernetes Secrets / ConfigMap

#### Configmap Example 1

##### Schritt 1: configmap vorbereiten

```
cd
mkdir -p manifests
cd manifests
mkdir configmaptests
cd configmaptests
nano 01-configmap.yml
```

```
01-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
 name: example-configmap
```

```
data:
 # als Wertepaare
 database: mongodb
 database_uri: mongodb://localhost:27017
 testdata: |
 run=true
 file=/hello/you
```

```
kubectl apply -f 01-configmap.yml
kubectl get cm
kubectl get cm example-configmap -o yaml
```

## Schritt 2: Beispiel als Datei

```
nano 02-pod.yml
```

```
kind: Pod
apiVersion: v1
metadata:
 name: pod-mit-configmap

spec:
 # Add the ConfigMap as a volume to the Pod
 volumes:
 # `name` here must match the name
 # specified in the volume mount
 - name: example-configmap-volume
 # Populate the volume with config map data
 configMap:
 # `name` here must match the name
 # specified in the ConfigMap's YAML
 name: example-configmap

 containers:
 - name: container-configmap
 image: nginx:latest
 # Mount the volume that contains the configuration data
 # into your container filesystem
 volumeMounts:
 # `name` here must match the name
 # from the volumes section of this pod
 - name: example-configmap-volume
 mountPath: /etc/config
```

```
kubectl apply -f 02-pod.yml
```

```
##Jetzt schauen wir uns den Container/Pod mal an
kubectl exec pod-mit-configmap -- ls -la /etc/config
kubectl exec -it pod-mit-configmap -- bash
ls -la /etc/config
```

## Schritt 3: Beispiel. ConfigMap als env-variablen

```
nano 03-pod-mit-env.yml
```

```
03-pod-mit-env.yml
kind: Pod
apiVersion: v1
metadata:
 name: pod-env-var
spec:
 containers:
 - name: env-var-configmap
 image: nginx:latest
 envFrom:
 - configMapRef:
 name: example-configmap
```

```
kubectl apply -f 03-pod-mit-env.yml
```

```
und wir schauen uns das an
##Jetzt schauen wir uns den Container/Pod mal an
kubectl exec pod-env-var -- env
kubectl exec -it pod-env-var -- bash
env
```

## Reference:

- <https://matthewpalmer.net/kubernetes-app-developer/articles/ultimate-configmap-guide-kubernetes.html>

## Secrets Example 1

### Übung 1 - ENV Variablen aus Secrets setzen

```
Schritt 1: Secret anlegen.
Diesmal noch nicht encoded - base64
vi 06-secret-unencoded.yml
apiVersion: v1
kind: Secret
metadata:
 name: mysecret
type: Opaque
stringData:
 APP_PASSWORD: "s3c3tp@ss"
 APP_EMAIL: "mail@domain.com"
```

```
Schritt 2: Apply'en und anschauen
kubectl apply -f 06-secret-unencoded.yml
ist zwar encoded, aber last_applied ist im Klartext
das könnte ich nur nur umgehen, in dem ich es encoded speichere
kubectl get secret mysecret -o yaml
```

```
Schritt 3:
vi 07-print-envs-complete.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-complete
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_PASSWORD
 valueFrom:
 secretKeyRef:
 name: mysecret
 key: APP_PASSWORD
 - name: APP_EMAIL
 valueFrom:
 secretKeyRef:
 name: mysecret
 key: APP_EMAIL
```

```
Schritt 4:
kubectl apply -f 07-print-envs-complete.yml
kubectl exec -it print-envs-complete -- bash
##env | grep -e APP_ -e MYSQL
```

## Änderung in ConfigMap erkennen und anwenden

- <https://github.com/stakater/Reloader>

## Kubernetes RBAC (Role based access control)

### RBAC Übung kubectl

#### Schritt 1: Nutzer-Account auf Server anlegen und secret anlegen / in Client

```
cd
mkdir -p manifests/rbac
cd manifests/rbac
```

#### Mini-Schritt 1: Definition für Nutzer

```
vi service-account.yml
apiVersion: v1
kind: ServiceAccount
metadata:
```

```
name: training<nr>
namespace: default
```

```
kubectl apply -f service-account.yml
```

#### Mini-Schritt 1.5: Secret erstellen

- From Kubernetes 1.25 tokens are not created automatically when creating a service account (sa)
- You have to create them manually with annotation attached
- <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/#create-token>

```
vi secret.yml
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
 name: trainingtoken<nr>
 namespace: default
 annotations:
 kubernetes.io/service-account.name: training<nr>
```

```
kubectl apply -f .
```

#### Mini-Schritt 2: ClusterRole festlegen - Dies gilt für alle namespaces, muss aber noch zugewiesen werden

```
Bevor sie zugewiesen ist, funktioniert sie nicht - da sie keinem Nutzer zugewiesen ist
```

```
vi pods-clusterrole.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: pods-clusterrole<nr>
rules:
- apiGroups: [""] # "" indicates the core API group
 resources: ["pods"]
 verbs: ["get", "watch", "list", "create"]
```

```
kubectl apply -f pods-clusterrole.yml
```

#### Mini-Schritt 3: Die ClusterRolle den entsprechenden Nutzern über RoleBinding zu ordnen

```
vi rb-training-ns-default-pods.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: rolebinding-ns-default-pods<nr>
 namespace: default
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: pods-clusterrole<nr>
subjects:
- kind: ServiceAccount
 name: training<nr>
 namespace: default
```

```
kubectl apply -f rb-training-ns-default-pods.yml
```

#### Mini-Schritt 4: Testen (klappt der Zugang)

```
kubectl auth can-i get pods -n default --as system:serviceaccount:default:training<nr>
```

### Schritt 2: Context anlegen / Credentials auslesen und in kubeconfig hinterlegen (ab Kubernetes-Version 1.25.)

#### Mini-Schritt 1: kubeconfig setzen

```
kubectl config set-context training-ctx --cluster do-fra1-bka-training --user training

extract name of the token from here

TOKEN=`kubectl -n default get secret trainingtoken<nr> -o jsonpath='{.data.token}' | base64 --decode`
echo $TOKEN
kubectl config set-credentials training --token=$TOKEN
kubectl config view
kubectl config use-context training-ctx
```

```
Hier reichen die Rechte nicht aus
kubectl get deploy
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kube-system:training" cannot list # resource
"pods" in API group "" in the namespace "default"
das funktioniert
kubectl get pods
```

### Mini-Schritt 3: Zurück zum alten Default-Context

```
kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
	cluster-user	cluster1	admin2	
*	training-ctx	cluster1	training2	

```
kubectl config use-context cluster-user
```

### Refs:

- <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingserviceaccttoken.htm>
- <https://micro8s.io/docs/multi-user>
- <https://faun.pub/kubernetes-rbac-use-one-role-in-multiple-namespaces-d1d08bb08286>

### Ref: Create Service Account Token

- <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/#create-token>

## Kubernetes Operator Konzept

### Ueberblick

#### Overview

```
o Possibility to extend functionality (new resource/object)
o Mainly to add new controllers to automate things
o Operator will control states
o Makes it easier to configure things.
 e.g. a crd prometheus could create a prometheus server, which consists of
 of different building blocks (Deployment, Service a.s.o)
```

### How to see CRD's (customresourcedefinitions)

```
kubectl get crd
Cilium, if present on the system
kubectl api-resources | grep cil
```

## Kubernetes Deployment Strategies

### Deployment green/blue,canary,rolling update

#### Canary Deployment

```
A small group of the user base will see the new application
(e.g. 1000 out of 100.000), all the others will still see the old version
```

```
From: a canary was used to test if the air was good in the mine
(like a test balloon)
```

#### Blue / Green Deployment

```
The current version is the Blue one
The new version is the Green one
```

```
New Version (GREEN) will be tested and if it works
the traffic will be switch completey to the new version (GREEN)
```

```
Old version can either be deleted or will function as fallback
```

#### A/B Deployment/Testing

```
2 Different versions are online, e.g. to test a new design / new feature
You can configure the weight (how much traffic to one or the other)
by the number of pods
```

#### Example Calculation

```
e.g. Deployment1: 10 pods
Deployment2: 5 pods

Both have a common label,
The service will access them through this label
```

## Praxis-Übung A/B Deployment

### Walkthrough

```
cd
cd manifests
mkdir ab
cd ab
```

```
vi 01-cm-version1.yml
apiVersion: v1
kind: ConfigMap
metadata:
 name: nginx-version-1
data:
 index.html: |
 <html>
 <h1>Welcome to Version 1</h1>
 </br>
 <h1>Hi! This is a configmap Index file Version 1 </h1>
 </html>
```

```
vi 02-deployment-v1.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deploy-v1
spec:
 selector:
 matchLabels:
 version: v1
 replicas: 2
 template:
 metadata:
 labels:
 app: nginx
 version: v1
 spec:
 containers:
 - name: nginx
 image: nginx:latest
 ports:
 - containerPort: 80
 volumeMounts:
 - name: nginx-index-file
 mountPath: /usr/share/nginx/html/
 volumes:
 - name: nginx-index-file
 configMap:
 name: nginx-version-1
```

```
vi 03-cm-version2.yml
apiVersion: v1
kind: ConfigMap
metadata:
 name: nginx-version-2
data:
 index.html: |
 <html>
 <h1>Welcome to Version 2</h1>
 </br>
 <h1>Hi! This is a configmap Index file Version 2 </h1>
 </html>
```

```
vi 04-deployment-v2.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deploy-v2
spec:
```



```

selector:
 matchLabels:
 version: v2
replicas: 2
template:
 metadata:
 labels:
 app: nginx
 version: v2
 spec:
 containers:
 - name: nginx
 image: nginx:latest
 ports:
 - containerPort: 80
 volumeMounts:
 - name: nginx-index-file
 mountPath: /usr/share/nginx/html/
 volumes:
 - name: nginx-index-file
 configMap:
 name: nginx-version-2

```

```

vi 05-svc.yml
apiVersion: v1
kind: Service
metadata:
 name: my-nginx
 labels:
 svc: nginx
spec:
 type: NodePort
 ports:
 - port: 80
 protocol: TCP
 selector:
 app: nginx

```

```

kubectl apply -f .
get external ip
kubectl get nodes -o wide
get port
kubectl get svc my-nginx -o wide
test it with curl apply it multiple time (at least ten times)
curl <external-ip>:<node-port>

```

## Kubernetes Monitoring

### Prometheus / blackbox exporter

#### Prerequisites

- prometheus setup with helm

#### Step 1: Setup

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install my-prometheus-blackbox-exporter prometheus-community/prometheus-blackbox-exporter --version 8.17.0 --namespace monitoring --create-namespace

```

#### Step 2: Find SVC

```

kubectl -n monitoring get svc | grep blackbox

```

my-prometheus-blackbox-exporter	ClusterIP	10.245.183.66	<none>	9115/TCP
---------------------------------	-----------	---------------	--------	----------

#### Step 3: Test with Curl

```

kubectl run -it --rm curltest --image=curlimages/curl -- sh

```

```

Testen nach google in shell von curl
curl http://my-prometheus-blackbox-exporter.monitoring:9115/probe?target=google.com&module=http_2xx

```

```

Looking for metric
probe_http_status_code 200

```

#### Step 4: Test apple-service with Curl

```
From within curlimages/curl pod
curl http://my-prometheus-blackbox-exporter.monitoring:9115/probe?target=apple-service.app&module=http_2xx
```

#### Step 5: Scrape Config (We want to get all services being labeled example.io/should\_be\_probed = true

```
prometheus:
 prometheusSpec:
 additionalScrapeConfigs:
 - job_name: "blackbox-microservices"
 metrics_path: /probe
 params:
 module: [http_2xx]
 # Autodiscovery through kube-api-server
 # https://prometheus.io/docs/prometheus/latest/configuration/configuration/#kubernetes_sd_config
 kubernetes_sd_configs:
 - role: service
 relabel_configs:
 # Example relabel to probe only some services that have "example.io/should_be_probed = true" annotation
 - source_labels: [__meta_kubernetes_service_annotation_example_io_should_be_probed]
 action: keep
 regex: true
 - source_labels: [__address__]
 target_label: __param_target
 - target_label: __address__
 replacement: my-prometheus-blackbox-exporter:9115
 - source_labels: [__param_target]
 target_label: instance
 - action: labelmap
 regex: __meta_kubernetes_service_label_(.+)
 - source_labels: [__meta_kubernetes_namespace]
 target_label: app
 - source_labels: [__meta_kubernetes_service_name]
 target_label: kubernetes_service_name
```

#### Step 6: Test with relabeler

- <https://relabeler.promlabs.com>

#### Step 7: Scrapeconfig einbauen

```
von kube-prometheus-grafana in values und ugraden
helm upgrade prometheus prometheus-community/kube-prometheus-stack -f values.yml --namespace monitoring --create-namespace --version 61.3.1
```

#### Step 8: annotation in service einfügen

```
kind: Service
apiVersion: v1
metadata:
 name: apple-service
 annotations:
 example.io/should_be_probed: "true"

spec:
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f service.yml
```

#### Step 9: Look into Status -> Discovery Services and wait

- blackbox services should now appear under blackbox\_microservices
- and not being dropped

#### Step 10: Unter <http://64.227.125.201:30090/targets?search=> gucken

- .. ob das funktioniert

#### Step 11: Hauptseite (status code 200)

- Metrik angekommen ?
- [http://64.227.125.201:30090/graph?g0.expr=probe\\_http\\_status\\_code&g0.tab=1&g0.display\\_mode=lines&g0.show\\_exemplars=0&g0.range\\_input=1h](http://64.227.125.201:30090/graph?g0.expr=probe_http_status_code&g0.tab=1&g0.display_mode=lines&g0.show_exemplars=0&g0.range_input=1h)

## Step 12: pod vom service stoppen

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: apple-deployment
spec:
 selector:
 matchLabels:
 app: apple
 replicas: 8
 template:
 metadata:
 labels:
 app: apple
 spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple-<dein-name>"
```

```
kubectl apply -f apple.yml # (deployment)
```

## Step 13: status\_code 0

- Metrik angekommen ?
- [http://64.227.125.201:30090/graph?g0.expr=probe\\_http\\_status\\_code&g0.tab=1&g0.display\\_mode=lines&g0.show\\_exemplars=0&g0.range\\_input=1h](http://64.227.125.201:30090/graph?g0.expr=probe_http_status_code&g0.tab=1&g0.display_mode=lines&g0.show_exemplars=0&g0.range_input=1h)

## Kubernetes Metrics Server verwenden

### Schritt 1: Trainer installs metrics-server

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm upgrade --install metrics-server metrics-server/metrics-server --namespace=metrics --create-namespace
Check it pods are running
kubectl -n metrics get pods
```

### Schritt 2: Use it

```
kubectl run nginx-data --image=nginx:1.27
how much does it use ?
kubectl top pods nginx-data
```

```

tln1@k8s-client:~/manifests$ kubectl top pods nginx-data
NAME CPU(cores) MEMORY(bytes)
nginx-data 6m 2Mi
tln1@k8s-client:~/manifests$
```

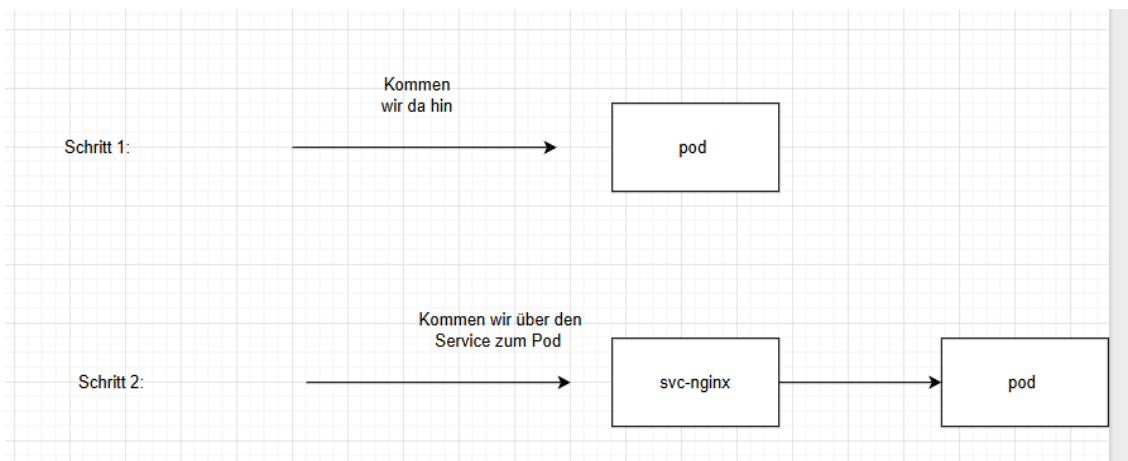
## Tips & Tricks

### Netzwerkverbindung zum Pod testen

#### Situation

Managed Cluster und ich kann nicht auf einzelne Nodes per ssh zugreifen

### Was wollen wir testen (auf der Verbindungsebene) ?



### Behelf: Eigenen Pod starten mit busybox

```
der einfachste Weg
kubectl run podtest --rm -it --image busybox
```

```
Alternative
kubectl run podtest --rm -it --image busybox -- /bin/sh
```

### Example test connection

```
wget befehl zum Kopieren
ping -c4 10.244.0.99
wget -O - http://10.244.0.99
```

```
-O -> Output (grosses O (buchstabe))
kubectl run podtest --rm -ti --image busybox -- /bin/sh
/ # wget -O - http://10.244.0.99
/ # exit
```

### Debug Container neben Container erstellen

#### Beispiel 1a: Walkthrough Debug Container

```
kubectl run ephemeral-demo --image=registry.k8s.io/pause:3.1 --restart=Never
kubectl exec -it ephemeral-demo -- sh

kubectl debug -it ephemeral-demo --image=ubuntu --target=<container-name>
```

#### Beispiel 1b: Walkthrough Debug Container with apple-app

```
cd
mkdir -p manifests
cd manifests
mkdir debugcontainer
cd debugcontainer
nano apple.yml
```

```
kind: Pod
apiVersion: v1
metadata:
 name: newapple-app
 labels:
 app: apple
spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple-jochen"
```

```
kubectl apply -f .
```

```
does not work
kubectl exec -it newapple-app -- bash
kubectl exec -it newapple-app -- sh
```

```
kubectl debug -it newapple-app --image=ubuntu

Kann ich von hier aus google erreichen
wget -O - http://www.google.de

Durch --target=apple-app sehe ich dann auch die Prozesse des anderen containers
kubectl debug -it newapple-app --image=ubuntu --target=apple-app
```

### Aufbauend auf 1b: copy des containers erstellen

```
kubectl debug newapple-app -it --image=busybox --share-processes --copy-to=newappleapp-debug
```

### Walkthrough Debug Node

```
kubectl get nodes
kubectl debug node/mynode -it --image=ubuntu
```

### Reference

- <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/#ephemeral-container>

### Debug Pod auf Node erstellen

```
node/<name-meines-nodes>
kubectl debug -it node/node-6icn1 --image=busybox
```

```
im pod
ip a
cd /host
ls -la
```

## Kubernetes Administration /Upgrades

### Kubernetes Administration / Upgrades

I. Schritt 1 (Optional aber zu empfehlen): Testsystem mit neuer Version aufsetzen (z.B. mit kind oder direkt in der Cloud)

II. Schritt 2: Manifeste auf den Stand bringen, dass sie mit den neuen Api's funktionieren, sprich ApiVersion anheben.

III. Control Plane upgraden.

Achtung: In dieser Zeit steht die API nicht zur Verfügung.  
Die Workloads im Cluster funktionieren nach wievor.

IV. Nodes upgraden wie folgt in 2 Varianten:

Variante 1: Rolling update

Jede Node wird gedrainet und die der Workload auf einer neuen Node hochgezogen.

Variante 2: Surge Update

Es werden eine Reihe von weiteren Nodes bereitgestellt, die bereits mit der neuen Version laufen.

Alle Workloads werden auf den neuen Nodes hochgezogen und wenn diese dort laufen, wird auf diese Nodes umgeschwitcht.

<https://medium.com/google-cloud/zero-downtime-gke-cluster-node-version-upgrade-and-spec-update-dad917e25b53>

### Terminierung von Container vermeiden

- <https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/>

```
preStop - Hook

Prozess läuft wie folgt:
```

```
Timeout before runterskalierung erfolgt ?
Was ist, wenn er noch rechnet ? (task läuft, der nicht beendet werden soll)
```

```
Timeout: 30 sec.
preStop
```

```
This is the process.
```

- a. State of pod is set to terminate
- b. preStop hook is executed, either exec or http after success.
- c. Terminate - Signal is sent to pod/container
- d. Wait 30 secs.
- e. Kill - Signal is set, if container did stop yet.

## Praktische Umsetzung RBAC anhand eines Beispiels (Ops)

### Enable RBAC in microk8s

```
This is important, if not enable every user on the system is allowed to do everything
microk8s enable rbac
```

### Wichtig:

```
Jeder verwendet seine eigene teilnehmer-nr z.B.
training1
training2
usw. ;o)
```

### Schritt 1: Nutzer-Account auf Server anlegen / in Client

```
cd
mkdir -p manifests/rbac
cd manifests/rbac
```

#### Mini-Schritt 1: Definition für Nutzer

```
vi service-account.yml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: training<nr> # <nr> entsprechend eintragen
 namespace: default

kubectl apply -f service-account.yml
```

#### Mini-Schritt 2: ClusterRolle festlegen - Dies gilt für alle namespaces, muss aber noch zugewiesen werden

```
Bevor sie zugewiesen ist, funktioniert sie nicht - da sie keinem Nutzer zugewiesen ist

vi pods-clusterrole.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: pods-clusterrole-<nr> # für <nr> teilnehmer - nr eintragen
rules:
- apiGroups: [""] # "" indicates the core API group
 resources: ["pods"]
 verbs: ["get", "watch", "list"]

kubectl apply -f pods-clusterrole.yml
```

#### Mini-Schritt 3: Die ClusterRolle den entsprechenden Nutzern über RoleBinding zu ordnen

```
vi rb-training-ns-default-pods.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: rolebinding-ns-default-pods<nr>
 namespace: default
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
```

```
name: pods-clusterrole-<nr> # <nr> durch teilnehmer nr ersetzen
subjects:
- kind: ServiceAccount
 name: training<nr> # nr durch teilnehmer - nr ersetzen
 namespace: default

kubectl apply -f rb-training-ns-default-pods.yml
```

#### Mini-Schritt 4: Testen (klappt der Zugang)

```
kubectl auth can-i get pods -n default --as system:serviceaccount:default:training<nr> # nr durch teilnehmer - nr ersetzen
```

### Schritt 2: Context anlegen / Credentials auslesen und in kubeconfig hinterlegen

#### Mini-Schritt 1: kubeconfig setzen

```
kubectl config set-context training-ctx --cluster microk8s-cluster --user training<nr> # <nr> durch teilnehmer - nr ersetzen

extract name of the token from here
TOKEN_NAME=`kubectl -n default get serviceaccount training<nr> -o jsonpath='{.secrets[0].name}'` # nr durch teilnehmer <nr>
ersetzen

TOKEN=`kubectl -n default get secret $TOKEN_NAME -o jsonpath='{.data.token}' | base64 --decode`
echo $TOKEN
kubectl config set-credentials training<nr> --token=$TOKEN # <nr> durch teilnehmer - nr ersetzen
kubectl config use-context training-ctx

Hier reichen die Rechte nicht aus
kubectl get deploy
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kube-system:training" cannot list # resource
"pods" in API group "" in the namespace "default"
```

#### Mini-Schritt 2:

```
kubectl config use-context training-ctx
kubectl get pods
```

#### Refs:

- <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingserviceaccttoken.htm>
- <https://microk8s.io/docs/multi-user>
- <https://faun.pub/kubernetes-rbac-use-one-role-in-multiple-namespaces-d1d08bb08286>

## Documentation (Use Cases)

### Case Studies Kubernetes

- <https://kubernetes.io/case-studies/>

### Use Cases

- <https://codilime.com/blog/harnessing-the-power-of-kubernetes-7-use-cases/>

## Interna von Kubernetes

### OCI, Container, Images Standards

#### Grundlagen

- Container und Images sind nicht docker-spezifisch, sondern folgen der OCI Spezifikation (Open Container Initiative)
- D.h. die "Bausteine" Image, Container, Registry sind standards
- Ich brauche kein Docker, um Images zu bauen, es gibt Alternativen:
  - z.B. buildah
- kubelet -> redet mit CRI (Container Runtime Interface) -> Redet mit Container Runtime z.B. containerd (Docker), CRI-O (Redhat)
  - [CRI](#)

#### Hintergründe

- Container Runtime (CRI-O, containerd)
- [OCI image \(Spezifikation\)](#)
- OCI container (Spezifikation)
- [Sehr gute Lernreihe zu dem Thema Container \(Artikel\)](#)

## Andere Systeme / Verschiedenes

### Kubernetes vs. Cloudfoundry

```
cloudfoundry hat als kleinsten Baustein, die application.
Der Entwickler entwickelt diese und pushed diese dann.
```

Dadurch wird der gesamte Prozess angetriggert  
(Es wird IMHO ein build pack verwendet) und das image wird gebaut.

Meiner Meinung nach verwendet es auch OCI für die Images  
(not sure)

Als Deployment platform für cloudfoundry kann auch kubernetes verwendet werden

Kubernetes setzt beim image an, das ist der kleinste Baustein.  
Kubernetes selbst ist nicht in der Lage images zu bauen.

Um diesen Prozess muss sich der Entwickler kümmern oder es wird eine Pipeline bereitgestellt, die das ermöglicht.

Kubernetes skaliert nicht out of the box, zumindest nicht so integriert wie das bei Cloudfoundry möglich ist.

Die Multi-Tenant möglichkeit geht nicht, wie ich das in Cloudfoundry verstehe out of the box.

Datenbanken sind bei Kubernetes nicht ausserhalb, sondern Teil von Kubernetes (bei Cloudfoundry ausserhalb)

Eine Verknüpfung der applikation mit der Datenbank erfolgt nicht automatisch

Quintessenz: Wenn ich Kubernetes verwende, muss ich mich um den Prozess "Von der Applikation zum Deployment/Image/Container" selbst kümmern, bspw. in dem ich eine Pipeline in gitlab baue

## Kubernetes Alternativen

docker-compose  
=====

Vorteile:  
>>>>>>>>  
Einfach zu lernen

Nachteile:  
>>>>>>>>  
Nur auf einem Host  
rudimentäre Features (kein loadbalancing)

Mittel der Wahl als Einstieg

docker swarm  
=====

Zitat Linux Magazin: Swarm ist das Gegenangebot zu Kubernetes für alle Admins, die gut mit den Docker-Konventionen leben können und den Umgang mit den Standard-Docker-APIs gewöhnt sind. Sie haben bei Swarm weniger zu lernen als bei Kubernetes.

Vorteile:  
>>>>>>>>  
Bereits in Docker integriert (gleiche Komandos)  
Einfacher zu lernen

Nachteile:  
>>>>>>>>  
Kleinere Community  
Kleineres Feature-Set als Kubernetes  
(Opinion): Bei vielen Containern wird es unhandlich

openshift 4 (Redhat)  
=====

- Verwendet als runtime: CRI-O (Redhat)

Vorteile:  
>>>>>>>>



```
Container laufen nicht als root (by default)
Viele Prozesse bereits mitgedacht als Tools
?? Applikation deployen ??
```

In OpenShift 4 - Kubernetes als Unterbau

```
Nachteile:
>>>>>>>>
o Lizenzgebühren (Redhat)
o kleinere Userbase
```

```
mesos
=====
```

Mesos ist ein Apache-Projekt, in das Mesospheres Marathon und DC/OS eingeflossen sind. Letzteres ist ein Container-Betriebssystem. Mesos ist kein Orchestrator im eigentlichen Sinne. Vielmehr könnte man die Lösung als verteiltes Betriebssystem bezeichnen, das eine gemeinsame Abstraktionsschicht der Ressourcen, auf denen es läuft, bereitstellt.

Vorteile:

Nachteile:

```
Rancher
=====
Graphical frontend, build on containers to deploy multiple kubernetes clusters
```

## Hyperscaler vs. Kubernetes on Premise

```
Neutral:
=====
o Erweiterungen spezifisch für die Cloud-Plattform
o Spezielle Kommandozeilen - Tools

Vorteile:
=====
o Kostenabrechnung nach Bedarf (Up- / Downscaling)
o Storage-Lösung (Clusterbasierte) beim CloudProvider.
o Backup mitgedacht.
o Leichter Upgrades zu machen
o wenig Operations-Aufwand durch feststehende Prozesse und Tools

Nachteile:
=====
o Gefahr des Vendor Logins
o Kosten-Explosion
o Erst_initialisierung: Aneignen von Spezial-Wissen für den jeweiligen Cloud-Provider
(Lernkurve und Invest)

Gibt es eine Abstraktionsschicht, die für alle Cloud-Anbieter verwenden kann.
```

## Lokal Kubernetes verwenden

### Kubernetes in ubuntu installieren z.B. innerhalb virtualbox

#### Walkthrough

```
sudo snap install microk8s --classic
Important enable dns // otherwise not dns lookup is possible
microk8s enable dns
microk8s status

Execute kubectl commands like so
microk8s kubectl
microk8s kubectl cluster-info

Make it easier with an alias
echo "alias kubectl='microk8s kubectl'" >> ~/.bashrc
```

```
source ~/.bashrc
kubectl
```

## Working with snaps

```
snap info microk8s
```

### Ref:

- <https://microk8s.io/docs/setting-snap-channel>

## minikube

### Decide for an hypervisor

e.g. hyperv

- <https://minikube.sigs.k8s.io/docs/drivers/hyperv/>

### Install minikube

- <https://minikube.sigs.k8s.io/docs/start/>

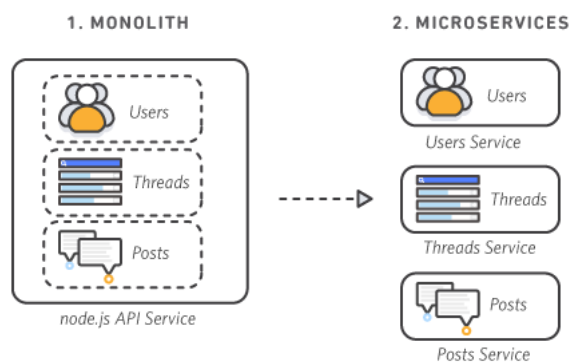
### rancher for desktop

- <https://github.com/rancher-sandbox/rancher-desktop/releases/tag/v1.9.1>

## Microservices

### Microservices vs. Monolith

#### Schaubild



Quelle: AWS Amazon

### Monolithische Architektur

- Alle Prozesse eng miteinander verbunden.
- Alles ist ein einziger Service
- Skalierung:
  - Gesamte Architektur muss skaliert werden bei Spitzen

### Herausforderung: Monolithische Architektur

- Verbesserung und Hinzufügen neuer Funktionen wird mit zunehmender Codebasis zunehmend komplexer
- Nachteil: Schwer zu experimentieren
- Nachteil: Hinderlich für die Umsetzung neuer Ideen/Konzepte

### Vorteile: Monolithische Architektur

- Gut geeignet für kleinere Konzepte und Teams
- Gut geeignet, wenn Projekt nicht stark wächst.
- Gut geeignet wenn Projekt durch ein kleines Team entwickelt wird.
- Guter Ausgangspunkt für ein kleineres Projekt
- Mit einer MicroService - Architektur zu starten, kann hinderlich sein.

### Microservices

- Jede Anwendung wird in Form von eigenständigen Komponenten erstellt.
- Jeder Anwendungsprozess wird als Service ausgeführt
- Services kommunizieren über schlanke API's miteinander
- Entwicklung in Hinblick auf Unternehmensfunktionen
- Jeder Service erfüllt eine bestimmte Funktion.

- Sie werden unabhängig voneinander ausgeführt, daher kann:
  - Jeder Service aktualisiert
  - bereitgestellt
  - skaliert werden

### Eigenschaften von microservices

- Eigenständigkeit
- Spezialisierung

### Vorteil: Microservices

- Agilität
  - kleines Team sind jeweils für einen Service verantwortlich
  - können schnell und eigenverantwortlich arbeiten
  - Entwicklungszyklus wird verkürzt.
- Flexible Skalierung
  - Jeder Service kann unabhängig skaliert werden.
- Einfache Bereitstellung
  - kontinuierliche Integration und Bereitstellung
  - einfach:
    - neue Konzepte auszuprobieren und zurückzunehmen, wenn etwas nicht funktioniert.
- Technologische Flexibilität
  - Die Teams haben die Freiheit, das beste Tool zur Lösung ihrer spezifischen Probleme auszuwählen.
  - Infolgedessen können Teams, die Microservices entwickeln, das beste Tool für die jeweilige Aufgabe wählen.
- Wiederverwendbarer Code
  - Die Aufteilung der Software in kleine, klar definierte Module ermöglicht es Teams, Funktionen für verschiedene Zwecke zu nutzen.
  - Ein Service/Funktion als Baustein
- Resilienz
  - Gut geplant/designed -> erhöht die Ausfallsicherheit
  - Monolithisch: Eine Komponente fällt aus, kann zum Ausfall der gesamten Anwendung führen.
  - Microservice: kompletter Ausfall wird vermieden, nur einzelnen Funktionalitäten sind betroffen

### Nachteile: Microservices

- Höhere Komplexität
- Bei schlechter / nicht automatischer Dokumentation kann man bei einer größeren Anzahl von Microservices den Überblick der Zusammenarbeit verlieren
- Aufwand: Architektur von Monolithisch nach Microservices IST Aufwand !
- Aufwand Wartung und Monitoring (Kubernetes)
- Erhöhte Knowledge bzgl. Debugging.
- Fallback-Aufwand (wenn ein Service nicht funktioniert, muss die Anwendung weiter arbeiten können, ohne dass andere Service nicht funktionieren)
- Erhöhte Anforderung an Konzeption (bzgl. Performance und Stabilität)
- Wichtiges Augenmerk (Netzwerk-Performance)

### Nachteile: Microservices in Kubernetes

- andere Anforderungen an Backups und Monitoring

### Monolith schneiden/aufteilen

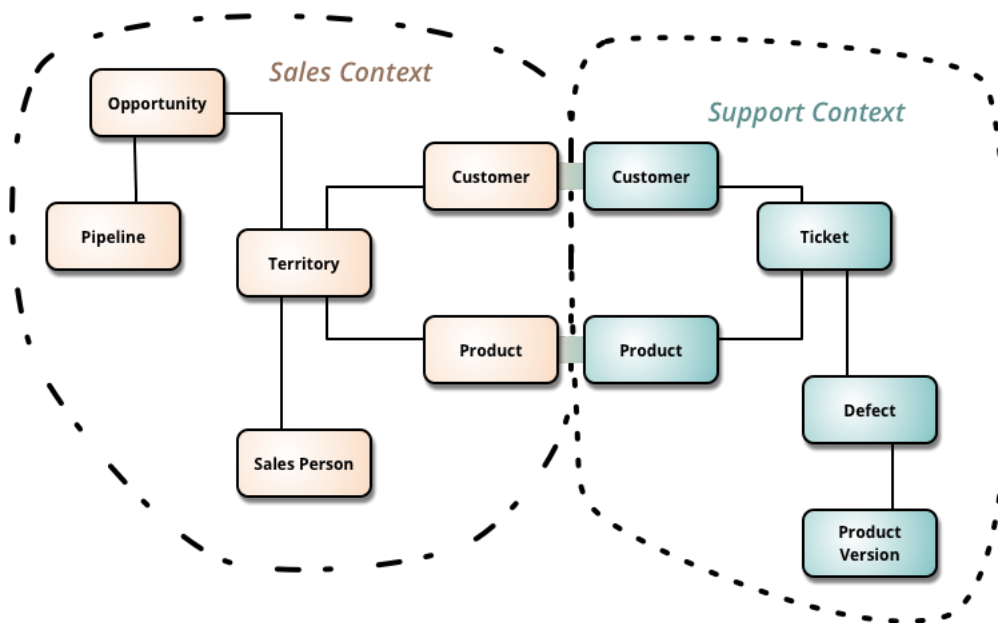
#### Wie kann ich schneiden (NOT's) ?

- Code-Größe
- Technische Schnitt
- Amazon: 2 Pizzas, wieviele können sich davon, wie gross kann man team
- Microserver wegschmeissen und er müsste in wenigen Tagen oder mehreren Wochen wieder herstellen

#### Wie kann ich schneiden (GUT) ?

- DDD (Domain Driven Design) - Welche Aufgaben gibt es innerhalb des sogenannten Bounded Context in meiner Domäne
- Domäne: Bibliothek
- In der Bibliothek
  - Leihe
  - Suche

### Bounded Context



## Zwei Merkmale mit den wir arbeiten

- Kohäsion (innerer Zusammenhalt des Fachbereichs) - innerhalb eines Services
- Bindung (lose Bindung) - zwischen den Services
- Jeder Service soll unabhängig sein

## Was heisst unabhängiger Service

1. Er muss funktionieren, auch wenn ein anderes Service nicht läuft (keine Abhängigkeit)
2. Er darf nicht DIREKT auf die Daten eines anderen Services zugreifen (maximal über Schnittstelle)
3. Jeder hat Service, ist völlig autark und seine eigene BusinessLogik und seine eigene Datenbank

## Regeln für das Design von Services

### Regel 1:

Es sollte eine große Kohäsion innerhalb des Services sein.  
(Bindung). Alles sollte möglichst benötigt werden.

(Ist eine schwache Kohäsion innerhalb des Services, sind Funktionen dort, die eigentlich in einen anderen Service gehören)

### Regel 2: lose Bindung (zwischen Services)

Es sollte eine lose Bindung zu anderen Services geben.  
(Ist die Bindung zu gross, sind entweder die Services zu klein konzipiert oder Funktionen sind an der falschen Stelle implementiert)

zu klein: zu viele Abfragen anderer Service ....

### Regel 3: unabhängigkeit

Jeder Service muss eigenständig sein und seine eigene Datenbank haben.

## Datenbanken

### Herangehensweise

heisst auch:  
o Kein großes allmächtiges Datenmodell, sondern viele kleine  
(nicht alles in jedem kleinen Datenmodell, sondern nur, was im jeweiligen Bounded Context benötigt wird)

## Eine Datenbank pro Service (eigenständig / abgespeckt)

### Warum ?

Axiom: Eine eigenständige Datenbank pro Service. Warum ?  
(Service will NEVER reach into another services database)

#### Punkt 1 : Jeder Service soll unabhängig laufen können

```
We want each service to run independently of other services

o no DB for everything (If DB goes down our service goes down)
o it easier to scale (if one service needs more capacity)
o more resilient. If one service goes down, our service will still work.
```

#### Punkt 2: Datenbank schemata könnten sich unerwartet ändern

```
o We (Service A) use data from Service B, directly retrieving it from the db.
o We (Service) want property name: Lisa
o Team of Service B changes this property to: firstName
 AND do not inform us.
 (This breaks our service !!) . OUR SERV
```

#### Punkt 3: Freiheit der Datenbankwahl

```
3.4.3 Some services might function more efficiently with different types
of DB's (sql vs. nosql)
```

#### Beispiel - Bounded

Der Bounded Context definiert den Einsatzbereich eines Domänenmodells.

Es umfasst die Geschäftslogik für eine bestimmte Fachlichkeit. Als Beispiel beschreibt ein Domänenmodell die Buchung von S-Bahn-Fahrkarten und ein weiteres die Suche nach S-Bahn-Verbindungen.

Da die beiden Fachlichkeiten wenig miteinander zu tun haben, sind es zwei getrennte Modelle. Für die Fahrkarten sind die Tarife relevant und für die Verbindung die Zeit, das Fahrziel und der Startpunkt der Reise.

```
oder z.B. die Domäne: Bibliothek
Bibliothek
 Leihe (bounded context 1)
 Suche (bounded context 2)
```

#### Strategic Patterns - wie monolith praktisch umbauen

##### Pattern: Strangler Fig Application

- Technik zum Umschreiben von Systemen

##### Wie umleitung, z.B.

- http proxy
- oder s.u. branch by extraction
- An- und Abschalten mit Feature Toggle
- Über message broker

##### http - proxy - Schritte

1. Schritt: Proxy einfügen
2. Schritt: Funktionalität migrieren
3. Schritt: Aufrufe umleiten

##### Message broker

- Monolith reagiert auf bestimmte Messages bzw. ignoriert bestimmte messages
- monolith bekommt bestimmte nachrichten gar nicht
- service reagiert auf bestimmte nachrichten

##### Pattern: Parallel Run

- Service und Teil im Monolith wird parallel ausgeführt
- Und es wird überprüft, ob das Ergebnis in beiden Systemen das gleiche ist (z.B. per batch job)

##### Pattern: Decorating Collaborator

- Ansteuerung als nachgelagerter Prozess über einen Proxy

##### Pattern Branch by Abstraction

- Beispiel Notification

##### Schritt 1: Abstraction der zu ersetzende Funktionalität erstellen

##### Schritt 2: Ändern sie die Clients der bestehenden Funktionalität so, dass sie die neue Abstraktion verwenden

##### Schritt 3: Neue Implementierung der Abstraktion

Erstellen Sie eine neue Implementierung der Abstraktion mit der überarbeiteten Funktionalität.

```
In unserem Fall wird diese neue Implementierung unseren neuen
Mikroservice aufrufen
```

#### Schritt 4: Abstraktion anpassen -> neue Implementierung

```
Abstraktion anpassen, dass sie unsere neue Implementierung verwendet
```

#### Schritt 5: Abstraktion aufräumen und alte Implementierung entfernen

#### Literatur von Monolith zu Microservices

- <https://www.amazon.de/Vom-Monolithen-Microservices-bestehende-umzugestalten/dp/3960091400/>

## Extras

### Install minikube on wsl2

#### Eventually update wsl

```
We need the newest version of wsl as of 09.2022
because systemd was included there
in powershell
wsl --shutdown
wsl --update
wsl
```

#### Walkthrough (Step 1) - in wsl

```
as root in wsl
sudo su -
echo "[boot]" >> /etc/wsl.conf
echo "systemd=true" >> /etc/wsl.conf
```

#### Walkthrough (Step 2) - restart wsl

```
in powershell
wsl --shutdown
takes a little bit longer now
wsl
```

#### Walkthrough (step 3) - Setup minikube

```
as unprivileged user, e.g. yourname
sudo apt-get install -y \
 apt-transport-https \
 ca-certificates \
 curl \
 software-properties-common

key for rep
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

sudo add-apt-repository \
 "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
 $(lsb_release -cs) \
 stable"

sudo apt-get update -y
sudo apt-get install -y docker-ce

sudo usermod -aG docker $USER && newgrp docker
sudo apt install -y conntrack

Download the latest Minikube
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

Make it executable
chmod +x ./minikube

Move it to your user's executable PATH
sudo mv ./minikube /usr/local/bin/

##Set the driver version to Docker
minikube config set driver docker

install minikube
```

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
and start it
minikube start

find out system pods
kubectl get pods -A

Note: kubernetes works within docker now
you can figure this out by
docker container ls
Now exec into the container you see: e.g acec
docker exec -it acec bash
within the container (docker runs within the container as well)
docker container ls
```

## Reference

- No need to install systemd mentioned here.
- <https://www.virtualizationhowto.com/2021/11/install-minikube-in-wsl-2-with-kubectl-and-helm/>

## kustomize - gute Struktur für größere Projekte

### Structure

Zum Beenden des Vollbildmodus **F11** drücken

```
tree apps/
apps/
├── README.md
├── app-proxy
│ ├── base
│ │ └── kustomization.yaml
│ └── overlays
│ └── aws-prod
│ ├── config.json
│ ├── ingress.yaml
│ └── kustomization.yaml
├── argocd-agent
│ ├── base
│ │ └── kustomization.yaml
│ └── overlays
│ └── aws-prod
│ ├── config.json
│ └── kustomization.yaml
├── aws-prod-apps
│ ├── aws-prod
│ │ └── config_dir.json
├── default-git-source
│ ├── aws-prod
│ │ └── config_dir.json
├── demoapp
│ ├── base
│ │ └── kustomization.yaml
│ └── overlays
│ └── aws-prod
│ ├── config.json
│ └── kustomization.yaml
├── events
│ ├── base
│ │ └── kustomization.yaml
│ └── overlays
│ └── aws-prod
│ ├── config.json
│ └── kustomization.yaml
```

- Source: [https://www.reddit.com/r/kubernetes/comments/sd50hk/kustomize\\_with\\_multiple\\_deployments\\_how\\_to\\_keep/](https://www.reddit.com/r/kubernetes/comments/sd50hk/kustomize_with_multiple_deployments_how_to_keep/)

#### kustomize with helm

- <https://fabianlee.org/2022/04/18/kubernetes-kustomize-with-helm-charts/>

## Documentation

### References

- <https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/deployment-v1/#DeploymentSpec>

### Tasks Documentation - Good one !

- <https://kubernetes.io/docs/tasks/>

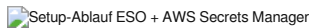
## AWS

### External Secrets Operator (ESO) mit AWS Secrets Manager + KMS einrichten

#### Hintergrund

Secrets liegen zentral in **AWS Secrets Manager** (dort automatisch durch **AWS KMS** verschlüsselt). Der **External Secrets Operator (ESO)** läuft als Controller im Cluster, holt sich die aktuellen Werte über eine eng begrenzte IAM-Rolle und legt daraus ein ganz normales Kubernetes- `Secret` an.





Setup-Ablauf ESO + AWS Secrets Manager

Die im Git-Manifest sichtbaren Ressourcen ( `SecretStore` , `ExternalSecret` ) enthalten **n**ie den Secret-Wert selbst — nur den Namen/ARN des AWS-Secrets.

## 1. AWS-Seite (einmalig, i.d.R. vom Cluster-Admin)

### Abkürzungen kurz erklärt

Kürzel	Bedeutung	Einfach gesagt
<b>IAM</b>	Identity and Access Management	AWS-Bereich, der regelt: wer darf was
<b>Policy</b>	—	Ein Zettel mit genau einer Erlaubnis ("darf X lesen")
<b>Role (Rolle)</b>	—	Ein "Ausweis", den man sich vorübergehend ausleihen kann
<b>ARN</b>	Amazon Resource Name	Die eindeutige "Adresse" einer AWS-Ressource (wie eine IBAN)
<b>OIDC</b>	OpenID Connect	Standard, mit dem sich der Kubernetes-Cluster bei AWS ausweisen kann
<b>IRSA</b>	IAM Roles for Service Accounts	Verfahren: ein Kubernetes-ServiceAccount bekommt eine IAM-Rolle geliehen
<b>STS</b>	Security Token Service	AWS-Dienst, der kurzlebige "Eintrittskarten" (Tokens) ausstellt
<b>JWT</b>	JSON Web Token	Ein digital signierter, fälschungssicherer "Ausweis" als Text

Die kurze Version: **Policy = was darf man, Role = wer darf es sich ausleihen, IRSA/OIDC/STS/JWT = wie das Ausleihen technisch funktioniert**, ohne dass irgendwo ein Passwort oder Access-Key gespeichert werden muss.

### 1.1 Secret in Secrets Manager anlegen

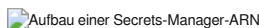
```
aws secretsmanager create-secret \
 --name prod/db-credentials \
 --secret-string '{"username":"app","password":"s3cr3t"}'
```

### 1.2 IAM Policy — Zugriff nur auf genau dieses Secret

**Warum überhaupt eine Policy?** Ohne Erlaubnis darf niemand in AWS irgendetwas lesen — auch ESO nicht. Die Policy ist die Erlaubnis, aber bewusst so eng wie möglich geschnitten: Sie erlaubt **nur** `GetSecretValue` (lesen, nicht ändern) und **nur** für die ARN (die "Adresse") von genau diesem einen Secret. Geht die Policy verloren oder wird sie missbraucht, ist der Schaden auf dieses eine Secret begrenzt — nicht auf ganz AWS.

**Warum der Name `eso-prod-db-credentials` ?** Reine Konvention, aber eine hilfreiche: `eso-` zeigt, wer die Policy nutzt (der External Secrets Operator), `prod-db-credentials` zeigt, für welches Secret sie gilt. Wer in der AWS-Konsole später 50 Policies sieht, findet die richtige auf einen Blick — der Name selbst hat keine technische Funktion.

**Was ist die ARN in der `Resource` -Zeile?** Die eindeutige "Adresse" des Secrets in AWS — siehe Aufbau unten. Wichtig: AWS hängt beim Anlegen automatisch 6 Zufallszeichen an den Namen an, deshalb steht am Ende ein Wildcard ( `-*` ).



Aufbau einer Secrets-Manager-ARN

```
vi 01-eso-iam-policy.json
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Action": ["secretsmanager:GetSecretValue"],
 "Resource": "arn:aws:secretsmanager:eu-central-1:123456789012:secret:prod/db-credentials-*"
 }
]
}
```

```
aws iam create-policy \
 --policy-name eso-prod-db-credentials \
 --policy-document file://01-eso-iam-policy.json
```

### 1.3 IAM-Rolle für IRSA (IAM Roles for Service Accounts)

**Warum reicht die Policy allein nicht?** Eine Policy ist nur der Erlaubnis-Zettel — irgendjemand muss ihn sich aber "anziehen" können. Das ist die **Rolle**: Sie bekommt die Policy angeheftet und kann dann von jemandem zeitweise übernommen ("assumed") werden. Der ESO-Pod im Cluster bekommt so, ohne je ein Passwort zu besitzen, für kurze Zeit genau diese eine Berechtigung geliehen.

**Warum eine eigene Rolle statt eines gespeicherten Access Keys?** Ein Access Key ist ein dauerhaftes Geheimnis, das irgendwo liegt und gestohlen werden kann. Die Rolle dagegen wird über **IRSA** genutzt: Der ServiceAccount im Cluster weist sich über den **OIDC**-Provider des Clusters bei AWS aus, AWS fragt seinen **STS**-Dienst, der ein kurzlebiges **JWT** ("digitaler Ausweis mit Ablaufdatum") ausstellt. Kein Passwort, keine Datei, nichts, was dauerhaft irgendwo liegt und geklaut werden könnte.

**Warum genau dieser Name/Bedingung in der Trust Policy?** Die Trust Policy regelt **wer** die Rolle überhaupt anziehen darf. Die `Condition` unten schränkt das auf **genau einen** ServiceAccount ein ( `system:serviceaccount:<namespace>:<name>` ). Ohne diese Einschränkung könnte theoretisch jeder Pod im Cluster versuchen, sich diese Rolle zu leihen — die Bedingung ist also die eigentliche Absicherung, nicht nur Formsache. Namespace und Name müssen dabei exakt zum ServiceAccount aus Schritt 2.2 passen, sonst schlägt das Ausleihen fehl.

```
vi 02-eso-trust-policy.json
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": {
 "Federated": "arn:aws:iam::123456789012:oidc-provider/oidc.eks.eu-central-1.amazonaws.com/id/EXAMPLE1234"
 },
 "Action": "sts:AssumeRoleWithWebIdentity",
 "Condition": {
 "StringEquals": {
 "oidc.eks.eu-central-1.amazonaws.com/id/EXAMPLE1234:sub": "system:serviceaccount:external-secrets:eso-aws-sa"
 }
 }
 }
]
}
```

```
aws iam create-role \
 --role-name eso-prod-db-credentials \
 --assume-role-policy-document file://02-eso-trust-policy.json

aws iam attach-role-policy \
 --role-name eso-prod-db-credentials \
 --policy-arn arn:aws:iam::123456789012:policy/eso-prod-db-credentials
```

#### FAQ: Brauche ich eine eigene Rolle pro Pod?

**Nein.** Die Rolle hängt am **ServiceAccount**, nicht am einzelnen Pod. Alle Replicas eines Deployments teilen sich denselben ServiceAccount — 100 Pod-Replicas brauchen also nicht 100 Rollen, sondern genau eine.

Wichtig: In diesem Setup hängt die Rolle sogar am ServiceAccount des **ESO-Controllers** selbst — ESO ruft AWS auf, nicht die Anwendungs-Pods direkt.

Bei mehreren Anwendungen/Teams mit unterschiedlichen Secrets gibt es zwei Muster:

Muster	Vorteil	Nachteil
Eine gemeinsame Rolle für ESO, Policy erlaubt mehrere Secret-ARNs	Einfach aufzusetzen	Weniger strenge Trennung zwischen Teams
Eine Rolle pro Team/Namespace, jeweils eigener <code>SecretStore</code> + eigener <code>ServiceAccount</code>	Team A kommt nicht an Secrets von Team B	Mehr Rollen/Policies zu pflegen

Faustregel: Die Granularität richtet sich nach **wer darf was sehen**, nicht nach der Anzahl der Pods.

## 2. Kubernetes-Seite

### 2.1 External Secrets Operator per Helm installieren

```
helm repo add external-secrets https://charts.external-secrets.io
helm repo update

helm install external-secrets external-secrets/external-secrets \
 --namespace external-secrets \
 --create-namespace \
 --wait
```

Das installiert den Operator **und** die CRDs `SecretStore`, `ClusterSecretStore` und `ExternalSecret` (`apiVersion: external-secrets.io/v1`).

### 2.2 ServiceAccount mit IRSA-Annotation

```
vi 03-eso-serviceaccount.yml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: eso-aws-sa
 namespace: external-secrets
 annotations:
 eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/eso-prod-db-credentials
```

```
kubectl apply -f 03-eso-serviceaccount.yml
```

### 2.3 SecretStore — Verbindung zu AWS Secrets Manager

```
vi 04-eso-secretstore.yml
apiVersion: external-secrets.io/v1
kind: SecretStore
```

```
metadata:
 name: aws-secretsmanager
 namespace: external-secrets
spec:
 provider:
 aws:
 service: SecretsManager
 region: eu-central-1
 auth:
 jwt:
 serviceAccountRef:
 name: eso-aws-sa
```

```
kubectl apply -f 04-eso-secretstore.yml
kubectl get secretstore -n external-secrets
```

## 2.4 ExternalSecret — welches Secret, wie oft, wohin

```
vi 05-eso-externalsecret.yml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
 name: db-credentials
 namespace: external-secrets
spec:
 refreshInterval: 1h
 secretStoreRef:
 name: aws-secretsmanager
 kind: SecretStore
 target:
 name: db-credentials
 creationPolicy: Owner
 data:
 - secretKey: DB_USERNAME
 remoteRef:
 key: prod/db-credentials
 property: username
 - secretKey: DB_PASSWORD
 remoteRef:
 key: prod/db-credentials
 property: password
```

```
kubectl apply -f 05-eso-externalsecret.yml
kubectl get externalsecret -n external-secrets
kubectl get secret db-credentials -n external-secrets -o yaml
```

### Was passiert dabei genau? (Laufzeit-Ablauf)

Der `kubectl apply` oben stößt im Hintergrund mehrere Schritte an, bis das Kubernetes- `Secret` tatsächlich existiert:

 Ablauf ExternalSecret bis Kubernetes Secret

Kurz gesagt: ESO merkt sich nichts dauerhaft selbst — bei jedem `refreshInterval` holt es sich den Wert frisch aus AWS und gleicht das Kubernetes- `Secret` ab.

## 3. Im Pod nutzen (env / envFrom)

Ab hier ist es ein ganz normales Kubernetes- `Secret` — die Wege aus der [Credentials-Übersicht](#) gelten unverändert:

```
vi 06-eso-demo-pod.yml
apiVersion: v1
kind: Pod
metadata:
 name: eso-demo
 namespace: external-secrets
spec:
 containers:
 - name: demo
 image: nginx
 envFrom:
 - secretRef:
 name: db-credentials
```

```
kubectl apply -f 06-eso-demo-pod.yml
kubectl exec eso-demo -n external-secrets -- env | grep DB_
```

## Aufräumen

```
kubectl delete namespace external-secrets
aws iam detach-role-policy --role-name eso-prod-db-credentials --policy-arn arn:aws:iam::123456789012:policy/eso-prod-db-credentials
aws iam delete-role --role-name eso-prod-db-credentials
aws iam delete-policy --policy-arn arn:aws:iam::123456789012:policy/eso-prod-db-credentials
aws secretsmanager delete-secret --secret-id prod/db-credentials --force-delete-without-recovery
```

### Kurz zusammengefasst

Ressource	apiVersion / Tool	Zweck
Helm Chart <code>external-secrets/external-secrets</code>	Helm Repo <code>https://charts.external-secrets.io</code>	Installiert Operator + CRDs
ServiceAccount mit <code>eks.amazonaws.com/role-arn</code>	Kubernetes	IRSA-Bindung an IAM-Rolle
SecretStore	<code>external-secrets.io/v1</code>	Verbindung zu AWS Secrets Manager
ExternalSecret	<code>external-secrets.io/v1</code>	Welches AWS-Secret → welches K8s-Secret

### ESO mit AWS Secrets Manager - Secret als Volume-Mount

Dieses Dokument baut auf [ESO mit AWS Secrets Manager + KMS einrichten](#) auf. Die komplette AWS-Seite (KMS, Secrets Manager, IAM-Policy, IAM-Rolle/IRSA) und die Kubernetes-Ressourcen `SecretStore` / `ExternalSecret` sind **identisch** — ESO erzeugt in beiden Fällen einfach ein normales Kubernetes- `Secret` . Der einzige Unterschied ist, **wie der Pod dieses Secret konsumiert**.

Warum überhaupt Volume statt ENV-Variable? Siehe [Credentials-Übersicht, Abschnitt "ENV-Variable vs. Datei"](#): kurz gesagt, ein Volume-Mount taucht nicht in `kubectl exec -`  
`- env` auf und wird nicht an Kindprozesse vererbt.

### Voraussetzung

Schritte 1 (AWS-Seite komplett) und 2.1–2.4 (Helm-Installation, ServiceAccount, `SecretStore` , `ExternalSecret` ) aus [ESO mit AWS Secrets Manager + KMS einrichten](#) sind bereits durchgeführt — es existiert also schon ein Kubernetes- `Secret` namens `db-credentials` im Namespace `external-secrets` .

### Pod mit Secret als Volume-Mount

```
vi 07-eso-demo-pod-volume.yml
apiVersion: v1
kind: Pod
metadata:
 name: eso-demo-volume
 namespace: external-secrets
spec:
 containers:
 - name: demo
 image: nginx
 volumeMounts:
 - name: db-credentials-volume
 mountPath: /etc/secrets/db
 readOnly: true
 volumes:
 - name: db-credentials-volume
 secret:
 secretName: db-credentials
 defaultMode: 0440
```

```
kubectl apply -f 07-eso-demo-pod-volume.yml
```

### Was bedeuten die Felder?

Feld	Bedeutung
<code>volumeMounts.mountPath</code>	Ordner <b>im Container</b> , unter dem die Secret-Keys als Dateien auftauchen (hier: <code>/etc/secrets/db/username</code> , <code>/etc/secrets/db/password</code> )
<code>volumeMounts.readOnly</code>	Container kann die Dateien nur lesen, nicht verändern
<code>volumes.secret.secretName</code>	Name des Kubernetes-Secret, das ESO angelegt hat
<code>volumes.secret.defaultMode</code>	Datei-Rechte in oktal (z.B. <code>0440</code> = nur Owner + Gruppe dürfen lesen, niemand schreiben)

### Testen

```
kubectl exec eso-demo-volume -n external-secrets -- ls -l /etc/secrets/db
kubectl exec eso-demo-volume -n external-secrets -- cat /etc/secrets/db/username
kubectl exec eso-demo-volume -n external-secrets -- env | grep -i db
-> zeigt nichts, das Secret taucht bewusst NICHT in den ENV-Variablen auf
```

### Aktualisiert sich das automatisch?

Ja — ändert ESO das Kubernetes- `Secret` (weil sich der Wert in AWS Secrets Manager geändert hat), aktualisiert Kubernetes die gemounteten Dateien im Pod **automatisch**, meist innerhalb von ca. 1 Minute (kubelet-Sync-Intervall) — ganz ohne Pod-Neustart. Bei ENV-Variablen ist das **nicht** der Fall: ein bereits laufender Prozess bekommt eine geänderte ENV-Variablen nie mehr mit, dafür ist ein Pod-Neustart nötig (siehe [Stakater Reloader](#)).

Aufräumen

```
kubectl delete -f 07-eso-demo-pod-volume.yml
```

Kurz zusammengefasst

	ENV-Variable	Volume-Mount
Sichtbar via <code>kubectl exec -- env</code>	Ja	Nein
Automatisches Update bei Secret-Änderung	Nein (Neustart nötig)	Ja (~1 Min., kein Neustart)
Passendes Dokument	<a href="#">eso-secrets-manager-setup.md</a>	dieses Dokument

Documentation for Settings right resources/limits

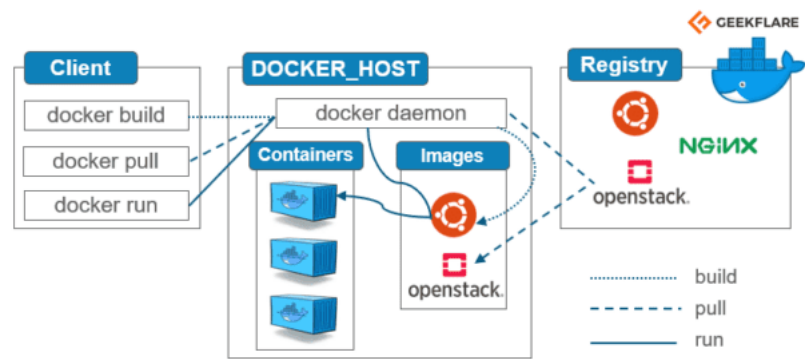
Goldilocks

- <https://www.fairwinds.com/blog/introducing-goldilocks-a-tool-for-recommending-resource-requests>

Kubernetes - Überblick

Allgemeine Einführung in Container (Dev/Ops)

Architektur



Was sind Docker Images

- Docker Image benötigt, um zur Laufzeit Container-Instanzen zu erzeugen
- Bei Docker werden Docker Images zu Docker Containern, wenn Sie auf einer Docker Engine als Prozess ausgeführt
- Man kann sich ein Docker Image als Kopiervorlage vorstellen.
  - Diese wird genutzt, um damit einen Docker Container als Kopie zu erstellen

Was sind Docker Container ?

```
- vereint in sich Software
- Bibliotheken
- Tools
- Konfigurationsdateien
- keinen eigenen Kernel
- gut zum Ausführen von Anwendungen auf verschiedenen Umgebungen

Weil :
- Container sind entkoppelt
- Container sind voneinander unabhängig
- Können über wohldefinierte Kommunikationskanäle untereinander Informationen austauschen

- Durch Entkopplung von Containern:
 o Unverträglichkeiten von Bibliotheken, Tools oder Datenbank können umgangen werden, wenn diese von den Applikationen in unterschiedlichen Versionen benötigt werden.
```

Container vs. VM

```
VM's virtualisieren Hardware
Container virtualisieren Betriebssystem
```

## Dockerfile

- Textdatei, die Linux - Kommandos enthält
  - die man auch auf der Kommandozeile ausführen könnte
  - Diese erledigen alle Aufgaben, die nötig sind, um ein Image zusammenzustellen
  - mit docker build wird dieses image erstellt

## Einfaches Beispiel eines Dockerfiles

```
FROM nginx:latest
COPY html /usr/share/nginx/html
```

```
beispiel
cd beispiel
ls
Dockerfile
docker build .
docker push
```

## Komplexeres Beispiel eines Dockerfiles

- <https://github.com/StefanScherer/whoami/blob/main/Dockerfile>

## Microservices (Warum ? Wie ?) (Devs/Ops)

### Was soll das ?

Ein mini-dienst, soll das minimale leisten, d.h. nur das wofür er da ist.

-> z.B. Webserver  
oder Datenbank-Server  
oder Dienst, der nur reports erstellt

### Wie erfolgt die Zusammenarbeit

Orchestrierung (im Rahmen der Orchestrierung über vorgefertigte Schnittstellen, d.h. auch feststehende Benennung)  
- Label

### Vorteile

```
##
Leichtere Updates von Microservices, weil sie nur einen kleinere Funktionalität
```

### Nachteile

```
* Komplexität
* z.B. in Bezug auf Debugging
* Logging / Backups
```

### Wann macht Kubernetes Sinn, wann nicht?

#### Wann nicht sinnvoll ?

- Anwendung, die ich nicht in Container "verpackt" habe
- Spielt der Dienstleister mit (Wartungsvertrag)
- Kosten / Nutzenverhältnis (Umstellen von Container zu teuer)
- Anwendung lässt sich nicht skalieren
  - z.B. Bottleneck Datenbank
  - Mehr Container bringen nicht mehr (des gleichen Typs)

#### Wo spielt Kubernetes seine Stärken aus ?

- Skalieren von Anwendungen.
- bessere Hochverfügbarkeit out-of-the-box
- Heilen von Systemen (neu starten von Containern)
- Automatische Überwachung (mit deklarativem Management) - ich beschreibe, was ich will
- Neue Versionen auszurollen (Canary Deployment, Blue/Green Deployment)

### Mögliche Nachteile

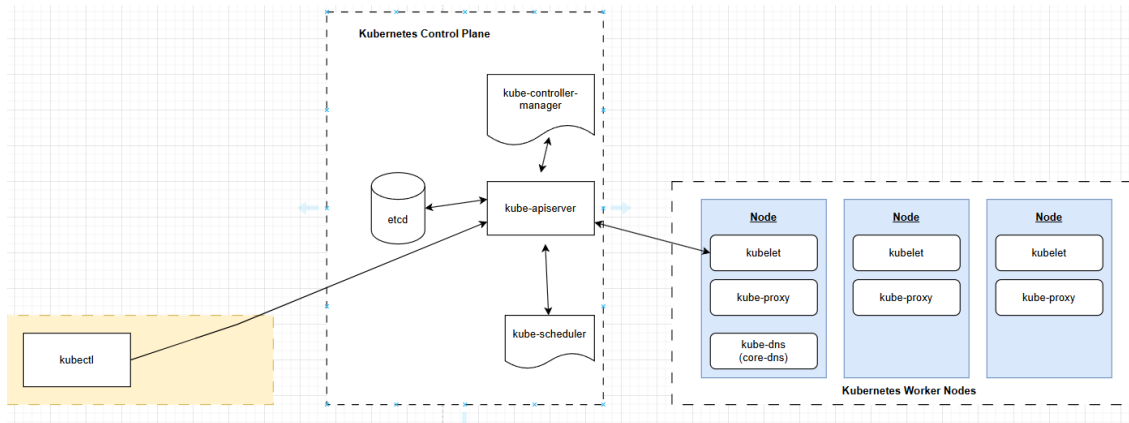
- Steigert die Komplexität.
- Debugging wird u.U. schwieriger
- Mit Kubernetes erkaufe ich mir auch, die Notwendigkeit.
  - Über adequate Backup-Lösungen nachzudenken (Moving Target, Kubernetes Aware Backups)
  - Bereitsstellung von Monitoring
  - Bereitsstellung Observability (Log-Aggregierungslösung, Tracing)

## Klassische Anwendungsfällen (wo Kubernetes von Vorteil)

- Webbasierte Anwendungen (z.B. auch API's bzw. Web)
- Ausser Problematik: Session StickyNess

## Aufbau Allgemein

### Schaubild



## Komponenten / Grundbegriffe

### Control Plane (Master)

#### Aufgaben

- Der Control Plane (Master) koordiniert den Cluster
- Der Control Plane (Master) koordiniert alle Aktivitäten in Ihrem Cluster
  - Planen von Anwendungen
  - Verwalten des gewünschten Status der Anwendungen
  - Skalieren von Anwendungen
  - Rollout neuer Updates.

### Komponenten des Masters

#### ETCD

- Verwalten der Konfiguration und des Status des Clusters (key/value - pairs)

#### KUBE-CONTROLLER-MANAGER

- Zuständig für die Überwachung der Stati im Cluster mit Hilfe von endlos loops.
- kommuniziert mit dem Cluster über die kubernetes-api (bereitgestellt vom kube-api-server)

#### KUBE-API-SERVER

- provides api-frontend for administration (no gui)
- Exposes an HTTP API (users, parts of the cluster and external components communicate with it)
- REST API

#### KUBE-SCHEDULER

- assigns Pods to Nodes.
- scheduler determines which Nodes are valid placements for each Pod in the scheduling queue ( according to constraints and available resources )
- The scheduler then ranks each valid Node and binds the Pod to a suitable Node.
- Reference implementation (other schedulers can be used)

### Nodes

- Nodes sind virtuelle oder physische Maschinen auf denen die notwendigen Kubernetes-System-Komponenten (Software)
- Worker Nodes (Knoten) sind die Arbeiter (Maschinen), die Anwendungen ausführen
- Ref: <https://kubernetes.io/de/docs/concepts/architecture/nodes/>

### Pod/Pods

- Pods sind die kleinste verwaltbare Einheit, die in Kubernetes erstellt und verwaltet werden können.
- Ein Pod (übersetzt Gruppe) ist eine Gruppe von einem oder mehreren Containern
  - gemeinsam genutzter Speicher- und Netzwerkressourcen
  - Befinden sich immer auf dem gleichen virtuellen Server

## Node (Minion) - components

### General

- On the nodes we will rollout the applications

### kubelet

Node Agent that runs on every node (worker)  
Er stellt sicher, dass Container in einem Pod ausgeführt werden.

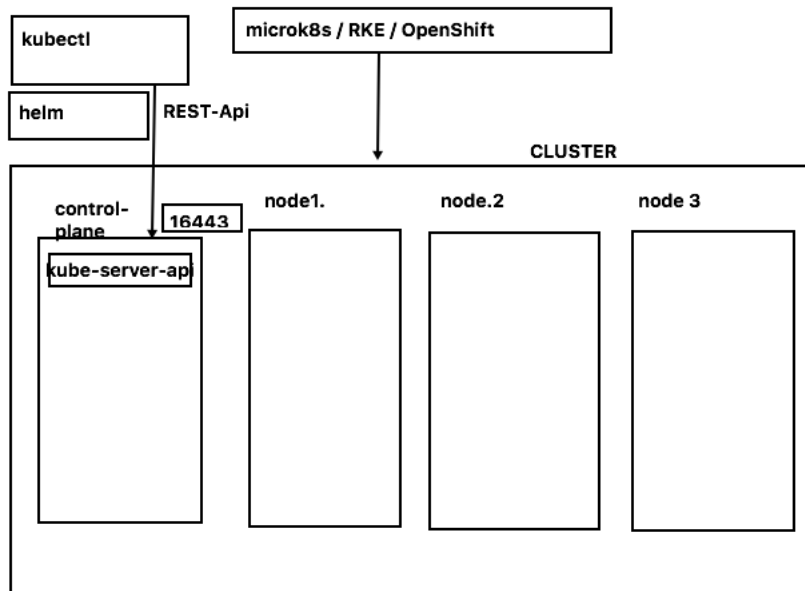
### Kube-proxy

- Läuft auf jedem Node
- = Netzwerk-Proxy für die Kubernetes-Netzwerk-Services.
- Kube-proxy verwaltet die Netzwerkkommunikation der Services innerhalb des Clusters

## Referenzen

- <https://www.redhat.com/de/topics/containers/kubernetes-architecture>

## Aufbau mit helm,OpenShift,Rancher(RKE),microk8s



## Welches System ? (minikube, micro8ks etc.)

## Überblick der Systeme

### General

kubernetes itself has not convenient way of doing specific stuff like creating the kubernetes cluster.

So there are other tools/distri around helping you with that.

### Kubeadm

#### General

- The official CNCF (<https://www.cncf.io/>) tool for provisioning Kubernetes clusters (variety of shapes and forms (e.g. single-node, multi-node, HA, self-hosted))
- Most manual way to create and manage a cluster

### microk8s

#### Prerequisites:

- at least 4 GB of ram per maschine

#### General

- Created by Canonical (Ubuntu)
- Runs on Linux
- Runs only as snap
- (In the meantime it is also available for Windows/Mac)
- HA-Cluster (control plane)

#### Production-Ready ?

- Short answer: YES

Quote canonical (2020):

MicroK8s is a powerful, lightweight, reliable production-ready Kubernetes distribution. It is an enterprise-grade Kubernetes distribution that has a small disk and memory footprint while offering carefully selected add-ons out-the-box, such as Istio, Knative, Grafana, Cilium and more. Whether you are running a production environment or interested in exploring K8s, MicroK8s serves your needs.



Ref: <https://ubuntu.com/blog/introduction-to-microk8s-part-1-2>

#### Advantages

- Easy to setup HA-Cluster (multi-node control plane)
- Easy to manage

#### Disadvantages

- Nicht so flexible wie kubeadm
- z.B. freie Wahl des CNI - Providers (z.B. Calico)
- nicht so flexibel bei speziell config (z.B. andere IP-Ranges)

#### minikube

##### Disadvantages

- Not usable / intended for production

##### Advantages

- Easy to set up on local systems for testing/development (Laptop, PC)
- Multi-Node cluster is possible
- Runs on Linux/Windows/Mac
- Supports plugin (Different name ?)

#### k3s (wsl oder virtuelle Maschine)

- sehr schlank.
- lokal installierbar (eine node, ca 5 minuten)
- ein einziges binary
- <https://docs.k3s.io/quick-start>

#### kind (Kubernetes-In-Docker)

##### General

- Runs in docker container

##### For Production ?

Having a footprint, where kubernetes runs within docker and the applications run within docker as docker containers it is not suitable for production.

#### Installation - Welche Komponenten from scratch

##### Step 1: Server 1 (manuell installiert -> microk8s)

```
Installation Ubuntu - Server

cloud-init script
s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers lttrainingdo per ssh)

Server 1 - manuell
Ubuntu 20.04 LTS - Grundinstallation

minimal Netzwerk - öffentlichen IP
nichts besonderes eingerichtet - Standard Digitalocean

Standard vo Installation microk8s
lo UNKNOWN 127.0.0.1/8 ::1/128
public ip / interne
eth0 UP 164.92.255.234/20 10.19.0.6/16 fe80::c:66ff:fec4:cbce/64
private ip
eth1 UP 10.135.0.3/16 fe80::8081:aaff:feaa:780/64

snap install microk8s --classic
Namensauflösung fuer pods
microk8s enable dns

Funktioniert microk8s
microk8s status
```

##### Steps 2: Server 2+3 (automatische Installation -> microk8s)

```
Was macht das ?
1. Basisnutzer (lttrainingdo) - keine Voraussetzung für microk8s
2. Installation von microk8s
##.>>>>>> microk8s installiert <<<<<<<<
- snap install --classic microk8s
>>>>>> Zuordnung zur Gruppe microk8s - notwendig für bestimmte plugins (z.B. helm)
```

```

usermod -a -G microk8s root
>>>>> Setzen des .kube - Verzeichnisses auf den Nutzer microk8s -> nicht zwingend erforderlich
chown -R -R microk8s ~/.kube
>>>>> REQUIRED .. DNS aktivieren, wichtig für Namensauflösungen innerhalb der PODS
>>>>> sonst funktioniert das nicht !!!
microk8s enable dns
>>>>> kubect1 alias gesetzt, damit man nicht immer microk8s kubect1 eingeben muss
- echo "alias kubect1='microk8s kubect1'" >> /root/.bashrc

cloud-init script
s.u. MITMICROK8S (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)
##cloud-config
users:
 - name: 11trainingdo
 shell: /bin/bash

runcmd:
 - sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config
 - echo " " >> /etc/ssh/sshd_config
 - echo "AllowUsers 11trainingdo" >> /etc/ssh/sshd_config
 - echo "AllowUsers root" >> /etc/ssh/sshd_config
 - systemctl reload sshd
 - sed -i '/11trainingdo/c
11trainingdo:6HeLUJW3a$4xSfDQjKWfAoGkZF3LFAxM4hgl3d6ATbr2kEu9zMOfwLxkYMO.AJF526mZONwdmsm9sg0tCBK1.SYbhS52u70:17476:0:99999:7:::
/etc/shadow
 - echo "11trainingdo ALL=(ALL) ALL" > /etc/sudoers.d/11trainingdo
 - chmod 0440 /etc/sudoers.d/11trainingdo

 - echo "Installing microk8s"
 - snap install --classic microk8s
 - usermod -a -G microk8s root
 - chown -f -R microk8s ~/.kube
 - microk8s enable dns
 - echo "alias kubect1='microk8s kubect1'" >> /root/.bashrc

Prüfen ob microk8s - wird automatisch nach Installation gestartet
kann eine Weile dauern
microk8s status

```

### Step 3: Client - Maschine (wir sollten nicht auf control-plane oder cluster - node arbeiten)

```

Weiteren Server hochgezogen.
Vanilla + BASIS

Installation Ubuntu - Server

cloud-init script
s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)

Server 1 - manuell
Ubuntu 20.04 LTS - Grundinstallation

minimal Netzwerk - öffentlichen IP
nichts besonderes eingerichtet - Standard Digitalocean

Standard vo Installation microk8s
lo UNKNOWN 127.0.0.1/8 ::1/128
public ip / interne
eth0 UP 164.92.255.232/20 10.19.0.6/16 fe80::c:66ff:fec4:cbce/64
private ip
eth1 UP 10.135.0.5/16 fe80::8081:aaff:feaa:780/64

Installation von kubect1 aus dem snap
NICHT .. keine microk8s - keine control-plane / worker-node
NUR Client zum Arbeiten
snap install kubect1 --classic

.kube/config
Damit ein Zugriff auf die kube-server-api möglich
d.h. REST-API Interface, um das Cluster verwalten.
Hier haben uns für den ersten Control-Node entschieden
Alternativ wäre round-robin per dns möglich

Mini-Schritt 1:
Auf dem Server 1: kubeconfig ausspielen
microk8s config > /root/kube-config
auf das Zielsystem gebracht (client 1)

```

```

scp /root/kubeconfig 11trainingdo@10.135.0.5:/home/11trainingdo

Mini-Schritt 2:
Auf dem Client 1 (diese Maschine) kubeconfig an die richtige Stelle bringen
Standardmäßig der Client nach eine Konfigurationsdatei sucht in ~/.kube/config
sudo su -
cd
mkdir .kube
cd .kube
mv /home/11trainingdo/kube-config config

Verbindungstest gemacht
Damit feststellen ob das funktioniert.
kubectl cluster-info

```

#### Schritt 4: Auf allen Servern IP's hinterlegen und richtigen Hostnamen überprüfen

```

Auf jedem Server
hostnamectl
evtl. hostname setzen
z.B. - auf jedem Server eindeutig
hostnamectl set-hostname n1.training.local

Gleiche hosts auf allen server einrichten.
Wichtig, um Traffic zu minimieren verwenden, die interne (private) IP

/etc/hosts
10.135.0.3 n1.training.local n1
10.135.0.4 n2.training.local n2
10.135.0.5 n3.training.local n3

```

#### Schritt 5: Cluster aufbauen

```

Mini-Schritt 1:
Server 1: connection - string (token)
microk8s add-node
Zeigt Liste und wir nehmen den Eintrag mit der lokalen / öffentlichen ip
Dieser Token kann nur 1x verwendet werden und wir auf dem ANDEREN node ausgeführt
microk8s join 10.135.0.3:25000/e9cdaa1b5d6d24461c8643cdf107837/bcad1949221a

Mini-Schritt 2:
Dauert eine Weile, bis das durch ist.
Server 2: Den Node hinzufügen durch den JOIN - Befehl
microk8s join 10.135.0.3:25000/e9cdaa1b5d6d24461c8643cdf107837/bcad1949221a

Mini-Schritt 3:
Server 1: token besorgen für node 3
microk8s add-node

Mini-Schritt 4:
Server 3: Den Node hinzufügen durch den JOIN-Befehl
microk8s join 10.135.0.3:25000/09c96e57ec12af45b2752fb45450530c/bcad1949221a

Mini-Schritt 5: Überprüfen ob HA-Cluster läuft
Server 1: (es kann auf jedem der 3 Server überprüft werden, auf einem reicht
microk8s status | grep high-availability
high-availability: yes

```

#### Ergänzend nicht notwendige Skripte

```

cloud-init script
s.u. BASIS (keine Voraussetzung - nur zum Einrichten des Nutzers 11trainingdo per ssh)

Digitalocean - unter user_data reingepastet beim Einrichten

##cloud-config
users:
 - name: 11trainingdo
 shell: /bin/bash

runcmd:
 - sed -i "s/PasswordAuthentication no/PasswordAuthentication yes/g" /etc/ssh/sshd_config
 - echo " " >> /etc/ssh/sshd_config
 - echo "AllowUsers 11trainingdo" >> /etc/ssh/sshd_config
 - echo "AllowUsers root" >> /etc/ssh/sshd_config
 - systemctl reload sshd

```

```
- sed -i '/11trainingdo/c
11trainingdo:6HeLUJW3a$4xSfDFQjKWfAoGkZF3LFAxM4hgl3d6ATbr2kEu9zMOfwLxkYMO.AJF526mZONwdmsm9sg0tCBK1.SYbhS52u70:17476:0:99999:7:::
/etc/shadow
- echo "11trainingdo ALL=(ALL) ALL" > /etc/sudoers.d/11trainingdo
- chmod 0440 /etc/sudoers.d/11trainingdo
```

## Kubernetes - microk8s (Installation und Management)

### Installation Ubuntu - snap

#### Walkthrough

```
sudo snap install microk8s --classic

Important enable dns // otherwise not dns lookup is possible
microk8s enable dns
microk8s status

Execute kubectl commands like so
microk8s kubectl
microk8s kubectl cluster-info

Make it easier with an alias
echo "alias kubectl='microk8s kubectl'" >> ~/.bashrc
source ~/.bashrc
kubectl
```

#### Working with snaps

```
snap info microk8s
```

#### Ref:

- <https://microk8s.io/docs/setting-snap-channel>

### Remote-Verbindung zu Kubernetes (microk8s) einrichten

```
on CLIENT install kubectl
sudo snap install kubectl --classic

On MASTER -server get config
als root
cd
microk8s config > /home/kurs/remote_config

Download (scp config file) and store in .kube - folder
cd ~
mkdir .kube
cd .kube # Wichtig: config muss nachher im verzeichnis .kube liegen
scp kurs@master_server:/path/to/remote_config config
z.B.
scp kurs@192.168.56.102:/home/kurs/remote_config config
oder benutzer 11trainingdo
scp 11trainingdo@192.168.56.102:/home/11trainingdo/remote_config config

Evtl. IP-Adresse in config zum Server aendern

Ultimative 1. Test auf CLIENT
kubectl cluster-info

or if using kubectl or alias
kubectl get pods

if you want to use a different kube config file, you can do like so
kubectl --kubeconfig /home/myuser/.kube/myconfig
```

### Create a cluster with microk8s

#### Walkthrough

```
auf master (jeweils für jedes node neu ausführen)
microk8s add-node

dann auf jeweiligem node vorigen Befehl der ausgegeben wurde ausführen
Kann mehr als 60 sekunden dauern ! Geduld...Geduld..Geduld
##z.B. -> ACHTUNG evtl. IP ändern
microk8s join 10.128.63.86:25000/567a21bdfc9a64738ef4b3286b2b8a69
```

## Auf einem Node addon aktivieren z.B. ingress

```
gucken, ob es auf dem anderen node auch aktiv ist.
```

## Add Node only as Worker-Node

```
microk8s join 10.135.0.15:25000/5857843e774c2ebe368e14e8b95bdf80/9bf3ceb70a58 --worker
Contacting cluster at 10.135.0.15

root@n41:~# microk8s status
This MicroK8s deployment is acting as a node in a cluster.
Please use the master node.
```

## Ref:

- <https://microk8s.io/docs/high-availability>

## Ingress controller in microk8s aktivieren

### Aktivieren

```
microk8s enable ingress
```

## Referenz

- <https://microk8s.io/docs/addon-ingress>

## Arbeiten mit der Registry

### Installation

```
node 1 - aktivieren
microk8s enable registry
```

## Creating an image mit docker

```
node 1 / nicht client
snap install docker

mkdir myubuntu
cd myubuntu
vi Dockerfile
FROM ubuntu:latest
RUN apt-get update; apt-get install -y inetutils-ping
CMD ["/bin/bash"]

docker build -t localhost:32000/myubuntu .
docker images
docker push localhost:32000/myubuntu
```

## Installation Kubernetes Dashboard

### Reference:

- <https://blog.tippybits.com/installing-kubernetes-in-virtualbox-3d49f666b4d6>

## Kubernetes Praxis API-Objekte

### Das Tool kubectl (Devs/Ops) - Spickzettel

#### Hilfe

```
Hilfe zu befehl
kubectl help config

Hilfe nächste Ebene
kubectl config set-context --help
```

#### Allgemein

```
Zeige Informationen über das Cluster
kubectl cluster-info

Welche Ressourcen / Objekte gibt es, z.B. Pod
kubectl api-resources
kubectl api-resources | grep namespaces
```

```
Hilfe zu object und eigenschaften bekommen
kubectl explain pod
kubectl explain pod.metadata
kubectl explain pod.metadata.name
```

## namespaces

```
kubectl get ns
kubectl get namespaces

namespace wechseln, z.B. nach Ingress
kubectl config set-context --current --namespace=ingress
jetzt werden alle Objekte im Namespace Ingress angezeigt
kubectl get all,configmaps

wieder zurückwechseln.
der standardmäßige Namespace ist 'default'
kubectl config set-context --current --namespace=default
```

## Arbeiten mit manifesten

```
kubectl apply -f nginx-replicaset.yml
Wie ist aktuell die hinterlegte config im system
kubectl get -o yaml -f nginx-replicaset.yml

Änderung in nginx-replicaset.yml z.B. replicas: 4
dry-run - was wird geändert
kubectl diff -f nginx-replicaset.yml

anwenden
kubectl apply -f nginx-replicaset.yml

Alle Objekte aus manifest löschen
kubectl delete -f nginx-replicaset.yml

Recursive Löschen
cd ~/manifests
multiple subfolders subfolders present
kubectl delete -f . -R
```

## Ausgabeformate / Spezielle Informationen

```
Ausgabe kann in verschiedenen Formaten erfolgen
kubectl get pods -o wide # weitere Informationen
im json format
kubectl get pods -o json
eine Werte rausziehen
key ist tls.crt (wichtig escapen => \.), sonst funktioniert das nicht
kubectl get secrets example-tls -o jsonpath='{.data.tls.crt}' | base64 -d

gilt natürluch auch für andere kommandos
kubectl get deploy -o json
kubectl get deploy -o yaml

Label anzeigen
kubectl get deploy --show-labels
```

## Zu den Pods

```
Start einen pod // BESSER: direkt manifest verwenden
kubectl run podname image=imagename
kubectl run nginx image=nginx

Pods anzeigen
kubectl get pods
kubectl get pod

Pods in allen namespaces anzeigen
kubectl get pods -A

Format weitere Information
kubectl get pod -o wide
Zeige labels der Pods
kubectl get pods --show-labels
```

```
Zeige pods mit einem bestimmten label
kubectl get pods -l app=nginx

Status eines Pods anzeigen
kubectl describe pod nginx

Pod löschen
kubectl delete pod nginx
Löscht alle Pods im eigenen Namespace bzw. Default
kubectl delete pods --all

Kommando in pod ausführen
kubectl exec -it nginx -- bash
```

## Deployments

```
kubectl -n ingress rollout restart deployment traefik
ik
```

## Alle Objekte anzeigen

```
Nur die wichtigsten Objekte werden mit all angezeigt
kubectl get all
Dies, kann ich wie folgt um weitere ergänzen
kubectl get all,configmaps

Über alle Namespaces hinweg
kubectl get all -A
```

## Logs

```
kubectl logs <container>
kubectl logs <deployment>
e.g.
kubectl logs -n namespace8 deploy/nginx
with timestamp
kubectl logs --timestamps -n namespace8 deploy/nginx
continuously show output
kubectl logs -f <pod>
letzten x Zeilen anschauen aus log anschauen
kubectl logs --tail=5 <your pod>
```

## CRD

```
kubectl get crd
```

## Referenz

- <https://kubernetes.io/de/docs/reference/kubectl/cheatsheet/>

## kubectl example with run

### Example (that does work)

```
Synopsis (most simplistic example
kubectl run NAME --image=IMAGE_EG_FROM_DOCKER
example
kubectl run nginx --image=nginx:1.23

kubectl get pods
on which node does it run ?
kubectl get pods -o wide
```

### Example (that does not work)

```
kubectl run testpod --image=foo2
ErrImagePull - Image konnte nicht geladen werden
kubectl get pods
Weitere status - info
kubectl describe pods testpod
```

## Ref:

- <https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#run>

## kubectl/manifest/pod

### Walkthrough

```
cd
mkdir -p manifests
cd manifests/
mkdir -p 01-web
cd 01-web
nano nginx-static.yml
```

```
vi nginx-static.yml
```

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx-static-web
 labels:
 webserver: nginx
spec:
 containers:
 - name: web
 image: nginx:1.23
```

```
kubectl apply -f nginx-static.yml
```

```
kubectl get pod/nginx-static-web -o wide
kubectl describe pod nginx-static-web
show config
kubectl get pod/nginx-static-web -o yaml
seitenweise anzeigen
kubectl get pod/nginx-static-web -o yaml | less
```

### Aufräumen

```
kubectl delete -f .
```

## kubectl/manifest/replicaset

### Walkthrough Erstellen

```
cd
mkdir -p manifests
cd manifests
mkdir 02-rs
cd 02-rs
nano rs.yml
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: nginx-replica-set
spec:
 replicas: 5
 selector:
 matchLabels:
 tier: frontend
 template:
 metadata:
 name: template-nginx-replica-set
 labels:
 tier: frontend
 spec:
 containers:
 - name: nginx
 image: nginx:1.23
 ports:
 - containerPort: 80
```

```
kubectl apply -f .
kubectl get all
kubectl get pods -l tier=frontend
kubectl get pods --show-labels
```



```
name anpassen
kubectl describe pod/nginx-replica-set-lpkbs
```

### Pod löschen, was passiert

```
kubectl delete po nginx-r<TAB>
einfach einen pod raussuchen und löschen
z.B.
kubectl delete po nginx-replica-set-xg8jp
```

```
gucken, welches sind die neuesten ?
kubectl get pods
```

### Walthrough Skalieren

```
nano rs.yml
```

```
Ändern
replicas: 5
-> ändern in
replicas: 8
```

```
kubectl apply -f .
kubectl get pods
```

### Aufräumen des replicaset

```
kubectl delete -f .
```

### kubectl/manifest/deployments

#### Prepare

```
cd
mkdir -p manifests
cd manifests
mkdir 03-deploy
cd 03-deploy
nano nginx-deployment.yml
```

```
vi nginx-deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
spec:
 selector:
 matchLabels:
 app: nginx
 replicas: 8 # tells deployment to run 8 pods matching the template
 template:
 metadata:
 labels:
 app: nginx
 spec:
 containers:
 - name: nginx
 image: nginxinc/nginx-unprivileged:1.28
 ports:
 - containerPort: 8080
```

```
kubectl apply -f .
```

#### Explore

```
kubectl get all
```

#### Optional: Change image - Version

```
nano nginx-deployment.yml
```

#### Version 1: (optical nicer)

```
Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
danach
kubectl apply -f . && watch kubectl get pods
```

#### Version 2:

```
Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
danach
kubectl apply -f . && kubectl get all && kubectl get pods -w
```

#### kubectl/manifest/service

##### Warum Services ?

- Wenn in einem Deployment bei einem Wechsel des images neue Pods erstellt werden, erhalten diese eine neue IP-Adresse
- Nachteil: Man müsste diese dann in allen Applikation ständig ändern, die auf die Pods zugreifen.
- Lösung: Wir schalten einen Service davor !

##### Hintergrund IP-Wechsel

```
tinl@client:~/manifests/04-services$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
web-nginx-69cc9f8d49-9r6wb	1/1	Running	0	5m37s	10.108.0.200	node
-p8s6u	<none>	<none>				
web-nginx-69cc9f8d49-z6fxf	1/1	Running	0	5m37s	10.108.0.25	node
-n8s6c	<none>	<none>				

- Image-Version wurde jetzt in Deployment geändert, Ergebnis:

```
tinl@client:~/manifests/04-services$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
web-nginx-754794c6d6-8pj99	1/1	Running	0	8s	10.108.1.42	node-p
8s68	<none>	<none>				
web-nginx-754794c6d6-swckk	1/1	Running	0	10s	10.108.0.121	node-p
8s6c	<none>	<none>				

##### Example I : Service with ClusterIP

###### Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 04-service
cd 04-service
```

###### Schritt 2: Deployment erstellen

```
nano deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: web-nginx
spec:
 selector:
 matchLabels:
 web: my-nginx
 replicas: 2
 template:
 metadata:
 labels:
 web: my-nginx
 spec:
 containers:
 - name: cont-nginx
 image: nginx:1.27
 ports:
 - containerPort: 80
```

```
nano service.yml
```

```

apiVersion: v1
kind: Service
metadata:
 name: svc-nginx
spec:
 type: ClusterIP
 ports:
 - port: 80
 protocol: TCP
 selector:
 web: my-nginx

```

```

kubectl apply -f .
wie ist die ClusterIP ?
kubectl get all
kubectl get svc svc-nginx
Find endpoints / did svc find pods ?
kubectl describe svc svc-nginx

```

### Schritt 3: Deployment löschen

```

kubectl delete -f deploy.yml
Keine endpunkte mehr
kubectl describe svc svc-nginx

```

### Schritt 4: Deployment wieder erstellen

```

kubectl apply -f .
Endpunkte wieder da
kubectl describe svc svc-nginx

```

### Example II : Short version (NodePort)

```

Wo sind wir ?
cd; cd manifests/04-service

```

```

nano service.yml
in Zeile type:
ClusterIP ersetzt durch NodePort

kubectl apply -f .
NodePort ab 30.000 ausfindig machen
kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
<u>svc-nginx</u>	NodePort	10.109.24.227	<none>	80:32708/TCP	18h

```

kubectl get nodes -o wide

```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
node-p8s68	Ready	<none>	25h	v1.33.1	10.135.0.72	139.59.128.36	Debian GNU/Linux 12 (bookworm)
node-p8s6c	Ready	<none>	25h	v1.33.1	10.135.0.73	64.226.125.3	Debian GNU/Linux 12 (bookworm)
node-p8s6u	Ready	<none>	25h	v1.33.1	10.135.0.74	159.223.24.231	Debian GNU/Linux 12 (bookworm)

```

im client Externe NodeIP und NodePort verwenden
curl http://164.92.193.245:32708

```

### Example II : Service with NodePort (long version)

```

nano deploy.yml

```

```

you will get port opened on every node in the range 30000+
apiVersion: apps/v1
kind: Deployment
metadata:

```

```

name: web-nginx
spec:
 selector:
 matchLabels:
 web: my-nginx
 replicas: 2
 template:
 metadata:
 labels:
 web: my-nginx
 spec:
 containers:
 - name: cont-nginx
 image: nginx
 ports:
 - containerPort: 80

```

```
nano service.yml
```

```

apiVersion: v1
kind: Service
metadata:
 name: svc-nginx
 labels:
 run: svc-my-nginx
spec:
 type: NodePort
 ports:
 - port: 80
 protocol: TCP
 selector:
 web: my-nginx

```

```
kubectl apply -f .
```

### Example III: Service mit LoadBalancer (ExternalIP)

```

cd; cd manifests/04-service
nano service.yml
in Zeile type:
NodePort ersetzt durch LoadBalancer

kubectl apply -f .
kubectl get svc svc-nginx

kubectl describe svc svc-nginx
hier heisst das nicht External-IP ->
sondern

```

```

LoadBalancer Ingress: 129.212.254.6 (VIP)
Port: <unset> 80/TCP

```

```

kubectl get svc svc-nginx -w
spätestens nach 5 Minuten bekommen wir eine externe ip
z.B. 41.32.44.45

curl http://41.32.44.45

```

### Example getting a specific ip from loadbalancer (if supported)

```

apiVersion: v1
kind: Service
metadata:
 name: svc-nginx2
spec:
 type: LoadBalancer
 # this line to get a specific ip if supported
 loadBalancerIP: 10.34.12.34
 ports:
 - port: 80
 protocol: TCP

```

```
selector:
 web: my-nginx
```

## Ref.

- <https://kubernetes.io/docs/concepts/services-networking/connect-applications-service/>

## Hintergrund Ingress

## Ref. / Dokumentation

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Documentation for default ingress nginx

- <https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/configmap/>

## Beispiel Ingress

## Prerequisites

```
Ingress Controller muss aktiviert sein
microk8s enable ingress
```

## Walkthrough

```
mkdir apple-banana-ingress

apple.yml
vi apple.yml
kind: Pod
apiVersion: v1
metadata:
 name: apple-app
 labels:
 app: apple
spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple"

kind: Service
apiVersion: v1
metadata:
 name: apple-service
spec:
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f apple.yml
```

```
banana
vi banana.yml
kind: Pod
apiVersion: v1
metadata:
 name: banana-app
 labels:
 app: banana
spec:
 containers:
 - name: banana-app
 image: hashicorp/http-echo
 args:
 - "-text=banana"

kind: Service
apiVersion: v1
metadata:
 name: banana-service
```

```
spec:
 selector:
 app: banana
 ports:
 - port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f banana.yml
```

```
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80
```

```
ingress
kubectl apply -f ingress.yml
kubectl get ing
```

## Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Find the problem

```
Hints

1. Which resources does our version of kubectl support
Can we find Ingress as "Kind" here.
kubectl api-resources

2. Let's see, how the configuration works
kubectl explain --api-version=networking.k8s.io/v1 ingress.spec.rules.http.paths.backend.service

now we can adjust our config
```

## Solution

```
in kubernetes 1.22.2 - ingress.yml needs to be modified like so.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - http:
 paths:
 - path: /apple
 pathType: Prefix
 backend:
 service:
 name: apple-service
 port:
 number: 80
 - path: /banana
 pathType: Prefix
 backend:
 service:
 name: banana-service
```

```
port:
 number: 80
```

## Beispiel mit Hostnamen

### Step 1: Walkthrough

```
cd
cd manifests
mkdir abi
cd abi
nano apple.yml
```

```
apple.yml
vi apple.yml
kind: Pod
apiVersion: v1
metadata:
 name: apple-app
 labels:
 app: apple
spec:
 containers:
 - name: apple-app
 image: hashicorp/http-echo
 args:
 - "-text=apple-<ever-name>"

kind: Service
apiVersion: v1
metadata:
 name: apple-service
spec:
 type: ClusterIP
 selector:
 app: apple
 ports:
 - protocol: TCP
 port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f apple.yml
```

```
nano banana.yml
```

```
banana
vi banana.yml
kind: Pod
apiVersion: v1
metadata:
 name: banana-app
 labels:
 app: banana
spec:
 containers:
 - name: banana-app
 image: hashicorp/http-echo
 args:
 - "-text=banana-<ever-name>"

kind: Service
apiVersion: v1
metadata:
 name: banana-service
spec:
 type: ClusterIP
 selector:
 app: banana
 ports:
 - port: 80
 targetPort: 5678 # Default port for image
```

```
kubectl apply -f banana.yml
```

## Step 2: Testing connection by podIP and Service

```
kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox
```

```
/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>
```

## Step 3: Walkthrough

```
nano ingress.yml
```

```
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - host: "<euername>.lab1.t3isp.de"
 http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80
```

```
ingress
kubectl apply -f ingress.yml
```

## Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

## Find the problem

### Schritt 1: api-version ändern

```
Welche Landkarten gibt es ?
kubectl api-versions
Auf welcher Landkarte ist Ingress jetzt
kubectl explain ingress
```

```
GROUP: networking.k8s.io
KIND: Ingress
VERSION: v1
```

```
ingress ändern ingress.yml
von
apiVersion: extensions/v1beta1
in
apiVersion: networking.k8s.io/v1
```

### Schritt 2: Fehler Eigenschaften beheben

```
Error from server (BadRequest): error when creating "ingress.yml": Ingress in version "v1" cannot be handled as a Ingress: strict decoding error: unknown field "spec.rules[0].http.paths[0].backend.serviceName", unknown field "spec.rules[0].http.paths[0].backend.servicePort", unknown field "spec.rules[0].http.paths[1].backend.serviceName", unknown field "spec.rules[0].http.paths[1].backend.servicePort"
```

```
Problem serviceName beheben
Was gibt es stattdessen
-> es gibt service aber keine serviceName
```



```
kubectl explain ingress.spec.rules.http.paths.backend
-> es gibt service.name
kubectl explain ingress.spec.rules.http.paths.backend.
```

```
Korrektur 2x in ingress.yaml: Inkl. servicePort (neu: service.port.number)
vorher
backend:
serviceName: apple-service
servicePort: 80
jetzt:
service:
 name: apple-service
 port:
 number: 80
```

```
kubectl apply -f .
```

### Schritt 3: pathType ergänzen

```
* spec.rules[0].http.paths[0].pathType: Required value: pathType must be specified
* spec.rules[0].http.paths[1].pathType: Required value: pathType must be specified
```

- Es wird festgelegt wie der Pfad ausgewertet

```
Wir müssen pathType auf der 1. Unterebene von paths einfügen
Entweder exact oder prefix
```

```
http:
 paths:
 - path: /apple
 pathType: Exact
 backend:
 service:
```

```
kubectl apply -f .
```

### Schritt 4: Testen aufrufen

```
curl http://<euername>.lab1.t3isp.de/apple
curl http://<euername>.lab1.t3isp.de/apple/ # Sollte nicht funktioniert
curl http://<euername>.lab1.t3isp.de/banana
curl http://<euername>.lab1.t3isp.de/banana/
curl http://<euername>.lab1.t3isp.de/banana/something
```

Das kann man auch so im Browser eingeben

### Solution

```
nano ingress.yml
```

```
in kubernetes 1.22.2 - ingress.yml needs to be modified like so.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
spec:
 ingressClassName: nginx
 rules:
 - host: "<euername>.lab1.t3isp.de"
 http:
 paths:
 - path: /apple
 pathType: Prefix
 backend:
 service:
 name: apple-service
 port:
 number: 80
 - path: /banana
 pathType: Prefix
```

```
 backend:
 service:
 name: banana-service
 port:
 number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
mit describe herausfinden, ob er die services gefunden
kubectl describe ingress example-ingress
```

### Achtung: Ingress mit Helm - annotations

#### Welcher wird verwendet, angeben:

Damit das Ingress Objekt welcher Controller verwendet werden soll, muss dieser angegeben werden:

```
kubernetes.io/ingress.class: nginx
```

```
Als ganzes Object:
Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
 name: example-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
 kubernetes.io/ingress.class: nginx
spec:
 rules:
 - http:
 paths:
 - path: /apple
 backend:
 serviceName: apple-service
 servicePort: 80
 - path: /banana
 backend:
 serviceName: banana-service
 servicePort: 80
```

#### Ref:

- <https://www.digitalocean.com/community/tutorials/how-to-set-up-an-nginx-ingress-on-digitalocean-kubernetes-using-helm>

### Permanente Weiterleitung mit Ingress

#### Example

```
redirect.yml
apiVersion: v1
kind: Namespace
metadata:
 name: my-namespace

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 annotations:
 nginx.ingress.kubernetes.io/permanent-redirect: https://www.google.de
 nginx.ingress.kubernetes.io/permanent-redirect-code: "308"
 name: destination-home
 namespace: my-namespace
spec:
 rules:
 - http:
 paths:
 - backend:
 service:
 name: http-svc
 port:
 number: 80
 path: /source
 pathType: ImplementationSpecific
```

```
eine node mit ip-adresse aufrufen
curl -I http://41.12.45.21/source
HTTP/1.1 308
Permanent Redirect
```

## Umbauen zu google ;o)

This annotation allows to return a permanent redirect instead of sending data to the upstream. For example `nginx.ingress.kubernetes.io/permanent-redirect: https://www.google.com` would redirect everything to Google.

### Refs:

- <https://github.com/kubernetes/ingress-nginx/blob/main/docs/user-guide/nginx-configuration/annotations.md#permanent-redirect>
- 

## ConfigMap Example

### Schritt 1: configmap vorbereiten

```
cd
mkdir -p manifests
cd manifests
mkdir configmaptests
cd configmaptests
nano 01-configmap.yml
```

```
01-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
 name: example-configmap
data:
 # als Wertepaare
 database: mongodb
 database_uri: mongodb://localhost:27017
 testdata: |
 run=true
 file=/hello/you
```

```
kubectl apply -f 01-configmap.yml
kubectl get cm
kubectl get cm example-configmap -o yaml
```

### Schritt 2: Beispiel als Datei

```
nano 02-pod.yml
```

```
kind: Pod
apiVersion: v1
metadata:
 name: pod-mit-configmap

spec:
 # Add the ConfigMap as a volume to the Pod
 volumes:
 # `name` here must match the name
 # specified in the volume mount
 - name: example-configmap-volume
 # Populate the volume with config map data
 configMap:
 # `name` here must match the name
 # specified in the ConfigMap's YAML
 name: example-configmap

 containers:
 - name: container-configmap
 image: nginx:latest
 # Mount the volume that contains the configuration data
 # into your container filesystem
 volumeMounts:
 # `name` here must match the name
 # from the volumes section of this pod
 - name: example-configmap-volume
 mountPath: /etc/config
```

```
kubectl apply -f 02-pod.yml
```

```
##Jetzt schauen wir uns den Container/Pod mal an
kubectl exec pod-mit-configmap -- ls -la /etc/config
kubectl exec -it pod-mit-configmap -- bash
ls -la /etc/config
```

### Schritt 3: Beispiel. ConfigMap als env-variablen

```
nano 03-pod-mit-env.yml
```

```
03-pod-mit-env.yml
kind: Pod
apiVersion: v1
metadata:
 name: pod-env-var
spec:
 containers:
 - name: env-var-configmap
 image: nginx:latest
 envFrom:
 - configMapRef:
 name: example-configmap
```

```
kubectl apply -f 03-pod-mit-env.yml
```

```
und wir schauen uns das an
##Jetzt schauen wir uns den Container/Pod mal an
kubectl exec pod-env-var -- env
kubectl exec -it pod-env-var -- bash
env
```

#### Reference:

- <https://matthewpalmer.net/kubernetes-app-developer/articles/ultimate-configmap-guide-kubernetes.html>

## Kubernetes - ENV - Variablen für den Container setzen

### ENV - Variablen - Übung

#### Übung 1 - einfach ENV-Variablen direkt setzen

```
mkdir envtests
cd envtest
vi 01-simple.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs
spec:
 containers:
 - name: env-print-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_FEATURES
 value: "backend,stats,reports"
```

```
kubectl apply -f 01-simple.yml
kubectl get pods
kubectl exec -it print-envs -- bash
env | grep APP
```

#### Übung 2 - ENV-Variablen von Feldern setzen (aus System)

```
erstmal falsch
und noch ein 2. versteckter Fehler
vi 02-feldref.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-fields
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
```

```

- name: APP_VERSION
 value: 1.21.1
- name: APP_FEATURES
 value: "backend,stats,reports"
- name: APP_POD_IP
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
- name: APP_POD_STATUS
 valueFrom:
 fieldRef:
 fieldPath: status.phase

```

```

kubectl apply -f 02-feldref.yml
Fehler, weil es das Objekt schon gibt und es so nicht geupdatet werden kann
Einfach zum Löschen verwenden
kubectl delete -f 02-feldref.yml
Nochmal anlegen.
Wieder fehler s.u.
kubectl apply -f 02-feldres.yml

```

```

Fehler
* spec.containers[0].env[3].valueFrom.fieldRef.fieldPath: Unsupported value: "status.phase": supported values: "metadata.name",
"metadata.namespace", "metadata.uid", "spec.nodeName", "spec.serviceAccountName", "status.hostIP", "status.podIP", "status.podIPs"

```

```

letztes Feld korrigiert
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-fields
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_FEATURES
 value: "backend,stats,reports"
 - name: APP_POD_IP
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: APP_POD_NODE
 valueFrom:
 fieldRef:
 fieldPath: spec.nodeName

```

```

kubectl apply -f 02-feldref.yml
kubectl exec -it print-envs -- bash
env | grep APP

```

#### Beispiel mit labels, die ich gesetzt habe:

```

vi 02-feldref.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-fields
 labels:
 app: foo
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_FEATURES
 value: "backend,stats,reports"
 - name: APP_POD_IP
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: LABEL_APP
 valueFrom:

```

```
fieldRef:
 fieldPath: metadata.labels['app']
```

### Übung 3 - ENV Variablen aus configMaps setzen.

```
Step 1: ConfigMap
03-matchmaker-config.yml
apiVersion: v1
kind: ConfigMap
metadata:
 name: app-config
 labels:
 app: matchmaker
data:
 MYSQL_DB: matchmaker
 MYSQL_USER: user_matchmaker
 MYSQL_DATA_DIR: /var/lib/mysql
```

```
Step 2: applying map
kubectl apply -f 03-matchmaker-config.yml
Das ist der Trostpreis !!
kubectl get configmap app-config
kubectl get configmap app-config -o yaml
```

```
Step 3: setup another pod to use it in addition
vi 04-matchmaker-app.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-multi
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_FEATURES
 value: "backend,stats,reports"
 - name: APP_POD_IP
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: APP_POD_NODE
 valueFrom:
 fieldRef:
 fieldPath: spec.nodeName
 envFrom:
 - configMapRef:
 name: app-config
```

```
kubectl apply -f 04-matchmaker-app.yml
kubectl exec -it print-envs-multi -- bash
env | grep -e MYSQL -e APP_
```

### Übung 4 - ENV Variablen aus Secrets setzen

```
Schritt 1: Secret anlegen.
Diesmal noch nicht encoded - base64
vi 06-secret-unencoded.yml
apiVersion: v1
kind: Secret
metadata:
 name: mysecret
type: Opaque
stringData:
 APP_PASSWORD: "s3c3tp@ss"
 APP_EMAIL: "mail@domain.com"
```

```
Schritt 2: Apply'en und anschauen
kubectl apply -f 06-secret-unencoded.yml
ist zwar encoded, aber last_applied ist im Klartext
das könnte ich nur umgehen, in dem ich es encoded speichere
kubectl get secret mysecret -o yaml
```

```

Schritt 3:
vi 07-print-envs-complete.yml
apiVersion: v1
kind: Pod
metadata:
 name: print-envs-complete
spec:
 containers:
 - name: env-ref-demo
 image: nginx
 env:
 - name: APP_VERSION
 value: 1.21.1
 - name: APP_FEATURES
 value: "backend,stats,reports"
 - name: APP_POD_IP
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: APP_POD_NODE
 valueFrom:
 fieldRef:
 fieldPath: spec.nodeName
 - name: APP_PASSWORD
 valueFrom:
 secretKeyRef:
 name: mysecret
 key: APP_PASSWORD
 - name: APP_EMAIL
 valueFrom:
 secretKeyRef:
 name: mysecret
 key: APP_EMAIL

 envFrom:
 - configMapRef:
 name: app-config

```

```

Schritt 4:
kubectl apply -f 07-print-envs-complete.yml
kubectl exec -it print-envs-complete -- bash
##env | grep -e APP_ -e MYSQL

```

## Kubernetes - Arbeiten mit einer lokalen Registry (microk8s)

### microk8s lokale Registry

#### Installation

```

node 1 - aktivieren
microk8s enable registry

```

#### Creating an image mit docker

```

node 1 / nicht client
snap install docker

mkdir myubuntu
cd myubuntu
vi Dockerfile
FROM ubuntu:latest
RUN apt-get update; apt-get install -y inetutils-ping
CMD ["/bin/bash"]

docker build -t localhost:32000/myubuntu .
docker images
docker push localhost:32000/myubuntu

```

## Kubernetes Praxis Scaling/Rolling Updates/Wartung

### Wartung mit drain / uncordon (Ops)

```

Achtung, bitte keine pods verwenden, dies können "ge"-drained (ausgetrocknet) werden
kubectl drain <node-name>
z.B.
Daemonsets ignorieren, da diese nicht gelöscht werden
kubectl drain n17 --ignore-daemonsets

Alle pods von replicaseten werden jetzt auf andere nodes verschoben
Ich kann jetzt wartungsarbeiten durchführen

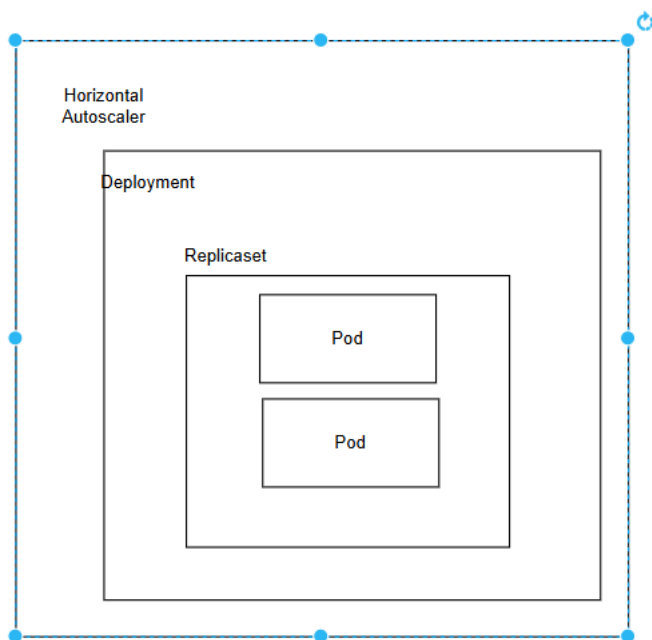
Wenn fertig bin:
kubectl uncordon n17

Achtung: deployments werden nicht neu ausgerollt, dass muss ich anstossen.
z.B.
kubectl rollout restart deploy/webserver

```

## Ausblick AutoScaling (Ops)

### Overview



### Example: newest version with autoscaling/v2 used to be hpa/v1

#### Prerequisites

- Metrics-Server needs to be running

```

Test with
kubectl top pods

```

```

Install with helm chart
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm upgrade --install metrics-server metrics-server/metrics-server --version 3.13.0 --create-namespace --namespace=metrics-server --reset-values

```

```

after that it will be available in kube-system namespace as pod
kubectl -n metrics-server get pods

```

#### Step 1: deploy app

```

cd
mkdir -p manifests
cd manifests
mkdir hpa

```



```
cd hpa
nano 01-deploy.yaml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: hello
spec:
 replicas: 3
 selector:
 matchLabels:
 app: hello
 template:
 metadata:
 labels:
 app: hello
 spec:
 containers:
 - name: hello
 image: k8s.gcr.io/hpa-example
 resources:
 requests:
 cpu: 100m

kind: Service
apiVersion: v1
metadata:
 name: hello
spec:
 selector:
 app: hello
 ports:
 - port: 80
 targetPort: 80

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: hello
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: hello
 minReplicas: 2
 maxReplicas: 20
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 80
```

```
kubectl apply -f .
```

## Step 2: Load Generator

```
nano 02-loadgenerator.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: load-generator
 labels:
 app: load-generator
spec:
 replicas: 100
 selector:
 matchLabels:
 app: load-generator
 template:
 metadata:
 name: load-generator
 labels:
 app: load-generator
```

```
spec:
 containers:
 - name: load-generator
 image: busybox
 command:
 - /bin/sh
 - -c
 - "while true; do wget -q -O- http://hello; done"
```

```
kubectl apply -f .
```

### Step 3: Zurücklehnen und genießen

```
watch kubectl get pods -l app=hello
```

```
2.Session aufmachen und ..
watch kubectl get nodes
```

### Downscaling

- Downscaling will happen after 5 minutes o

```
Adjust down to 1 minute
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: hello
spec:
 # change to 60 secs here
 behavior:
 scaleDown:
 stabilizationWindowSeconds: 60
 # end of behaviour change
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: hello
 minReplicas: 2
 maxReplicas: 20
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 80
```

For scaling down the stabilization window is 300 seconds (or the value of the --horizontal-pod-autoscaler-downscale-stabilization flag if provided)

### Reference

- <https://docs.digitalocean.com/tutorials/cluster-autoscaling-ca-hpa/>
- <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/#autoscaling-on-more-specific-metrics>
- <https://medium.com/expedia-group-tech/autoscaling-in-kubernetes-why-doesnt-the-horizontal-pod-autoscaler-work-for-me-5f0094694054>

## Autoscaling

### Example:

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
 name: busybox-1
spec:
 scaleTargetRef:
 kind: Deployment
 name: busybox-1
 minReplicas: 3
 maxReplicas: 4
 targetCPUUtilizationPercentage: 80
```

### Reference

- <https://medium.com/expedia-group-tech/autoscaling-in-kubernetes-why-doesnt-the-horizontal-pod-autoscaler-work-for-me-5f0094694054>

## Kubernetes Storage

### Praxis. Beispiel (Dev/Ops)

#### Create new server and install nfs-server

```
on Ubuntu 20.04LTS
apt install nfs-kernel-server
systemctl status nfs-server

vi /etc/exports
adjust ip's of kubernetes master and nodes
kmaster
/var/nfs/ 192.168.56.101(rw,sync,no_root_squash,no_subtree_check)
knode1
/var/nfs/ 192.168.56.103(rw,sync,no_root_squash,no_subtree_check)
knode 2
/var/nfs/ 192.168.56.105(rw,sync,no_root_squash,no_subtree_check)

exportfs -av
```

#### On all nodes (needed for production)

```
##
apt install nfs-common
```

#### On all nodes (only for testing) (Version 1)

```
Please do this on all servers (if you have access by ssh)
find out, if connection to nfs works !

for testing
mkdir /mnt/nfs
192.168.56.106 is our nfs-server
mount -t nfs 192.168.56.106:/var/nfs /mnt/nfs
ls -la /mnt/nfs
umount /mnt/nfs
```

#### Setup PersistentVolume and PersistentVolumeClaim in cluster

##### Schritt 1:

```
cd
cd manifests
mkdir -p nfs; cd nfs
nano 01-pv.yml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
 # any PV name
 name: pv-nfs-tln<nr>
 labels:
 volume: nfs-data-volume-tln<nr>
spec:
 capacity:
 # storage size
 storage: 1Gi
 accessModes:
 # ReadWriteMany(RW from multi nodes), ReadWriteOnce(RW from a node), ReadOnlyMany(R from multi nodes)
 - ReadWriteMany
 persistentVolumeReclaimPolicy:
 # retain even if pods terminate
 Retain
 nfs:
 # NFS server's definition
 path: /var/nfs/tln<nr>/nginx
 server: 10.135.0.7
 readOnly: false
 storageClassName: ""
```

```
kubectl apply -f 01-pv.yml
```

##### Schritt 2:

```
nano 02-pvc.yml
```

```
vi 02-pvc.yml
now we want to claim space
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: pv-nfs-claim-tln<nr>
spec:
 storageClassName: ""
 volumeName: pv-nfs-tln<nr>
 accessModes:
 - ReadWriteMany
 resources:
 requests:
 storage: 1Gi
```

```
kubectl apply -f 02-pvc.yml
```

### Schritt 3:

```
nano 03-deploy.yml
```

```
deployment including mount
vi 03-deploy.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
spec:
 selector:
 matchLabels:
 app: nginx
 replicas: 4 # tells deployment to run 4 pods matching the template
 template:
 metadata:
 labels:
 app: nginx
 spec:

 containers:
 - name: nginx
 image: nginx:latest
 ports:
 - containerPort: 80

 volumeMounts:
 - name: nfsvol
 mountPath: "/usr/share/nginx/html"

 volumes:
 - name: nfsvol
 persistentVolumeClaim:
 claimName: pv-nfs-claim-tln<nr>
```

```
kubectl apply -f 03-deploy.yml
```

```
nano 04-service.yml
```

```
now testing it with a service
cat 04-service.yml
apiVersion: v1
kind: Service
metadata:
 name: service-nginx
 labels:
 run: svc-my-nginx
spec:
 type: NodePort
 ports:
 - port: 80
 protocol: TCP
 selector:
 app: nginx
```

```
kubectl apply -f 04-service.yml
```

#### Schritt 4

```
connect to the container and add index.html - data
kubectl exec -it deploy/nginx-deployment -- bash
in container
echo "hello dear friend" > /usr/share/nginx/html/index.html
exit

get external ip
kubectl get nodes -o wide

now try to connect
kubectl get svc

connect with ip and port
kubectl run -it --rm curly --image=curlimages/curl -- /bin/sh
curl http://<cluster-ip>
exit

oder alternative von extern (Browser) auf Client
http://<ext-ip>:30154 (Node Port) - ext-ip -> kubectl get nodes -o wide

now destroy deployment
kubectl delete -f 03-deploy.yml

Try again - no connection
kubectl run -it --rm curly --image=curlimages/curl -- /bin/sh
curl http://<cluster-ip>
exit
```

#### Schritt 5

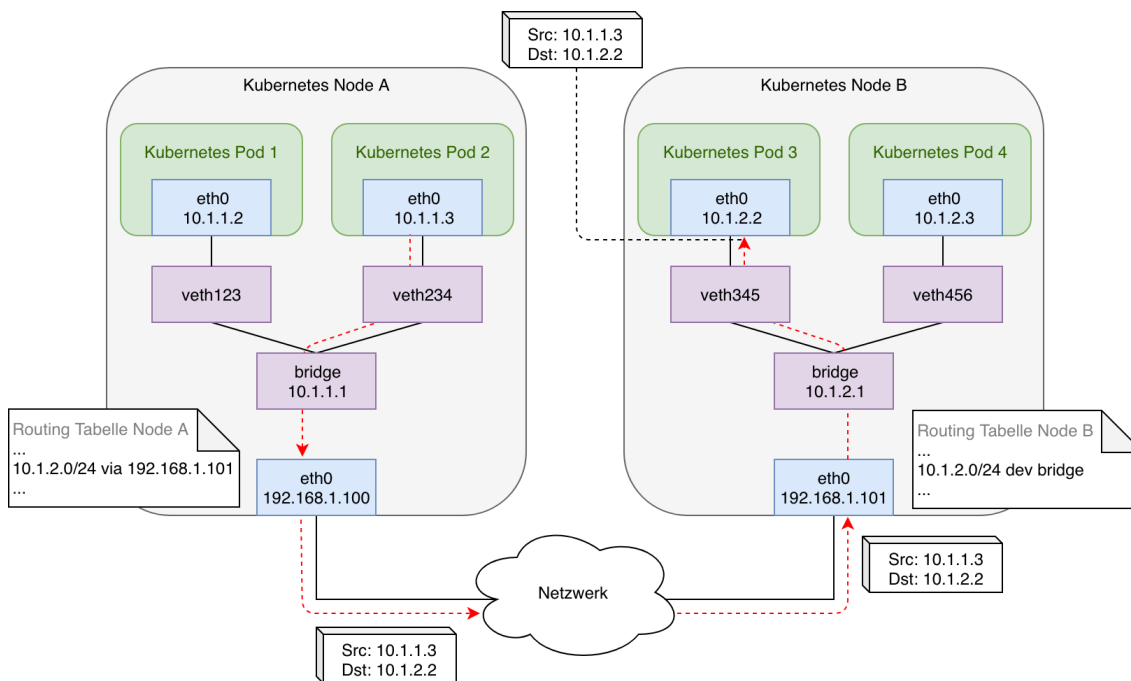
```
now start deployment again
kubectl apply -f 03-deploy.yml

and try connection again
kubectl run -it --rm curly --image=curlimages/curl -- /bin/sh
curl http://<cluster-ip>:<port> # port -> > 30000
exit
```

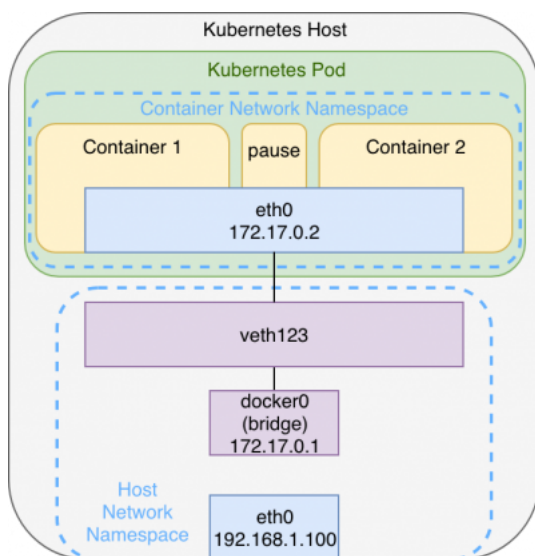
## Kubernetes Networking

### Überblick

### Show us



### Die Magie des Pause Containers



### CNI

- Common Network Interface
- Feste Definition, wie Container mit Netzwerk-Bibliotheken kommunizieren

### Docker - Container oder andere

- Container wird hochgefahren -> über CNI -> zieht Netzwerk - IP hoch.
- Container wird runtergefahren -> über CNI -> Netzwerk - IP wird released

### Welche gibt es ?

- Flannel
- Canal
- Calico
- Cilium

### Flannel

### Overlay - Netzwerk

- virtuelles Netzwerk was sich oben drüber und eigentlich auf Netzwerkebene nicht existiert
- VXLAN

#### Vorteile

- Guter einfacher Einstieg
- reduziert auf eine Binary flanneld

#### Nachteile

- keine Firewall - Policies möglich
- keine klassischen Netzwerk-Tools zum Debuggen möglich.

#### Canal

##### General

- Auch ein Overlay - Netzwerk
- Unterstützt auch policies

#### Calico

##### Generell

- klassische Netzwerk (BGP)

##### Vorteile gegenüber Flannel

- Policy über Kubernetes Object (NetworkPolicies)

##### Vorteile

- ISTIO integrierbar (Mesh - Netz)
- Performance etwas besser als Flannel (weil keine Encapsulation)

##### Referenz

- <https://projectcalico.docs.tigera.io/security/calico-network-policy>

#### Cilium

##### Generell

##### microk8s Vergleich

- <https://microk8s.io/compare>

```

snap.microk8s.daemon-flanneld
Flannel is a CNI which gives a subnet to each host for use with container runtimes.

Flanneld runs if ha-cluster is not enabled. If ha-cluster is enabled, calico is run instead.

The flannel daemon is started using the arguments in ${SNAP_DATA}/args/flanneld. For more information on the configuration, see
the flannel documentation.
```

#### Beispiel NetworkPolicies

##### Um was geht es ?

- Wir wollen Firewall-Regeln mit Kubernetes machen (NetworkPolicy)
- Firewall in Kubernetes -> Network Policies

##### Gruppe mit eigenem cluster

```

<tln> = nix
z.B.
policy-demo<tln> => policy-demo
```

##### Gruppe mit einem einzigen Cluster

```

<tln> = Teilnehmernummer
z.B.
policy-demo<tln> => policy-demo1
```

#### Walkthrough

```

Schritt 1:
kubectl create ns policy-demo<tln>
kubectl create deployment --namespace=policy-demo<tln> nginx --image=nginx
kubectl expose --namespace=policy-demo<tln> deployment nginx --port=80
lassen einen 2. pod laufen mit dem auf den nginx zugreifen
kubectl run --namespace=policy-demo<tln> access --rm -ti --image busybox -- /bin/sh
```

```

innerhalb der shell
wget -q nginx -O -
```

## Schritt 2: Policy festlegen, dass kein Ingress Traffic erlaubt ist

```
cd
cd manifests
mkdir network
cd network
nano 01-policy.yml
```

```
Deny Regel
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
 name: default-deny
 namespace: policy-demo<tln>
spec:
 podSelector:
 matchLabels: {}
```

```
kubectl apply -f 01-policy.yml
```

```
lassen einen 2. pod laufen mit dem auf den nginx zugreifen
kubectl run --namespace=policy-demo<tln> access --rm -ti --image busybox -- /bin/sh
```

```
innerhalb der shell
kein Zugriff möglich
wget -O - nginx
```

## Schritt 3: Zugriff erlauben von pods mit dem Label run=access

```
cd
cd manifests
cd network
nano 02-allow.yml
```

```
Schritt 3:
02-allow.yml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
 name: access-nginx
 namespace: policy-demo<tln>
spec:
 podSelector:
 matchLabels:
 app: nginx
 ingress:
 - from:
 - podSelector:
 matchLabels:
 run: access
```

```
kubectl apply -f 02-allow.yml
```

```
lassen einen 2. pod laufen mit dem auf den nginx zugreifen
pod hat durch run -> access automatisch das label run:access zugewiesen
kubectl run --namespace=policy-demo<tln> access --rm -ti --image busybox -- /bin/sh
```

```
innerhalb der shell
wget -q nginx -O -
```

```
kubectl run --namespace=policy-demo<tln> no-access --rm -ti --image busybox -- /bin/sh
```

```
in der shell
wget -q nginx -O -
```

```
kubectl delete ns policy-demo<tln>
```

## Ref:

- <https://projectcalico.docs.tigera.io/security/tutorials/kubernetes-policy-basic>
- <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- <https://docs.cilium.io/en/latest/security/policy/language/#http>



## Kubernetes Paketmanagement (Helm)

### Warum ? (Dev/Ops)

```
Ein Paket für alle Komponenten
Einfaches Installieren, Updaten und deinstallieren
Konfigurations-Values-Files übergeben zum Konfigurieren
Feststehende Struktur
Versionierung (jedes Chart hat ein Version)
In meinem Kubernetes-Cluster kann ich sehen, welche Version des Charts/der Charts installiert wurde
Ein Chart für viele Kunden und für viele Umgebungen (Chart und passe das mit Konfigurationswerten an)
```

### Grundlagen / Aufbau / Verwendung (Dev/Ops)

#### Wo kann ich Helm-Charts suchen ?

- Im Telefonbuch von helm <https://artifacthub.io/>

### Komponenten

#### Chart

- beeinhaltet Beschreibung und Komponenten

#### Chart - Bereitstellungsformen

- url
- .tgz (abkürzung tar.gz) - Format
- oder Verzeichnis

```
Wenn wir ein Chart installieren, wird eine Release erstellen
(parallel: image -> container, analog: chart -> release)
```

### Installation

#### Was brauchen wir ?

- helm client muss installiert sein

#### Und sonst so ?

```
Beispiel ubuntu
snap install --classic helm

Cluster auf das ich zugreifen kann und im client -> helm und kubectl
Voraussetzung auf dem Client-Rechner (helm ist nichts als anderes als ein Client-Programm)
Ein lauffähiges kubectl auf dem lokalen System (welches sich mit dem Cluster verbinden.
-> saubere -> .kube/config

Test
kubectl cluster-info
```

### Praktisches Beispiel bitnami/mysql (Dev/Ops)

#### Prerequisites

- helm needs a config-file (kubeconfig) to know how to connect and credentials in there
- Good: helm (as well as kubectl) works as unprivileged user as well - Good for our setup
- install helm on ubuntu (client) as root: snap install --classic helm
  - this installs helm3
- Please only use: helm3. No server-side components needed (in cluster)
  - Get away from examples using helm2 (hint: helm init) - uses tiller

#### Simple Walkthrough (Example 0: Step 1)

```
Repo hinzufügen
helm repo add bitnami https://charts.bitnami.com/bitnami
gecachte Informationen aktualisieren
helm repo update

helm search repo bitnami
helm install release-name bitnami/mysql
```

#### Simple Walkthrough (Example 0: Step 2: for learning - pull)

```
helm pull bitnami/mysql
tar xvfz mysql*
```

#### Simple Walkthrough (Example 0: Step 3: install)

```

helm install my-mysql bitnami/mysql
Chart runterziehen ohne installieren
helm pull bitnami/mysql

Release anzeigen zu lassen
helm list

Status einer Release / Achtung, heisst nicht unbedingt nicht, dass pod läuft
helm status my-mysql

weitere release installieren
helm install neuer-release-name bitnami/mysql

```

## Under the hood

```

Helm speichert Informationen über die Releases in den Secrets
kubectl get secrets | grep helm

```

## Example 1: - To get know the structure

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update
helm pull bitnami/mysql
tar xzvf mysql-9.0.0.tgz

Show how the template would look like being sent to kube-api-server
helm template bitnami/mysql

```

## Example 2: We will setup mysql without persistent storage (not helpful in production ;o)

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update

helm install my-mysql bitnami/mysql

```

## Example 2 - continue - fehlerbehebung

```

helm uninstall my-mysql
Install with persistentStorage disabled - Setting a specific value
helm install my-mysql --set primary.persistence.enabled=false bitnami/mysql

just as notice
helm uninstall my-mysql

```

## Example 2b: using a values file

```

mkdir helm-mysql
cd helm-mysql
vi values.yml
primary:
 persistence:
 enabled: false

```

```

helm uninstall my-mysql
helm install my-mysql bitnami/mysql -f values.yml

```

## Example 3: Install wordpress

### Example 3.1: Setting values with --set

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress \
 --set wordpressUsername=admin \
 --set wordpressPassword=password \
 --set mariadb.auth.rootPassword=secretpassword \
 bitnami/wordpress

```

### Example 3.2: Setting values with values.yml file

```
cd
mkdir -p manifests
cd manifests
mkdir helm-wordpress
cd helm-wordpress
nano values.yml
```

```
values.yml
wordpressUsername: admin
wordpressPassword: password
mariadb:
 auth:
 rootPassword: secretpassword
```

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress -f values.yml bitnami/wordpress
```

## Referenced

- <https://github.com/bitnami/charts/tree/master/bitnami/mysql/#installing-the-chart>
- <https://helm.sh/docs/intro/quickstart/>

## Kustomize

### Beispiel ConfigMap - Generator

#### Walkthrough

```
External source of truth
Create a application.properties file
vi application.properties
USER=letterman
ORG=it

No use the generator
the name need to be kustomization.yaml
```

```
kustomization.yaml
configMapGenerator:
- name: example-configmap-1
 files:
 - application.properties
```

```
See the output
kubectl kustomize ./

run and apply it
kubectl apply -k .
configmap/example-configmap-1-k4dmb9cbmb created
```

#### Ref.

- <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

### Beispiel Overlay und Patching

#### Konzept Overlay

- Base + Overlay = Gepatchtes manifest
- Sachen patchen.
- Die werden drübergelegt.

#### Example 1: Walkthrough

```
cd
mkdir -p manifests
cd manifests
mkdir kexample
cd kexample
```

```
Step 1:
Create the structure
kustomize-example1
L base
| - kustomization.yml
L overlays
##. L dev
- kustomization.yml
```

```
##. L prod
##. - kustomization.yml
mkdir -p kustomize-example1/base
mkdir -p kustomize-example1/overlays/prod
cd kustomize-example1
```

```
Step 2: base dir with files
now create the base kustomization file
vi base/kustomization.yml
resources:
- service.yml
```

```
Step 3: Create the service - file
vi base/service.yml
kind: Service
apiVersion: v1
metadata:
 name: service-app
spec:
 type: ClusterIP
 selector:
 app: simple-app
 ports:
 - name: http
 port: 80
```

```
See how it looks like
kubectl kustomize ./base
```

```
Step 4: create the customization file accordingly
##vi overlays/prod/kustomization.yaml
bases:
- ../../base
patches:
- path: service-ports.yaml
```

```
Step 5: create overlay (patch files)
vi overlays/prod/service-ports.yaml
kind: Service
apiVersion: v1
metadata:
 #Name der zu patchenden Ressource
 name: service-app
spec:
 # Changed to Nodeport
 type: NodePort
 ports: #Die Porteinstellungen werden überschrieben
 - name: https
 port: 443
```

```
Step 6:
kubectl kustomize overlays/prod/

or apply it directly
kubectl apply -k overlays/prod/
```

```
Step 7:
mkdir -p overlays/dev
vi overlays/dev/kustomization
bases:
- ../../base
```

```
Step 8:
statt mit der base zu arbeiten
kubectl kustomize overlays/dev
```

#### Example 2: Advanced Patching with patchesJson6902 (You need to have done example 1 firstly)

```
DEPRECATED ---- use below version
Schritt 1:
Replace overlays/prod/kustomization.yml with the following syntax
bases:
- ../../base
patchesJson6902:
- target:
```

```
version: v1
kind: Service
name: service-app
path: service-patch.yaml
```

```
Schritt 1:
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
patches:
- path: service-patch.yaml
 target:
 kind: Service
 name: service-app
 version: v1
```

```
Schritt 2:
vi overlays/prod/service-patch.yaml
- op: remove
 path: /spec/ports
 value:
 - name: http
 port: 80
- op: add
 path: /spec/ports
 value:
 - name: https
 port: 443
```

```
Schritt 3:
kubectl kustomize overlays/prod
```

### Special Use Case: Change the metadata.name

```
Same as Example 2, but patch-file is a bit different
vi overlays/prod/service-patch.yaml
- op: remove
 path: /spec/ports
 value:
 - name: http
 port: 80

- op: add
 path: /spec/ports
 value:
 - name: https
 port: 443

- op: replace
 path: /metadata/name
 value: svc-app-test
```

```
kubectl kustomize overlays/prod
```

### Ref:

- <https://blog.ordix.de/kubernetes-anwendungen-mit-kustomize>

### Resources

#### Where ?

- Used in base

```
base/kustomization.yml
which resources to use
e.g
resources:
- my-manifest.yml
```

#### Which ?

- URL
- filename
- Repo (git)

### Example:

```
kustomization.yaml
resources:
a repo with a root level kustomization.yaml
- github.com/Liujingfang1/mysql
a repo with a root level kustomization.yaml on branch test
- github.com/Liujingfang1/mysql?ref=test
a subdirectory in a repo on branch repoUrl2
- github.com/Liujingfang1/kustomize/examples/helloWorld?ref=repoUrl2
a subdirectory in a repo on commit `7050a45134e9848fca214ad7e7007e96e5042c03`
- github.com/Liujingfang1/kustomize/examples/helloWorld?ref=7050a45134e9848fca214ad7e7007e96e5042c03
```

## Kubernetes Rechteverwaltung (RBAC)

### Wie aktivieren?

#### Generell

Es muss das `flat --authorization-mode=RBAC` für den Start des Kube-API-Server gesetzt werden

Dies ist bei jedem Installationssystem etwas anders (microk8s, Rancher etc.)

### Wie ist es bei microk8s

Auf einem der Node:

```
microk8s enable rbac
```

ausführen

Wenn ich ein HA-Cluster (control-planes) eingerichtet habe, ist dies auch auf den anderen Nodes (Control-Planes) aktiv.

### Praktische Umsetzung anhand eines Beispiels (Ops)

#### Enable RBAC in microk8s

```
This is important, if not enable every user on the system is allowed to do everything
microk8s enable rbac
```

#### Wichtig:

```
Jeder verwendet seine eigene teilnehmer-nr z.B.
training1
training2
usw. ;o)
```

### Schritt 1: Nutzer-Account auf Server anlegen / in Client

```
cd
mkdir -p manifests/rbac
cd manifests/rbac
```

#### Mini-Schritt 1: Definition für Nutzer

```
vi service-account.yml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: training<nr> # <nr> entsprechend eintragen
 namespace: default

kubectl apply -f service-account.yml
```

#### Mini-Schritt 2: ClusterRolle festlegen - Dies gilt für alle namespaces, muss aber noch zugewiesen werden

```
Bevor sie zugewiesen ist, funktioniert sie nicht - da sie keinem Nutzer zugewiesen ist

vi pods-clusterrole.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: pods-clusterrole-<nr> # für <nr> teilnehmer - nr eintragen
rules:
- apiGroups: [""] # "" indicates the core API group
```

```
resources: ["pods"]
verbs: ["get", "watch", "list"]
```

```
kubectl apply -f pods-clusterrole.yml
```

#### Mini-Schritt 3: Die ClusterRolle den entsprechenden Nutzern über RoleBinding zu ordnen

```
vi rb-training-ns-default-pods.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: rolebinding-ns-default-pods<nr>
 namespace: default
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: pods-clusterrole-<nr> # <nr> durch teilnehmer nr ersetzen
subjects:
- kind: ServiceAccount
 name: training<nr> # nr durch teilnehmer - nr ersetzen
 namespace: default

kubectl apply -f rb-training-ns-default-pods.yml
```

#### Mini-Schritt 4: Testen (klappt der Zugang)

```
kubectl auth can-i get pods -n default --as system:serviceaccount:default:training<nr> # nr durch teilnehmer - nr ersetzen
```

### Schritt 2: Context anlegen / Credentials auslesen und in kubeconfig hinterlegen

#### Mini-Schritt 1: kubeconfig setzen

```
kubectl config set-context training-ctx --cluster microk8s-cluster --user training<nr> # <nr> durch teilnehmer - nr ersetzen

extract name of the token from here
TOKEN_NAME=`kubectl -n default get serviceaccount training<nr> -o jsonpath='{.secrets[0].name}'` # nr durch teilnehmer <nr>
ersetzen

TOKEN=`kubectl -n default get secret $TOKEN_NAME -o jsonpath='{.data.token}' | base64 --decode`
echo $TOKEN
kubectl config set-credentials training<nr> --token=$TOKEN # <nr> druch teilnehmer - nr ersetzen
kubectl config use-context training-ctx

Hier reichen die Rechte nicht aus
kubectl get deploy
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kube-system:training" cannot list # resource
"pods" in API group "" in the namespace "default"
```

#### Mini-Schritt 2:

```
kubectl config use-context training-ctx
kubectl get pods
```

#### Refs:

- <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingserviceaccttoken.htm>
- <https://microk8s.io/docs/multi-user>
- <https://faun.pub/kubernetes-rbac-use-one-role-in-multiple-namespaces-d1d08bb08286>

## Kubernetes Backups

### Kubernetes Backup

#### Background

- Belongs to veeam (one of the major companies for backup software)

#### What does Kubernetes Native Backup mean ?

- It is tight into the control plane, so it knows about the objects
- Uses the api to find out about Kubernetes

#### Setup a storage class (Where to store backup)

- <https://docs.kasten.io/latest/install/storage.html#direct-provider-integration>

#### Inject backup into a namespace to be used by app

- <https://docs.kasten.io/latest/install/generic.html#using-sidecars>

#### Restore:

```
Restore is done on the K10 - Interface
```

### Creating MYSQL - Backup / Restore with Kasten

- TODO: maybe move this to a separate page
- <https://blog.kasten.io/kubernetes-backup-and-restore-for-mysql>

### Ref:

- <https://www.kasten.io>
- [Installation DigitalOcean](#)
- [Installation Kubernetes \(Other distributions\)](#)

### Kasten.io overview

- <https://docs.kasten.io/latest/usage/overview.html>

## Kubernetes Monitoring

### Debugging von Ingress

#### 1. Schritt Pods finden, die als Ingress Controller fungieren

```
-A alle namespaces
kubectl get pods -A | grep -i ingress

jetzt sollten die pods zu sehen
Dann logs der Pods anschauen und gucken, ob Anfrage kommt
Hier steht auch drin, wo sie hin geht (zu welcher PodIP)
microk8s -> namespace ingress
Frage: HTTP_STATUS_CODE welcher ? z.B. 404
kubectl logs -n ingress <controller-ingress-pod>
```

#### 2. Schritt Pods analysieren, die Anfrage bekommen

```
Dann den Pod herausfinden, wo die Anfrage hinging
anhand der IP
kubectl get pods -o wide

Den entsprechenden pod abfragen bzgl. der Logs
kubectl logs <pod-name-mit-ziel-ip>
```

### Ebenen des Loggings

- container-level logging
- node-level logging
- Cluster-Ebene (cluster-wide logging)

### Working with kubectl logs

#### Logs

```
kubectl logs <container>
kubectl logs <deployment>
e.g.
kubectl logs -n namespace8 deploy/nginx
with timestamp
kubectl logs --timestamp -n namespace8 deploy/nginx
continuously show output
kubectl logs -f <container>
```

### Built-In Monitoring tools - kubectl top pods/nodes

#### Warum ? Was macht er ?

```
Der Metrics-Server sammelt Informationen von den einzelnen Nodes und Pods
Er bietet mit

kubectl top pods
kubectl top nodes

ein einfaches Interface, um einen ersten Eindruck über die Auslastung zu bekommen.
```

### Walkthrough

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm -n kube-system upgrade --install metrics-server metrics-server/metrics-server --version 3.13.0
```



```
Es dauert jetzt einen Moment bis dieser aktiv ist auch nach der Installation
Auf dem Client
kubectl top nodes
kubectl top pods
```

## Kubernetes

- <https://kubernetes-sigs.github.io/metrics-server/>
- kubectl apply -f <https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml>

## Protokollieren mit Elasticsearch und Fluentd (Devs/Ops)

### Installieren

```
microk8s enable fluentd

Zum anzeigen von kibana
kubectl port-forward -n kube-system service/kibana-logging 8181:5601
in anderer Session Verbindung aufbauen mit ssh und port forwarding
ssh -L 8181:127.0.0.1:8181 11trainingdo@167.172.184.80

Im browser
http://localhost:8181 aufrufen
```

### Konfigurieren

```
Discover:
Innerhalb von kibana -> index erstellen
auch nochmal in Grafiken beschreiben (screenshots von kibana)
https://www.digitalocean.com/community/tutorials/how-to-set-up-an-elasticsearch-fluentd-and-kibana-efk-logging-stack-on-kubernetes
```

## Long Installation step-by-step - Digitalocean

- <https://www.digitalocean.com/community/tutorials/how-to-set-up-an-elasticsearch-fluentd-and-kibana-efk-logging-stack-on-kubernetes>

## Setting up metrics-server - microk8s

### Warum ? Was macht er ?

```
Der Metrics-Server sammelt Informationen von den einzelnen Nodes und Pods
Er bietet mit

kubectl top pods
kubectl top nodes

ein einfaches Interface, um einen ersten Eindruck über die Auslastung zu bekommen.
```

### Walkthrough

```
Auf einem der Nodes im Cluster (HA-Cluster)
microk8s enable metrics-server

Es dauert jetzt einen Moment bis dieser aktiv ist auch nach der Installation
Auf dem Client
kubectl top nodes
kubectl top pods
```

## Kubernetes

- <https://kubernetes-sigs.github.io/metrics-server/>
- kubectl apply -f <https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml>

## Kubernetes Monitoring (CheckMK)

### CheckMK RAW - Kubernetes Monitoring einrichten

### CheckMK Enterprise Features für Kubernetes

#### Prerequisites

- Use at least version 2.1 of checkmk

#### Spoiler Alert

- Kubernetes läßt sich in checkmk nur gut monitoren mit der checkmk Enterprise - Version

### Was hat die checkmk Enterprise gegenüber der Raw-Version besonderes für Kubernetes

- Spezielle Kubernetes Dashboard, bereits fix und fertig. (Ansonsten sehr viel Arbeit nachzubauen)
- Noch wichtiger: Dynamic Host Management

### Dynamic Host Management

- Neues Pods werden automatisch erkannt (auch Services, Deployments usw.)
- Ressourcen, die nicht mehr da sind, werden automatisch rausgelöscht.

### Alternative Dynamic Host Management: Ich müsste das händisch machen bzw. selbst scripten

- Ich müsste rest-api call ausführen, um dies über die api von check zu setzen.
- Neue Hosts müßte ich discovern, alte rauslöschen (automatisch)

### Alternative: Dashboards

- Dashboards selber bauen
- Grafana mit checkmk - Daten (Aus meiner Sicht wenig praktikabel, dann entweder ganz Grafana + Prometheus oder checkmk Enterprise)

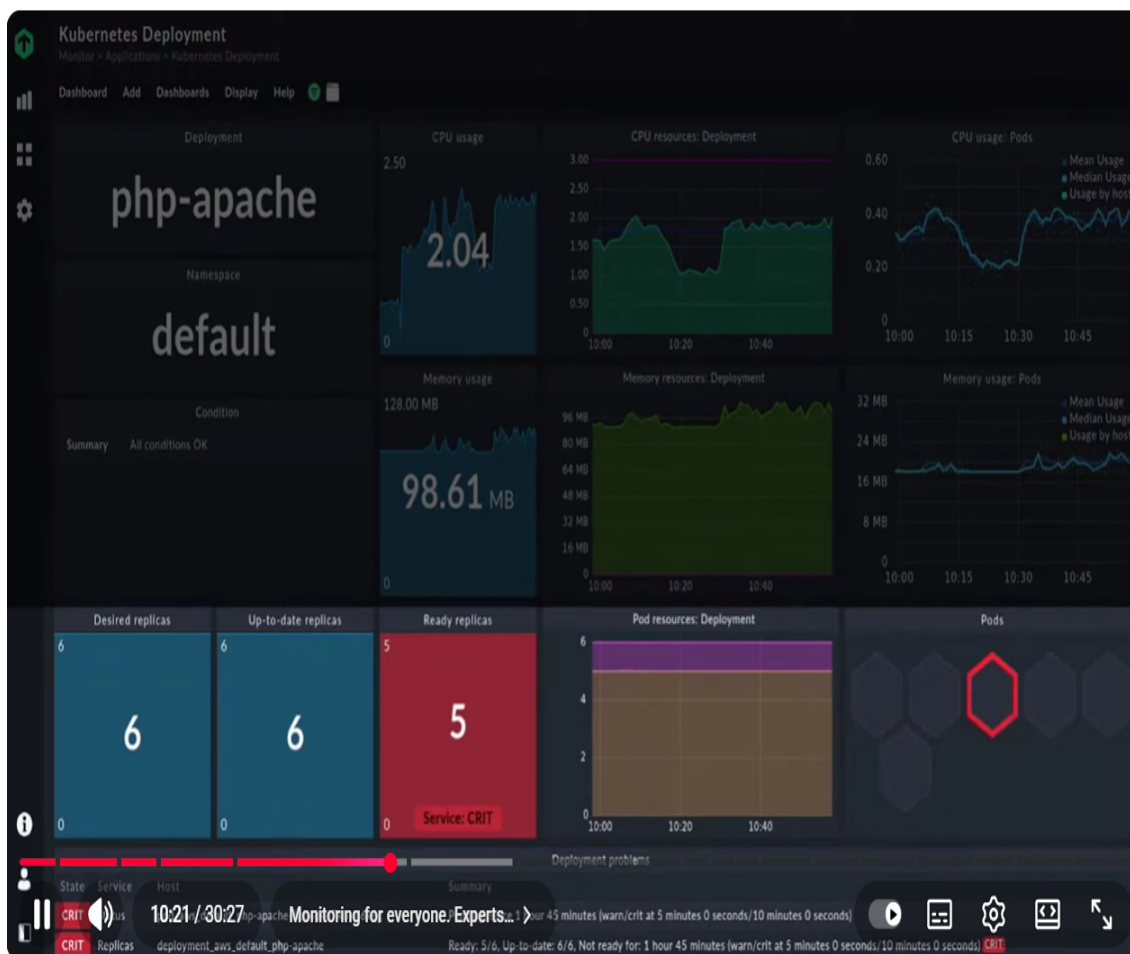
### Kubernetes Dashboards in CheckMK

- These dashboards are only available in Checkmk Enterprise (so not in community version: CheckMk Raw)
- But, you could also create them yourself (not recommended)
  - Ref: <https://docs.checkmk.com/latest/en/dashboards.html>

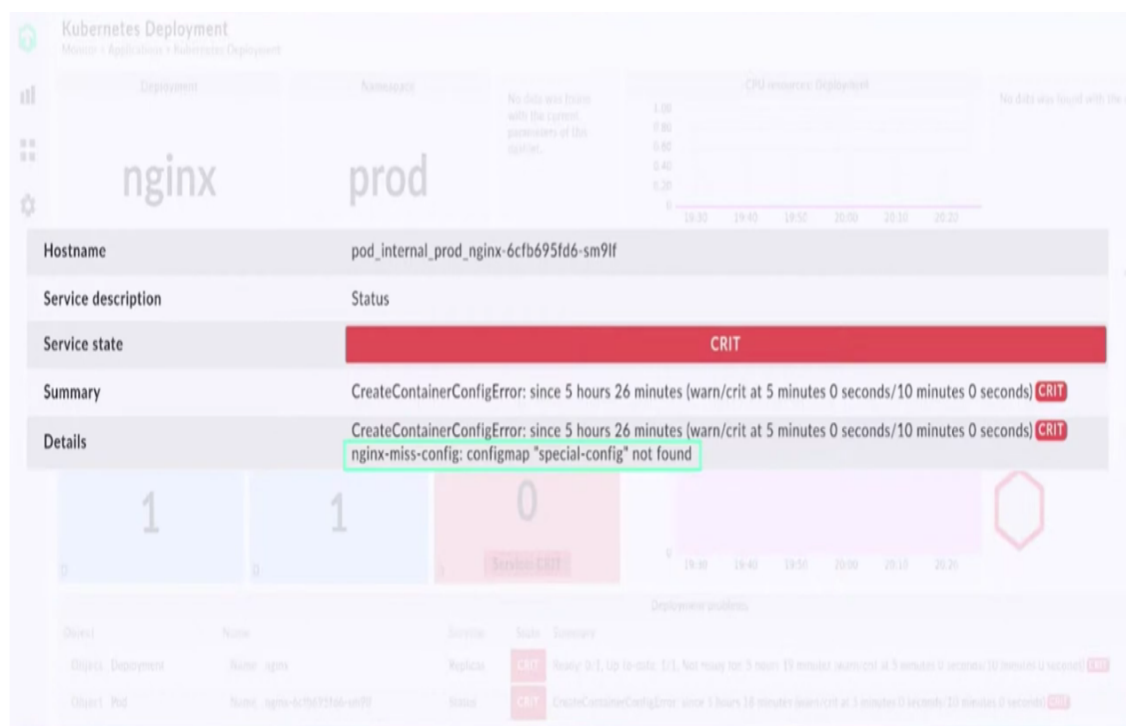
### Cluster Dashboard



### Deployment Dashboard



#### Detail



## References:

- [Checkmk about new Kubernetes Integration \(since 2022\)](#)

## Kubernetes Security

### Grundlagen und Beispiel (Praktisch)

#### PSA (Pod Security Admission)

Policies defined by namespace.  
e.g. not allowed to run container as root.

Will complain/deny when creating such a pod with that container type

#### Example (seccomp / security context)

A. seccomp - profile  
<https://github.com/docker/docker/blob/master/profiles/seccomp/default.json>

```
apiVersion: v1
kind: Pod
metadata:
 name: audit-pod
 labels:
 app: audit-pod
spec:
 securityContext:
 seccompProfile:
 type: Localhost
 localhostProfile: profiles/audit.json

 containers:

 - name: test-container
 image: hashicorp/http-echo:0.2.3
 args:
 - "-text=just made some syscalls!"
 securityContext:
 allowPrivilegeEscalation: false
```

#### SecurityContext (auf Pod Ebene)

```
kubectl explain pod.spec.containers.securityContext
```

#### NetworkPolicy

```
Firewall Kubernetes
```

## Grundlagen Security

### Geschichte

- Namespaces sind die Grundlage für Container
- LXC - Container

### Grundlagen

- letztendlich nur ein oder mehreren laufenden Prozesse im Linux - Systeme

#### Seit: 1.2.22 Pod Security Admission

- 1.2.22 - Alpha - D.h. ist noch nicht aktiviert und muss als Feature Gate aktiviert (Kind)
- 1.2.23 - Beta -> d.h. aktiviert

#### Vorgefertigte Regelwerke

- privileges - keinerlei Einschränkungen
- baseline - einige Einschränkungen
- restricted - sehr streng

#### Praktisches Beispiel für Version ab 1.2.23 - Problemstellung

```
Schritt 1: Namespace anlegen

mkdir manifests/security
cd manifests/security
vi 01-ns.yml
```

```
apiVersion: v1
kind: Namespace
metadata:
 name: test-ns<tln>
 labels:
 pod-security.kubernetes.io/enforce: baseline
 pod-security.kubernetes.io/audit: restricted
 pod-security.kubernetes.io/warn: restricted
```

```
kubectl apply -f 01-ns.yml
```

```
Schritt 2: Testen mit nginx - pod
vi 02-nginx.yml
```

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 namespace: test-ns<tln>
spec:
 containers:
 - image: nginx
 name: nginx
 ports:
 - containerPort: 80
```

```
a lot of warnings will come up
kubectl apply -f 02-nginx.yml
```

```
Schritt 3:
Anpassen der Sicherheitseinstellung (Phase1) im Container
```

```
vi 02-nginx.yml

apiVersion: v1
kind: Pod
metadata:
 name: nginx
 namespace: test-ns<tln>
spec:
 containers:
 - image: nginx
 name: nginx
 ports:
 - containerPort: 80
 securityContext:
 seccompProfile:
 type: RuntimeDefault
```

```
kubectl delete -f 02-nginx.yml
kubectl apply -f 02_pod.yml
kubectl -n test-ns<tln> get pods
```

```
Schritt 4:
Weitere Anpassung runAsNotRoot
vi 02-nginx.yml
```

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 namespace: test-ns12
spec:
 containers:
 - image: nginx
 name: nginx
 ports:
 - containerPort: 80
 securityContext:
 seccompProfile:
 type: RuntimeDefault
 runAsNonRoot: true
```

```
pod kann erstellt werden, wird aber nicht gestartet
kubectl delete -f 02_pod.yml
kubectl apply -f 02_pod.yml
```

```
kubectl -n test-ns<tl> get pods
kubectl -n test-ns<tl> describe pods nginx
```

### Praktisches Beispiel für Version ab 1.2.23 -Lösung - Container als NICHT-Root laufen lassen

- Wir müssen ein image, dass auch als NICHT-Root kaufen kann
- .. oder selbst eines bauen (:o)) o bei nginx ist das bitnami/nginx

```
vi 03-nginx-bitnami.yml
apiVersion: v1
kind: Pod
metadata:
 name: bitnami-nginx
 namespace: test-ns12
spec:
 containers:
 - image: bitnami/nginx
 name: bitnami-nginx
 ports:
 - containerPort: 80
 securityContext:
 seccompProfile:
 type: RuntimeDefault
 runAsNonRoot: true
```

```
und er läuft als nicht root
kubectl apply -f 03_pod-bitnami.yml
kubectl -n test-ns<tl> get pods
```

## Kubernetes GUI

### Rancher

#### Was ist Rancher ?

- Eine GUI für Kubernetes
- Neben dem Kubernetes Cluster, gibt es den Rancher-Server eine Web-Oberfläche zum Verwalten des Cluster und dafür Anwendungen auszurollen
- Verwendet k3s als Kubernetes-Distribution (<https://rancher.com/docs/k3s/latest/en/architecture/>)

#### Reference

- Nette kurze Beschreibung
  - <https://www.dev-insider.de/container-orchestrierung-mit-rancher-a-886962/>
- Hintergründe:
  - <https://rancher.com/why-rancher>

### Kubernetes Dashboard

#### Setup / Walkthrough

##### Step 1: Enable Dashboard

```
Auf Node 1:
microk8s enable dashboard

Wenn rbac aktiviert ist, einen Nutzer mit Berechtigung einrichten
microk8s status | grep -i rbac
```

##### Step 2: Create a user and bind it to a specific role

```
Wir verwenden die Rolle cluster-admin, die standardmäßig alles darf
kubectl -n kube-system get ClusterRole cluster-admin -o yaml

Wir erstellen einen System-Account (quasi ein Nutzer): admin-user
mkdir manifests/dashboard
cd manifests/dashboard
```

```
vi dashboard-admin-user.yml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: admin-user
 namespace: kube-system
```

```
Apply'en
kubectl apply -f dashboard-admin-user.yml
```

```
Jetzt erfolgt die Zuordnung des Users zur Rolle
adminuser-rolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: admin-user
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: cluster-admin
subjects:
- kind: ServiceAccount
 name: admin-user
 namespace: kube-system
```

```
Und anwenden
kubectl apply -f adminuser-rolebinding.yaml
```

```
Damit wir zugreifen können, brauchen wir jetzt den Token für den Service - Account
kubectl -n kube-system describe secret $(kubectl -n kube-system get secret | grep admin-user | awk '{print $1}')
Diesen kopieren wir in das Clipboard und brauche ihn dann demnächst zum Anmelden
```

- Tricky to find a good solution because of different namespace
- Ref: <https://www.linkedin.com/pulse/9-steps-enable-kubernetes-dashboard-microk8s-hendri-t/>

### Step 3: Verbindung aufbauen

```
Auf Client proxy starten
kubectl proxy

Wenn Client, nicht Dein eigener Rechner ist, dann einen Tunnel von Deinem eigenen Rechner zum Client aufbauen
ssh -L localhost:8001:127.0.0.1:8001 tln1@138.68.92.49

In Deinem Browser auf Deinem Rechner folgende URL öffnen
http://localhost:8001/api/v1/namespaces/kube-system/services/https:kubernetes-dashboard:/proxy/

Jetzt kannst Du Dich einloggen - verwende das Token von oben, dass Du ins clipboard kopiert hast.
```

## Kubernetes CI/CD (Optional)

## Tipps & Tricks

### Ubuntu client aufsetzen

```
Now let us do some generic setup
echo "Installing kubectl"
snap install --classic kubectl

echo "Installing helm"
snap install --classic helm

apt-get update
apt-get install -y bash-completion
source /usr/share/bash-completion/bash_completion
is it installed properly
type _init_completion

activate for all users
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl > /dev/null

Activate syntax - stuff for vim
Tested on Ubuntu
echo "hi CursorColumn cterm=NONE ctermbg=lightred ctermfg=white" >> /etc/vim/vimrc.local
echo "autocmd FileType y?ml setlocal ts=2 sts=2 sw=2 ai number expandtab cursorline cursorcolumn" >> /etc/vim/vimrc.local

Activate Syntax highlighting for nano
cd /usr/local/bin
git clone https://github.com/serialhex/nano-highlight.git
Now set it generically in /etc/nanorc to work for all
echo 'include "/usr/local/bin/nano-highlight/yaml.nanorc"' >> /etc/nanorc
```

### bash-completion

### Walkthrough

```

apt install bash-completion
source /usr/share/bash-completion/bash_completion
is it installed properly
type _init_completion

activate for all users
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl > /dev/null

verifizieren - neue login shell
su -

zum Testen
kubectl g<TAB>
kubectl get

```

### Alternative für k als alias für kubectl

```

source <(kubectl completion bash)
complete -F __start_kubectl k

für permanente Nutzungn
eintragen in ~/.bash_profile oder anlegen, wenn datei nicht vorhanden
alias k=kubectl
complete -o default -F __start_kubectl k

und danach rausgehen und entweder
Session neu starten
pder su - <benutzername>

```

### Reference

- <https://kubernetes.io/docs/tasks/tools/included/optional-kubectl-configs-bash-linux/>

### Alias in Linux kubectl get -o wide

```

cd
echo "alias kgw='kubectl get -o wide'" >> .bashrc
for it to take immediately effect or relogin
bash
kgw pods

```

### vim einrückung für yaml-dateien

#### Ubuntu (im Unterverzeichnis /etc/vim - systemweit)

```

hi CursorColumn cterm=NONE ctermbg=lightred ctermfg=white
autocmd FileType yaml setlocal ts=2 sts=2 sw=2 ai number expandtab cursorline cursorcolumn

```

### Testen

```

vim test.yaml
Eigenschaft: <return> # springt eingerückt in die nächste Zeile um 2 spaces eingerückt

evtl funktioniert vi test.yaml auf manchen Systemen nicht, weil kein vim (vi improved)

```

### kubectl spickzettel

#### Hilfe

```

Hilfe zu befehl
kubectl help config
Hilfe nächste Ebene
kubectl config set-context --help

```

#### Allgemein

```

Zeige Informationen über das Cluster
kubectl cluster-info

Welche Ressourcen / Objekte gibt es, z.B. Pod
kubectl api-resources
kubectl api-resources | grep namespaces

Hilfe zu object und eigenschaften bekommen

```



```
kubectl explain pod
kubectl explain pod.metadata
kubectl explain pod.metadata.name
```

## namespaces

```
kubectl get ns
kubectl get namespaces

namespace wechseln, z.B. nach Ingress
kubectl config set-context --current --namespace=ingress
jetzt werden alle Objekte im Namespace Ingress angezeigt
kubectl get all,configmaps

wieder zurückwechseln.
der standardmäßige Namespace ist 'default'
kubectl config set-context --current --namespace=default
```

## Arbeiten mit manifesten

```
kubectl apply -f nginx-replicaset.yml
Wie ist aktuell die hinterlegte config im system
kubectl get -o yaml -f nginx-replicaset.yml

Änderung in nginx-replicaset.yml z.B. replicas: 4
dry-run - was wird geändert
kubectl diff -f nginx-replicaset.yml

anwenden
kubectl apply -f nginx-replicaset.yml

Alle Objekte aus manifest löschen
kubectl delete -f nginx-replicaset.yml

Recursive Löschen
cd ~/manifests
multiple subfolders subfolders present
kubectl delete -f . -R
```

## Ausgabeformate / Spezielle Informationen

```
Ausgabe kann in verschiedenen Formaten erfolgen
kubectl get pods -o wide # weitere Informationen
im json format
kubectl get pods -o json
eine Werte rausziehen
key ist tls.crt (wichtig escapen => \.), sonst funktioniert das nicht
kubectl get secrets example-tls -o jsonpath='{.data.tls\.crt}' | base64 -d

gilt natürluch auch für andere kommandos
kubectl get deploy -o json
kubectl get deploy -o yaml

Label anzeigen
kubectl get deploy --show-labels
```

## Zu den Pods

```
Start einen pod // BESSER: direkt manifest verwenden
kubectl run podname image=imagename
kubectl run nginx image=nginx

Pods anzeigen
kubectl get pods
kubectl get pod

Pods in allen namespaces anzeigen
kubectl get pods -A

Format weitere Information
kubectl get pod -o wide
Zeige labels der Pods
kubectl get pods --show-labels

Zeige pods mit einem bestimmten label
```

```
kubectl get pods -l app=nginx

Status eines Pods anzeigen
kubectl describe pod nginx

Pod löschen
kubectl delete pod nginx
Löscht alle Pods im eigenen Namespace bzw. Default
kubectl delete pods --all

Kommando in pod ausführen
kubectl exec -it nginx -- bash
```

## Deployments

```
kubectl -n ingress rollout restart deployment traefik
ik
```

## Alle Objekte anzeigen

```
Nur die wichtigsten Objekte werden mit all angezeigt
kubectl get all
Dies, kann ich wie folgt um weitere ergänzen
kubectl get all,configmaps

Über alle Namespaces hinweg
kubectl get all -A
```

## Logs

```
kubectl logs <container>
kubectl logs <deployment>
e.g.
kubectl logs -n namespace8 deploy/nginx
with timestamp
kubectl logs --timestamps -n namespace8 deploy/nginx
continuously show output
kubectl logs -f <pod>
letzten x Zeilen anschauen aus log anschauen
kubectl logs --tail=5 <your pod>
```

## CRD

```
kubectl get crd
```

## Referenz

- <https://kubernetes.io/de/docs/reference/kubectl/cheatsheet/>

## Alte manifests migrieren

### What is about?

- Plugins needs to be installed seperately on Client (or where you have your manifests)

### Walkthrough

```
curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl-convert"
Validate the checksum
curl -LO "https://dl.k8s.io/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl-convert.sha256"
echo "${<kubectl-convert.sha256> kubectl-convert" | sha256sum --check
install
sudo install -o root -g root -m 0755 kubectl-convert /usr/local/bin/kubectl-convert

Does it work
kubectl convert --help

Works like so
Convert to the newest version
kubectl convert -f pod.yaml
```

## Reference

- <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/#install-kubectl-convert-plugin>

## X-Forward-Header-For setzen in Ingress

```
Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: apache-ingress
 annotations:
 ingress.kubernetes.io/rewrite-target: /
 nginx.ingress.kubernetes.io/configuration-snippet: |
 more_set_headers "X-Forwarded-For $http_x_forwarded_for";

spec:
 rules:
 - http:
 paths:
 - path: /project
 pathType: Prefix
 backend:
 service:
 name: svc-apache
 port:
 number: 80
```

#### Refs:

- <https://stackoverflow.com/questions/62337379/how-to-append-nginx-ip-to-x-forwarded-for-in-kubernetes-nginx-ingress-controller>
- <https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/annotations/#configuration-snippet>

## Übungen

### Training 23.–25.09.2026: Pad-Zusammenfassung (alle Übungen)

**Zeitraum:** 23.09. – 25.09.2026 **Quelle:** [yppad.eu/p/kubernetes](https://yppad.eu/p/kubernetes) (Stand 25.09.2026)

Dieses Dokument fasst das gemeinsame Etherpad des Trainings zusammen: alle Übungen und Infos in der Reihenfolge, in der wir sie durchgegangen sind, plus Zeitplan und Agenda-Status.

#### Organisatorisches

Was	Wo
Dokumentation / Agenda	<a href="#">README.md</a>
Jochen bewerten	<a href="https://reviews.appv2.t3isp.de/E4U6">https://reviews.appv2.t3isp.de/E4U6</a>
Newsletter	<a href="mailto:j.metzger@t3company.de">j.metzger@t3company.de</a>
Bastion-Client (SSH)	client-aok.do.t3isp.de, Port 22
SSH-Client für Windows	<a href="#">PuTTY</a>

Die Zuordnung der Teilnehmer zu den Logins `tl1` – `tl9` steht im Pad und wird hier bewusst nicht veröffentlicht.

#### Zeitplanung

Zeit	Block
09:00 – 10:30	Block I
10:30 – 10:45	Frühstück
10:45 – 12:15	Block II
12:15 – 13:15	Mittag
13:15 – 14:45	Block III
14:45 – 15:00	Teatime
15:00 – 16:30	Block IV

### Tag 1 – Container, Kubernetes-Architektur, Pods, Deployments, Services

#### Info 1.1 – Docker-Container auf dem Linux-System

Jochen malt: Wie ein Container auf dem Linux-System aufgebaut ist (Namespaces, Cgroups, Layer).

#### Info 1.2 – Docker Container

- [Was ist ein Container?](#)

#### Info 1.3 – Docker Images

- [Was sind Container Images?](#)

#### Info 1.4 – Schaubild Architektur

- [Übersicht Architektur](#)

#### Info 1.5 – Kubernetes Aufbau

- [Kubernetes Architektur](#)

#### Übung 1.6 – Zugang zum Trainings-Client

Mit PuTTY oder `ssh` auf den Bastion-Client verbinden:

```
ssh tln<x>@client-aok.do.t3isp.de
```

#### Übung 1.7 – kubectl einrichten

- [kubectl einrichten mit Namespace](#)

#### Übung 1.8 – Pod mit `kubectl run`

- [kubectl run – Beispiel](#)

#### Info 1.9 – Applikation aus Standardobjekten bauen

- [Bauen einer Webanwendung mit Resource-Objekten](#)

#### Info 1.10 – Anatomie einer Webanwendung

- [Anatomie einer Webanwendung](#)

#### Übung 1.11 – Pod mit Manifest

- [Pod nginx \(Manifest\)](#)

#### Übung 1.12 – Walkthrough: ReplicaSet erstellen

- [ReplicaSet – Walkthrough Erstellen](#)

#### Übung 1.13 – ReplicaSet erstellen

Nur den Punkt „Erstellen“ bearbeiten.

- [ReplicaSet nginx](#)

#### Übung 1.14 – Deployment

- [Deployment nginx](#)

#### Übung 1.15 – Services (ClusterIP)

- [Service – Example I: ClusterIP](#)

#### Übung 1.16 – Netzverbindung testen

```
IP-Adresse eines Pods aus dem Deployment ermitteln
kubectl get pods -o wide

IP des Services
kubectl get svc svc-nginx -o wide

Test-Pod starten (wird nach dem Beenden automatisch gelöscht)
kubectl run podtest --rm -it --image busybox
```

Innerhalb des Test-Pods:

```
ping -c4 <pod-ip>
wget -O - <pod-ip>
ping -c4 <cluster-ip>
wget -O - <cluster-ip>
exit
```

#### Übung 1.17 – NodePort

- [Service – Example II: NodePort](#)

#### Übung 1.18 – LoadBalancer (Example III)

- [Service – Example III: LoadBalancer / ExternalIP](#)

#### Übung 1.19 – DNS-basierte Verbindungen

- [DNS Resolution – Services](#)

#### Info 1.20 – Ingress Controller

- [Traefik Ingress Controller mit Helm installieren](#)

## Tag 2 – Ingress, ConfigMaps, Secrets, Vault, Helm

### Übung 2.1 – Ingress mit Hostnamen (Schritt 1: Deployment und Services)

Vorher aufräumen:

```
cd
cd manifests/04-service
```

```
kubectl delete -f .
```

- [Ingress mit Traefik und Hostnamen](#)

## Übung 2.2 – Ingress bis Schritt 4.1 (inklusive)

Thema: `kind` und die „Landkarte“ der API-Groups.

## Übung 2.3 – Ingress mit SSL / TLS

- [Ingress-Objekt mit TLS erstellen \(Schritt 3\)](#)

## Übung 2.4 – ConfigMap mit MariaDB (Schritt 1 + 2)

- [ConfigMap Example MariaDB](#)

## Übung 2.5 – Umbau auf Secret

Secret-Manifest per Dry-Run erzeugen:

```
kubectl create secret generic mariadb-secret \
 --from-literal=MARIADB_ROOT_PASSWORD=<passwort> \
 --dry-run=client -o yaml > 01-secrets.yml
```

Dann in `02-deploy.yml` anpassen:

- `configMapRef:` → `secretRef:`
- `mariadb-configmap` → `mariadb-secret`

```
kubectl apply -f .
kubectl get pods # Läuft der Pod?
```

## Info 2.6 – Vault

- [HashiCorp Vault – Architektur einfach erklärt](#)
- Ausführlich im Advanced-Workshop: [workshop-kubernetes-advanced-2026-Q3](#)

## Info 2.7 – Helm Grundlagen

- [Helm Grundlagen](#)

## Übung 2.8 – Helm Chart installieren (Schritt 1)

- [MariaDB mit Helm \(cloudpirates\)](#)

## Übung 2.9 – Helm Chart Upgrade (Schritt 2)

- [Upgrade auf neue Version](#)

## Übung 2.10 – Upgrade mit Deinstallation (Schritt 3)

- [Upgrade mit Deinstallation](#)

---

## Tag 3 – CNCF, Storage, RBAC, QoS, Autoscaling, StatefulSets, Monitoring

### Übung 3.1 – CNCF Landscape

- <https://landscape.cncf.io>

### Übung 3.2 – NFS mit Pod (CSI)

- [NFS Exercise – Persistent Volume Claim \(ab Step 3\)](#)

### Übung 3.3 – Traefik: RBAC-Analyse

```
kubectl get ns # Gibt es einen Namespace "ingress"?
kubectl -n ingress get sa
kubectl -n ingress get sa traefik
```

**Quizfrage:** Ihr wollt herausfinden, ob im Traefik-Pod wirklich der ServiceAccount `traefik` eingehängt ist. Wie seht ihr das?

```
kubectl -n ingress get pod <traefik-pod> -o yaml | grep serviceAccount
```

Weiter mit Rollen und ClusterRoles:

```
kubectl -n traefik get roles # Rolle traefik?
kubectl get clusterroles | grep traefik
kubectl get clusterrole <clusterrole-fuer-traefik> -o yaml
```

Siehe auch: [Kubernetes RBAC – was darf Traefik](#)

### Übung 3.4 – RoleBinding / ClusterRoleBinding

```
kubectl get clusterrolebinding | grep traefik-ingress
```

Info 3.5 – „Ich darf alles“: ClusterRole

Diskussion: Was bedeutet eine ClusterRole mit vollen Rechten, und warum sollte man das vermeiden.

Übung 3.6 – Pod suchen und Quality of Service ablesen

```
kubectl describe pods <name-des-pods> | grep QoS
```

- Hintergrund: [Quality of Service – evict pods](#)

Übung 3.7 – Horizontal Pod Autoscaler

- [Kubernetes Autoscaling](#)

Übung 3.8 – StatefulSet

- [Example StatefulSet](#)

Info 3.9 – Monitoring

- [Prometheus Monitoring Server \(Overview\)](#)
- [Prometheus / Grafana Slack installieren](#)

Übung 3.10 – Prometheus GUI

- [Übung: Prometheus UI und PromQL](#)

Übung 3.11 – Metrics-Endpoint direkt scrapen

```
kubectl run metrics-check -it --rm --image=curlimages/curl \
--restart=Never -- \
curl -s http://node-exporter.monitoring.svc.cluster.local:9100/metrics \
| head -40
```

- [Schritt 2: Metrics direkt ansehen](#)

Übung 3.12 – Readiness Probe

- [Übung: Readiness Probe mit HTTP](#)

Offene Themen / Wünsche aus dem Training

- Kong Gateway Operator
- PostgreSQL mit CloudNativePG → [HA mit dem Postgres Operator](#)

Agenda-Status (Abgleich mit der offiziellen Agenda)

Legende: ✓ behandelt · laufend · □ offen

Kubernetes Grundlagen

Thema	Status
Motivation für Container und Möglichkeiten der Containertechnologie	✓
Einführung in Containertechnologie und das Arbeiten mit Containern	✓
Docker Ecosystem	✓
Linux Kernelfunktionen	✓
Vergleich Systemvirtualisierung und Container	✓
Design-Prinzipien für Cloud-Native-Anwendungen	□

Einführung in Kubernetes

Thema	Status
Motivation für eine Orchestrierungsplattform	✓
Vorteile und Kosten von Kubernetes	✓
Eigenschaften von Kubernetes im Überblick	□

Kubernetes Architektur und Konzept

Thema	Status
System-Übersicht mit allen Komponenten (API Server, Controller Manager, Scheduler)	✓
Installations-Optionen (Cloud, Minikube, etc.)	□

Setup der Arbeitsumgebung und Nutzen der CLI

Thema	Status
Config-File und der Arbeitsbereich (Context)	✓

CLI-Tool (kubectl)	
Imperatives und deklaratives Management	✓

#### Pod-Konzept

Thema	Status
Pod-Konzept	✓

#### Flexibles Anwendungsdeployment

Thema	Status
Arbeiten mit Labels und Label-Selektoren	✓

#### Workloads

Thema	Status
Pods	✓
Deployments	✓
StatefulSets	✓ (Übung 3.8)
DaemonSets	□
Jobs	□

#### Datenspeicher bereitstellen

Thema	Status
Einfache Volumes	□
Persistente Volumes	✓ (Übung 3.2)

#### Konfigurationsdaten und Secrets bereitstellen

Thema	Status
ConfigMaps	✓
Secrets	✓

#### Netzwerkverbindungen bereitstellen

Thema	Status
Architektur des Kubernetes-Netzwerks	□
Verbindungen zwischen Containern, Verbindungen nach außen	✓
Load Balancing und NodePort	✓
DNS-basierte Verbindungen	✓
Ingress	✓ (Übungen 2.1 – 2.3)

#### Steuerung, Überwachung und Kontrolle von Anwendungen

Thema	Status
Quality Class	✓ (Übung 3.6)
Health Checks für Pods (Container)	✓ (Übung 3.12)
Scheduling steuern (Taints und Tolerations)	□

#### Komplexe Anwendungen einfach deployen: Der Helm-Paketmanager

Thema	Status
Paketformat	✓
Anwendungsdeployment vereinfachen	✓
Anwendungsdeployment flexibel gestalten	✓
Lifecycle-Management: Upgrade, Rollback und mehr	✓ (Übungen 2.8 – 2.10)
Helm Charts und die Community	✓

#### Troubleshooting

--	--

Thema	Status
Zugriff auf einen Pod ( <code>kubectl exec</code> )	<input type="checkbox"/> fehlt noch
Netzwerkverbindungen testen	<input checked="" type="checkbox"/>
Logging / Event-Infos des CLI-Tools	<input type="checkbox"/>

#### Zugriffskontrolle

Thema	Status
Rollenbasierte Zugriffskontrolle	<input checked="" type="checkbox"/> (Übungen 3.3 – 3.5)
Richtlinien	<input type="checkbox"/>
Service Accounts	<input checked="" type="checkbox"/> (Übung 3.3)

#### Dashboard und andere GUI

Thema	Status
Dashboard und andere GUI	<input type="checkbox"/>

#### Cluster-Erweiterungen

Thema	Status
Monitoring und Logging (Fluentd, Elastic, Prometheus)	<input checked="" type="checkbox"/> (Übungen 3.9 – 3.11)
Cluster DNS	<input checked="" type="checkbox"/>
CNCF und Ausblick → <a href="https://landscape.cncf.io">landscape.cncf.io</a>	<input checked="" type="checkbox"/> (Übung 3.1)

#### übung Tag 3

2) Übung

a) Deployed ein apache-server

```
-> hub.docker.com -> httpd
DocumentRoot (Pfad der Dokumente)
/usr/local/apache2/htdocs
```

b) Volume einhängen

```
/var/nfs/tln<x>/apache/
Im Container einhängen wie unter a) genannt ... apache2/htdocs usw.
```

-> Testen

C) Service bereitstellen ohne NodePort  
(ClusterIP)

-> Testen

D) Ingress-Config bereitstellen

```
/project
```

ACHTUNG: Struktur auf dem WebServer so angelegt sein muss  
wie auf nfs, (was den Unterordner betrifft)

-> Testen

#### übung Tag 4

Verwendet das nachfolgende Deployment und  
baut MYSQL\_ROOT\_PASSWORD so um, dass  
es aus secret kommt, welches aus einem  
sealed secret erstellt wird.

Stellt einen Service svc-mysql bereit, der auf einem  
NodePort lauscht.

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mysql
spec:
 selector:
```



```

 matchLabels:
 app: mysql
 strategy:
 type: Recreate
 template:
 metadata:
 labels:
 app: mysql
 spec:
 containers:
 - image: mysql:8.0
 name: mysql
 env:
 - name: MYSQL_ROOT_PASSWORD
 value: password
 ports:
 - containerPort: 3306
 name: mysql

```

## Fragen

### Q and A

#### Wieviele Replicaset beim Deployment zurückbehold / Löschen von Replicaset

```

kubectl explain deployment.spec.revisionHistoryLimit

apiVersion: apps/v1
kind: Deployment
...
spec:
 # ...
 revisionHistoryLimit: 0 # Default to 10 if not specified
 # ...

```

#### Wo dokumentieren, z.B. aus welchem Repo / git

Labels can be used to select objects and to find collections of objects that satisfy certain conditions. In contrast, annotations are not used to identify and select objects.

- <https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>
- <https://kubernetes.io/docs/reference/labels-annotations-taints/>

#### Wie groß werden die Logs der einzelnen Pods maximal ?

```

10 mb. max
Wird im kubelet konfiguriert.
containerMaxLogSize

```

## Kubernetes und Ansible

### Warum ?

- Hilft mir mein Cluster auszurollen (Infrastruktur)
- Verwalten der gesamten Applikation (manifeste etc.) über Ansible

### Für Infrastruktur

- Hervorragende Lösung. Erleichtert die Deployment-Zeit.
- Möglichst schlank und einfach mit Module halten,
  - z.B. [https://docs.ansible.com/ansible/latest/collections/community/aws/aws\\_eks\\_cluster\\_module.html](https://docs.ansible.com/ansible/latest/collections/community/aws/aws_eks_cluster_module.html)

### Empfehlungen Applikation

- Eigenes Repos mit manifesten (losgelöst von ansible playbooks)
- Vorteil: Entwickler und andere Teams können das gleiche Repo verwenden
- Kein starkes Solution-LockIn.
- Denkbar: Das dann ansible darauf zugreift.

### Fragen Applikation

- Zu klären: Wie läuft der LifeCycle.
- Wie werden neue Versionen ausgerollt ? -> Deployment - Prozess

### Empfehlung Image

- Bereitstellen über Registry (nicht repo ansible)
- Binaries gehören nicht in repos (git kann das nicht so gut)

#### **Alternativ bzw. Ergänzung**

- Terraform

#### **Documentation**

##### **Kubernetes mit VisualStudio Code**

- <https://code.visualstudio.com/docs/azure/kubernetes>

##### **Kube Api Ressourcen - Versionierungsschema**

##### **Wie ist die deprecation policy ?**

- <https://kubernetes.io/docs/reference/using-api/deprecation-policy/>

##### **Was ist wann deprecated ?**

- <https://kubernetes.io/docs/reference/using-api/deprecation-guide/>

##### **Reference:**

- <https://kubernetes.io/docs/reference/using-api/>

##### **Kubernetes Labels and Selector**

- <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>

#### **Documentation - Sources**

##### **controller manager**

- <https://github.com/kubernetes/kubernetes/tree/release-1.29/cmd/kube-controller-manager/app/options>